

3X-UI 面板用户手册

 العربية ·  English ·  Español ·  فارسی ·  Bahasa Indonesia ·  日本語 ·  Português ·  Русский · 
Türkçe ·  Українська ·  Tiếng Việt ·  简体中文 ·  繁體中文

3X-UI 版本：3.5.0。 本手册依据该版本编写，内容与之对应。3.5.0 相对于 3.4.2 的变更摘要，请参阅「[3.5.0 新特性](#)」章节。

本手册为 **3X-UI** 网页面板（管理 Xray-core）的详细使用指南，涵盖各项功能、配置与运维说明，并对界面中的每个字段和开关逐一解析。

名称和标签与面板界面保持一致。*inbound* / *outbound* 两词不作翻译。

目录

- 3.5.0 新特性
- 1. 简介、系统要求与安装
 - 1.1. 什么是 3X-UI
 - 1.2. 支持的操作系统与架构
 - 1.3. 安装方式
 - 1.4. 首次启动与默认凭据
 - 1.5. 文件位置
 - 1.6. `x-ui` 管理命令（脚本菜单）
 - 1.7. `x-ui` 子命令（非交互式）
 - 1.8. SQLite 迁移至 PostgreSQL
- 2. 登录面板与访问安全
 - 2.1. 登录表单
 - 2.2. 双因素认证（2FA / TOTP）
 - 2.3. 登录尝试限制（login limiter / 暴力破解防护）
 - 2.4. 修改管理员登录名和密码
 - 2.5. 密钥路径（URI 路径 / webBasePath）和面板端口
 - 2.6. 会话生存时间（超时）
 - 2.7. LDAP（同步与认证）
- 3. 概览 / 仪表盘
 - 3.1. 数据采集的基本原则
 - 3.2. CPU
 - 3.3. 内存（RAM）
 - 3.4. 交换空间（Swap）
 - 3.5. 磁盘（Storage）
 - 3.6. 系统运行时间（Uptime）
 - 3.7. 系统负载（Load average）
 - 3.8. 网络：速率与总流量
 - 3.9. 服务器 IP 地址
 - 3.10. TCP/UDP 连接
 - 3.11. Xray 状态与进程管理
 - 3.12. 面板更新（3X-UI）
 - 3.13. 地理文件更新（GeoIP / GeoSite）
 - 3.14. 数据库备份与恢复
 - 3.15. 其他界面元素
- 4. Inbounds：创建与通用参数
 - 4.1. 表单通用字段
 - 4.2. Sniffing（嗅探）
 - 4.3. Allocate（端口分配策略）
 - 4.4. External Proxy（外部代理）
 - 4.5. Fallbacks（Fallback）
 - 4.6. 周期性流量重置
 - 4.7. 入站 JSON（高级）
 - 4.8. inbound 操作：QR / Edit / Reset / Delete 及统计

- 5. 协议
 - 5.1. 支持的协议列表
 - 5.2. 哪些协议支持 TLS / REALITY / 传输
 - 5.3. VLESS
 - 5.4. VMess
 - 5.5. Trojan
 - 5.6. Shadowsocks
 - 5.7. Dokodemo-door / Tunnel (透明转发器)
 - 5.8. SOCKS / HTTP (mixed 协议)
 - 5.9. WireGuard (inbound)
 - 5.10. Hysteria (默认 v2)
 - 5.11. MTPROTO (Telegram 代理)
 - 5.12. 协议选择速查表
- 6. 传输层 (Stream Settings)
 - 6.1. 选择传输网络
 - 6.2. RAW / TCP (tcpSettings)
 - 6.3. mKCP (kcpSettings)
 - 6.4. WebSocket (wsSettings)
 - 6.5. gRPC (grpcSettings)
 - 6.6. HTTPUpgrade (httpupgradeSettings)
 - 6.7. XHTTP / SplitHTTP (xhttpSettings)
 - 6.8. Hysteria 传输 (hysteriaSettings)
 - 6.9. 相关参数
- 7. 连接安全: TLS、XTLS 与 REALITY
 - 7.1. 区别: TLS vs XTLS vs REALITY
 - 7.2. 「无」模式 (none)
 - 7.3. TLS 模式
 - 7.4. REALITY 模式
 - 7.5. 配置实践建议
- 8. 客户端
 - 8.1. 客户端字段
 - 8.2. 绑定 inbound
 - 8.3. 客户端操作
 - 8.4. 批量操作
 - 8.5. 搜索、筛选和排序
 - 8.6. 图标和状态
- 9. 客户端分组
 - 9.1. 什么是客户端分组以及为什么需要它
 - 9.2. 分组与客户端、inbound、节点和协议的关系
 - 9.3. 分组字典与「空」分组
 - 9.4. 分组的字段与列
 - 9.5. 创建分组
 - 9.6. 重命名分组
 - 9.7. 向分组添加客户端
 - 9.8. 将客户端移出分组 (不删除客户端本身)

- 9.9. 重置分组流量
- 9.10. 删除分组与删除分组客户端
- 9.11. 与「客户端」页面的关联
- 9.12. API 端点汇总
- 9.13. 按分组统计的流量
- 10. 订阅 (Subscription)
 - 10.1. 什么是 subId 以及链接的构成方式
 - 10.2. 订阅服务器设置
 - 10.3. 输出格式
 - 10.4. 订阅信息页面与 QR 码
 - 10.5. 自定义订阅页面模板
- 11. Xray: 路由、outbounds、DNS 与扩展
 - 11.1. 编辑器结构: 选项卡/模式
 - 11.2. 主要设置 (General)
 - 11.3. 路由规则 (routing)
 - 11.4. Outbounds (出站连接)
 - 11.5. 负载均衡器 (Balancers)
 - 11.6. DNS
 - 11.7. Fake DNS
 - 11.8. WireGuard / WARP / NordVPN
 - 11.9. Reverse 代理与 TUN
 - 11.10. 日志与统计 (Stats, metrics)
 - 11.11. 保存、重启与自动转换
 - 11.12. 订阅 outbound (自动更新)
 - 11.13. WARP IP 轮换
- 12. 节点 (多面板, master/slave)
 - 12.1. 列表顶部的汇总信息
 - 12.2. 添加和编辑节点
 - 12.3. TLS 验证 (用于 https 节点)
 - 12.4. 每个节点显示的信息
 - 12.5. 节点操作
 - 12.6. 指标历史
 - 12.7. inbound 和客户端如何同步
 - 12.8. 节点链 (子节点/传递节点)
 - 12.9. 3.5.0 中的修复 (节点稳定性)
 - 12.10. 节点: 3.3.0 中的新功能
- 13. 面板设置
 - 13.1. 保存与重启面板
 - 13.2. 通用设置 («面板» 选项卡 / *General*)
 - 13.3. 面板访问: IP、端口、路径、域名、证书
 - 13.4. 会话、面板代理与受信任代理 («代理与服务器» 选项卡 / *Proxy and Server*)
 - 13.5. Telegram 机器人 («Telegram 机器人» 选项卡 / *Telegram Bot*)
 - 13.6. 日期与时间 («日期与时间» 选项卡 / *Date and Time*)
 - 13.7. 外部流量与 Xray 行为 («外部流量» 选项卡 / *External Traffic*)
 - 13.8. 其他: Xray 配置模板与测试 URL
 - 13.9. 管理员账户与 API 令牌

- 13.10. 3.3.0 中的 API 变更 (对集成很重要)
- 14. Telegram 机器人
 - 14.1. 启用与配置机器人
 - 14.2. 主菜单与按钮
 - 14.3. 机器人命令
 - 14.4. 客户端管理 (仅管理员)
 - 14.5. 通知与报告
 - 14.6. 备份与日志
 - 14.7. 工作特性
- 15. 地理数据库 (geoip / geosite 及自定义)
 - 15.1. 什么是 geoip.dat 和 geosite.dat
 - 15.2. 标准地理文件及其更新
 - 15.3. 通过 Xray 自动更新地理数据 (Geodata Auto-Update)
 - 15.4. 校验和限制
 - 15.5. 面板启动时的自动检查
 - 15.6. 在路由规则中使用地理数据库
- 16. 日常运维: 备份、日志、更新、CLI
 - 16.1. 数据库备份与恢复
 - 16.2. 查看日志
 - 16.3. Xray 日志级别与配置
 - 16.4. 管理 Xray: 停止与重启
 - 16.5. 重启与更新面板
 - 16.6. 定时任务 (cron)
 - 16.7. 控制台菜单与 CLI (`x-ui`)
 - 16.8. 卸载面板
 - 16.9. `x-ui migrateDB` 命令

3.5.0 新特性

3.5.0 是一次重大发布：MTProto 改用多客户端模型（mtg-multi 引擎、每客户端独立密钥、配额与 ad-tag），托管主机改为分组式（一条记录内含多个 inbound 和多个地址），PostgreSQL 面板的恢复功能可接受 SQLite 备份，outbound 新增「Target Strategy」、「Real delay」测试以及 Egress/Country 列，负载均衡器可将另一个负载均衡器用作 fallback。随附 Xray 核心 26.7.11。以下列出相对于 3.4.2 的变更，按手册章节分组。

第 1 节的更改 – 简介、系统要求与安装

- Xray 核心已更新至 **26.7.11**。随之而来的自动迁移：Shadowsocks 的 `none/plain` 与 VMess 的 `none/zero` 加密方式已从核心中移除（已保存的配置会自动改写），指向公网地址的未加密 VLESS/Trojan outbound 在保存时会被拒绝。
- 新增 `x-ui pgclient` [版本] 命令和 PostgreSQL 菜单中的 **10. Install/Upgrade client tools (pg_dump/pg_restore)** 菜单项——用于安装/升级 PostgreSQL 客户端工具。
- 脚本修复：RHEL 系（EPEL）上的 PostgreSQL 与 fail2ban 安装、Arch 不再执行完整的 `pacman -Syu`、32 位 ARM 上 Xray 二进制文件名已修正（`xray-linux-arm32`）、签发 IP 证书前确认自动检测到的 IPv4，并修复了选择默认 ACME 端口时误报的「Your input is invalid」。

第 2 节的更改 – 登录面板与访问安全

- IP 限制：「死」连接现在只封禁一次，而不是在每次 10 秒扫描时重复封禁——fail2ban 计数器不再虚增，无需再调高 `maxretry`。

第 4 节的更改 – Inbounds：创建与通用参数

- inbound 列表新增**搜索**（按备注、端口和协议），节点下拉列表（「部署到」、「节点」筛选器）也支持搜索。

第 5 节的更改 – 协议

- **MTProto 改用多客户端模型**（mtg-multi 引擎）：MTProto 用户现在是普通客户端，拥有自己的密钥、配额、有效期、ad-tag 和专属 `tg://proxy` 链接。inbound 级别的「Secret」字段已移除（现有 inbound 会自动转换），「FakeTLS domain」成为新密钥的默认域名。新的 inbound 字段：**Max connections**（连接数限制）、**Public IPv4/IPv6**（用于 ad-tag middle proxy）。客户端更改「热」应用，不会中断其他用户的 Telegram 会话。
- WireGuard：inbound 菜单获得完整的客户端操作集（Export All URLs、绑定/解绑、分组），导出拆分为 **Config** 和 **Links** 两个选项卡，「WireGuard 允许的 IP」字段变为可编辑（多个条目以逗号分隔），节点 inbound 的客户端配置中 `Endpoint` 现在指向节点地址。

第 7 节的更改 – 连接安全：TLS、XTLS 与 REALITY

- **Finalmask + REALITY** 组合在保存时会被拒绝（该组合会导致 Xray-core 在首次连接时崩溃）；`minClientVer` 占位符已更新为 26.3.27。

- Finalmask 新增 TCP 掩码类型——**XMC (Minecraft)**：将流量伪装成 Minecraft 流量 (Hostname、Usernames, 以及必填、可自动生成的 Password)。

第 8 节的更改 – 客户端

- 新增**「速度」**列——每个客户端的实时速度 (↑/↓, 约 5 秒滑动平均值)。
- 客户端搜索恢复支持按 **Telegram ID** 查找；客户端表单的绑定列表中隐藏已禁用的 inbound；修复了「解绑」窗口中重复条目累积的问题。
- MTProto 客户端拥有专属字段：**「MTProto secret」** (可重新生成) 和**「Ad-tag (赞助频道)」** (32 个十六进制字符)；配额和有效期现在对 MTProto 真正生效。

第 9 节的更改 – 客户端分组

- 客户端信息窗口现在会显示其所属分组。

第 10 节的更改 – 订阅 (Subscription)

- 托管主机改为分组式：一条记录可覆盖多个 **inbound** (多选) 和多个**地址** (标签式输入, 每个条目可带自己的 `:??`；地址自动补全；留空则继承 inbound 地址)。列表的列以芯片形式显示地址和 inbound (带「+N」), 操作和 API 均以分组 (`groupId`) 为单位, 并新增了批量端点 `POST /panel/api/hosts/bulk/add`。主机排序现在是全局的 (先按排序序号, 再按备注)。
- 公告文本 (`subAnnounce`) 现在会以横幅形式显示在订阅信息页面上；自定义模板中可使用 `announce` 变量。
- 现在通过 **JSON/Clash 链接** (而不仅是主链接) 也能在浏览器中打开信息页面。
- 主机设置 **Final Mask** 与 **Allow insecure** 现在分别也作用于 raw 链接 (`fm=`) 和 **Hysteria2** (`insecure=1 / skip-cert-verify: true`)。
- 「订阅更新间隔」 (`subUpdates`) 的取值范围修正为 **0-525600** (此前界面上限 168 会在从 2.x 升级后阻止保存设置)。
- 原生 **WireGuard** 客户端现在会进入 **Clash** 和 **JSON** 订阅 (此前仅进入 raw)。

第 11 节的更改 – Xray: 路由、outbounds、DNS 与扩展

- Outbound 编辑器：新增**「Target Strategy」字段 (从 **AsIs** 到 **ForceIPv4** 共 11 个值)、**「Real delay」**测试模式 (包含隧道建立在内的完整耗时；HTTP 模式现在按「热」连接测量), 以及 HTTP/Real 测试后的 **Egress** 列 (出口 IP, 藏在「眼睛」图标后) 和 **Country** 列 (旗帜 + 国家名称, WARP 标记)。
- 负载均衡器的 **Fallback** 可以指向另一个负载均衡器：面板会自动构建隐藏的 loopback 对象 (`_bl_...`), 并防止循环依赖及删除正在使用的负载均衡器；`_bl_` 前缀已被保留。
- 「基本路由」选项卡新增**「Default Outbound」**选择器——决定由哪个 outbound 处理未命中任何规则的流量 (所选 outbound 会移至首位)。
- 私有 IP 上的 DNS 服务器不再被 `geoip:private` 规则拦截——面板会自行维护一条受管的放行规则。
- dial 设置 (sockopt) 中的 Happy Eyeballs 现在能真正启用；「Try delay」默认 250 毫秒, 显式设置的 0 会被保留。
- outbound 订阅导入：带 `?plugin=` 结尾 / 的 `ss://` 链接现在能正确解析端口。

第 12 节的更改 – 节点（多面板，master/slave）

- 一组修复：未做修改地保存客户端不再中断节点 inbound 的实时流量；节点的 Host 覆盖会在首次接受时导入主面板；自动续期会开启新的配额窗口；在主面板删除客户端会将其从节点上完全删除；节点的 inbound 在首次接受前不会被清除；单个异常 inbound 不再阻断节点流量同步；端口冲突检查仅限于所在节点。

第 14 节的更改 – Telegram 机器人

- 机器人命令菜单新增 `/usage`、`/inbound`、`/restart`，以及新的管理员命令 `/clearall`（重置所有客户端流量，需确认）。
- 在线客户端列表以 `email - inbound` 标注；备份和封禁日志消息包含主机名；按 Telegram ID 搜索不受设置格式化方式影响。

第 16 节的更改 – 日常运维：备份、日志、更新、CLI

- PostgreSQL 面板的恢复功能可接受 SQLite 文件：普通 `.db` 备份或迁移 `.dump` 可直接导入 PostgreSQL（单个事务，并在停止 Xray 前完成检查）。两种数据库引擎的文件选择对话框均接受 `.dump, .db`；「下载迁移文件」仅保留在 PostgreSQL 面板上。
- 恢复 `pg_dump` 归档前，面板会检查转储文件的可读性，版本不匹配时会提示准确的 `x-ui pgclient <[?]>` 命令。
- 启动时自动修复：溢出的流量计数器会被钳制并恢复；移除 inbound 端口上过时的 UNIQUE 约束（它曾妨碍 multi-node）。
- Xray 日志：新的定时任务每 10 分钟运行一次，在 access 或 error 日志超过 **64 MiB** 时进行截断；每日清理现在两者都会清理。
- Docker：证书自动续期已修复（crond 会启动，acme.sh 状态保存在卷中）。

1. 简介、系统要求与安装

1.1. 什么是 3X-UI

3X-UI 是一款面向 [Xray-core](#) 服务器的开源 Web 管理面板。该面板提供统一的多语言 Web 界面，用于部署、配置和监控各类代理及 VPN 协议——从单台 VPS 到多节点分布式部署均可胜任。

3X-UI 是原版 X-UI 项目的增强型 fork，新增了对更多协议的支持、更高的稳定性、按客户端的精细流量统计以及大量实用功能。

主要功能：

- **多协议 inbound** —— VLESS、VMess、Trojan、Shadowsocks、WireGuard、Hysteria2、HTTP、SOCKS (Mixed)、Dokodemo-door / Tunnel、TUN 以及 **MTPProto** (Telegram 代理，3.3.0 版本新增)。
- **现代传输与加密** —— TCP (Raw)、mKCP、WebSocket、gRPC、HTTPUpgrade 和 XHTTP，可通过 TLS、XTLS 和 REALITY 进行保护。
- **Fallback** —— 通过 Xray 的 fallback 机制在同一端口上服务多个协议（例如，在 443 端口同时运行 VLESS 和 Trojan）。
- **按客户端管理** —— 流量配额、到期日期、IP 限制、在线状态显示、一键邀请链接、二维码和订阅。
- **流量统计** —— 按每个 inbound、客户端和 outbound 统计，支持重置。
- **多节点支持** —— 从单一面板管理和扩展至多台服务器。
- **Outbound 与路由** —— WARP、NordVPN、自定义路由规则、负载均衡器、代理链。
- **内置订阅服务器**，支持多种输出格式。
- **Telegram 机器人**，用于远程监控和管理。
- **REST API**，内置 Swagger 文档。
- **灵活的存储** —— SQLite (默认) 或 PostgreSQL。
- **13 种界面语言**，支持深色和浅色主题。
- **集成 Fail2ban**，按客户端实施 IP 限制。

重要提示：本项目仅供个人使用，不建议将其用于违法目的或生产环境。

1.2. 支持的操作系统与架构

操作系统

安装脚本通过读取 `/etc/os-release`（或 `/usr/lib/os-release`）中的 `ID` 字段来识别发行版。官方支持的系统包括：

Ubuntu、Debian、Armbian、Fedora、CentOS、RHEL、AlmaLinux、Rocky Linux、Oracle Linux、Amazon Linux、Virtuozzo、Arch、Manjaro、Parch、openSUSE (Tumbleweed / Leap)、Alpine 以及 Windows。

Alpine 系列使用 OpenRC 服务（`rc-service` / `rc-update`），其他系统使用 systemd。CentOS 7 通过 `yum` 安装软件包，较新版本使用 `dnf`。如果无法识别发行版，脚本默认尝试使用 `apt-get` 包管理器。

处理器架构

架构由 `uname -m` 的输出确定，并映射到以下支持的值之一：

uname -m 的值	3X-UI 架构
x86_64, x64, amd64	amd64
i*86, x86	386
armv8*, arm64, aarch64	arm64
armv7*, arm	armv7
armv6*	armv6
armv5*	armv5
s390x	s390x

如果架构不在此列表中，脚本将输出"Unsupported CPU architecture!"并终止安装。

基础依赖

安装面板前，脚本会自动安装一组基础软件包（各发行版名称有所不同）：`cron` / `cronie` / `dcron`、`curl`、`tar`、`tzdata` / `timezone`、`socat`、`ca-certificates`、`openssl`。

1.3. 安装方式

方式一：安装脚本（推荐）

以 root 身份运行一条命令即可完成安装：

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh)
```

该脚本必须以 root 权限运行：若非 root 用户执行，将输出"Please run this script with root privilege"并报错退出。

安装程序的逐步操作说明：

1. 识别操作系统和架构。
2. 安装基础依赖。
3. 下载发布归档包 `x-ui-linux-<arch>.tar.gz` 并解压到 `/usr/local/x-ui` 目录。
4. 下载管理脚本 `x-ui.sh` 并将其安装为 `/usr/bin/x-ui` 命令。
5. 创建日志目录 `/var/log/x-ui`。
6. 启动初始配置：选择数据库、生成凭据、选择端口、可选配置 SSL。
7. 安装并启动自动启动服务（systemd 单元 `x-ui.service` 或 Alpine 的 OpenRC init 脚本）。

安装时选择数据库。 安装程序提供以下选项：

- 1) SQLite（默认，客户端数量少于 500 时推荐）-- 单一文件 `/etc/x-ui/x-ui.db`，无需额外配置。
- 2) PostgreSQL（客户端数量较多或有多个节点时推荐）。PostgreSQL 可本地安装（创建名为 `xui` 的专用用户和数据库），也可指定已有服务器的 DSN。连接参数以 `XUI_DB_TYPE` 和 `XUI_DB_DSN` 变量的形式写入服务环境文件（根据发行版不同，路径为 `/etc/default/x-ui`、`/etc/conf.d/x-ui` 或 `/etc/sysconfig/x-ui`）。

示例：将 PostgreSQL 参数写入服务环境文件。 选择 PostgreSQL 并指定 DSN 后，安装程序会向环境文件中添加类似如下的内容：

```
XUI_DB_TYPE=postgres
XUI_DB_DSN=postgres://xui:S3cretPass@127.0.0.1:5432/xui?sslmode=disable
```

其中 `xui` 是用户名和数据库名，`127.0.0.1:5432` 是服务器地址和端口，`sslmode=disable` 适用于本地连接（远程服务器通常使用 `require`）。

安装特定（旧版）版本。 可以显式指定版本标签，安装程序将下载相应版本：

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/v2.4.0/install.sh)
v2.4.0
```

此类安装支持的最低版本为 `v2.3.5`；若指定更旧的版本，将输出 "Please use a newer version (at least v2.3.5)"。

安装开发版。 除版本标签外，安装程序还接受 `dev-latest`（别名 `dev`）参数，用于安装基于 `main` 分支最新提交的滚动开发版：

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh) dev-
latest
```

开发版是每次提交对应的预发布版本（标签 `dev-latest`），而非稳定版，因此不执行最低版本检查。运行时显示警告："Installing the rolling dev build (tag: dev-latest). This is a per-commit pre-release, not a stable version."。不带参数时，安装程序将安装最新稳定版。仅在需要验证尚未正式发布的修复时才使用开发版；日常使用请安装稳定版。

方式二：Docker

使用默认 SQLite 数据库启动：

```
docker compose up -d
```

若要使用内置 PostgreSQL 服务，需在 `docker-compose.yml` 中取消 `XUI_DB_*` 行的注释，并使用 `profile` 启动：

```
docker compose --profile postgres up -d
```

镜像内置 Fail2ban（默认启用），用于按客户端实施 IP 限制。Fail2ban 通过 iptables 封锁违规者，这需要 NET_ADMIN 能力。docker-compose.yml 中已通过 cap_add 提供该能力。自 3.5.0 起，容器中通过 SSL 菜单签发的证书自动续期已修复：entrypoint 会启动 crond 并重新注册 acme.sh 的 cron 任务，docker-compose.yml 新增了用于保存 acme.sh 状态的独立卷——续期在容器重建后依然有效（此前证书会在约 90 天后悄然过期）。若通过 docker run 手动启动，需自行添加该能力，否则封锁操作只会记录日志而不会实际生效：

示例：完整的 docker run 命令。 最简配置，包含面板端口映射、网络能力和持久化数据库卷：

```
docker run -d \
  --name 3x-ui \
  --restart unless-stopped \
  --cap-add=NET_ADMIN --cap-add=NET_RAW \
  -v $PWD/db:/etc/x-ui \
  -v $PWD/cert:/root/cert \
  -p 2053:2053 \
  ghcr.io/mhsanaei/3x-ui:latest
```

/etc/x-ui 卷用于在容器重启之间保留 x-ui.db 文件，否则配置和账户数据将丢失。

```
docker run -d --cap-add=NET_ADMIN --cap-add=NET_RAW ... ghcr.io/mhsanaei/3x-ui
```

在 Docker 中，面板是容器的主进程：自动启动由容器重启策略（例如 restart: unless-stopped）控制，而不是由容器内的服务控制。

1.4. 首次启动与默认凭据

首次安装时（仍使用默认凭据时），安装程序会随机生成用户名、密码、Web 路径和端口：

参数	安装时的生成方式	备注
用户名 (Username)	10 个字符的随机字符串	自动生成
密码 (Password)	10 个字符的随机字符串	自动生成
面板 Web 路径 (WebBasePath)	18 个字符的随机字符串	防止通过根 URL 发现面板
面板端口 (Port)	默认为 1024-62000 范围内的随机端口；如有需要也可手动指定	webPort 的“出厂”值为 2053，但安装程序会覆盖该值

安装结束时，脚本会输出汇总信息：用户名、密码、端口、Web 路径、API 令牌以及形如以下格式的访问链接 (Access URL)：

```
https://<??IP>:<??>/<Web??>
```

如果未配置 SSL 证书，链接将使用 http://，脚本会提示需要配置 SSL（菜单第 19 项）。

必须更改凭据。由于登录名和密码是随机生成的，请在**安装后立即保存**。可随时通过菜单中的"Reset Username & Password"选项（见下文）或在面板 Web 界面的设置中进行更改。重置后脚本会提醒："Please use the new login username and password to access the X-UI panel. Also remember them!"。

安装完成后，使用 `x-ui` 命令打开管理菜单（见 1.6 节）。

1.5. 文件位置

路径	用途
<code>/usr/local/x-ui/</code>	面板安装目录（二进制文件 <code>x-ui</code> 、脚本 <code>x-ui.sh</code> ）
<code>/usr/local/x-ui/bin/xray-linux-<arch></code>	Xray-core 二进制文件（在 armv5/armv6/armv7 上重命名为 <code>xray-linux-arm</code> ）
<code>/usr/bin/x-ui</code>	管理脚本（ <code>x-ui</code> 命令）
<code>/etc/x-ui/x-ui.db</code>	SQLite 数据库文件（默认）
<code>/var/log/x-ui/</code>	面板日志目录
<code>/etc/systemd/system/x-ui.service</code>	服务的 systemd 单元（Alpine 除外）
<code>/etc/init.d/x-ui</code>	OpenRC init 脚本（仅 Alpine）
<code>/etc/default/x-ui</code> · <code>/etc/conf.d/x-ui</code> · <code>/etc/sysconfig/x-ui</code>	服务环境变量文件（路径取决于发行版）； <code>XUI_DB_TYPE</code> / <code>XUI_DB_DSN</code> 写入此处

数据库目录可通过环境变量 `XUI_DB_FOLDER` 覆盖（默认 `/etc/x-ui`），Xray 二进制文件目录可通过 `XUI_BIN_FOLDER` 覆盖（默认为面板目录下的 `bin`）。数据库文件名为 `x-ui.db`。

示例：将数据库移至独立磁盘。 若要将 `x-ui.db` 存储在已挂载的磁盘 `/data` 上而非 `/etc/x-ui`，请在服务环境文件中设置变量并重启面板：

```
echo 'XUI_DB_FOLDER=/data/x-ui' >> /etc/default/x-ui
mkdir -p /data/x-ui
systemctl restart x-ui
```

数据库的完整路径将变为 `/data/x-ui/x-ui.db`。

主要环境变量

变量	用途	默认值
<code>XUI_DB_TYPE</code>	数据库后端： <code>sqlite</code> 或 <code>postgres</code>	<code>sqlite</code>
<code>XUI_DB_DSN</code>	PostgreSQL 连接字符串（当 <code>XUI_DB_TYPE=postgres</code> 时）	—
<code>XUI_DB_FOLDER</code>	SQLite 数据库文件目录	<code>/etc/x-ui</code>

变量	用途	默认值
XUI_INIT_WEB_BASE_PATH	Web 面板的初始 URI 路径（仅首次初始化时有效）	/
XUI_DB_MAX_OPEN_CONNS	最大打开连接数（PostgreSQL 连接池）	—
XUI_DB_MAX_IDLE_CONNS	最大空闲连接数（PostgreSQL 连接池）	—
XUI_ENABLE_FAIL2BAN	是否通过 Fail2ban 启用 IP 限制	true
XUI_LOG_LEVEL	日志级别（debug、info、warning、error）	info
XUI_DEBUG	调试模式	false

示例：临时启用详细日志。若要诊断问题，将日志级别提升至 debug 并重启服务：

```
echo 'XUI_LOG_LEVEL=debug' >> /etc/default/x-ui
systemctl restart x-ui
x-ui log      #  ? ? ? ? ? ?
```

诊断完成后请将其恢复为 info，以防日志文件过度增长。

通过环境变量设置初始 Web 路径。 变量 XUI_INIT_WEB_BASE_PATH 用于在首次初始化设置时指定 Web 面板的 URI 路径（webBasePath）。在 Docker 或 systemd 部署时，这便于预先固定面板的访问路径。该值会自动规范化——必要时会添加前导和尾部斜杠，空值或纯空白值将被忽略（此时使用默认路径 /）。该变量仅在首次初始化时生效：若设置已存在，可通过 Web 界面或菜单中的“Reset Web Base Path”选项修改路径。

1.6. x-ui 管理命令（脚本菜单）

安装完成后，以 root 身份运行 x-ui 命令将打开“3X-UI Panel Management Script”交互式菜单。通过输入对应编号（范围 0-27）选择菜单项。许多菜单项也可作为子命令在脚本中直接调用（见 1.7 节）。

菜单按功能分为以下几个模块。

安装与更新

- **1. Install** —— 安装面板（运行 install.sh）。安装前会检查面板是否已安装。
- **2. Update** —— 将所有 x-ui 组件更新至最新版本。数据不会丢失；更新后面板自动重启。需要确认。
- **3. Update Menu** —— 仅将管理脚本（x-ui.sh / x-ui 命令）更新至最新版本，无需重新安装面板。
- **4. Legacy Version** —— 安装指定的（旧版）面板。脚本会要求输入版本号（例如 2.4.0）并下载相应版本。
- **5. Uninstall** —— 完全卸载面板及 Xray。停止并禁用服务，删除 /etc/x-ui/ 和 /usr/local/x-ui/ 目录、服务环境文件以及管理脚本本身。需要确认（默认为“否”）。

凭据与设置

- **6. Reset Username & Password** —— 重置面板的用户名和密码。可输入自定义值，也可留空以随机生成（随机用户名 10 个字符，随机密码 18 个字符）。若已配置双因素认证（2FA），还会提示是否禁用。重置后面板重启。

- **7. Reset Web Base Path** — 重置面板 Web 路径：生成新的随机路径（18 个字符），面板重启。若原路径已泄露或遗忘，可使用此项。
- **8. Reset Settings** — 将面板所有设置重置为默认值。**凭据（用户名和密码）及账户数据不会丢失。** 需要确认；重置后面板重启。
- **9. Change Port** — 更改 Web 面板端口。输入端口号（1-65535）；设置后需要重启才能生效。
- **10. View Current Settings** — 查看当前设置（`x-ui setting -show`）。显示所用数据库后端（SQLite 或 PostgreSQL，DSN 中的密码已脱敏）以及访问链接（Access URL）。若未配置 SSL，会提示为 IP 地址申请 Let's Encrypt 证书。

服务管理

- **11. Start** — 启动面板服务。若面板已在运行，会提示无需重复启动。
- **12. Stop** — 停止面板服务。
- **13. Restart** — 重启面板服务。
- **14. Restart Xray** — 仅重启 Xray-core 内核，不重启面板本身（通过 `systemctl reload x-ui`，在 Docker 中则向面板进程发送 `USR1` 信号）。
- **15. Check Status** — 检查服务状态（`systemctl status x-ui` 或 `rc-service x-ui status`）。
- **16. Logs Management** — 日志管理：查看调试日志（Debug Log，通过 `journalctl`），以及（Alpine 除外）清除所有日志（Clear All logs）。

自动启动

- **17. Enable Autostart** — 开启系统启动时自动启动面板（`systemctl enable x-ui` 或 `rc-update add`）。
- **18. Disable Autostart** — 禁用系统启动时的自动启动。

在 Docker 中，自动启动由容器重启策略控制，因此这两个选项仅显示相应的提示信息。

安全与网络

- **19. SSL Certificate Management** — 通过 `acme.sh` 管理 SSL 证书：为域名申请证书、吊销、强制续期、查看已有域名、指定面板证书路径，以及为 IP 地址申请短期证书（有效期约 6 天，自动续期）。
- **20. Cloudflare SSL Certificate** — 通过 Cloudflare DNS 验证申请 SSL 证书。
- **21. IP Limit Management** — 管理按客户端的 IP 数量限制（基于 Fail2ban）：查看和解除封锁等。
- **22. Firewall Management** — 管理防火墙（开放/关闭端口及查看规则）。
- **23. SSH Port Forwarding Management** — 配置 SSH 端口转发，以便通过 SSH 隧道从本地计算机访问面板。

性能与维护

- **24. Enable BBR** — 启用/禁用 TCP BBR 拥塞控制算法（子菜单包含 Enable BBR / Disable BBR 选项）。
- **25. Update Geo Files** — 更新 geo 数据库（`.dat` 文件），可选择数据源：Loyalsoldier（`geoip.dat`、`geosite.dat`）、chocolate4u（`geoip_IR.dat`、`geosite_IR.dat`）、runetfreedom（`geoip_RU.dat`、`geosite_RU.dat`）或 All（全部）。更新后面板重启。
- **26. Speedtest by Ookla** — 通过 Speedtest by Ookla 运行网络速度测试。

- **27. PostgreSQL Management** —— 管理内置/关联的 PostgreSQL 实例（启用及相关操作）。
- **0. Exit Script** —— 退出菜单。

1.7. x-ui 子命令（非交互式）

为便于在脚本中使用，`x-ui` 命令支持直接子命令（不带参数运行 `x-ui` 将打开菜单）：

命令	操作
<code>x-ui</code>	打开管理菜单
<code>x-ui start</code>	启动面板
<code>x-ui stop</code>	停止面板
<code>x-ui restart</code>	重启面板
<code>x-ui restart-xray</code>	重启 Xray
<code>x-ui status</code>	查看当前服务状态
<code>x-ui settings</code>	查看当前设置
<code>x-ui enable</code>	开启系统启动时自动启动
<code>x-ui disable</code>	禁用自动启动
<code>x-ui log</code>	查看日志
<code>x-ui banlog</code>	查看 Fail2ban 封锁日志
<code>x-ui update</code>	更新面板
<code>x-ui update-all-geofiles</code>	更新所有 geo 文件
<code>x-ui migrateDB [file]</code>	转换 <code>.db</code> 到 <code>.dump</code> (SQLite)
<code>x-ui legacy</code>	安装旧版本
<code>x-ui install</code>	安装面板
<code>x-ui uninstall</code>	卸载面板

1.8. SQLite 迁移至 PostgreSQL

可将现有的 SQLite 安装迁移至 PostgreSQL：

```
x-ui migrate-db --dsn "postgres://xui:password@127.0.0.1:5432/xui?sslmode=disable"
# /etc/default/x-ui XUI_DB_TYPE XUI_DB_DSN
systemctl restart x-ui
```

原 SQLite 文件保持不变——仅在确认新后端正常运行后，再手动删除该文件。

示例：验证是否已切换至 PostgreSQL。 迁移后，通过查看设置命令确认面板确实运行在新后端上——输出中应显示 PostgreSQL（DSN 中的密码已脱敏）：


```
x-ui settings | grep -i -E 'db|dsn'
```

若面板可正常访问且账户数据完整，即可删除原 `x-ui.db`。

2. 登录面板与访问安全

本节介绍与 3X-UI 面板管理员认证相关的所有内容：登录表单、双因素认证（TOTP）、密码暴力破解防护、修改账户凭据、修改面板的密钥路径和端口、会话生存时间，以及通过 LDAP 进行同步/认证。

2.1. 登录表单

登录页面通过面板密钥路径（webBasePath）的根路径提供。如果用户已经授权，会自动重定向到 `.../panel/`。页面上有主题切换、界面语言选择以及登录表单本身。

表单字段：

字段	提示/标题 (RU)	必填	说明
用户名	«Имя пользователя»	是	管理员登录名。空值会在客户端被拒绝，在服务器端则以消息「请输入用户名」拒绝。
密码	«Пароль»	是	管理员密码。空值会以消息「请输入密码」拒绝。
2FA 代码	«Код 2FA»	仅在启用 2FA 时	该字段仅在面板启用双因素认证时出现。来自认证应用的 6 位代码。

按钮 «Войти»（登录）将表单提交至 `POST /login`。

行为与消息：

- 登录成功时显示「登录成功」并跳转至 `.../panel/`。
- 当账户凭据有任何错误或 2FA 代码不正确时，服务器返回统一消息：「账户数据错误。」（英文：*Invalid username or password or two-factor code.*）。这是有意为之——面板不会提示具体哪一项错误（登录名、密码还是代码），以免为暴力破解提供便利。
- 面板根据 `POST /getTwoFactorEnable` 请求来显示或隐藏「2FA 代码」字段，该请求在授权之前就返回当前的 2FA 状态。
- 如果服务器会话已过期，则在下一次请求时显示「会话已过期。请重新登录」，并将用户重定向到登录页面。

关于 CSRF 的说明：客户端在提交表单之前会获取一个 CSRF 令牌（`GET /csrf-token`）；`/login` 和 `/logout` 请求受 CSRF 校验保护。

示例：通过 API 登录。 当 2FA 关闭时，只需登录名和密码即可；启用 2FA 时会增加 `twoFactorCode` 字段：

```
# Без 2FA
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=ВашПароль'

# С включённой 2FA — добавляется 6-значный код
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=ВашПароль&twoFactorCode=123456'
```

成功时服务器会返回带有会话 cookie 的 `Set-Cookie` ——正是它需要在后续对 `/panel/api/...` 的请求中传递。

2.2. 双因素认证 (2FA / TOTP)

3X-UI 中的 2FA 按照 **TOTP** 标准实现，并兼容任何认证应用（Google Authenticator、Aegis、FreeOTP 等）。参数被固定写死：算法 **SHA1**、6 位数字、周期 **30** 秒、发行方（issuer）`3x-ui`、标签 `Administrator`。

示例：编码到 QR 码中的 otpauth-URI。 如果认证应用无法用相机扫描，可以通过以下链接手动添加令牌（用你自己的 Base32 密钥替换 `JBSWY3DPEHPK3PXP`）：

```
otpauth://totp/3x-ui:Administrator?secret=JBSWY3DPEHPK3PXP&issuer=3x-ui&algorithm=SHA1&digits=6&period=30
```

参数 `algorithm=SHA1`、`digits=6`、`period=30` 与面板写死的值一致——无需更改。

设置位于 **Настройки** → **Учетная запись** 部分的 **«Двухфакторная аутентификация»** 选项卡中。

元素	文本 (RU)	说明
开关	«Включить 2FA»	启用/关闭双因素认证。
说明	«Добавляет дополнительный уровень аутентификации для повышения безопасности.»	开关下方的提示。

如何启用 2FA

启用开关时，面板会在本地生成一个**新密钥**——一个 Base32 编码的随机字符串（字母表为 `A-Z` 和 `2-7`）。随后会打开「启用双因素认证」窗口，并附带分步说明：

1. **«Отсканируйте этот QR-код в приложении для аутентификации или скопируйте токен рядом с QR-кодом и вставьте его в приложение»。** QR 码下方以文本形式显示密钥本身——点击 QR 码即可将密钥复制到剪贴板（弹出「已复制」）。
2. **«Введите код из приложения»** ——需要输入应用生成的 6 位代码。代码在浏览器端校验：面板会根据刚刚生成的密钥自行计算当前的 TOTP，并与输入值比较。如果代码不正确——「代码错误」；该字段只接受恰好 6 位数字。

只有在成功确认之后，密钥和启用标志才会被保存。保存时显示「双因素认证已成功设置」。

重要：设置部分的更改需通过通用按钮 **«Сохранить»** 应用，之后通常需要重启面板（「保存更改并重启面板以使其生效」）。首次启用 2FA 时，服务器还会额外**使所有活动会话失效**（递增「login epoch」），因此应用设置后需要重新登录——这次要带上 2FA 代码。

如何关闭 2FA

再次点击开关会打开「关闭双因素认证」窗口，并附带提示「输入应用中的代码以关闭双因素认证。」。输入正确代码后（自 3.4.2 起在**服务器端**校验），标志和密钥会被清除，并显示「双因素认证已成功删除」。

登录时的代码校验

登录时，服务器取出已保存的密钥，并将当前的 TOTP 与传入的 2FA 代码比较。不匹配会被视为登录失败，但向用户显示的是同一条统一消息「账户数据错误。」。

恢复访问 (recovery)

3X-UI 中没有单独的「恢复代码」机制。如果丢失了对认证应用的访问，无法通过面板界面恢复登录。唯一的办法是直接在服务器的数据库中关闭 2FA：将设置表中的 `twoFactorEnable` 键重置为 `false`（必要时清空 `twoFactorToken`），之后重启面板。因此建议在启用 2FA 时把密钥（Base32 令牌）保存在安全的地方。

示例：在服务器上紧急关闭 2FA。 通过 SSH 获取服务器访问权限后，停止面板，重置设置表中的键，然后再次启动面板：

```
x-ui stop
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='false' WHERE
key='twoFactorEnable';"
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='' WHERE key='twoFactorToken';"
x-ui start
```

此后只需用登录名和密码即可登录，如有需要可重新配置 2FA。

与修改凭据的关联：修改登录名/密码时（见 2.4），服务器上的 2FA 会**自动关闭**，以免旧密钥阻挡新账户下的访问。

2.3. 登录尝试限制 (login limiter / 暴力破解防护)

面板内置了失败登录限制器（相当于应用层面的 fail2ban）。参数在代码中设定，**无法通过界面配置**：

参数	值	用途
最大失败次数	5	在窗口期内允许多少次失败尝试。
计数窗口	5 分钟	累计失败次数的滑动窗口（更早的会被丢弃）。
封锁 (cooldown)	15 分钟	超过阈值后该键被封锁多长时间。

工作原理：

- 封锁键由「IP + 登录名」组合构成（登录名转为小写，去除空格）。也就是说，封锁针对具体的「地址 + 用户名」一对，而非整个面板。
- 每次失败尝试（登录名/密码错误或 2FA 代码错误）都会使计数器增长。在 5 分钟内达到 5 次失败后，该键被封锁 15 分钟。封锁期间，该对的任何尝试都会被以同样的消息「账户数据错误。」立即拒绝，即使数据正确。
- 成功登录会立即重置计数器并解除该对的封锁。
- 客户端 IP 地址在考虑可信代理（见 `trustedProxyCIDRs`）的情况下确定：仅当请求来自可信地址时才接受 `X-Real-IP` 和 `X-Forwarded-For` 头。否则使用连接的真实地址，若无法获取则使用字符串 `unknown`。

所有尝试都会被记录。对于失败的尝试，会在服务器日志中写入一条警告，包含用户名、IP、原因，以及封锁时的 `blocked_until` 时间。如果启用了通过 Telegram 机器人的登录通知（`tgNotifyLogin` ——「登录通知」），管理员还会额外收到成功、失败和被封锁尝试的用户名、IP 和时间。

示例：Telegram 登录通知。启用 `tgNotifyLogin` 后，每次尝试之后管理员会收到大致如下的消息：

```
Уведомление о входе
Пользователь: admin
IP: 203.0.113.45
Время: 2026-06-10 14:32:07
Статус: успешно
```

对于被封锁的「IP + 登录名」对，状态中会指明该尝试已被限制器拒绝。

2.4. 修改管理员登录名和密码

Настройки → Учетная запись 部分的 «Учетные данные администратора» 选项卡。字段：

字段	文本 (RU)	说明
当前登录名	«Текущий логин»	现行用户名。必须与当前登录名一致，否则更改会被拒绝。
当前密码	«Текущий пароль»	用于确认身份的现行密码。
新登录名	«Новый логин»	新用户名。不能为空。
新密码	«Новый пароль»	新密码。不能为空。

更改通过按钮 «Подтвердить» 应用，并提交至 `POST /panel/setting/updateUser`。

服务器逻辑与消息：

- 如果「当前登录名」与实际不符或「当前密码」错误——「修改管理员凭据时发生错误。」并附说明「用户名或密码错误」。
- 如果新登录名或新密码为空——说明「新用户名和新密码必须填写」。
- 成功时——「您已成功修改管理员凭据。」。密码以 `bcrypt` 哈希形式存储。

2FA 代码确认（自 3.4.2 起）。 如果面板启用了双因素认证，修改登录名/密码还需要输入身份验证器应用中的当前代码。会打开**「修改凭据」**窗口，并附带提示「输入应用中的代码以修改管理员凭据。」；代码（`twoFactorCode`）在服务器端校验，代码错误或为空时不会应用修改。关闭 2FA 时现在也需要同样的确认（参见 2.2）。

示例：通过 API 修改凭据。 该请求需要有效的会话 cookie（登录时获得）以及对当前登录名/密码的确认：

```
curl -X POST https://panel.example.com:2053/мой-секрет/panel/setting/updateUser \
  -b 'session=БАША_СЕССИОННАЯ_КОOKIE' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'oldUsername=admin&oldPassword=СтарыйПароль&newUsername=root&newPassword=НовыйСложныйПароль'
```

成功后当前会话会失效——需要用新数据重新登录。

修改凭据的重要影响：

- **所有现有会话都会失效**（用户的 `login_epoch` 计数器递增），因此修改后面板会自动退出登录并重定向到登录页面——需要重新登录。
- 如果修改时启用了 **2FA**，**它会自动关闭**（标志和密钥被重置）。修改登录名/密码后需要重新配置双因素认证。

如果启用了 2FA，在提交表单之前会打开「修改凭据」窗口，并附带提示「输入应用中的代码以修改管理员凭据。」——只有确认当前 2FA 代码后才能修改凭据。

2.5. 密钥路径（URI 路径 / webBasePath）和面板端口

这些参数位于 **Настройки → Панель** 部分，直接影响面板的「隐蔽性」和可达性。在保存并重启面板后生效。

字段	文本 (RU)	默认值	说明
面板端口	«Порт панели» (<code>panelPort</code>)，提示「面板运行的端口」	2053	Web 界面的 TCP 端口。
URI 路径	«URI-путь» (<code>panelUrlPath</code>)，提示「必须以 '/' 开头并以 '/' 结尾」	/	密钥基础路径 (<code>webBasePath</code>)。面板只能通过它访问（例如 <code>/мой-секрет/</code> ）。
面板管理 IP 地址	«IP-адрес для управления панелью» (<code>panelListeningIP</code>)，提示「留空以允许任意 IP 连接」	空	面板监听的地址。空 = 所有接口。
面板域名	«Домен панели» (<code>panelListeningDomain</code>)，提示「留空以允许任意域名和 IP 连接。」	空	按域名 (Host) 限制访问。
面板证书公钥路径	<code>publicKeyPath</code> ，提示「输入以 '/' 开头的完整路径」	空	用于 HTTPS 访问面板的 TLS 证书。

字段	文本 (RU)	默认值	说明
面板证书私钥路径	<code>privateKeyPath</code> , 相同提示	空	TLS 私钥。

基础路径 (`webBasePath`) 的行为：

- 该值会被自动规范化：如果不以 `/` 开头，则在开头添加该字符；如果不以 `/` 结尾，则在末尾添加。也就是说路径实际上总是 `/.../` 形式。
- 基础路径应用于面板本身、资源以及会话 cookie (cookie 仅针对该路径下发) 。

安全建议 (「安全警告」部分)：如果配置「过于公开」，面板会自行显示警告：

- 「面板通过普通 HTTP 运行——请为生产环境配置 TLS。」
- 「标准端口 2053 广为人知——请改为一个随机端口。」
- 「默认基础路径 `/` 广为人知——请改为一个随机路径。」

换言之，对于生产服务器应当设置非标准端口、非平凡的 URI 路径以及 TLS 证书。

示例：生产环境的「隐蔽」面板配置。在 **Настройки** → **Панель** 部分设置大致如下的值：

字段	值
面板端口	<code>34571</code> (随机, 替代 2053)
URI 路径	<code>/aXf9Qm2/</code> (非平凡, 以 <code>/</code> 开头和结尾)
面板证书公钥路径	<code>/etc/letsencrypt/live/panel.example.com/fullchain.pem</code>
面板证书私钥路径	<code>/etc/letsencrypt/live/panel.example.com/privkey.pem</code>

保存并重启后，面板将只能通过 `https://panel.example.com:34571/aXf9Qm2/` 访问，而安全警告会消失。

2.6. 会话生存时间 (超时)

字段 **«Продолжительность сессии»** (`sessionMaxAge`) 位于面板/间隔设置之中。

字段	文本 (RU)	默认值	单位	说明
会话时长	«Продолжительность сессии», 提示「系统中的会话时长 (值: 分钟)」	360	分钟	管理员会话 cookie 的生存时间。

行为：

- 该值以分钟为单位设定 (默认 360 分钟 = 6 小时)，在设置 cookie 时换算为秒。

- 如果该值大于 0，会话 cookie 会被设置相应的 MaxAge。超过该期限后 cookie 失效，下一次请求时用户会收到「会话已过期。请重新登录」。
- 会话也会在修改凭据或首次启用 2FA 时提前失效（通过 login_epoch 机制，见 2.4 和 2.2），以及在显式退出（POST /logout）时失效。
- 会话 cookie 标记为 HttpOnly，策略为 SameSite=Lax；在直接通过 HTTPS 访问面板时会设置 Secure 标志。

除超时本身外，还有一个相关通知：«Задержка уведомления об истечении сессии»（expireTimeDiff，提示「在达到阈值之前收到会话过期通知（值：天）」，默认 0）——可以提前收到警告。

2.7.LDAP（同步与认证）

LDAP 部分提供两项功能：(1) 当本地密码不匹配时通过 LDAP 认证管理员登录；(2) 定期从目录同步客户端状态（VLESS 标志的启用/禁用）。

登录时如何使用：服务器先校验本地的 bcrypt 密码哈希。如果它不匹配且 LDAP 已启用，面板会尝试在目录中认证用户：在设定了 Bind DN 的情况下执行一次服务性 bind，然后按过滤器和属性查找用户记录，并尝试用输入的密码以找到的 DN 进行 bind。成功即表示登录。（LDAP 认证成功后，如果启用了 2FA，仍会校验 TOTP 代码。）

部分字段：

字段	文本 (RU)	默认值	说明
启用 LDAP 同步	«Включить LDAP-синхронизацию» (enable)	false	LDAP 集成的主开关。
LDAP 主机	«LDAP-хост» (host)	空	LDAP 服务器地址。
LDAP 端口	«Порт LDAP» (port)	389	端口。LDAPS 通常为 636。
使用 TLS (LDAPS)	«Использовать TLS (LDAPS)» (useTls)	false	启用后使用 ldaps:// 方案（默认校验服务器证书）。
跳过 TLS 证书校验	«Пропускать проверку TLS-сертификата» (ldapInsecureSkipVerify)	false	3.4.2 新增。在 LDAPS 下禁用服务器证书校验——用于内部/自签名 CA。提示：「不安全——禁用服务器证书校验。仅在内部/不受信任的 CA 下使用」。该开关仅在启用「使用 TLS (LDAPS)」时可用。

字段	文本 (RU)	默认值	说明
Bind DN	«Bind DN» (bindDn)	空	用于首次 bind/查找的服务账户 DN。若为空——不执行 bind (匿名查找)。
Bind 密码	提示: 「已配置; 留空以保留当前密码。」 / 「未配置。」 / 「已配置——输入新值以替换」	空	Bind DN 的密码。单独存储; 如需保留原值, 将该字段留空。
Base DN	«Base DN» (baseDn)	空	进行查找的子树根 (递归查找, 遍历整个子树)。
用户过滤器	«Фильтр пользователя» (userFilter)	(objectClass=person)	选取账户的 LDAP 过滤器。认证时登录名会经转义后代入过滤器。
用户属性 (username/email)	«Атрибут пользователя (username/email)» (userAttr)	mail	与客户端的登录名/标识符相匹配的属性 (例如 mail 或 uid)。
VLESS 标志属性	«Атрибут VLESS-флага» (vlessField)	vless_enabled	决定客户端 VLESS 访问是否应启用的属性。
通用标志属性 (可选)	«Общий атрибут флага (опц.)» (flagField), 提示「若设置, 将覆盖 VLESS 标志——例如 shadowInactive。」	空	若设置, 则代替 vless_enabled 使用。
Truthy 值	«Truthy-значения» (truthyValues), 提示「以逗号分隔; 默认: true,1,yes,on」	true,1,yes,on	标志属性中视为「已启用」的值列表。
反转标志	«Инвертировать флаг» (invertFlag), 提示「当属性表示『已禁用』时启用 (例如 shadowInactive)。」	false	反转标志的含义。
同步计划	«Расписание синхронизации» (syncSchedule), 提示「类 cron 字符串, 例如 @every 1m」	@every 1m	以类 cron 格式表示的同步周期。
inbound 标签	«Теги входящих» (inboundTags), 提示「LDAP 同步可在其上自动创建或自动删除客户端的 inbound。」	空	限定在哪些 inbound 上允许自动操作。如果没有 inbound: 「未找到 inbound。请先创建一个 inbound。」

字段	文本 (RU)	默认值	说明
自动创建客户端	«Авто-создание клиентов» (<code>autoCreate</code>)	<code>false</code>	当客户端出现在目录中时，在指定的 <code>inbound</code> 中创建该客户端。
自动删除客户端	«Авто-удаление клиентов» (<code>autoDelete</code>)	<code>false</code>	当客户端从目录中消失时删除该客户端。
默认流量 (GB)	«Объём по умолчанию (ГБ)» (<code>defaultTotalGb</code>)	<code>0</code>	自动创建客户端的流量限制 (0 = 无限制)。
默认期限 (天)	«Срок по умолчанию (дни)» (<code>defaultExpiryDays</code>)	<code>0</code>	自动创建客户端的有效期 (0 = 永久)。
默认 IP 限制	«Лимит IP по умолчанию» (<code>defaultIpLimit</code>)	<code>0</code>	同时在线 IP 数量限制 (0 = 无限制)。

同步标志逻辑的特点：读取标志属性 (`flagField` , 默认 `vless_enabled`) 时，若其值在 `truthy` 值列表中则视为「已启用」；启用反转后结果取反。用户属性 (`userAttr`) 用作匹配键 (email/名称) —— 没有该属性值的记录会被跳过。

安全：建议启用 **TLS (LDAPS)**，以免 bind 密码和被校验的密码以明文传输；并为 Bind DN 使用一个仅具备最低必要读取权限的账户。

示例：典型的 LDAP 同步配置 (Active Directory)。针对某个目录填写本部分字段，该目录的访问状态保存在一个类似 `userAccountControl` 的标志属性中，匹配按邮箱进行：

字段	值
LDAP 主机	<code>ldap.example.com</code>
LDAP 端口	<code>636</code>
使用 TLS (LDAPS)	已启用
Bind DN	<code>CN=svc-3xui,OU=Service,DC=example,DC=com</code>
Base DN	<code>OU=Users,DC=example,DC=com</code>
用户过滤器	<code>(objectClass=person)</code>
用户属性 (username/email)	<code>mail</code>
VLESS 标志属性	<code>vless_enabled</code>
Truthy 值	<code>true,1,yes,on</code>
同步计划	<code>@every 5m</code>

在这种配置下，面板每 5 分钟遍历一次 `OU=Users` 子树，按 `mail` 匹配客户端，并根据 `vless_enabled` 的值启用/关闭 VLESS 访问。

3. 概览 / 仪表盘

仪表盘（"Дашборд", 英文界面为 *Overview*）是面板的起始页面，实时显示服务器和 Xray 进程的状态。所有指标均来自服务器端。后台调度器每 **2 秒** 重新采集一次快照并通过 WebSocket 推送至所有已打开的标签页；每分钟将累积的指标行刷入磁盘。HTTP 端点 `GET /status` 返回最新的缓存快照。

以下将逐一说明页面上的每个指标和每个控件。

自 3.4.2 起，面板侧边菜单（以及移动端抽屉）中有一个**「文档」按钮（书本图标）——打开官方文档 <https://docs.sanaei.dev/>。此外，3.4.2 进行了全面的可访问性**改进：图标按钮和可点击元素都获得了 aria 标签，并可通过 Enter/Space 激活（面向屏幕阅读器和键盘导航）。

3.1. 数据采集的基本原则

- 快照由 `gopsutil` 库采集。若某项测量失败，该字段保持为零，并向日志写入警告（`get cpu percent failed`、`get uptime failed` 等）——这不会导致整个仪表盘崩溃，只是相应的卡片显示 0/N-A。
- "瞬时"速率（CPU %、网络、磁盘 I/O）的计算方式为：当前快照与上一次快照之差除以时间间隔（秒）。因此，在页面首次加载时，速率值可能为零，需等待第二次采样后才有数据。
- 历史记录可在"系统历史"（*System History*）部分查看——图表基于下文描述的同一批数据行构建（见第 3.12 节）。

3.2. CPU

"CPU"卡片（*CPU*）显示当前 CPU 使用率百分比以及处理器参数。

指标	说明
CPU 使用率 (%)	上一个时间间隔内已占用的处理器时间占比。使用指数移动平均（EMA，系数 <code>alpha = 0.3</code> ）进行平滑，以避免指示器因骤变而抖动。值始终限制在 0–100% 范围内。在首次采样时返回 0（基准点初始化）。
逻辑处理器	逻辑核心数——即含 Hyper-Threading 的数量。
物理核心	物理核心数。
频率	CPU 基础频率（MHz）。采用惰性请求并缓存：首次成功采样的值会被保存，重试频率不超过每 5 分钟一次，且每次请求有 1.5 秒超时（某些系统上频率查询响应较慢）。

CPU 使用率的算法：若存在原生平台实现则使用之，否则通过处理器时间计数器的差值（`busy / total`）进行计算。Guest 和 GuestNice 时间被排除在外，以避免重复计算。

3.3. 内存（RAM）

"内存"卡片（*RAM*）显示已使用量和总量，以"已使用 / 总量"形式及/或使用率百分比展示。历史记录中记录百分比。

3.4. 交换空间 (Swap)

"交换空间"卡片 (Swap) 显示已使用量和总量。若未配置交换文件/分区 (总量 = 0)，指标显示为零；无 swap 时历史行写入 0。

3.5. 磁盘 (Storage)

"磁盘"卡片 (Storage) 显示已使用量和总量，仅统计根分区 `/`。"磁盘使用率" (Disk Usage) 历史中记录使用率百分比。另外单独采集磁盘 I/O (读取/写入，字节/秒)，作为时间间隔内的计数器差值——显示在历史的"磁盘 I/O"标签页中。

3.6. 系统运行时间 (Uptime)

"系统运行时间"指标 (Uptime) 表示整台服务器自启动以来的时间 (秒)，而非面板或 Xray 的运行时间。Xray 进程的运行时间单独存储 (见第 3.9 节)，面板的线程数也单独记录 ("线程" / Threads)。

面板占用的内存

在面板进程指标旁边，显示 3X-UI 进程本身占用的内存量。该值取自进程的实际 RSS (操作系统所见)，与系统工具所显示的一致。随着内存释放，数值会下降。以前面板显示的是 Go 内部计数器，会高估内存占用 (例如，只有一个客户端的空闲服务器显示约 300 MB)，且从不减少——现在这一问题已消除。此外，定期运行的后台进程会将未使用的内存归还给操作系统，使指标反映实际消耗。

3.7. 系统负载 (Load average)

"系统负载"块 (System Load) ——由三个数字组成的数组 [Load1, Load5, Load15]。提示说明："系统在过去 1、5 和 15 分钟内的平均负载" (System load average for the past 1, 5, and 15 minutes)。历史图表名为"系统平均负载 (1 / 5 / 15 分钟)"。历史行中各值分别记录：load1、load5、load15。

这是标准的 Unix 指标：处于运行队列中的进程平均数量。参考标准——与核心数对比：若负载持续超过物理核心数，表明系统过载。

3.8. 网络：速率与总流量

仅统计物理接口。虚拟接口和隧道接口被排除：包括 `lo / lo0`，以及所有以 `loopback`、`docker`、`br-`、`veth`、`virbr`、`tun`、`tap`、`wg`、`tailscale`、`zt` 开头的接口。所有其余接口的值累加求和。

总体速率 (Overall Speed) ——瞬时速率，为时间间隔内的计数器差值：

指标	说明
上传 / 发送 (标签"上传" / Upload)	发送速率，字节/秒。
下载 / 接收 (标签"下载" / Download)	接收速率，字节/秒。

总流量 (Total Data) ——自系统启动以来的累计计数器：

指标	说明
已发送（标签"已发送" / <i>Sent</i> ）	累计发送字节数。
已接收（标签"已接收" / <i>Received</i> ）	累计接收字节数。

另外还采集数据包速率（包/秒）和数据包总计数器——显示在历史的“网络数据包”（*Network Packets*）标签页中。网络历史行：`netUp`、`netDown`、`pktUp`、`pktDown`。

3.9. 服务器 IP 地址

“服务器 IP 地址”块（*IP Addresses*）显示 `IPv4` 和 `IPv6`。外部地址通过第三方服务获取（IPv4 使用 `api4.ipify.org`、`ipv4.icanhazip.com`、`v4.api.ipinfo.io/ip`、`ipv4.myexternalip.com/raw`、`4.ident.me`、`check-host.net/ip`，IPv6 使用类似的服务）。列表按顺序尝试，直到第一个成功响应；每个请求的超时为 3 秒。

注意事项：

- 结果在进程生命周期内**缓存**：成功获取的地址不会再次请求。
- 若所有服务均未响应，字段显示 `N/A`。对于 IPv6，首次出现 `N/A` 后将完全禁用 IPv6 请求，以避免在没有 IPv6 的网络上浪费时间。
- 旁边有一个“眼睛”按钮用于隐藏/显示地址——提示“切换服务器 IP 地址的可见性”（*Toggle visibility of the IP*）。这只是界面上的视觉隐藏（例如用于截图），不影响地址本身。

3.10. TCP/UDP 连接

“连接统计”块（*Connection Stats*）显示服务器上活跃的 TCP 和 UDP 连接总数（整个系统范围，不仅限于 Xray）。历史图表为“活跃连接（TCP / UDP）”（*Active Connections*），行名 `tcpCount`、`udpCount`。

3.11. Xray 状态与进程管理

“Xray”卡片显示 Xray-core 进程的状态，并提供管理控件。

状态

值	标签	含义	触发时机
<code>running</code>	“运行中”	<i>Running</i>	Xray 进程已启动。
<code>stop</code>	“已停止”	<i>Stopped</i>	进程未运行，且无已记录的启动错误。
<code>error</code>	“错误”	<i>Error</i>	进程未运行，且已记录启动错误。错误信息显示在标题为“运行 Xray 时发生错误”（ <i>An error occurred while running Xray</i> ）的弹出窗口中。
—	“未知”	<i>Unknown</i>	尚未获取到状态时显示。

状态旁边显示 **Xray 版本**。

管理按钮

- **停止 (Stop)**。调用 `POST /stopXrayService`。成功时，面板通过 WebSocket 广播新状态 `stop` 及通知“Xray 已成功停止” (*Xray service has been stopped*)；出错时广播状态 `error` 及错误信息。注意：若面板通过 Xray 本身访问，停止 Xray 可能中断与面板的连接——直接连接面板则不会有此问题。
- **重启 (Restart)**。调用 `POST /restartXrayService`。操作前显示确认提示“重启 xray? ”并说明“使用已保存的配置重新加载 xray 服务”。成功时显示状态 `running` 及通知“Xray 已成功重启” (*Xray service has been restarted successfully*)。重启会应用当前已保存的配置——在修改设置后使用此功能。

说明：本 fork 在仪表盘中为所有授权类型添加了完整的 Start / Stop / Restart 管理功能；原版 3x-ui 界面没有单独的“启动”按钮——启动通过重启来完成。

查看 Xray 日志按钮

Xray 卡片上有一个查看 Xray 日志的按钮 (*Logs*)。仅当 Xray 配置中启用了 access 日志时该按钮才会出现：内置查看器读取的正是该文件，因此在没有 access 日志的情况下按钮会隐藏。按钮的可见性绑定到独立标志 `accessLogEnable`，不再依赖 IP 限制——即使没有 access 日志，在线列表和 IP 地址限制仍然正常工作（见第 8 节）。

选择 Xray 版本

“版本选择” (*Version*) 部分允许将 Xray-core 切换到其他发行版。版本列表通过 `GET /getXrayVersion` 加载：

- 数据来源为 GitHub API 的 `XTLS/Xray-core` 仓库 (`/releases`)。请求缓存 **15 分钟**；GitHub 故障时返回最后一次成功获取的列表，确保选择器不为空。
- 列表仅包含 `X.Y.Z` 格式且不早于 **26.4.25** 的发行版。

提示：“选择您要切换的版本” (*Choose the version you want to switch to.*) 以及警告“注意：旧版本可能不支持当前配置” (*Choose carefully, as older versions may not be compatible with current configurations.*)。

切换方式：`POST /installXray/:version`。场景说明：

示例。 切换到指定版本的 Xray-core（需已通过认证获取 cookie）：

```
curl -X POST 'https://panel.example.com:2053/xpanel/installXray/v25.6.8' \
  -b cookie.txt
```

其中 `v25.6.8` 为 `GET /getXrayVersion` 返回列表中的标签。版本必须存在于该列表中，否则面板将拒绝请求。自 3.4.2 起，允许安装的最低 Xray 版本已提升至 **26.6.27**，而 3.5.0 随附的核心为 **Xray 26.7.11**。核心的随附变更：Shadowsocks 的 `none/plain` 与 VMess 的 `none/zero` 加密方式已被移除（已保存的配置会自动改写：SS 改为受支持的加密方式，VMess 改为 `auto`），指向公网地址的未加密 VLESS/Trojan outbound 在保存时会被拒绝——带有此类配置的核心本就无法启动。

1. 所选版本会在当前发行版列表中进行验证（否则拒绝）。

2. Xray 停止。
3. 根据当前操作系统和架构从 GitHub 下载 `Xray-<os>-<arch>.zip`（支持 amd64/64、arm64-v8a、arm32-v7a/v6/v5、386/32、s390x；Windows 下为 `xray.exe`）。压缩包和二进制文件大小限制为 200 MB。
4. 二进制文件原子性替换（通过临时文件 + 重命名），并标记为可执行。
5. Xray 重新启动。

切换前显示对话框“切换 Xray 版本”（*Do you really want to change the Xray version?*），说明“这将把 Xray 版本更改为 #version#”。成功时显示通知“Xray 已成功更新”（*Xray updated successfully*）。

3.12. 面板更新（3X-UI）

面板更新检查块。数据通过 `GET /getPanelUpdateInfo` 获取：

字段	说明
当前面板版本	已安装面板的版本。
最新面板版本	从 GitHub 获取的 3x-ui 最新发行版。
有可用更新	表示最新版本比当前版本更新。若无需更新，则显示“面板已是最新版本”/“已更新”。

更新面板按钮（*Update Panel*）触发 `POST /updatePanel`。提示：“这将把 3X-UI 更新到最新发行版并重启面板服务”。执行前显示确认“您真的要更新面板吗？”，内容为“这将把 3X-UI 更新到 #version# 版本并重启面板服务”。

注意事项与限制：

- 自动更新仅支持 **Linux**（其他操作系统返回错误）。
- 更新脚本从官方仓库下载（`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`，限制 2 MB），通过 `bash` 执行，条件允许时通过 `systemd-run` 进行隔离。
- 成功启动后显示“面板更新已开始”（*Panel update started*）；若更新检查失败则显示“面板更新检查失败”。安装期间显示警告“安装进行中，请勿刷新页面”。

3.13. 地理文件更新（GeoIP / GeoSite）

地理数据库更新按钮/对话框调用 `POST /updateGeofile`（所有文件）或 `POST /updateGeofile/:fileName`（单个文件）。更新严格按照文件名和来源白名单执行：

文件	来源
<code>geoip.dat</code> 、 <code>geosite.dat</code>	<code>Loyalsoldier/v2ray-rules-dat</code> (latest)
<code>geoip_IR.dat</code> 、 <code>geosite_IR.dat</code>	<code>chocolate4u/Iran-v2ray-rules</code> (latest)
<code>geoip_RU.dat</code> 、 <code>geosite_RU.dat</code>	<code>runetfreedom/russia-v2ray-rules-dat</code> (latest)

行为：

- 文件名经过验证：禁止 `..`、斜杠、绝对路径；仅允许 `[a-zA-Z0-9._-]+.dat`。白名单以外的文件不会被下载。

- 使用条件请求 `If-Modified-Since`：若源服务器上文件未更改（HTTP 304），则不重复下载，仅更新时间戳。
- 下载完成后，Xray 重启以加载新数据库。

示例。仅更新俄罗斯地理数据库，不影响其他文件：

```
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geoip_RU.dat' -b cookie.txt
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geosite_RU.dat' -b cookie.txt
```

要同时更新白名单中的所有文件，请不带文件名调用 `POST /updateGeofile`。

- 对话框内容：“您真的要更新地理文件吗？”，单个文件说明“这将更新文件 #filename#”，“全部更新”按钮说明“这将更新所有地理文件”。成功后显示“地理文件已成功更新”。

3.14. 数据库备份与恢复

“备份与恢复”块（*Backup & Restore*）。行为取决于所用数据库（默认为 SQLite 或 PostgreSQL）。

导出数据库（备份）

“导出数据库”/“备份”按钮（*Back Up*）调用 `GET /getDb`，以附件形式返回文件：

- **SQLite**：首先执行 checkpoint（刷写 WAL），然后下载 `x-ui.db` 文件。提示：“点击下载包含当前数据库备份的 .db 文件……”。
- **PostgreSQL**：下载自定义格式的转储文件 `x-ui.dump`（`pg_dump --format=custom --no-owner --no-privileges`）。服务器上需安装 PostgreSQL 客户端工具；否则返回缺少 `pg_dump` 的错误。

导入数据库（恢复）

“导入数据库”/“恢复”按钮（*Restore*）通过 `POST /importDB`（表单字段 `db`）上传文件。提示：“点击选择并上传 .db 文件……以从备份恢复数据库”。

SQLite 的恢复流程安全且可回滚：

1. 验证文件为 SQLite 格式并保存至临时文件，然后检查完整性。
2. 停止 Xray，关闭当前数据库并重命名为 `*.backup`（回滚用）。
3. 新文件替换当前数据库，执行初始化和迁移。若出现错误则恢复备份文件。
4. 重新启动 Xray。

对于 **PostgreSQL**，自 3.5.0 起“恢复”接受三种文件（类型按内容而非扩展名判断）：`pg_dump` 归档（PGDMP）——通过 `pg_restore --clean --if-exists --single-transaction ...` 应用；**SQLite** 数据库 `.db`（普通备份）和 **SQLite 迁移** `.dump` ——它们会先经过校验、必要时重组，然后由与 `x-ui migrate-db` 相同的引擎以单个事务导入 PostgreSQL（出错时 PostgreSQL 保持不变）。两种数据库引擎的文件选择对话框均接受 `.dump`, `.db`；将 `pg_dump` 归档上传到 SQLite 面板会得到明确的错误。提示明确警告：“这将替换所有当前数据”。

提示信息：“数据库导入成功”、“导入数据库时发生错误”、“.....读取数据库时”、“.....获取数据库时”。

迁移文件（SQLite 与 PostgreSQL 之间）

“下载迁移文件”按钮（*Download Migration*）调用 `GET /getMigration`。自 3.5.0 起，它仅在 **PostgreSQL** 面板上显示，并下载 `x-ui.db` ——从 PostgreSQL 数据构建、随时可用的 SQLite 数据库（“迁回 SQLite”的路径）。SQLite 面板上的该按钮已作为冗余功能移除：普通 `.db` 备份现在本就可以直接恢复到 PostgreSQL 面板。

3.15. 其他界面元素

- **在线客户端指示器**。仪表盘维护 `online` 行（*Online Clients* / “在线客户端”）——有活跃连接的客户端数量。在 Xray 运行时计算（否则为 0），并以相同的 2 秒节拍写入历史。图表为“在线”标签页。
- **系统历史（图表）**。“图表”按钮/部分 → “系统历史”，包含以下标签页：“带宽”、“数据包”、“磁盘 I/O”、“在线”、“负载”、“连接”、“磁盘使用率”。数据通过 `GET /history/:metric/:bucket` 获取；允许的聚合间隔（bucket，秒）：**2、30、60、180、360、720、1440、2880、10080**，每个标签页最多返回 60 个数据点。页面上的时间范围选择器提供以下按钮：**2m、1h、3h、6h、12h、24h、2d、7d**（对应 bucket `2` `60` `180` `360` `720` `1440` `2880` `10080`）。在较长的 **2d** 和 **7d** 范围内，时间轴标签额外显示 `MM-DD HH:MM` 格式的日期。存储采用三级降采样（rollup）：最新数据以 2 秒步长保留最近一小时，然后以 1 分钟步长平均保留 **48 小时**，再以 10 分钟步长保留 **7 天**。因此，图表（CPU、RAM、流量、数据包、连接、磁盘、在线、负载）可查看**最多 7 天**的数据（之前最多 48 小时），时间越久远，精度越粗。允许的指标：`cpu`、`mem`、`swap`、`netUp`、`netDown`、`pktUp`、`pktDown`、`diskRead`、`diskWrite`、`diskUsage`、`tcpCount`、`udpCount`、`online`、`load1`、`load5`、`load15`。标签“最近 2 分钟”对应 `bucket = 2`（实时模式）。

示例。 获取最近约 2 分钟的 CPU 使用率序列（`bucket = 2` 秒，最多 60 个数据点）以及以 5 分钟聚合的同一序列（`bucket = 300` 秒）：

```
curl 'https://panel.example.com:2053/xpanel/history/cpu/2' -b cookie.txt
curl 'https://panel.example.com:2053/xpanel/history/cpu/300' -b cookie.txt
```

指标可替换为任意允许值（`mem`、`netUp`、`tcpCount`、`load1` 等）。白名单 `2` `30` `60` `180` `360` `720` `1440` `2880` `10080` 之外的 bucket 将被拒绝。

- **Xray 指标**——独立块，显示 Xray 的内存占用和垃圾回收情况（行 `xrAlloc`、`xrSys`、`xrHeapObjects`、`xrNumGC`、`xrPauseNs`）以及“Observatory”（outbound 连接状态）。仅当 Xray 配置中设置了 `metrics` 块（`listen 127.0.0.1:11111`，标签 `metrics_out`）时才有效；否则显示“Xray 指标端点未配置”。Xray 指标窗口有独立的时间范围选择器，提供按钮 **2m、1h、3h、6h、12h**（对应 bucket `2` `60` `180` `360` `720`）。

示例：启用 Xray 指标卡片的配置块。Xray 设置部分需同时包含带标签的 `metrics` 和监听该标签的 `inbound`：

```
{
  "metrics": {
    "tag": "metrics_out"
  },
  "inbounds": [
    {
      "listen": "127.0.0.1",
      "port": 11111,
      "protocol": "dokodemo-door",
      "settings": { "address": "127.0.0.1" },
      "tag": "metrics_out"
    }
  ]
}
```

地址 `127.0.0.1:11111` 有意不对外暴露——面板在本地对其进行轮询。

- **深色主题切换。** 位于公共菜单/顶栏中，而非仪表盘本身。选项：“主题”（*Theme*），可选“深色”和“极深色”（*Ultra Dark*）。这是纯视觉外观设置，不影响面板运行。
- **仪表盘周边的其他链接**（来自菜单/底部栏）：“日志”、“配置”——查看 Xray 最终 JSON（`GET /getConfigJson`）、“文档”。

4. Inbounds：创建与通用参数

「入站」（英文 *Inbounds*）一节是 Xray 所有入口点的列表，客户端通过这些入口点进行连接。每个 inbound 既保存「面板」字段（备注、流量限额、重置计划），也保存 Xray 配置的原始 JSON 块（`settings`、`streamSettings`、`sniffing`）。

创建通过**「创建连接」（*Add Inbound*）按钮完成，编辑通过「修改连接」**（*Modify Inbound*）按钮完成。两个操作分别发送到 API 端点 `POST /add` 和 `POST /update/:id`。

下面讲解表单中不属于具体协议设置（客户端、加密、REALITY/TLS）、也不属于传输/流（「流」、**「安全」**选项卡）的所有字段——这些是其他章节的主题。

4.1. 表单通用字段

Remark（备注）

参数	值
字段	<code>remark</code>
类型	字符串
默认	空

inbound 的可读名称，显示在列表中和对话框标题里（「删除连接 "{remark}"？」等）。字段标签为**「备注」**。它不影响 Xray 的运行，仅用于方便管理；建议设置唯一且有意义的名称，因为它们会被填入导出文件的名称以及批量操作的确认信息中。

Protocol（协议）

参数	值
字段	<code>protocol</code>
标签	「协议」
校验	<code>required,oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel</code>

inbound 协议的下拉列表。允许的值：

值	备注
<code>vmess</code>	
<code>vless</code>	
<code>trojan</code>	

值	备注
shadowsocks	
wireguard	
hysteria	Hysteria v2 即带 streamSettings.version = 2 的 hysteria，没有单独的协议
http	
mixed	同一端口上的 socks/http
tunnel	
tun	校验器接受，但没有单独的协议常量

该字段为必填（required）。协议的选择决定了哪些客户端设置字段以及哪种传输可用（见各协议专属章节）。

重要：保存时服务会对 streamSettings 进行规范化。传输设置仅对 vmess、vless、trojan、shadowsocks、hysteria 协议保留；对其余协议（http、mixed、tunnel、wireguard、tun），streamSettings 字段会被强制清空。

对于 tunnel/TProxy 类型的 inbound，如果其 streamSettings 块不含 security 键（无传输变体），表单可以正常打开和保存，不会出现 streamSettings.security Invalid input 校验错误。

Listen IP（监听 IP）

参数	值
字段	listen
类型	字符串
默认	空 → Xray 监听 0.0.0.0（所有 IP）

inbound 接收连接的 IP 地址。字段提示：

「留空以监听所有 IP 地址」。

在生成 Xray 配置时，空值会被替换为 0.0.0.0。除 IP 外，该字段还接受 Unix 套接字路径——提示：

「也可以指定 Unix 套接字路径（例如 /run/xray/in.sock）或带 @ 前缀的抽象套接字名称（例如 @xray/in.sock），以监听套接字而非 TCP 端口——这种情况下请将端口设为 0」。

因此该字段接受两种 Unix 套接字形式：文件系统中的路径（/run/xray/in.sock）和带 @ 前缀的抽象套接字名称（@xray/in.sock）。两种情况下都要把 Port 设为 0。

当需要把 inbound 限制在单个接口上时（例如把仅作为 Nginx 后端 fallback 目标的 inbound 设为 127.0.0.1），或当 inbound 监听 Unix 套接字时，才会修改此字段。

示例。 仅监听本地接口（典型的 Nginx 后端 fallback 目标）和 Unix 套接字的 inbound：

```
listen = 127.0.0.1    port = 8443
listen = /run/xray/in.sock    port = 0
```

Port（端口）

参数	值
字段	port
标签	「端口」
校验	gte=0,lte=65535
默认	–（由用户设置）

TCP/UDP 监听端口。允许的值从 0 到 65535。值 0 仅在配合监听 Unix 套接字时使用（见上文）。

保存时服务会检查端口冲突：两个 inbound 不能同时占用同一传输（TCP/UDP）上相互重叠的 listen:port。传输由协议和 streamSettings/settings 推算：例如 hysteria 和 wireguard 始终占用 UDP，kcp/quic 占用 UDP，而其余大多数占用 TCP。发生冲突时保存会被拒绝并报错。

此外，面板不允许占用内部 Xray API 的保留端口（标签 api，默认 127.0.0.1 上的 62789）：本地 TCP inbound 若其监听地址在 loopback 上与该端口重叠，会以同样的端口冲突错误被拒绝。API 的真实端口从 Xray 配置模板中读取（回退值为 62789）。在节点（nodes）上此限制不生效——它们有自己的 Xray。

Xray 标签（Tag，唯一）会从端口和传输自动生成，格式为 in-<?>-<tcp|udp|tcpudp|any>；对于部署在节点上的 inbound，会加上 n<nodeId>- 前缀。发生冲突时会向标签追加 -2、-3 等。用户通常不编辑标签。

Total traffic（总流量，GB）

参数	值
字段	total（以字节计）
标签	「总用量」
默认	0

inbound 的总流量限额。表单中以吉字节输入，数据库中以字节存储。字段提示：

「= 无限制。（单位：GB）」。

也就是说，`0` 表示无限制。这是整个 inbound 层级（而非单个客户端）的限额；实际已用流量保存在 `up`（已发送）和 `down`（已接收）字段中，并与 `total` 比较。

Expiry date / Duration（到期日期 / 时长）

参数	值
字段	<code>expiryTime</code> （Unix 时间戳）
标签	「到期日期」（英文 <i>Duration</i> ）
默认	空 / <code>0</code>

inbound 的有效期。提示：

「留空表示永久有效」。

空值（`0`）表示无期限的 inbound。该值以 Unix 时间戳存储；表单既允许设置具体日期，也允许设置以天为单位的时长（从当前时刻相对计算——英文字段标签 *Duration*）。

Enabled（启用）

参数	值
字段	<code>enable</code>
标签	「启用」（英文 <i>Enabled</i> ）
默认	创建时设置

inbound 的活动状态标志。在列表中切换此标志由专门的「轻量」端点 `POST /setEnabled/:id` 处理，而非完整更新——这样设计是为了避免在每次点击拥有上千客户端的 inbound 开关时都重新序列化整个 `settings` 块（所有客户端）。关闭 inbound 时会将其从运行中的 Xray 移除，启用时则重新加入。

Node / Deploy to（节点 / 部署到）

参数	值
字段	<code>nodeId</code>
标签	「部署到」、「本地面板」
默认	空（本地面板）

选择 inbound 实际运行的位置：本地面板或某个已注册的节点。自 3.5.0 起，「部署到」列表（以及 inbound 列表上方的「节点」筛选器）支持输入搜索，inbound 列表上方还新增了搜索框（按备注、端口和协议搜索；可与节点筛选器配合使用）。实现上的特点：`nodeId = 0` 会被规范化为 `nil`，因为 `0` 不是有效的节点 id，而是表单绑定的产物；`nil / 0` 表示本地面板。在离线节点上保存 inbound 时可能出现提示

「更改将在节点重新连接时同步」。自 3.4.2 起，该提示仅在节点确实离线或已关闭时显示（此前在保存到在线节点时也可能出现）。

分享链接的地址策略（Share address strategy）

参数	值
字段	策略 + （可选）自定义地址
标签	「分享链接的地址策略」（英文 <i>Share address strategy</i> ）
默认	「inbound 监听地址」（ <i>Inbound listen</i> ）

下拉列表决定将哪个地址填入该 inbound 的导出分享链接和二维码。可选值：

值	标签	填入的内容
node	「节点地址」（ <i>Node address</i> ）	inbound 所运行节点的地址
listen	「inbound 监听地址」（ <i>Inbound listen</i> ）	inbound 自身的监听地址
custom	「自定义」（ <i>Custom</i> ）	来自**「分享链接自定义地址」**（ <i>Custom share address</i> ）字段的自定义地址

选择**「自定义」时会出现「分享链接自定义地址」字段；在其中输入主机或 IP，不含协议和端口（该值会被校验）。「节点地址」选项仅当存在可运行该 inbound 的已启用节点时才显示在列表中；否则它会被隐藏，且取值回退为「inbound 监听地址」**。

此策略仅影响直接的分享链接和二维码，不影响订阅的输出——那里的地址仍由面板的常规逻辑决定。

4.2. Sniffing（嗅探）

「嗅探」选项卡编辑 Xray 配置的 `sniffing` 块，该块以原始 JSON 存储。Sniffing 允许 Xray「窥探」连接内部的真实域名/协议，用于路由目的。

子字段	标签	用途
<code>enabled</code>	（选项卡开关）	为 inbound 开启/关闭嗅探
<code>destOverride</code>	—	要拦截目标地址的协议列表： <code>http</code> 、 <code>tls</code> 、 <code>quic</code> 、 <code>fakedns</code>
<code>metadataOnly</code>	「仅元数据」	仅使用连接元数据，不读取有效负载
<code>routeOnly</code>	「仅路由」	嗅探结果仅用于路由，不重写目标地址
<code>domainsExcluded</code>	「排除的域名」	从嗅探中排除的域名
（排除的 IP）	「排除的 IP」	从嗅探中排除的 IP 地址

- `destOverride` —— 嗅探器集合：`http`（从 HTTP Host 头识别域名）、`tls`（从 SNI 识别）、`quic`（从 QUIC ClientHello 识别）、`fakedns`（与 FakeDNS 池匹配）。通常为识别域名会开启 `http` 和 `tls`。

sniffing 块示例（通过 HTTP 和 TLS 识别域名，结果仅用于路由，不触碰本地网络）：

```
{
  "enabled": true,
  "destOverride": ["http", "tls"],
  "routeOnly": true,
  "domainsExcluded": ["courier.push.apple.com"]
}
```

- **metadataOnly** —— 开启时，Xray 不读取首个数据包的内容，仅依赖元数据；这有助于不破坏那些不能「窥探」数据的协议。
- **routeOnly** —— 嗅探结果仅供路由规则使用；此时 outbound 中的连接地址不会被重写为识别出的域名。

注意：面板将 **sniffing** 作为不透明的 JSON 块存储，保存时不向其添加任何内容——这些复选框的所有默认值都由客户端应用一侧生成。原始块可通过「入站 JSON」一节编辑（见下文）。

4.3. Allocate（端口分配策略）

streamSettings 中的 **allocate** 块控制 Xray 如何分配监听端口。这是 Xray 配置的一部分；面板将其作为 **streamSettings** /inbound JSON 的一部分存储和传递。参数（按 Xray-core 术语）：

子字段	用途	值 / 默认
strategy	端口分配策略	always —— 始终监听指定端口（默认）； random —— 周期性在范围内变更监听端口
refresh	random 时端口变更间隔（分钟）	整数分钟（建议 5；最小为 2）
concurrency	random 时同时保持开放的端口数	整数（默认 3；不超过端口范围宽度的三分之一）

strategy: always 让 inbound 固定在单个端口上（标准模式）。**strategy: random** 用于反封锁场景，此时 inbound 周期性在端口范围内「跳动」；这种情况下 **refresh** 和 **concurrency** 才有意义。只有在有意使用随机端口模式时才需要修改这些值。

streamSettings 中的 **allocate** 块示例（随机端口模式：保持 3 个端口开放，每 5 分钟变更一次）：

```
{
  "allocate": {
    "strategy": "random",
    "refresh": 5,
    "concurrency": 3
  }
}
```

要使其生效，inbound 的 **port** 需设为一个范围（例如 **20000-20100**）。

4.4. External Proxy（外部代理）

「External Proxy」字段属于邀请链接生成的设置，存储在 inbound 的 `streamSettings` 中。它设置一组备用外部地址（主机/端口，必要时带强制 TLS——「强制 TLS」），这些地址会被填入客户端链接，替代 inbound 真实的 `listen:port`。

当客户端不应直接连接服务器、而要经由外部代理/反代/CDN 连接时使用：此时分享链接中填的是该前端的公网地址。这不影响 Xray 接收连接的过程——它只是生成链接的「外观修饰」。相关表单字段：「强制 TLS」、「Fingerprint」、每条记录的标签。

4.5. Fallbacks（Fallback）

「Fallback」一节设置对未匹配 inbound 任何客户端的连接进行转发的规则。仅对 TLS 传输上的主 inbound（VLESS/Trojan TCP-TLS）可用。通过端点 `GET /:id/fallbacks` / `POST /:id/fallbacks` 管理。

该节提示：

「当此入站上的连接未匹配任何客户端时，会被转发到别处。在下方选择一个子入站，路由字段（SNI / ALPN / Path / xver）将自动从其传输填入；或将选择留空并直接指定 Dest（例如 8080 或 127.0.0.1:8080），以转发到外部服务器，如 Nginx。每个子入站都应在 127.0.0.1 上以 `security=none` 监听」。

fallback 一节仅对基于 RAW（TCP）且安全为 TLS 或 REALITY 的 VLESS/Trojan inbound 显示。新建的 inbound 以 `security=none` 启动，因此该节起初可能看起来不存在。在此状态下（VLESS/Trojan、RAW/TCP、安全尚未配置），会显示一段内嵌提示代替该节：在**「安全」**选项卡选择 TLS 或 Reality 后，fallback 才会可用。

fallback 行的字段

字段	默认	描述
（子 inbound）	—	选择子 inbound（标签**「选择入站」**）。若已选择，Name/Alpn/Path/Dest 字段可从其传输自动填入
Name	空（= 任意）	按名称（SNI/名称）的匹配条件。「任意」标签为**「任意」**
Alpn	空	按 ALPN 的匹配条件
Path	空	按路径的匹配条件（用于子 inbound 的 WS/HTTP 传输）
Dest	自动	转发目标。占位符**「自动（子项的 listen:端口）」**。可指定端口（8080）或 <code>host:port</code> （127.0.0.1:8080）
Xver	0	PROXY 协议版本（「Xver」）：0 —— 关闭，1 或 2 —— 对应的 PROXY protocol 版本
（顺序）	按位置	规则的应用顺序；通过**「上移」/「下移」**按钮设置

保存逻辑：主 inbound 的整个 fallback 列表被原子替换。既没有选择子 inbound（`childId <= 0`）、也没有指定 `Dest` 的行会被跳过。若所选子 inbound 等于主 inbound 自身的 id，则将其清零。生成最终 JSON 时：若 `Dest` 为空，则从子 inbound 计算为 `listen:port`，其中 `0.0.0.0/::/::0` 被替换为 `127.0.0.1`；`name/alpn/path` 为空的字段不会进入输出 JSON；`xver` 仅在大于 0 时才添加。

最终 `settings.fallbacks` 示例（带 `alpn=h2` 的流量经路径 `/ws` 转发到 WS 目标，其余全部转发到端口 8080 上的本地 Nginx）：

```
{
  "fallbacks": [
    { "alpn": "h2", "path": "/ws", "dest": "127.0.0.1:2001", "xver": 1 },
    { "dest": 8080 }
  ]
}
```

最后一行不含 `name/alpn/path`，是捕获其余所有流量的「默认」规则。

fallback 一节的按钮和提示

- 「添加 fallback」—— 添加一行；「暂无 fallback」—— 空状态。
- 「快速添加所有合适项」 / 「全部添加」—— 为每个尚未连接的合适 inbound 添加一行 fallback。结果：「已添加 {n} 个 fallback」或「没有新的合适入站」。
- 「从子项填充」—— 从所选子 inbound 的传输重新拉取路由字段（SNI/ALPN/Path/xver）；执行后显示「已从子项填充」。
- 「修改路由字段」 / 「隐藏高级项」—— 显示/隐藏行的细节字段。
- 「路由条件」和「默认——捕获其余所有流量」标签说明了每行的触发条件。

保存 fallback 后，服务器会触发 Xray 重启，使新的 `settings.fallbacks` 生效。

4.6. 周期性流量重置

「流量重置」块按计划配置 inbound 流量计数器的自动重置。描述：

「按指定间隔自动重置流量计数器」。

参数	值
字段	<code>trafficReset</code>
校验	<code>omitempty,oneof=never hourly daily weekly monthly</code>
默认	<code>never</code>
关联字段	<code>lastTrafficResetTime</code> —— 上次重置的时间戳（标签**「上次重置」**）

下拉列表：

值	标签
never	「从不」
hourly	「每小时」
daily	「每天」
weekly	「每周」
monthly	「每月」

每个周期都注册了一个 cron 作业，按相应计划运行（@hourly、@daily、@weekly、@monthly）。作业会选出所有设置了对应 trafficReset 的 inbound，并对每个 inbound 重置其自身的计数器（up=0、down=0）以及其所有客户端的流量。也就是说，周期性重置同时影响 inbound 及其客户端。

字段值示例。 要让计数器在每月一号清零，表单中选择**「每月」**，保存为：

```
{ "trafficReset": "monthly" }
```

值 never（默认）完全关闭自动重置。

4.7. 入站 JSON（高级）

「入站 JSON 分区」一节提供对 inbound 原始 JSON 块的直接访问。描述：

「完整的入站 JSON 以及针对 settings、sniffing 和 streamSettings 的独立编辑器」。

可用的编辑器：

选项卡	标签	编辑内容
全部	「在一个编辑器中包含所有字段的完整入站对象」	整个 Inbound 对象
设置	「Xray settings 块的封装」	settings 字段
Sniffing	「Xray sniffing 块的封装」	sniffing 字段
Stream	「Xray stream 块的封装」	streamSettings 字段

这些字段被序列化为嵌套的 JSON 对象：空块以 null 返回，而非有效 JSON 的文本会被包装成字符串，以免数据丢失。保存时的解析错误会带**「高级 JSON」**前缀显示。

「入站 JSON」查看窗口和 inbound 导入窗口一样，使用带 JSON 语法高亮的完整代码编辑器（而非普通文本框）：查看配置时为高亮的只读模式，导入时为可编辑模式，从而便于阅读和修改。

4.8. inbound 操作：QR / Edit / Reset / Delete 及统计

在列表和 inbound 卡片中可执行以下操作（**「菜单」**菜单）：

流量统计

显示 inbound 的汇总流量：「已发送/已接收」（up / down 字段）、「总流量」、「总连接数」。卡片中还有**「创建于」、「更新于」、「到期日期」**。

inbounds 列表中有一个 **Speed** 列，显示每个 inbound 当前的流量速度（上传/下载），它由两次轮询间计数器的增量计算得出；同样的实时速度也显示在 inbound 统计窗口中。当某次轮询没有增量时，速度值会被重置。

在 inbounds 页面的客户端汇总中，状态按「已耗尽/已结束」优先级判定：到期或流量耗尽（并被自动任务取消了 enable）的客户端归入**「已耗尽/已结束」（*Depleted/Ended*）状态，而非灰色的「已禁用」（*Disabled*），且不会被重复计数。该分类与客户端自身卡片中显示的一致，并正确处理绑定到多个 inbound 的客户端。

二维码与复制链接

- 「详情」—— 展开连接和订阅链接。
- 客户端二维码：提示**「点击二维码即可复制」**。
- 「复制链接」（英文 *Copy URL*）、「导出链接」。自 3.4.2 起，页面级操作**「导出所有链接」**（GET /panel/api/inbounds/allLinks；「导出 inbound 所有链接」窗口，文件名「All-Inbounds」）通过订阅引擎生成链接——为每个客户端套用备注模板，并优先使用受管 Host 端点（此前使用固定备注 inbound-email）。

Edit（修改）

「修改连接」—— 打开编辑表单（POST /update/:id）。更新时服务会重新读取现有行，迁移已变更的字段，必要时重新生成标签（若旧标签是自动生成的），并同步 Xray 运行时。成功时显示提示**「连接更新成功」**。

Reset Traffic（重置流量）

「重置流量」—— 将此 inbound 的 up / down 计数器清零（POST /:id/resetTraffic，设为 up=0, down=0）。确认：

「重置 "{remark}" 的流量？」 / 「将此连接的发送/接收计数器重置为 0」。

重置 inbound 流量不触动其客户端的计数器（客户端有单独的「重置客户端流量」操作）。重置后会触发 Xray 重启。成功时显示提示**「入站流量已重置」。还有批量版本——「重置所有连接的流量」**（POST /resetAllTraffics）。

Delete（删除）

「删除连接」（POST /del/:id）。确认：

「删除连接 "{remark}"? 」 / 「该连接及其所有客户端都将被删除。此操作无法撤销」。

删除会将 inbound 从运行中的 Xray 移除（必要时伴随重启）。成功时显示提示**「连接删除成功」**。批量删除为 `POST /bulkDel`，带逐项报告，且最多只重启一次 Xray。

inbound 客户端的其他操作

菜单中还提供：「克隆」（带新端口和空客户端列表的 inbound 副本）、「删除所有客户端」（`POST /:id/delAllClients` —— 删除所有客户端，inbound 本身保留）、「删除已禁用的客户端」、「绑定/解绑客户端」、「导入」/「导出连接」（`POST /import`）。客户端操作的细节属于客户端章节。

5. 协议

创建 inbound 时，首先需要选择协议（"Protocol"）。协议决定了 Xray-core 对该 inbound 采用何种身份验证和流量加密方式、`settings` 中需要填写哪些字段，以及该 inbound 支持哪些传输方式（`network`）和安全类型（TLS / REALITY）。

协议字段在创建 inbound 时设置一次，**编辑时不可更改**（编辑表单中的下拉列表处于禁用状态）。如需更改协议，需创建新的 inbound。

5.1. 支持的协议列表

服务端接受以下 `Protocol` 字段值：

```
oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun mtproto
```

从版本 **3.3.0** 起，列表中新增了 `mtproto` 值（Telegram 代理）。

配置中的值	用途	客户端模型
<code>vless</code>	主要代理协议（创建 inbound 时的默认值）	使用 UUID 的客户端，支持 flow 和量子加密
<code>vmess</code>	Xray 经典代理协议	使用 UUID 和 <code>security</code> 参数的客户端
<code>trojan</code>	伪装成普通 HTTPS 的代理	使用密码的客户端
<code>shadowsocks</code>	Shadowsocks 代理（包括 SIP022 / 2022-blake3）	单用户或多用户（2022）
<code>wireguard</code>	WireGuard inbound	peer（而非客户端）
<code>hysteria</code>	Hysteria inbound（默认版本 2）	使用 <code>auth</code> 令牌的客户端
<code>http</code>	经典 HTTP 代理（正向代理）	用户名/密码账户，不计流量
<code>mixed</code>	合并的 SOCKS + HTTP 代理	用户名/密码账户
<code>tunnel</code>	透明转发器（xray dokodemo-door）	无客户端
<code>tun</code>	TUN 接口（仅用于渲染现有配置）	无客户端
<code>mtproto</code>	Telegram 代理（MTPROTO），3.3.0 新增；由独立进程 <code>mtg</code> 而非 Xray 处理	无客户端（通过密钥访问）

关于 `tun`：该值保留在列表中是为了兼容性和显示先前保存的 inbound，但在当前版本的后端中不建议创建，支持已标记为废弃。创建此类型的新 inbound 没有意义。

关于 Hysteria 2：不存在单独的“hysteria2”协议。这是 hysteria 协议，其 streamSettings.version = 2。在生成分享链接时，如果流版本为 2，则自动选择 hysteria2:// 链接方案。

并非所有协议都支持分发到节点（nodes）。可部署到节点的协议仅有：vless、vmess、trojan、shadowsocks、hysteria、wireguard。协议 http、mixed、tunnel、tun、mtproto 仅在本地面板上运行。

5.2. 哪些协议支持 TLS / REALITY / 传输

能否启用某种安全层和传输方式取决于协议和所选网络（streamSettings.network）：

功能	可用协议	允许的网络（network）
TLS	vmess、vless、trojan、shadowsocks（以及 hysteria 始终可用）	tcp、ws、http、grpc、httpupgrade、xhttp
REALITY	vless、trojan	tcp、http、grpc、xhttp
flow（xtls-rprx-vision）	仅 vless	仅 tcp，且 security = tls 或 reality
Stream / 传输（“传输”选项卡）	vmess、vless、trojan、shadowsocks、hysteria	—

对于 http、mixed、tunnel、tun、wireguard 协议，传输选项卡不可用——它们没有 Xray 的 stream 设置。

5.3. VLESS

用途：主要的现代代理协议。支持 XTLS-Vision（flow）、REALITY，以及 VLESS 自身级别的后量子加密（字段 decryption / encryption）。新建 inbound 时默认使用。

settings 块的字段：

字段	默认值	说明
clients	[]	客户端列表。每个客户端包含：id（UUID）、email（必填）、flow、限制（limitIp、totalGB、expiryTime）、enable、tgId、subId、comment、reset
decryption	none	服务端解密参数。界面标签：“Decryption”
encryption	none	配对的加密参数（写入客户端链接）。标签：“Encryption”
fallbacks	[]	fallback 列表（参见 fallback 相关章节）；当 network = tcp 且 security 为 TLS 或 REALITY 时可用
testseed	（4 个数值：900,500,900,256）	“Vision testseed”——4 个正整数，用于 XTLS-Vision padding。仅对 flow 为 xtls-rprx-vision 的客户端生效，否则忽略

flow (xtls-rprx-vision)

flow 在客户端上设置，而非 inbound，可取以下三个值之一：

值	含义
"" (空)	不使用 XTLS-flow (默认)
xtls-rprx-vision	XTLS-Vision——推荐在 VLESS over TCP+TLS/REALITY 时使用
xtls-rprx-vision-udp443	同 Vision，但增加了对 UDP/443 (QUIC) 的处理

只有满足以下所有条件时，flow 字段才可选：协议为 vless、network = tcp 且 security 为 tls 或 reality。Vision testseed 字段也在相同条件下显示。

XHTTP 的例外：在 VLESS over network = xhttp 且启用了 VLESS 后量子身份验证 (encryption / decryption, vlessenc) 时，flow xtls-rprx-vision 也是允许的——无论安全层如何，包括与 REALITY 搭配使用。此时面板会正确地将 xtls-rprx-vision 传入分享链接和订阅（包括 Clash/Mihomo 格式），客户端将获得包含 Vision 的配置。

解密 / 加密 (VLESS 后量子身份验证)

decryption 和 encryption 字段是 VLESS 自身级别的身份验证（独立于传输层 TLS/REALITY）。默认情况下两者均为 none。表单中这些字段下方有一个***"密钥生成"块——包含模式下拉列表和"生成"按钮（旁边有"清除"按钮）。下拉列表包含六个选项：X25519 (native)、X25519 (xorpub)、X25519 (random)、ML-KEM-768 (native)、ML-KEM-768 (xorpub)、ML-KEM-768 (random)——即两种密钥类型（经典 X25519 和后量子 ML-KEM-768），各有三种模式：

- native——所选类型的基本密钥对；
- xorpub——对公钥进行额外处理的派生模式；
- random——带随机分量的派生模式。

从列表中选择所需模式，然后点击***"生成"：面板将用该模式下的一对值填充两个字段（decryption 和 encryption）。"清除"按钮将两个字段重置为 none。

字段下方显示状态行***已选择：...***，它会根据生成的字符串识别密钥类型（X25519 或 ML-KEM-768）和模式（native / xorpub / random）并予以显示。空字段或 none 显示为"None"。

从技术上讲，按钮调用 GET /panel/api/server/getNewVlessEnc（通过 xray vlessenc 生成密钥）并用类似 mlkem768x25519plus.native.<rtt>.<role> 的配对值填充两个字段（例如 decryption = mlkem768x25519plus.native.600s.server-x25519，encryption = mlkem768x25519plus.native.0rtt.client-x25519）。decryption 参数保留在服务端，encryption 进入客户端链接。

重要提示：在为 Xray 生成 inbound 配置时，面板会移除多余内容：如果 settings 中存在 encryption（属于客户端侧），它将从服务端配置中被移除。服务端只保留 decryption。

何时选择 VLESS：这是新建 inbound 时的推荐默认选项，尤其适合与 REALITY（无需自有证书）或 TLS + XTLS-Vision 搭配使用。

示例：包含单个客户端和 XTLS-Vision 的 VLESS inbound 的 `settings` 块。 `flow` 字段在客户端上，`decryption` 保留在服务端：

```
{
  "clients": [
    {
      "id": "d342d11e-d424-4583-b36e-524ab1f0afa4",
      "email": "user1",
      "flow": "xtls-rprx-vision",
      "limitIp": 2,
      "totalGB": 0,
      "expiryTime": 0,
      "enable": true
    }
  ],
  "decryption": "none"
}
```

与 REALITY 搭配时，对应的 `streamSettings` 块（"Transport"选项卡 → Security: REALITY）如下：

```
{
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "dest": "www.microsoft.com:443",
    "serverNames": ["www.microsoft.com"],
    "privateKey": "<X25519 [?]>",
    "shortIds": ["", "6ba85179e30d4fc2"]
  }
}
```

5.4. VMess

用途：Xray 经典代理协议。通过 UUID 进行身份验证，客户端侧可额外配置载荷加密方式（`security`）。

`settings` 块的字段：

字段	默认值	说明
<code>clients</code>	<code>[]</code>	客户端列表

每个 VMess 客户端（除了公共字段 `email`、限制、`enable`、`tgId`、`subId`、`comment`、`reset` 之外）：

客户端字段	默认值	说明
<code>id</code>	—	客户端 UUID

客户端字段	默认值	说明
security	auto	VMess 载荷加密方式。允许的值：aes-128-gcm、chacha20-poly1305、auto、none、zero

security 的值：

- auto ——Xray 根据平台自动选择加密算法（推荐）；
- aes-128-gcm、chacha20-poly1305 ——固定的 AEAD 加密算法；
- none ——不加密载荷（仅在 TLS 之上有意义）；
- zero ——不加密也不验证载荷。

历史兼容性：旧记录可能存储了 security: "" ——读取时空字符串会被转换为 auto。在生成服务端配置时，VMess 客户端的 security 字段会从 settings 中删除，因为 inbound 不需要该字段。

何时选择 VMess：用于与旧客户端或现有配置兼容。对于新部署，通常优先选择 VLESS。

5.5. Trojan

用途：模拟普通 HTTPS 流量的代理。通过密码进行身份验证。与 VLESS 一样，支持 fallback，且在 network = tcp 时支持 REALITY/TLS。

settings 块的字段：

字段	默认值	说明
clients	[]	客户端列表
fallbacks	[]	fallback 列表（在 network = tcp 且使用 TLS/REALITY 时可用）

每个 Trojan 客户端的关键字段：

客户端字段	默认值	说明
password	—	客户端密码（必填，至少 1 个字符）
email	—	客户端唯一标识符

其余客户端字段为公共字段（limitIp、totalGB、expiryTime、enable、tgId、subId、comment、reset）。

何时选择 Trojan：需要在 443 端口伪装成 HTTPS 时，包括对非预期连接设置 fallback 到 Web 服务器（Nginx）的场景。

示例：包含 fallback 到本地 Web 服务器的 Trojan settings 块。未携带有效密码的非预期连接将转发至监听 127.0.0.1:8080 的 Nginx：

```
{
  "clients": [
    { "password": "S3cret-Pass-1", "email": "user1" }
  ],
  "fallbacks": [
    { "dest": "127.0.0.1:8080" }
  ]
}
```

使用 fallback 需要 `network = tcp` 且 Security 为 TLS 或 REALITY；否则 fallbacks 字段不可用。

5.6. Shadowsocks

用途：轻量级 Shadowsocks 代理。支持旧版 AEAD 加密算法和新版 SIP022 方式（2022-blake3-*）。可在单用户或多用户模式下运行。

settings 块的字段：

字段	默认值	说明
method	2022-blake3-aes-256-gcm	inbound 加密方式。界面标签：“Encryption method”
password	``	inbound 密码（2022 方式下会根据所选方式自动生成）
network	tcp,udp	传输方式。标签：“Network”。选项：tcp,udp（TCP,UDP）、tcp、udp
clients	[]	客户端列表
ivCheck	false（关闭）	“ivCheck”开关——防止 IV 重复使用

加密方式（method）

允许的值：

方式	类别
aes-256-gcm	旧版 AEAD
chacha20-poly1305	旧版 AEAD
chacha20-ietf-poly1305	旧版 AEAD
xchacha20-ietf-poly1305	旧版 AEAD
2022-blake3-aes-128-gcm	SS 2022（推荐）
2022-blake3-aes-256-gcm	SS 2022（默认）
2022-blake3-chacha20-poly1305	SS 2022，单用户

面板关于方式的处理逻辑：

- **2022 方式**（`2022-blake3-*`）被视为"SS 2022"。方式 `2022-blake3-chacha20-poly1305` 为**单用户**（不支持多用户）；其他 2022 方式支持多个客户端。密码字段（带有生成按钮，会根据方式调整密钥长度）仅在 2022 方式下显示在表单中。
- **旧版加密算法**（`aes-*`、`chacha20-*`）按照经典的"单方式 + 单密码"方案运行。

运行 Xray 前的规范化处理：对于旧版加密算法，每个客户端的 `method` 必须与 `inbound` 的方式一致（否则 Xray 会报"unsupported cipher method:"错误）。对于 2022 方式则相反——客户端的 `method` 字段必须为空（否则 Xray 会拒绝 `inbound`，报"users must have empty method"）。切换方式时，面板会自动将数据整理规范。

更换密钥大小时重新生成客户端密钥：对于 Shadowsocks-2022，当加密方式切换到密钥大小不同的方式时（例如在 `2022-blake3-aes-256-gcm` 和 `2022-blake3-aes-128-gcm` 之间切换），面板在保存 `inbound` 时会自动重新生成客户端 PSK 以适应新长度。否则旧密钥将保持原有长度，Xray 会拒绝它们。结果：受影响的客户端需要重新获取订阅——旧链接将无法连接。

何时选择 Shadowsocks：适用于无需 TLS 伪装的简单部署；现代推荐选择 2022-blake3 方式。

示例：2022-blake3 方式下的 Shadowsocks settings 块（多用户模式）。 `inbound` 有自己的密码（所需长度的 base64 密钥），每个客户端有自己的密码，客户端的 `method` 字段为空：

```
{
  "method": "2022-blake3-aes-256-gcm",
  "password": "d2hhdGV2ZXItMzItYnl0ZS1iYXNlnjQta2V5LWlcmU=",
  "network": "tcp,udp",
  "clients": [
    {
      "email": "user1",
      "password": "Y2xpZW50LWtleS0zMilieXRlcy1iYXNlnjQtaGVyZQ==",
      "method": ""
    }
  ]
}
```

对于旧版加密算法（`aes-256-gcm` 等）则相反：`inbound` 只有一个密码，客户端的 `method` 必须与 `inbound` 的方式一致。

5.7. Dokodemo-door / Tunnel（透明转发器）

用途：透明转发器（在面板中为 `tunnel` 协议，实现 `dokodemo-door` 的行为）。接收流量并将其转发到指定的地址/端口，无需身份验证和客户端。

`settings` 块的字段：

字段	默认值	说明
<code>rewriteAddress</code>	(无)	"Rewrite address"—流量被重定向到的目标地址
<code>rewritePort</code>	(无)	"Rewrite port"—目标端口 (0-65535)
<code>allowedNetwork</code>	<code>tcp,udp</code>	"Allowed network"。选项: <code>tcp,udp</code> 、 <code>tcp</code> 、 <code>udp</code>
<code>portMap</code>	<code>{}</code>	"端口映射"—端口到端口的映射 (Record<string,string>)
<code>followRedirect</code>	<code>false</code> (关闭)	"Follow redirect"—使用从拦截连接中获取的原始目标地址

Tunnel 的"Transport"选项卡: 此类型的 inbound 有***"Transport"***选项卡, 仅限配置 `sockopt` ——这足以支持 **TProxy** 模式 (通过 `sockopt.tproxy` 进行透明代理/重定向)。传输选择下拉列表 (`network`) 和 Tunnel 的"Security"选项卡已隐藏, 因为此类型不支持 TLS/REALITY。

何时选择: 用于将端口透明代理/重定向到内部服务。

"Rewrite port" (`rewritePort`) 字段可以留空: 清除后该值只是从 inbound 设置中排除, 不会导致保存错误。(以前清除该字段会导致 `settings.rewritePort` 验证错误并阻止保存, 包括通过 JSON 选项卡保存。)

5.8. SOCKS / HTTP (`mixed` 协议)

此版本中没有单独的 `socks` 协议——SOCKS 和 HTTP 代理合并为 `mixed` 协议 (合并的 SOCKS + HTTP)。此外还有单独的纯 `http` 代理。

5.8.1. Mixed (SOCKS + HTTP)

`settings` 块的字段:

字段	默认值	说明
<code>auth</code>	<code>password</code>	"Auth"—身份验证模式。选项: <code>password</code> (用户名/密码) 或 <code>noauth</code> (无需授权)
<code>accounts</code>	(可选)	"账户"—用户名/密码对列表。 <code>auth = noauth</code> 时不写入配置
<code>udp</code>	<code>false</code> (关闭)	"UDP"开关——通过 SOCKS 支持 UDP
<code>ip</code>	<code>127.0.0.1</code>	"UDP IP"—UDP 关联的本地地址。仅在启用 <code>udp</code> 时显示

账户通过"添加"按钮添加; 添加时会生成随机用户名 (8 个字符) 和密码 (12 个字符), 可自行编辑。

5.8.2. HTTP (纯代理)

用途: 经典的 HTTP 正向代理。在 Xray 层面不将客户端作为"计费"用户追踪 (无 email/限制) ——只有账户列表。

settings 块的字段：

字段	默认值	说明
accounts	[]	"账户"—用户名/密码对列表（两个字段均必填）
allowTransparent	false（关闭）	"Allow transparent"—以原始 Host 头转发请求

何时选择 SOCKS/HTTP：用于无需复杂伪装的本地或服务性代理访问。mixed 的优点在于单个端口同时服务 SOCKS 和 HTTP 客户端。

5.9. WireGuard (inbound)

用途：WireGuard inbound——WireGuard VPN 隧道，而非伪装代理。传输和 TLS/REALITY 对其不适用。

自 3.4.2 起，WireGuard 改用多客户端模型（与 VLESS/VMess/Trojan/Shadowsocks/Hysteria 一样）。inbound 本身只保存服务端部分；peer 设备现在作为普通客户端添加（参见第 8 节）——隧道内地址自动分配，支持订阅、流量/有效期限限制和分组。inbound 表单中原先内联的「Peer / 添加 peer」列表已移除：新建 inbound 时不含 peer，每个客户端的地址都会自动分配。

inbound 字段（settings 块）：

字段	默认值	说明
secretKey	—	服务端私钥（必填）。旁边有生成按钮；公钥（Public Key）由其自动推导（只读；原先单独保存的 pubKey 已移除）
mtu	1420	接口 MTU
dns	1.1.1.1, 1.0.0.1	3.4.2 新增。写入客户端 .conf 的 DNS = 行的 DNS
noKernelTun	false（关闭）	"No-kernel TUN"—使用用户空间 TUN 而非内核 TUN
domainStrategy	（可选）	"Domain Strategy": ForceIP、ForceIPv4、ForceIPv4v6、ForceIPv6、ForceIPv6v4

客户端的 WireGuard 参数——位于客户端添加/编辑窗口的**「凭据」**选项卡（当绑定的 inbound 为 WireGuard 时显示）：

字段	说明
「WireGuard 私钥」	客户端私钥（可编辑，带「重新生成」按钮）；输入后会由其自动推导公钥
「WireGuard 公钥」	只读；由私钥计算得出
「WireGuard 预共享密钥」（PSK）	可选的额外预共享密钥
「WireGuard 允许的 IP」	自 3.5.0 起可编辑（占位符 10.0.0.2/32，提示「留空以自动分配；多个条目以逗号分隔」）。留空——地址自动分配；服务器会校验每个条目（IP 或 CIDR），将单个地址规范化为 /32，并拒绝已被该 inbound 其他客户端占用的地址

隧道内地址由服务器从 inbound 的子网中自动分配（默认 10.0.0.0/24：服务器占用 .1，客户端从 .2 开始）；如果现有客户端使用其他 /24，则新地址在同一子网中分配。

Domain Strategy 与 Transport 选项卡

除上述字段外，WireGuard inbound 还有 **Domain Strategy** 字段（域名解析策略：ForceIP、ForceIPv4、ForceIPv4v6、ForceIPv6、ForceIPv6v4）。该字段为可选，仅在设置后才写入配置。

Workers 字段（workers，工作线程数）已从 WireGuard 表单（包括 inbound 和 outbound）中移除：从 xray-core v26.6.22 起，引擎不再使用该字段，而是依赖 wireguard-go 的内部机制。之前保存的配置无需修改即可正常工作——解析时该字段会被忽略，无需迁移。

WireGuard 也有“Transport”选项卡——但功能有限：只能配置 sockopt 和 Finalmask 混淆。传输选择下拉列表（network）已隐藏，因为 WireGuard 始终监听 UDP。在 Finalmask 的噪声记录（noise）中，Rand Range（字节范围 0-255，带验证）作为单独字段设置，对于 WireGuard 和 Hysteria，可用的混淆方法还包括 Salamander。

inbound 操作与导出（自 3.5.0 起）

WireGuard inbound 的行菜单现在包含完整的「多客户端」操作集——「导出所有链接」、按订阅导出、绑定/解绑客户端、加入分组和删除所有客户端（此前只有导出/重置/克隆/删除），列表中还会显示客户端计数。对于 WireGuard，「导出所有链接」窗口拆分为两个选项卡：**Config**（拼接在一起的 .conf 块，每个客户端一段）和 **Links**（专属 wireguard:// 链接）；「复制」和「下载」作用于当前打开的选项卡。对于部署在节点上的 inbound，客户端 .conf /QR 中的 Endpoint 现在指向节点地址，而非主面板地址。

WireGuard 客户端配置与分享

WireGuard 客户端在「信息」/QR 窗口中提供一张可折叠的配置卡片：完整的 .conf（[Interface] 含 PrivateKey / Address / DNS / MTU，[Peer] 含 PublicKey / PresharedKey / AllowedIPs = 0.0.0.0/0, ::/0 / Endpoint / PersistentKeepalive），并带有复制、下载和 QR 按钮。同时还支持 wireguard:// / wg:// 形式的链接，客户端应用可将其解析为完整的 .conf。

何时选择 WireGuard：当需要的是真正的 WireGuard VPN 隧道，而非伪装代理时。

5.10. Hysteria（默认 v2）

用途：基于 QUIC 的 Hysteria inbound。面板默认使用版本 2。每个客户端使用 auth 令牌而非 UUID/密码进行身份验证。Hysteria 始终可用 TLS（参见 5.2 中的功能对照表）。

settings 块的字段：

字段	默认值	说明
version	2	协议版本（最小值 1；面板默认为 2）

字段	默认值	说明
clients	[]	客户端列表

每个客户端的关键字段为 `auth`（令牌，必填），以及公共字段（`email`、限制、`enable`、`tgId`、`subId`、`comment`、`reset`）。

附加参数在 `streamSettings.hysteriaSettings` 中设置：

字段	值 / 选项	说明
version	固定为 2（字段已锁定）	"Version"
udpIdleTimeout	（整数 ≥ 1，秒）	"UDP idle timeout (s)"——UDP 空闲超时
masquerade	默认关闭	"Masquerade"——对非预期请求伪装成普通 Web 服务器

启用 `masquerade` 后，可选择类型（`type`）：

- `""`——默认（404 页面）；
- `proxy`——反向代理（字段："Upstream URL"、"Rewrite Host"、"Skip TLS verify"）；
- `file`——提供目录文件（字段："目录"，例如 `/var/www/html`）；
- `string`——固定响应（字段："状态码"、"Body"、"Headers"）。

何时选择 Hysteria：在需要 QUIC 传输和在不稳定/移动网络上保持稳定性时；伪装功能可提高入口点的隐蔽性。

5.11. MTPROTO（Telegram 代理）

在版本 3.3.0 中新增。协议值为 `mtpROTO`。

MTPROTO 是 Telegram 自有代理协议。在 3X-UI 中，此类 inbound 不由 Xray 处理，而由独立进程 `mtg` 处理，面板负责管理该进程。面板会定期将已启用的 MTPROTO inbound 与正在运行的 `mtg` 进程进行比对：启动缺失的进程，停止多余的进程，并从 `mtg` 的指标中读取流量计数。因此，此类 inbound 的流量统计与普通协议一样正常工作。

自 3.5.0 版本起，MTPROTO 是多客户端协议（引擎更换为分支 `mtg-multi`）：一个 inbound 可服务多个用户，每个用户都是普通客户端（参见第 8 节），拥有自己的 FakeTLS 密钥、流量配额、有效期、启用开关、可选的 `ad-tag` 以及专属的 `tg://proxy` 链接。原先“一个 inbound = 一个密钥”的模型已废弃：表单中 inbound 级别的“Secret”字段已移除，面板升级时现有的单个密钥会自动转换为该 inbound 的第一个客户端（无需手动操作）。

其影响：

- “Transport”（Stream Settings）选项卡不适用于此 inbound；客户端现在则像其他协议一样管理——绑定/解绑、限制、分组、订阅。

- 客户端更改（添加、删除、更换密钥、启用/禁用、ad-tag）通过 `mtg` 的 management API “热”应用，不会中断其他用户的 Telegram 会话；只有 inbound 的结构性更改（地址、domain fronting、连接数限制、公网 IP）才会重启进程。
- MTPROTO inbound 仅在主面板上运行；不会部署到子节点（nodes）（带有指定 NodeID 的 inbound 会被跳过）。
- MTPROTO 的“Sniffing”选项卡已隐藏——该协议由 `mtg` 进程处理而非 Xray，因此嗅探不适用。

字段。存储在 inbound 的 `settings` 中：

界面字段	键名	说明
Remark	<code>remark</code>	inbound 标签。
Listen IP	<code>listen</code>	监听 IP（为空表示所有接口）。
Port	<code>port</code>	代理端口。
伪装域名 (FakeTLS)	<code>settings.fakeTlsDomain</code>	默认为 <code>www.cloudflare.com</code> 。自 3.5.0 起是为新客户 端生成密钥的默认域名（提示：“Default FakeTLS domain used to generate a new client's secret. Each client can front its own domain”）；每个客户端都可以通过自己 的密钥伪装成自己的域名。
Max connections	<code>settings.throttleMaxConnections</code>	3.5.0 新增。 对所有用户的并发连接总数限制，公平分 配； <code>0</code> 表示不限制。
Public IPv4 / Public IPv6	<code>settings.publicIpv4</code> / <code>settings.publicIpv6</code>	3.5.0 新增。 用于 ad-tag middle proxy 的服务器公网地 址（占位符 <code>1.2.3.4</code> / <code>2001:db8::1</code> ）；留空则由 <code>mtg</code> 自行检测。

密钥格式 (FakeTLS)。 密钥现在归每个客户端所有（客户端表单中的 “MTPROTO secret” 字段，带重新生成按钮；旁边是可选的 “Ad-tag (赞助频道)”，恰好 32 个十六进制字符）。密钥格式不变：`ee` + 32 个十六进制字符 + 伪装域名的十六进制编码（`ee<hex32><hex(??)>`）；将新客户绑定到 MTPROTO inbound 时，密钥会根据 inbound 的默认域名自动生成。自 3.5.0 起，客户端的流量配额和有效期真正生效（通过 `mtg-multi` 的限制）：耗尽或过期的客户端会被断开连接，重置流量后立即恢复访问。

域前置和 mtg 扩展选项

MTPROTO inbound 还有 `mtg` 进程的附加参数。**Domain fronting IP**、**Domain fronting port** 和 **Domain fronting PROXY protocol** 字段指定 `mtg` 将非 Telegram 流量（例如发送到伪造的 NGINX 网站）转发到何处：IP 留空则通过 DNS 使用 FakeTLS 域名，端口默认为 `443`。此外还有 **Accept PROXY protocol**（用于监听器）、**IP preference**（`prefer-ipv6` / `prefer-ipv4` / `only-ipv6` / `only-ipv4`）和 **Debug logging**。每个值仅在设置后才写入 `mtg-<id>.toml` 文件。

通过 Xray 路由 Telegram 流量

“Route through Xray”开关（默认关闭）和可选的 **Outbound** 字段允许将 Telegram 的出口流量交由 Xray 路由器处理。启用后，面板会在 Xray 配置中嵌入一个带有 inbound 自身标签的本地 SOCKS 桥接，`mtg` 则通

过该桥接发送 Telegram 流量。之后可以在"Routing"选项卡中用规则匹配该流量，或通过 **Outbound** 字段强制将其导向指定的 outbound 或负载均衡器（若字段为空，则由路由规则决定）。

如何分发给用户。面板会为 MTProto inbound 生成邀请链接：

示例：FakeTLS 密钥和完整链接。 若伪装域名为 `www.cloudflare.com`，密钥构造为 `ee` + 32 个十六进制字符 + 域名的十六进制编码，例如：

```
secret = ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

完整邀请链接（将其和二维码通过 Telegram 发送给用户）：

```
tg://proxy?
server=203.0.113.10&port=443&secret=ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f75646
66c6172652e636f6d
```

```
tg://proxy?server=<[?]>&port=<[?]>&secret=<[?]>
```

（等效链接——`https://t.me/proxy?server=...&port=...&secret=...`）。自 3.5.0 起该链接是**专属的**——由具体客户端的密钥构成，且其中的 `#remark` 片段已被移除（不严格的 Telegram 解析器会把它与密钥拼接在一起，导致导入失败）；备注改为在信息窗口中以单独的标签显示。将此链接和二维码发送给 Telegram 用户——打开后代理会立即添加到应用中。该链接也可通过订阅服务器提供。

何时使用。 这是绕过 Telegram 封锁的标准方式；FakeTLS 伪装（伪装域名）使流量看起来像是对指定网站的普通访问。

5.12. 协议选择速查表

- **VLESS**——默认首选；与 REALITY 或 TLS + XTLS-Vision 搭配最佳，支持后量子身份验证。
- **Trojan**——伪装成 HTTPS，可设置 fallback 到 Web 服务器。
- **VMess**——与旧客户端兼容。
- **Shadowsocks**——无需 TLS 伪装的简单代理；现代推荐使用 `2022-blake3-*` 方式。
- **Hysteria**——QUIC 传输，在网络条件差的情况下表现稳定。
- **mixed / http**——服务性 SOCKS/HTTP 代理。
- **WireGuard**——完整的 VPN 隧道。
- **tunnel**——透明端口转发。
- **MTProto**——绕过 Telegram 封锁的代理（FakeTLS）；独立进程 `mtg`。

6. 传输层 (Stream Settings)

传输层（在面板界面中为「**Транспорт**」字段，英文 *Transmission*）决定了 Xray-core 在 inbound 内部传输数据的方式：在 TLS/Reality 之上使用何种网络协议，以及流量具体如何成帧。这些参数会保存到 Xray 配置的 `streamSettings` 对象中，并在 inbound 编辑器的传输层选项卡中设置。加密（TLS / Reality）在单独的章节中讨论——此处仅描述网络的选择及其参数。

6.1. 选择传输网络

网络在「**Транспорт**」下拉列表（`streamSettings.network`）中选择。默认值为 `tcp`（在列表中显示为 **RAW**）。可用的选项如下：

列表中的值	<code>network</code> 字段	传输层
RAW	<code>tcp</code>	普通 TCP（在新版 Xray 中已重命名为 RAW），可选 HTTP 混淆
mKCP	<code>kcp</code>	可靠的 UDP 传输 mKCP
WebSocket	<code>ws</code>	基于 HTTP(S) 的 WebSocket
gRPC	<code>grpc</code>	gRPC 隧道 (HTTP/2)
HTTPUpgrade	<code>httpupgrade</code>	HTTP Upgrade
XHTTP	<code>xhttp</code>	XHTTP / SplitHTTP——现代的可复用多路传输

切换该值时，面板会清空上一个网络的设置块，并用其结构定义中的默认值填充新网络的设置块，因此子表单中的每个字段始终都有合理的初始值。

重要。 在本版面板中，列表里没有 **HTTP/2 (h2) 传输**——它已从网络集合中移除；如需双向的类 HTTP/2 隧道，请使用 gRPC，而现代的 HTTP 伪装传输请使用 XHTTP。**Hysteria** 传输 (`hysteria`) 不通过此列表选择：它与 Hysteria 协议硬绑定，当 inbound 本身以 Hysteria 协议创建时会自动出现（见第 6.8 节）。

下面分别解析每种网络及其每个字段。

6.2. RAW / TCP (`tcpSettings`)

基础 TCP 传输。默认情况下流量按「原样」传输；可选地将其伪装成普通的 HTTP/1.1 交互。

字段	默认值	说明
Proxy Protocol (<code>acceptProxyProtocol</code>)	<code>false</code> (关闭)	接收来自上游负载均衡器/代理的 PROXY protocol 头
HTTP 混淆 (<code>header.type</code>)	<code>none</code> (关闭)	启用将流量伪装为 HTTP/1.1

Proxy Protocol

「Proxy Protocol」开关（`acceptProxyProtocol`）。启用后，Xray 会在入站连接上预期收到 PROXY protocol 头，并从中提取客户端的真实 IP。仅当面板前面有添加该头的反向代理/负载均衡器（例如 HAProxy 或带 `send-proxy` 的 nginx）时才启用。默认关闭。

HTTP 混淆（camouflage）

「HTTP Обфускация」开关。控制 `header` 字段：

- 关闭 → `header.type = "none"`（在传输线路上 `header` 字段直接不存在）。纯 TCP。
- 开启 → `header.type = "http"`。流量按 HTTP/1.1 请求和响应的样式成帧。启用时，面板会立即用默认值填充 `request` 和 `response` 子对象。

启用 HTTP 混淆后，会出现用于配置模拟请求和响应的字段。

请求头（`header.request`）：

字段	键	默认值	说明
请求版本	<code>request.version</code>	1.1	请求起始行中的 HTTP 版本
请求方法	<code>request.method</code>	GET	模拟请求的 HTTP 方法
请求路径	<code>request.path</code>	/	路径（可多个）。以逗号分隔的值列表输入；在传输线路上是字符串数组。若留空，则填入 /
请求头	<code>request.headers</code>	{ } (空)	HTTP 头的「名称/值」表。存储为映射 <code>[[key] → [value]]</code> （一个名称可对应多个值）

响应头（`header.response`）：

字段	键	默认值	说明
响应版本	<code>response.version</code>	1.1	响应起始行中的 HTTP 版本
响应状态	<code>response.status</code>	200	模拟响应的 HTTP 状态码
响应原因	<code>response.reason</code>	OK	状态的文本描述（reason-phrase）
响应头	<code>response.headers</code>	{ } (空)	响应头的「名称/值」表（映射 <code>[[key] → [value]]</code> ）

头部字段按行编辑——每一行设置头名称（Имя）及其值（Значение）。这些参数仅用于伪装流量的外观；它们不影响加密。默认值（GET / HTTP/1.1，响应 200 OK）适用于大多数场景——只有在需要模拟特定网站/服务时才值得修改。

RAW 带 HTTP 混淆的 `streamSettings` 示例：

```
{
  "network": "tcp",
  "tcpSettings": {
    "acceptProxyProtocol": false,
    "header": {
      "type": "http",
      "request": {
        "version": "1.1",
        "method": "GET",
        "path": ["/"],
        "headers": {
          "Host": ["www.example.com"]
        }
      },
      "response": {
        "version": "1.1",
        "status": "200",
        "reason": "OK"
      }
    }
  }
}
```

请注意：在传输线路上 `path` 是字符串数组，每个头都是值的数组（`Host` → `["www.example.com"]`）。

6.3. mKCP（`kcpSettings`）

mKCP 是基于 UDP 的可靠传输。在丢包率高、延迟大的链路上很有用，但会产生更高的额外开销流量。所有默认值都与 xray-core 中推荐的值一致。

字段	键	默认值	允许范围	说明
MTU	<code>mtu</code>	1350	576–1460	最大数据包大小（字节）。出现分片问题时调小
TTI（毫秒）	<code>tti</code>	20	10–100	传输间隔（ms）。越小延迟越低，但额外开销越高
Uplink（MБ/c）	<code>uplinkCapacity</code>	5	≥ 0	上行的估算吞吐能力（MБ/c）
Downlink（MБ/c）	<code>downlinkCapacity</code>	20	≥ 0	下行的估算吞吐能力（MБ/c）
CWND 乘数	<code>cwndMultiplier</code>	1	≥ 1	拥塞窗口（congestion window）乘数
最大发送窗口	<code>maxSendingWindow</code>	2097152	≥ 0	发送窗口的最大大小

字段说明：

- **Uplink / Downlink capacity** 决定 mKCP 占用链路的激进程度。应按实际链路带宽设置：值过高会导致多余流量，值过低则导致链路未被充分利用。
- **TTI** 直接影响「延迟」与「额外开销」的权衡：较小的值会降低延迟，但会增加额外开销数据包的数量。
- **MTU** 限制单个 mKCP 数据包的大小；调低有助于在大型 UDP 包被截断或丢弃的链路上工作。

在本版中，mKCP 子表单内的「seed」字段（mKCP 混淆密码）和头部类型/混淆下拉列表（none、srtp、utp、wechat-video、dtls、wireguard）**未作为单独字段提供**——传输层混淆已整合进通用的「FinalMask」机制（包括 mkcp-legacy 模式），在相应章节中描述。作为单独复选框的「congestion」参数也未提供；拥塞控制通过 cwndMultiplier 和 maxSendingWindow 设置。

6.4. WebSocket（wsSettings）

基于 HTTP(S) 的 WebSocket 传输。能很好地穿越 CDN 和反向代理，伪装成普通的 Web 流量。

字段	键	默认值	说明
Proxy Protocol	acceptProxyProtocol	false	接收来自上游代理的 PROXY protocol 头（见第 6.2 节）
Хост	host	""（空）	HTTP 头 Host 的值。通过 CDN/域前置工作时指定
Путь	path	/	WebSocket 握手请求行中的路径
heartbeat 周期	heartbeatPeriod	0	发送 heartbeat 帧的间隔（秒）。0 表示禁用 heartbeat
Заголовки	headers	{ }（空）	握手的附加 HTTP 头。映射「名称 → 值」（仅字符串值，无数组）

说明：

- **Путь** 必须在服务端（inbound）和客户端一致。通常会在 Web 服务器一侧用该路径来伪装入口点。
- **Хост** 在 inbound 位于 CDN 之后或使用域前置时才有意义；否则可以留空。
- **heartbeat 周期** 使连接在穿越会积极断开非活动会话的代理/CDN 时保持「存活」。默认（0）禁用 heartbeat。
- 与 RAW 不同，WebSocket 的头部表使用「扁平」格式 `key → value`（每个头一行值）。

WebSocket 位于 CDN 之后的 streamSettings 示例：

```
{
  "network": "ws",
  "wsSettings": {
    "acceptProxyProtocol": false,
    "host": "cdn.example.com",
    "path": "/ray",
    "heartbeatPeriod": 0,
    "headers": {
      "User-Agent": "Mozilla/5.0"
    }
  }
}
```

host 和 path 的值必须与客户端一致；与 RAW 不同，这里头的值是普通字符串，而非数组。

6.5. gRPC (`grpcSettings`)

参数数量最「精简」的传输。在 gRPC 调用内部（基于 HTTP/2）隧道化流量；与支持 gRPC 的 CDN 兼容性良好。没有头部混淆。

字段	键	默认值	说明
服务名 (Service Name)	<code>serviceName</code>	<code>""</code> (空)	gRPC 服务名（实际上是隧道的「路径」）。服务端和客户端必须一致
Authority	<code>authority</code>	<code>""</code> (空)	伪头 <code>:authority</code> 的值（HTTP/2 中相当于 <code>Host</code> ）。通过 CDN/域工作时指定
Multi Mode	<code>multiMode</code>	<code>false</code> (关闭)	启用在单个连接内多路复用多个并行 gRPC 流

说明：

- **Service Name**——gRPC 通道的主要标识符；它在两端必须相同。空值是允许的，但通常会设置一个不明显的字符串用于伪装。
- **Authority** 影响 HTTP/2 帧中发送的 `:authority`；首先在通过 CDN 代理时需要。
- **Multi Mode** 允许多个逻辑流通过单个物理连接；在服务端和客户端都支持时启用以提升性能。

gRPC 的 `streamSettings` 示例：

```
{
  "network": "grpc",
  "grpcSettings": {
    "serviceName": "GunService",
    "authority": "grpc.example.com",
    "multiMode": false
  }
}
```

`serviceName`（此处为 `GunService`）扮演隧道「路径」的角色，必须在服务端和客户端一致。

6.6. HTTPUpgrade (`httpupgradeSettings`)

基于 HTTP Upgrade 机制的传输（与 WebSocket 类似，但不含 WebSocket 协议本身）。同样能很好地穿越代理和 CDN。字段集与 WebSocket 相同，但不含 heartbeat 周期。

字段	键	默认值	说明
Proxy Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	接收来自上游代理的 PROXY protocol 头
Хост	<code>host</code>	<code>""</code> (空)	HTTP 头 <code>Host</code> 的值
Путь	<code>path</code>	<code>/</code>	带 <code>Upgrade</code> 头的 HTTP 请求路径

字段	键	默认值	说明
Заголовки	headers	<code>{}</code> (空)	附加 HTTP 头。「扁平」映射 <code>{key: value} → [key, value]</code> (与 WebSocket 相同)

Хост、**Путь** 和 **Заголовки** 字段的用途与 WebSocket (第 6.4 节) 相同。HTTPUpgrade 没有 heartbeat——这是 WebSocket 特有的。

6.7. XHTTP / SplitHTTP (`xhttpSettings`)

XHTTP (又称 SplitHTTP) 是 xray-core 的现代可复用多路 HTTP 传输。它将上行流和下行流拆分成单独的 HTTP 请求，非常适合 CDN 以及对连接持续时间有限制的环境。编辑器中并非所有字段都会同时显示：其中一部分会根据所选模式 (`mode`) 和启用的开关而出现。

基础字段 (始终可见)

字段	键	默认值	说明
Хост	host	<code>""</code> (空)	HTTP 头 <code>Host</code> 的值
Путь	path	<code>/</code>	HTTP 请求的基础路径
模式 (<code>Mode</code>)	mode	auto	传输模式 (见下文)
Server Max Header Bytes	serverMaxHeaderBytes	0	服务端请求头大小限制 (字节)。0 为 xray-core 的默认值
Padding Bytes	xPaddingBytes	100-1000	随机「填充」padding 的范围 (字节, 格式 <code>[min]-[max]</code>), 用于干扰大小分析
Заголовки	headers	<code>{}</code> (空)	附加 HTTP 头。「扁平」映射 <code>{key: value} → [key, value]</code>
Uplink 的 HTTP 方法	uplinkHTTPMethod	<code>""</code> (Default = POST)	上行请求的 HTTP 方法。可选: 空 (默认为 POST)、POST、PUT、GET (最后一项仅在 packet-up 模式下可用)
Padding Obfs Mode	xPaddingObfsMode	false (关闭)	启用增强的 padding 混淆并展开附加字段 (见下文)
No SSE Header	noSSEHeader	false (关闭)	不发送 <code>Content-Type: text/event-stream</code> (SSE) 头。当它妨碍穿越中间节点时启用

「模式」字段 (`mode`)

下拉列表, 取值:

值	说明
auto	自动选择模式 (默认)

值	说明
packet-up	上行流被拆分成单独的 HTTP 请求（每个请求一个数据包）
stream-up	上行流通过一个长时持续的流式请求传输
stream-one	一个共用的双向流式请求

模式的选择决定了哪些附加字段会变为可见。

仅在 `mode = packet-up` 时可见的字段：

字段	键	默认值	说明
最大缓冲上传数	scMaxBufferedPosts	30	上行流同时缓冲的 POST 请求的最大数量
最大上传大小 (字节)	scMaxEachPostBytes	1000000	单个上行 POST 请求的最大大小 (字节)
Uplink Data Placement	uplinkDataPlacement	"" (Default = body)	上行流数据的放置位置: body、header、cookie、query
Uplink Data Key	uplinkDataKey	""	uplink 数据的键名/头名。仅当 uplinkDataPlacement 已设置且不等于 body 时出现

仅在 `mode = stream-up` 时可见的字段：

字段	键	默认值	说明
Stream-Up Server	scStreamUpServerSecs	20-80	服务端流式连接的保持时间范围 (秒, 格式 [?][?]-[?][?])

Padding 混淆字段 (在 `xPaddingObfsMode = 开启` 时可见)

字段	键	默认值	说明
Padding Key	xPaddingKey	"" (占位符 x_padding)	padding 的键名
Padding Header	xPaddingHeader	"" (占位符 X-Padding)	传递 padding 所用的 HTTP 头名称
Padding Placement	xPaddingPlacement	"" (Default = queryInHeader)	padding 的放置位置: queryInHeader、header、cookie、query
Padding Method	xPaddingMethod	"" (Default = repeat-x)	padding 的生成方法: repeat-x 或 tokenish

会话与序列的放置（始终可见）

字段	键	默认值	说明
Session ID Placement	<code>sessionIDPlacement</code>	<code>""</code> (Default = path)	传递会话标识符的位置: <code>path</code> 、 <code>header</code> 、 <code>cookie</code> 、 <code>query</code>
Session ID Key	<code>sessionIDKey</code>	<code>""</code> (占位符 <code>x_session</code>)	会话键名。仅当 <code>sessionIDPlacement</code> 已设置且不等于 <code>path</code> 时出现
Session ID Table	<code>sessionIDTable</code>	<code>""</code> (占位符 <code>Base62</code>)	用于生成会话标识符的字符集。可从自动补全下拉列表中选择预定义值 (<code>ALPHABET</code> 、 <code>Alphabet</code> 、 <code>BASE36</code> 、 <code>Base62</code> 、 <code>HEX</code> 、 <code>alphabet</code> 、 <code>base36</code> 、 <code>hex</code> 、 <code>number</code>)，或输入任意 ASCII 字符串。空——xray-core 的默认值
Session ID Length	<code>sessionIDLength</code>	<code>""</code> (空)	所生成标识符的长度或范围 (例如 8-16)。仅在已设置 <code>Session ID Table</code> 时显示；最小值必须大于 0
Sequence Placement	<code>seqPlacement</code>	<code>""</code> (Default = path)	传递数据包序号的位置: <code>path</code> 、 <code>header</code> 、 <code>cookie</code> 、 <code>query</code>
Sequence Key	<code>seqKey</code>	<code>""</code> (占位符 <code>x_seq</code>)	序列键名。仅当 <code>seqPlacement</code> 已设置且不等于 <code>path</code> 时出现

会话字段在 xray-core v26.6.22 中已重命名：之前称为 **Session Placement / Session Key**

(`sessionPlacement` / `sessionKey`) ——现在是 **Session ID Placement / Session ID Key**

(`sessionIDPlacement` / `sessionIDKey`)；内核已不再识别旧名称。更新前保存的 inbound 会自动迁移到新键——无需重新保存。

建议：

- 对于大多数安装，只需保持 **模式 = auto**，设置 **Путь/Хост**，并（在通过 CDN 工作时）与客户端协调一致即可。
- 放置字段 (`*Placement` / `*Key`) 和 padding 混淆仅在针对特定反 DPI/CDN 场景进行精细调整时才需要；当值为空时，使用括号中标注的 xray-core 默认值。
- 与客户端/outbound 一侧相关的参数（例如重复 POST 的间隔、分块大小）不会出现在 inbound 表单中——监听服务器会忽略它们。相反，XMUX 多路复用器在 inbound 表单中是可用的（见下文）。
- 不会写入额外开销的默认值。面板不再向 XHTTP 配置写入额外开销默认值 `scMaxEachPostBytes` 和 `scMinPostsIntervalMs` ——会应用 xray-core 的内部值。这消除了一个固定的 DPI 特征（字面量 `scMinPostsIntervalMs=30`），此前流量可能因此被封锁。对于已保存的 inbound，与 xray-core 默认值相同的值不会出现在链接和订阅中（无需重新保存 inbound）；手动设置的值会被保留。

XHTTP (auto 模式) 的 streamSettings 示例：

```
{
  "network": "xhttp",
  "xhttpSettings": {
    "host": "xhttp.example.com",
    "path": "/yourpath",
    "mode": "auto",
    "xPaddingBytes": "100-1000"
  }
}
```

对于大多数安装，这四个字段就足够了；会话/序列放置字段和 padding 混淆字段留空即可——这样会使用 xray-core 的默认值。

XMUX（连接多路复用）

XMUX 开关（`enableXmux`）启用多路复用层，它将并行请求分配到一个小的物理连接池上。启用后会展开六个可配置字段（保存在 `xhttpSettings.xmux` 中）：

字段	键	默认值	说明
Max Concurrency	<code>maxConcurrency</code>	16-32	单个连接上的最大并发请求数（范围 <code>??-??</code> ）
Max Connections	<code>maxConnections</code>	6	物理连接的最大数量（0 表示无限制）。自 3.4.2 起，新建 inbound 默认值为 6；已有 inbound 保留其原值。
Max Reuse Times	<code>cMaxReuseTimes</code>	""（空）	连接可重复使用的次数
Max Request Times	<code>hMaxRequestTimes</code>	600-900	单个连接上的最大请求数（范围）
Max Reusable Secs	<code>hMaxReusableSecs</code>	1800-3000	连接可用于重复使用的时长（秒，范围）
Keep Alive Period	<code>hKeepAlivePeriod</code>	""（空）	用于保持连接的 keep-alive 周期

重要。 不能同时设置 **Max Connections** 和 **Max Concurrency**——xray-core 会拒绝这样的配置。默认情况下，启用 XMUX 时面板会设置 `Max Concurrency = 16-32`；如果你设置了 **Max Connections**（值大于 0），面板会移除默认的 **Max Concurrency** 值以避免冲突。

6.8. Hysteria 传输（`hysteriaSettings`）

Hysteria 传输不在「Транспорт」列表中选择：当 inbound 以 Hysteria 协议创建时它会自动激活，而对其他协议则隐藏（离开 Hysteria 协议时，网络会被强制重置为 `tcp`）。参数：

字段	键	默认值	说明
版本	version	2（已固定，字段被锁定）	Hysteria 版本。仅支持 Hysteria 2
UDP idle timeout（秒）	udpIdleTimeout	60	UDP 会话空闲超时（秒）。允许范围为 2–600；xray-core 在启动时会拒绝该区间之外的值
Masquerade	masquerade	关闭（不存在）	在被探测时启用将监听器伪装成 HTTP/3 服务器

启用 **Masquerade** 后会出现类型（type）选择以及依赖于它的字段：

- **"" – default (404 page)**：返回标准的 404 页面（无需额外字段）。
- **proxy (reverse proxy)**：反向代理到外部网站。
 - url（Upstream URL，占位符 https://www.example.com）——目标地址；
 - rewriteHost（Переписать Host，默认 false）——替换 Host 头；
 - insecure（Пропустить TLS verify，默认 false）——不校验上游的 TLS 证书。
- **file (serve directory)**：从目录提供文件。
 - dir（Директория，占位符 /var/www/html）。
- **string (fixed body)**：固定的 HTTP 响应。
 - statusCode（Код статуса，默认 0，范围 0–599）；
 - content（Body）——响应体；
 - headers（Заголовки）——映射 [?] → [?]

Masquerade 让基于 Hysteria 的 inbound 在主动探测中看起来像普通的 HTTP/3 服务器，从而提高隐蔽性。默认情况下伪装是关闭的。

带反向代理（masquerade → proxy）的 hysteriaSettings 示例：

```
{
  "version": 2,
  "udpIdleTimeout": 60,
  "masquerade": {
    "type": "proxy",
    "url": "https://www.example.com",
    "rewriteHost": true,
    "insecure": false
  }
}
```

此处在被探测时，监听器会返回来自 https://www.example.com 的响应，将自己伪装成普通的 HTTP/3 网站。

6.9. 相关参数

除了选择网络之外，同一选项卡上还有两个不依赖于具体传输的通用块（详见相应章节）：

- **External Proxy**（externalProxy）——一组外部地址/端口，会在订阅链接中替换面板自身的地址。
- **Sockopt**（sockopt）——底层套接字选项（TCP Fast Open、mark、域策略、透明代理等）。

Real client IP（识别 CDN/中继后的真实 IP）

当 inbound 位于中间方（如 Cloudflare 这样的 CDN、L4 隧道/中继或另一个面板）之后时，Xray 看到的是中间方的地址，而非真实访问者的地址。该地址会进入在线客户端列表，并据此计算每客户端的 IP 限制，这导致两者在代理之后都失去意义。为了恢复真实 IP，inbound 表单的 **Sockopt** 区块中有一个 **Real client IP** 预设选择，它整合了 `acceptProxyProtocol` 和 `trustedXForwardedFor` 的设置：

预设	作用	何时使用
Off / direct	清空两个字段。	inbound 对客户端直接可达
Cloudflare CDN	设置 <code>sockopt.trustedXForwardedFor = ["CF-Connecting-IP"]</code> 。	位于 Cloudflare CDN（橙色云朵）之后的 WebSocket / HTTPUpgrade / XHTTP / gRPC
L4 relay / Spectrum (PROXY)	启用 <code>acceptProxyProtocol = true</code> 。	inbound 前面有 L4 隧道/中继或 Cloudflare Spectrum

各预设相互排斥：选择其中一个会清空另一个的字段，因此过时的 `trustedXForwardedFor` 不会覆盖通过 PROXY 协议恢复的 IP。预设下方仍可见「原始」的 **Proxy Protocol** 开关和 **Trusted X-Forwarded-For** 列表——预设只是替你填写它们，必要时可手动修改。如果所选预设不被当前传输支持（例如在 mKCP 上使用 PROXY 协议），表单会显示警告。这些字段仅与服务端相关，并且绝不会在订阅中发送给客户端。

只用其中一个。`acceptProxyProtocol` 从 L4 的 PROXY 协议头中读取真实 IP，而 `trustedXForwardedFor` 从 HTTP 请求头中读取；只有当你的中间方链路有此要求时，才值得手动将两者混用。

- **FinalMask**（`finalmask`）——通用的传输层混淆机制（包括 mKCP 的 legacy 混淆），它取代了各网络子表单内单独的「seed」/「header type」字段。

7. 连接安全：TLS、XTLS 与 REALITY

每个支持通过传输流传递数据的 inbound（VMess、VLESS、Trojan、Shadowsocks、Hysteria）在编辑器中都有「安全」选项卡。该选项卡用于配置传输通道的加密和混淆方式。共有三种模式，通过单选按钮切换：

模式	UI 标签	可用条件
none	无	始终可用（Hysteria 除外，该协议强制要求 TLS）
tls	TLS	适用于 VMess/VLESS/Trojan/Shadowsocks 在 tcp、ws、http、grpc、httpupgrade、xhttp 网络上；对 Hysteria 始终可用
reality	Reality	仅适用于 VLESS/Trojan 在 tcp、http、grpc、xhttp 网络上

如果协议是 Hysteria，无按钮不会显示（该协议强制要求 TLS）。Reality 按钮仅在协议与网络类型组合合法时显示（见上表）。

切换模式时，面板会完全重建 streamSettings 块：移除前一模式的 tlsSettings 和 realitySettings，并填入所选模式的默认值。特别是选择 Reality 时，面板会自动：从内置热门域名列表中随机填入一对 target + serverNames（SNI），生成随机 shortIds，并向服务器请求最新的 X25519 密钥对（privateKey/publicKey）。

7.1. 区别：TLS vs XTLS vs REALITY

- **TLS** – 基于 TLS 1.2/1.3 协议的经典传输加密。服务器需要有效的证书（自有域名 + 证书链）。流量表现为普通 HTTPS，但对于主动审查者而言，其 TLS 握手特征可被识别；若通过 SNI 屏蔽或缺少受信任证书，连接将被封锁或报错。
- **XTLS (Vision)** – 这并非「安全」列表中的独立模式，而是运行于 TLS 或 Reality 之上的 flow 机制。通过 inbound 客户端侧的 Flow 字段设置为 xtls-rprx-vision（或 xtls-rprx-vision-udp443）来启用。Vision 适用于网络为 tcp 且 security = tls 或 security = reality 的 VLESS，以及启用 VLESS 加密（vlessenc / ML-KEM）的 xhttp 传输上的 VLESS——此时 Flow 字段同样可设置为 xtls-rprx-vision，该值会正确写入 vless:// 链接（flow=xtls-rprx-vision）。Vision 通过在握手后直接传递有效载荷来减少「双重加密」（TLS-in-TLS），从而提升传输性能并改善流量混淆效果。因此，**VLESS + Reality + Flow xtls-rprx-vision** 被视为当前推荐的现代配置。

Vision flow 自动恢复。 如果 VLESS/XHTTP-inbound 的加密功能（ML-KEM，decryption/encryption 字段）是在已添加客户端之后才启用的，该 inbound 就具备了 flow 条件。在这种情况下，面板会自动为应当拥有 flow = xtls-rprx-vision 但 Flow 字段为空的客户端恢复该值。以往在这种场景下，Vision 会悄悄从配置文件、分享链接和订阅中消失（在节点 inbound 上尤为明显）。无需任何手动操作：修复会在保存 inbound 时自动应用，并在面板更新时执行一次。该行为是保守的——面板不会凭空添加 flow，也不会覆盖客户端显式设置的值。

- **REALITY** – 无需自有证书的流量伪装机制。服务器「借用」真实第三方网站（target / serverNames）的 TLS 握手，因此对观察者而言，连接与访问该网站无异，且完全不需要证书。认证基于 X25519 密钥

对和 `shortIds` 集合。REALITY 能抵抗主动探测（active probing）和 SNI 封锁，因为 SNI 指向的是真实的外部域名。代价是配置要求更严格（`target` 必须含端口、密钥须与客户端同步）。

简短选择建议：

- 有自有域名和有效证书，需要普通 HTTPS 外观 → **TLS**（尽可能配合 Vision）；
- 没有域名/证书，或需要对 DPI 最大程度隐蔽 → **REALITY**（配合 Vision 用于 VLESS/TCP）。

7.2. 「无」模式（`none`）

传输不带 TLS 包裹：`streamSettings` 中的 `tlsSettings` 和 `realitySettings` 块被移除。该模式无额外字段。适用场景：

- inbound 仅监听 `127.0.0.1`，作为 fallback 目标使用（按面板规则，fallback 子 inbound 应监听 `127.0.0.1` 且 `security=none`）；
- 加密/混淆由外部层提供（例如面板前的 Nginx 反向代理）；
- 在内部网络中使用具有自身加密机制的协议（Shadowsocks）。

对于向外网开放的 inbound，不建议使用「无」模式。

示例：**tcp 网络上 TLS 的 `streamSettings` 块**（VLESS/Trojan/VMess）。这是选择 **TLS** 模式并填写 SNI 和证书路径后的结果：

```
"streamSettings": {
  "network": "tcp",
  "security": "tls",
  "tlsSettings": {
    "serverName": "vpn.example.com",
    "minVersion": "1.2",
    "maxVersion": "1.3",
    "alpn": ["h2", "http/1.1"],
    "settings": { "fingerprint": "chrome" },
    "certificates": [
      {
        "certificateFile": "/root/cert/vpn.example.com.crt",
        "keyFile": "/root/cert/vpn.example.com.key",
        "ocspStapling": 3600,
        "usage": "encipherment"
      }
    ]
  }
}
```

7.3. TLS 模式

`tlsSettings` 块的字段。默认值取自面板模式。

基本参数

字段 (标签)	默认值	说明
SNI (<code>serverName</code>)	<code>""</code> (空)	Server Name Indication – TLS 握手中提供的域名。必须与证书域名一致。英文占位提示: 「Server Name Indication」。
Cipher Suites (<code>cipherSuites</code>)	<code>""</code> → 自动	允许的密码套件列表。默认为空——选择权交给 Xray/Go (自动选项)。仅在需要显式限制密码套件时更改。
最小/最大版本 (<code>minMaxVersion</code>)	min = 1.2 , max = 1.3	TLS 最低和最高版本。可选值: 1.0、1.1、1.2、1.3。建议保持 1.2 – 1.3; 不建议将最低版本降至 1.0/1.1 (已过时, 不安全)。
uTLS (<code>settings.fingerprint</code>)	chrome (表单中可选 None = <code>""</code>)	模拟的客户端 TLS 握手指纹 (uTLS fingerprint), 使握手看起来像主流浏览器。见下方列表。在 TLS 中, 列表第一项为 None (<code>""</code>), 用于禁用指纹模拟。
ALPN (<code>alpn</code>)	<code>["h2", "http/1.1"]</code>	TLS 中协商的应用层协议列表 (多选)。可选值: h3、h2、http/1.1。默认提供 h2 和 http/1.1。

uTLS fingerprint 可选值 (TLS 和 REALITY 相同): chrome、firefox、safari、ios、android、edge、360、qq、random、randomized、randomizednoalpn、unsafe。TLS 表单中额外提供空选项 **None** (不应用指纹模拟)。

Cipher Suites 可选值 (TLS 1.3 及 ECDHE 套件): TLS_AES_128_GCM_SHA256、TLS_AES_256_GCM_SHA384、TLS_CHACHA20_POLY1305_SHA256、TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA、TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA、TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA、TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA、TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256、TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384、TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256、TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384、TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256、TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256。

TLS 开关

开关	默认值	说明
拒绝未知 SNI (<code>rejectUnknownSni</code>)	关 (false)	启用后, 当客户端提供的 SNI 与证书名称不匹配时, 服务器断开连接。提高隐蔽性 (服务器不响应「陌生」请求), 但要求客户端 SNI 精确匹配。
禁用系统根证书 (<code>disableSystemRoot</code>)	关 (false)	禁止使用系统受信任根证书存储。
会话恢复 (<code>enableSessionResumption</code>)	关 (false)	启用 TLS 会话恢复 (session resumption / session tickets)。

TLS 高级参数 (vcn、曲线、密钥日志、ECH Sockopt)

TLS 基本设置下方提供了附加字段。

字段（标签）	默认值	说明
Verify Peer Cert By Name (settings.verifyPeerCertByName)	""	客户端用于验证服务器证书的名称（逗号分隔），代替 SNI 进行验证。这是 Xray 在 2026-06-01 后移除的 allowInsecure 字段的现代替代方案。该值仅用于面板端：不写入服务器 xray 配置，但会传入分享链接和订阅（vcn=...），供客户端自行应用。占位提示：example.com。
Curve Preferences (curvePreferences)	""	TLS 密钥交换曲线的限制和优先顺序（例如 X25519MLKEM768、X25519）。为空时使用 Xray-core 默认值。
Master Key Log (masterKeyLog)	""	以 SSLKEYLOGFILE 格式记录 TLS 主密钥的文件路径（用于在 Wireshark 中解密流量以进行调试）。占位提示：/path/to/sslkeylog.txt。生产环境请留空——该文件可解密所有流量。
ECH Sockopt (echSockopt)	关	用于 Xray 请求 ECH 配置列表的连接套接字参数开关。启用后可配置： Dialer Proxy (dialerProxy - 通过指定标签的 outbound 路由该请求)、 Domain Strategy (domainStrategy)、 TCP Fast Open (tcpFastOpen)、 Multipath TCP (tcpMptcp)。无需要时保持关闭。

verifyPeerCertByName、curvePreferences、masterKeyLog 和 echSockopt 字段位于 tlsSettings 顶层，在保存配置时不会被面板字段裁剪。

证书

SSL 证书（UI 中标题为「SSL 证书」）以列表形式配置：点击 + 按钮添加新证书条目，点击 - 删除 按钮移除（删除按钮仅在条目多于一项时可用）。启用 TLS 时默认创建一条空条目。

每条条目的输入模式开关（useFile）：

- **证书路径**（useFile = true，默认）– 填写服务器上的文件路径：
 - **公钥** (certificateFile) – 证书文件路径（.crt / .pem）；
 - **私钥** (keyFile) – 私钥文件路径（.key）。
- **证书内容**（useFile = false）– 直接将内容粘贴到字段中（多行文本区域）：
 - **公钥** (certificate) – PEM 格式的证书内容；
 - **私钥** (key) – PEM 格式的密钥内容。

「证书路径」模式下字段下方提供两个按钮：

- **使用面板证书** – 将面板自身 SSL 证书的路径填入字段。对于主面板上的 inbound，使用面板证书（POST /panel/setting/all → webCertFile/webKeyFile）；对于分配给节点的 inbound，使用该节点自身的证书（GET /panel/api/nodes/webCert/{nodeId}），因为主面板的路径在节点上不存在。若未配置证书，将显示警告：「面板未配置证书。请先在设置中配置。」（面板证书本身在「设置 → 安全」部分配置）。
- **清除** – 清空两个路径字段。

每条证书条目的附加字段：

字段	默认值	说明
OCSP Stapling (ocspStapling)	0 (关)	OCSP stapling 更新间隔 (秒, 最小值 0)。新 inbound 默认关闭 (0)：可避免 xray 日志中出现不支持 OCSP 响应器的证书 (例如已停止 OCSP 的 Let's Encrypt) 产生的错误。仅对支持 stapling 的证书启用。
单次加载 (oneTimeLoading)	关 (false)	启用后, 证书在启动时从磁盘读取一次, 文件变更后不重新读取。
用途选项 (usage)	encipherment	证书用途。可选: encipherment (加密——普通服务器证书)、verify (验证)、issue (签发——服务器自行签名/颁发证书)。
Build Chain (buildChain)	关 (false)	仅在 usage = issue 时显示。构建证书链。

编辑器中没有单独的自签名证书按钮: 面板不会为 inbound 动态生成自签名证书。证书需通过路径/内容指定, 或通过「使用面板证书」按钮从面板设置获取。面板自身 SSL 证书的申请/获取 (包括文件上传和域名绑定) 在 **设置** → **安全** 部分进行; 此处没有针对 inbound 的 ACME/Let's Encrypt 端点。

ECH 与证书固定 (TLS 高级字段)

字段	默认值	说明
ECH key (echServerKeys)	""	Encrypted Client Hello 服务器密钥。
ECH config (settings.echConfigList)	""	ECH 配置列表 (客户端部分, 写入链接)。
对端证书 SHA-256 (settings.pinnedPeerCertSha256)	[]	对端证书的 SHA-256 哈希 (十六进制字符串, 逗号分隔)。字段提示原文: 「对端证书的 SHA-256 哈希, 以十六进制字符串形式 (如 e8e2d3...), 逗号分隔。仅用于面板——不写入 xray 服务器配置, 但包含在分享链接中, 供客户端固定证书。」

按钮: **对端证书 SHA-256** 字段旁提供两个自动填充按钮:

- **Fill from this inbound's certificate** (盾牌图标) — 填入该 inbound 自身证书的 SHA-256 哈希 (通过 `getCertHash` 端点在本地获取)。
- **Fetch the hash by pinging the SNI (xray tls ping)** (下载图标) — 通过对指定 SNI 发起 TLS 连接来获取服务器实时证书哈希 (服务器端调用 `getRemoteCertHash`)。**SNI** (serverName) 字段必须已填写——否则显示提示「Set the SNI (serverName) first to ping the remote certificate.」

获取到的哈希会以逗号分隔方式追加到字段中, 并写入分享链接, 供客户端固定证书。

- **获取新 ECH 证书** — 向服务器请求当前 SNI 对应的新 ECH 密钥对 (POST /panel/api/server/getNewEchCert, 服务器端执行 `xray tls ech --serverName <SNI>`) ; 填充 **ECH key** 和 **ECH config** 字段。
- **清除** — 清空两个 ECH 字段。

7.4. REALITY 模式

`realitySettings` 块的字段。REALITY 不使用 SSL 证书：取而代之的是借用的外部域名 TLS 握手和 X25519 密钥对。

伪装参数

字段 (标签)	默认值	说明
显示 (show)	关 (false)	在 Xray 日志中输出 REALITY 调试信息。通常保持关闭。
Xver (xver)	0	传递给后端的 PROXY 协议版本 (0 – 关闭)。最小值 0。
uTLS (settings.fingerprint)	chrome	模拟的 TLS 指纹 (与 TLS 中的列表相同，但无空的 None 选项)。
目标 (target)	"" (自 3.4.2 起，启用 REALITY 时保持为空)	必填字段。 REALITY 借用其 TLS 握手的真实域名。字段提示原文：「必填。必须包含端口 (例如 <code>example.com:443</code>)。不含端口时 <code>Xray-core</code> 无法启动。」面板验证会检查端口的存在性和合法性；否则显示错误「REALITY 目标为必填项」/「REALITY 目标必须包含端口...」/「REALITY 目标端口无效」。字段旁边有**「扫描」按钮 (「实时」检查当前目标) 和「查找目标」**按钮 (打开 REALITY 目标扫描器)；见下文。
SNI (serverNames)	[] (与目标一起填入)	允许的 SNI 列表 (以标签方式多值输入)。必须与 目标 中的域名对应。成功扫描目标后，SNI 会根据其证书自动填入。
最大时钟偏差 (毫秒) (maxTimediff)	0	客户端与服务器时钟最大允许偏差 (毫秒，0 表示无限制)。最小值 0。
最低客户端版本 (minClientVer)	""	最低 Xray 客户端版本 (自 3.5.0 起占位符为 26.3.27)。为空时无限制。
最高客户端版本 (maxClientVer)	""	最高 Xray 客户端版本。为空时无限制。
Short IDs (shortIds)	[] (启用时自动生成)	用于区分客户端的短标识符 (十六进制) 列表。以标签方式多值输入；刷新按钮生成随机集合。
SpiderX (settings.spiderX)	/	「蜘蛛」路径 (REALITY 客户端部分)，用于模拟访问外部网站。写入分享链接。

自 3.4.2 起，启用 REALITY 时 目标 (target) 和 SNI (serverNames) 不再自动填入——两个字段均保持为空，目标由实时扫描器挑选 (见下文)。请选择流量大、稳定、支持 TLS 1.3 和 HTTP/2 的第三方 HTTPS 网站，且该网站不应部署在您的服务器上。

REALITY 目标查找 (实时扫描器)

自 3.4.2 起，原先的「随机目标」按钮 (带静态内置列表) 被 目标 字段旁的两个操作取代：

- 「扫描」——「实时」检查当前目标：建立 TLS 连接，若目标可用，则根据其证书填入 SNI。提示：「目标可用——已填入目标和 SNI。」/「目标可达，但不适用于 REALITY。」/「无法扫描 REALITY 目标。」。

- 「查找目标」——打开**「REALITY 目标扫描器」窗口：可指定域名、IP 或 CIDR 范围（此时面板根据存活主机的证书查找目标），或留空（检测内置候选）。结果会被排序；列包括「状态」/「可用」、「密钥交换」、「证书」、「TLS」、「ALPN」、「延迟」。点击「使用」**按钮可将选中行填入 inbound 字段。

当服务器协商 TLS 1.3、HTTP/2 (ALPN h2)、X25519/X25519MLKEM768 密钥交换，并提供受信任（非通配符）证书时，目标被视为可用。扫描在面板服务器端执行（POST /panel/api/server/scanRealityTarget 和 .../scanRealityTargets）。

示例：tcp 网络上 REALITY 的 streamSettings 块（VLESS）。无需证书——以借用的域名和 X25519 密钥对代替：

```
"streamSettings": {
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "show": false,
    "xver": 0,
    "dest": "www.nvidia.com:443",
    "serverNames": ["www.nvidia.com"],
    "privateKey": "YOUR_X25519_PRIVATE_KEY",
    "shortIds": [""],
    "settings": {
      "publicKey": "YOUR_X25519_PUBLIC_KEY",
      "fingerprint": "chrome",
      "spiderX": "/"
    }
  }
}
```

此处面板中的 目标（target）字段对应 Xray 最终配置中的 dest。如果 REALITY inbound 是用旧版面板、通过 API 或外部工具以 dest 键创建的，面板在解析时会将 dest 规范化为 target（当 target 为空时）——因此该 inbound 可正确加载，目标 字段不会为空，重新保存也不会破坏正常运行的 REALITY。

REALITY 密钥 (X25519)

字段	默认值	说明
公钥 (settings.publicKey)	""	X25519 公钥（客户端将其写入自身配置/链接）。
私钥 (privateKey)	""	X25519 私钥（仅存于服务器端）。

密钥下方的按钮：

- 获取新证书 — 向服务器请求新密钥对（GET /panel/api/server/getNewX25519Cert；服务器端执行 xray x25519），填充 私钥 和 公钥。首次启用 REALITY 模式时也会自动生成该密钥对。

示例：通过 API 获取 X25519 密钥对（在表单之外，例如用于脚本）。请求返回私钥和公钥：

```
curl -s -b cookie.txt https://your-panel:2053/panel/api/server/getNewX25519Cert
# [?][?][?]
# {"success":true,"obj":{"privateKey":"...","publicKey":"..."}}
```

cookie.txt — 通过 POST /login 登录后获取的会话 cookie 文件。

- **清除** — 清空两个密钥字段。

后量子签名 ML-DSA-65 (mldsa65)

REALITY 的附加（可选）后量子认证层：

字段	默认值	说明
mldsa65 Seed (mldsa65Seed)	""	ML-DSA-65 密钥的服务器端 seed。
mldsa65 Verify (settings.mldsa65Verify)	""	验证值（客户端部分，写入链接）。

按钮：

- **获取新 Seed** — 请求新密钥对（GET /panel/api/server/getNewmldsa65；服务器端执行 xray mldsa65），填充 **mldsa65 Seed** 和 **mldsa65 Verify**。
- **清除** — 清空两个字段。

REALITY fallback 限速与密钥日志

REALITY 设置中提供了 fallback 流量限速功能——防止主动探测者将服务器作为免费通道访问借用的域名。该设置分为两个方向分别配置——**Limit Fallback Upload** 和 **Limit Fallback Download**（limitFallbackUpload / limitFallbackDownload），每个方向的字段相同：

字段（标签）	默认值	说明
After Bytes (afterBytes)	0	开始限速前以全速传输的字节数。0 表示从第一个字节起即限速。
Bytes Per Sec (bytesPerSec)	0	超过阈值后 fallback 流量的速度上限（字节/秒）。0 表示无限制（禁用该方向）。
Burst Bytes Per Sec (burstBytesPerSec)	0	超出固定速率的短时突发余量（令牌桶大小）。若小于 Bytes Per Sec，则自动提升至该值。

同一位置还提供 **Master Key Log**（masterKeyLog）字段——用于以 SSLKEYLOGFILE 格式记录 TLS 主密钥的文件路径，供 Wireshark 调试使用；生产环境请留空。

7.5. 配置实践建议

1. **VLESS + Reality（推荐）**：在 tcp 网络上创建 VLESS inbound，在「安全」选项卡中选择 **Reality**——面板会自动填入随机 target /SNI、shortIds 并生成 X25519 密钥。如需使用自己的密钥对，点击「获取新证书」。为 VLESS 客户端启用 **Flow = xtls-rprx-vision**（XTLS Vision）——可获得最佳性能与隐蔽性。

示例：**VLESS + Reality + Vision** 的客户端链接。面板为该 inbound 生成的分享链接如下所示（密钥/ID 仅为示意）：

```
vless://uuid-клиента@1.2.3.4:443?  
type=tcp&security=reality&pbk=ПУБЛИЧНЫЙ_КЛЮЧ&fp=chrome&sni=www.nvidia.com&sid=6ba85179e3  
0d4fc2&spx=%2F&flow=xtls-rprx-vision#my-reality
```

其中 pbk 为 X25519 公钥，sni 为目标中借用的域名，sid 为 **Short IDs** 之一，flow=xtls-rprx-vision 表示已启用 XTLS Vision。2. 使用自有域名的 TLS：选择 **TLS**，在 **SNI** 中填入域名，添加证书（通过文件路径或内容），或点击「使用面板证书」（前提是域名和证书已在「设置 → 安全」中配置）。将 **最小/最大版本** 保持为 1.2 - 1.3，**uTLS** 设为 chrome 以模拟普通浏览器流量。3. 不要对向外网开放的 inbound 使用 **无** 模式——该模式仅适用于本地 fallback 目标（127.0.0.1）或由外部代理提供 TLS 的场景。4. 界面建议：大多数高级字段都有提示「建议保留默认设置」——请仅在充分了解影响的情况下更改。

8. 客户端

客户端是用户的 VPN 账户：一组凭据（UUID 或密码），绑定到一个或多个 inbound，拥有独立的流量配额、有效期和同时连接数限制。在此分支中，客户端是独立的实体（`clients` 表）：同一个客户端可以同时绑定到多个 inbound，共享相同的 UUID/密码和共同的流量计数器。**客户端**页面显示面板中所有账户，不受 inbound 限制，支持搜索、筛选、排序和批量操作。

8.1. 客户端字段

以下逐一介绍**添加客户端 / 编辑客户端**编辑器中的各个字段。

客户端表单分为两个选项卡：**基本**（email、inbound 绑定、限制、有效期、分组、备注、反向标签）和**凭据**（UUID/密码/auth、Flow、VMess Security）。字段标签中配额显示为**流量限制 (GB)**，有效期显示为**时长 (天)**和**自动续期 (天)**；**流量限制 (GB)**和**IP 限制**字段均有提示说明，`0` 表示“无限制”。编辑已有客户端时，生成随机 email 的按钮会隐藏，IP 日志按钮则直接显示在**IP 限制**字段旁，并显示已记录的地址数量。

字段	JSON 键	默认值	说明
Email	<code>email</code>	—（必填）	客户端的唯一标识符
UUID	<code>id</code>	自动生成	VMess/VLESS 协议标识符
密码	<code>password</code>	自动生成	Trojan/Shadowsocks 密码
授权	<code>auth</code>	自动生成	Hysteria 密码
Flow	<code>flow</code>	空	Flow control (XTLS)，仅适用于 VLESS
VMess Security	<code>security</code>	<code>auto</code>	VMess 加密方式
IP 限制	<code>limitIp</code>	<code>0</code> （无限制）	同时连接的最大 IP 数
总上传/下载 (GB)	<code>totalGB</code>	<code>0</code> （无限制）	流量配额
到期时间	<code>expiryTime</code>	<code>0</code> （永不过期）	账户到期日期
自动续期	<code>reset</code>	<code>0</code> （关闭）	流量重置周期，单位：天
Telegram 用户 ID	<code>tgId</code>	<code>0</code> （无）	数字型 Telegram ID
订阅 ID	<code>subId</code>	自动生成	订阅链接标识符
分组	<code>group</code>	空	逻辑分组标签
备注	<code>comment</code>	空	任意说明文字
已启用	<code>enable</code>	<code>true</code>	账户是否激活

Email（标识符）

Email 字段是客户端的主要必填标识符。尽管名称如此，它不必是电子邮件地址：任何文本标签（用户名、编号）均可。该值在面板内必须**唯一**；尝试创建具有相同 email 的第二个客户端会被拒绝（`email already in use`），除非 `subId` 也相同（此时视为绑定同一客户端）。

Email **不能为空**（client email is required），且**不能包含空格、/、\ 或控制字符**（"Email 不能包含空格、/、\" 或控制字符"）。Email 用于流量统计、IP 日志、在线列表以及操作名称，因此不建议事后修改。

UUID / 密码 / 授权（凭据）

具体的凭据字段取决于客户端所绑定的 inbound 协议。若字段留空，系统将自动填充：

- **UUID**（字段 `id`）——用于 **VMess** 和 **VLESS** 协议。若未指定，则生成随机 UUID v4。
- **密码**（字段 `password`）——用于 **Trojan** 和 **Shadowsocks**。Trojan 默认生成不含连字符的 UUID。Shadowsocks 根据 inbound 的加密方式生成相应长度的 Base64 密钥：`2022-blake3-aes-128-gcm` 为 16 字节，`2022-blake3-aes-256-gcm` 和 `2022-blake3-chacha20-poly1305` 为 32 字节；其他方式则生成不含连字符的 UUID。若手动输入的密钥不符合 2022-blake3 方式的要求，将被替换为自动生成的密钥。
- **授权**（字段 `auth`）——**Hysteria** 的密码，默认为不含连字符的 UUID。

由于同一个客户端可以绑定到不同协议的 inbound，客户端记录中可以同时包含 UUID、密码和 auth——每个协议使用各自对应的字段。

示例：客户端凭据在不同 inbound 的 settings 中的呈现方式。 同一个客户端在 VLESS inbound 中通过 `id` 标识，在 Trojan 中通过 `password`，在 Shadowsocks 中通过 `password`（Base64 密钥）：

```
// VLESS inbound settings.clients
{ "id": "b831381d-6324-4d53-ad4f-8cda48b30811", "email": "user-a", "flow": "xtls-rprx-vision" }

// Trojan inbound
{ "password": "b831381d63244d53ad4f8cda48b30811", "email": "user-a" }

// Shadowsocks inbound 2022-blake3-aes-256-gcm
{ "password": "GPY0aA3f7C00az53eaQ8eqMfRDjmBl0h7v1u3+Z+pHk=", "email": "user-a" }
```

Flow

Flow（字段 `flow`）——XTLS 流量控制。**仅适用于 VLESS**，且仅当 inbound 配置为 XTLS Vision 时有效：即 VLESS 使用 **TCP** 传输并将 `security` 设置为 **tls** 或 **reality**。有效值为 `xtls-rprx-vision`（以及历史值 `xtls-rprx-vision-udp443`）；空值表示不使用 flow。

若 inbound 不支持 XTLS flow，则在保存客户端时，已设置的 flow 将被**静默清除**：对于同一个绑定到多个 inbound 的客户端，flow 仅在支持的 inbound 上生效。仅在明确使用 VLESS-Vision 时才需修改此项。

VMess Security

VMess Security（字段 `security`）——VMess 的载荷加密方式。默认值为 `auto`（由 Xray 自动选择加密算法）。有效值为 VMess 的标准选项：`auto`、`aes-128-gcm`、`chacha20-poly1305`、`none`、`zero`。其他协议不使用此字段。

IP 限制（同时连接数）

IP 限制（字段 `limitIp`）——客户端可同时连接的不同 **IP 地址** 的最大数量。默认值为 `0`，表示**无限制**。若设置为正数，面板将跟踪客户端的活跃 IP，超出限制时由后台任务禁用该账户。（从 **3.3.1** 起，IP 统计通过 Xray 内核的 `online-stats` API 进行，**无需**访问日志；对于旧版内核，面板回退到读取访问日志，此时访问日志必须启用。）可用此功能防止同一订阅被多台设备共用：例如 `2` 表示允许两台设备。

IP 限制通过 **Fail2ban** 实现，因此 **IP 限制** 字段仅在 Fail2ban 已安装并正常运行时有效（面板通过 `GET /panel/api/server/fail2banStatus` 检查其状态）。若未安装 Fail2ban，客户端编辑表单（及批量添加表单）中的此字段将被禁用，鼠标悬停时会显示提示，建议从 `x-ui bash` 菜单安装 Fail2ban（“Fail2ban is not installed, so the IP limit cannot be enforced. Install Fail2ban from the x-ui bash menu to enable this option.”）；Windows 系统的提示则说明 Fail2ban 在该平台不可用（“Fail2ban is not available on Windows, so the IP limit cannot be enforced.”），若服务器上该功能已禁用，则显示 “The IP limit feature is disabled on this server.”。更新面板时，若服务器上未安装 Fail2ban，客户端已保存的 IP 限制值将通过一次性迁移被清零，因为该值本来也无法生效。

数值示例： `limitIp: 0` 表示无限制；`limitIp: 1` 表示同时只允许一台设备；`limitIp: 3` 表示最多允许三个不同 IP。当第四个活跃 IP 出现时，后台任务将禁用客户端（`enable = false`），直到执行**重置 IP 限制**。

相关操作： **IP 日志** 显示客户端已记录的 IP 列表；每条记录除 IP 本身外，还包含最后访问时间以及节点标签（@ `???`），标明是通过哪个节点记录的连接——在多面板配置中，可以看到客户端是通过哪个节点连接的。**重置 IP 限制** 可清除已积累的 IP 日志，使客户端无需等待记录自然过期即可重新连接。

总上传/下载 (GB)——流量配额

总上传/下载 (GB)（字段 `totalGB`）——总流量配额（上传 + 下载）。默认值 `0` 表示**无限制**。达到配额（`up + down >= total`）后，客户端被视为**已耗尽**（depleted）并被禁用。在界面中通常以 GB 为单位输入；在数据库中以字节存储。

在客户端列表中，**流量** 列以彩色进度条显示使用情况：已用流量、限制标签（无限制时显示 ∞ ），以及鼠标悬停时的详细提示（包含上传/下载分项和剩余量）。在手机上，客户端卡片中也会显示相同的简洁指示器。

到期时间 (Expiry)

到期时间（字段 `expiryTime`）设置账户的到期时刻。该字段有三种模式：

- **永不过期** —— `0`。客户端永不因时间到期。
- **指定日期** —— 正数 Unix 时间戳（毫秒）。到期后（`expiryTime <= ???`），客户端被视为已过期（expired）并被禁用。在界面中通常通过日期选择器或天数（**时长**，单位**天**）设置。
- **首次使用后开始计时** —— 负数值，表示时长编码。客户端未传输任何流量时，到期时间保持为负数（“延迟启动”）。当首次有流量统计时，面板将其转换为绝对日期：`???` + `|??|`。这样可以出售例如“首次连接后 30 天”的套餐，而无需预先知道客户端何时激活。转换针对每个 email 只执行一次，确保所有绑定的 inbound 获得相同的到期时间。

到期时间编码示例： 固定日期 2026 年 3 月 1 日 00:00 UTC → `expiryTime: 1772323200000`（毫秒正时间戳）。"首次连接后 30 天" → `expiryTime: -2592000000`（负值， $30 \times 24 \times 60 \times 60 \times 1000$ ）；首次有流量时面板将其替换为 `???? + 2592000000`。永不过期 → `expiryTime: 0`。

自动续期（客户端流量重置周期）

自动续期字段（字段 `reset`）——以天为单位的自动续期/重置周期。提示："到期后自动续期。（0 = 关闭）（单位：天）"。

- `0` —— 自动续期关闭（默认值）。到期后客户端仅变为已耗尽状态。
- `> 0` —— 后台任务在到期时将流量计数器重置为零（`up = down = 0`），将到期时间向前推移 `reset` 天（如有需要，推移多个周期，直到新的到期时间在未来），并在必要时重新启用客户端。这实现了周期性订阅（例如按月）。自动续期不适用于节点上的 `inbound`（`node_id IS NOT NULL`）。

重要注意事项： `reset > 0` 的客户端不会被标记为"已耗尽"，因此在批量删除操作中会被排除——其流量/到期时间预期由自动续期重置，而非将账户标记为待删除候选。

Telegram 用户 ID

Telegram 用户 ID（字段 `tgId`）——用于绑定到面板内置 Telegram 机器人的数字型 Telegram 用户标识符（用于通知和自助查看统计）。提示："数字型 Telegram 用户 ID（0 = 无）"。默认值 `0` 表示未绑定。可按此字段筛选（有 / 无）。

订阅 ID（subId）

订阅 ID（字段 `subId`）——客户端被包含在订阅（subscription）中的标识符。所有具有相同 `subId` 的客户端通过同一个订阅链接提供。若创建时留空，面板将自动生成随机 `subId`（UUID）。该值在不同 email 的客户端之间必须唯一（`subId already in use`），并遵循与 email 相同的字符限制（"订阅 ID 不能包含空格、'/'、"或控制字符"）。

若无 `subId`，客户端的订阅链接将不可用（"该客户端没有 `subId`，共享链接不可用。"）。

Links 选项卡（外部链接和订阅）

除**基本**和**凭据**选项卡外，客户端编辑器还有第三个选项卡 **Links**（提示："Add third-party share links and remote subscription URLs to include in this client's subscription."）。在此选项卡中，可通过 **Add External Link** 按钮添加第三方分享链接（`vless://`、`vmess://`、`trojan://`、`ss://`、`hysteria2://`、`wireguard://`），通过 **Add External Subscription** 按钮添加远程订阅 URL（例如 `https://provider.example/sub/...`）。

所有这些内容都会混入该客户端的订阅输出（raw、JSON 和 Clash 格式）：链接原样添加，而远程订阅则由面板定期抓取（带缓存和短超时）并与自有配置合并。这样，在客户端的单个订阅链接中，可以同时提供自有服务器和外部配置。

分组

分组（字段 `group`）——用于将相关客户端归类的逻辑标签。提示：“用于将相关客户端分组的逻辑标签（例如团队、客户、地区）。可从工具栏筛选。”，占位符为“例如 customer-a”。字段可选（默认为空）。可按分组筛选列表并执行批量操作；使用**取消分组**操作可清除客户端的分组标签。

也可以直接在单个客户端的编辑器中取消分组：若清空**分组**字段并保存，标签将被正确清除，该客户端将不再显示在原分组下。

备注

备注（字段 `comment`）——管理员用的任意文本说明（默认为空）。内容参与搜索，并可按是否有备注进行筛选（有/无）。

已启用

已启用（字段 `enable`）——账户激活标志。默认为**启用**（`true`）；创建时即使未传递此标志，面板也会强制设置为 `true`。已禁用的客户端（`enable = false`）无法连接，在统计概览中归类为**非活跃**（`deactive`）。面板会自动禁用已耗尽配额、已过期或超出 IP 限制的客户端。

只读字段

客户端卡片中还显示服务性字段：**创建时间**（`created_at`）和**更新时间**（`updated_at`）——创建和最后修改的时间戳，自动填充，不可编辑。**反向标签**字段（`reverse`）——用于简单 VLESS 反向代理的可选 Reverse tag（“可选 Reverse 标签”）。

8.2. 绑定 inbound

每个客户端必须至少绑定到一个 inbound——创建时至少需要绑定一个（`at least one inbound is required`）。在编辑器中，此字段为**绑定入站**，提示为**选择一个或多个入站**。

- **绑定**——将客户端添加到所选 inbound（共享相同的 UUID/密码和流量统计）。已有绑定保持不变。
- **解绑**——从所选 inbound 中移除客户端。客户端记录本身保留（完全删除请使用**删除**）。未绑定的配对将被静默跳过。

保存绑定到多个 inbound 的客户端时，与特定协议/传输不兼容的字段（例如 VLESS-Vision 之外的 Flow）将自动调整为每个 inbound 的合法值。

在 inbound 选择列表上方（客户端表单、批量添加客户端以及批量绑定/解绑窗口中）有**全选**和**清除**按钮。这些列表中每个 inbound 使用其备注（`remark`）作为标签（若已设置），否则使用 inbound 的 `tag`。

8.3. 客户端操作

针对单个客户端（通过**客户端信息**卡片或**操作**上下文菜单）可执行以下操作：

查看信息、二维码和链接

- **客户端信息** —— 包含所有字段、已用/剩余流量（**剩余**）、有效期和已绑定 inbound 的卡片。自 3.5.0 起，当客户端设有分组时，卡片还会显示其**分组**（带标签的「分组」行，位于「备注」之上）。

通过 API 查询客户端（GET /panel/api/clients/get/:email）时，在 client 和 inboundIds 字段之外还会额外返回 usedTraffic —— 实际已消耗的流量（上传 + 下载，含节点数据），便于将消耗量与 totalGB 配额进行对比。

- **二维码和链接** —— 用于导入客户端应用的客户端配置链接。根据所有绑定的支持协议 inbound 生成（GET /links/:email）。若没有合适的链接：“没有可共享的链接——请先将客户端绑定到使用支持协议的入站。”
- **订阅链接** —— 基于 subId 的订阅 URL（GET /subLinks/:subId）。仅在客户端有 subId 且订阅服务在**面板设置** → **订阅**中已启用时可用（否则显示“订阅服务已禁用。”）。同时提供 **JSON 订阅 URL**。

重置流量

重置流量（POST /resetTraffic/:email）将特定客户端的上传/下载计数器（up、down）清零。配额（totalGB）和有效期**不受影响**——仅重置已用流量。提示：“流量已重置”。若客户端未绑定任何 inbound：“请先将此客户端绑定到入站。”

重置流量按钮也可在客户端编辑表单底部访问——位于**取消 / 保存**旁边（重置前会请求确认）。若客户端因流量耗尽而被禁用，单次或批量重置将自动重新启用该客户端（enable = true），并立即将此变更推送到节点——无需再在主面板和各节点上手动重新启用客户端。

重置 IP 限制

清除客户端已积累的 IP 日志（POST /clearIps/:email），以解除因超出同时连接数限制导致的临时封锁。提示：“日志已清除”。

删除

删除（POST /del/:email）——完全删除客户端。确认对话框标题：“删除客户端 {email}？”，内容：“客户端将从所有已绑定的入站中移除，其流量记录将被销毁。此操作无法撤销。”。删除将从**所有** inbound 中移除客户端并销毁其流量记录。提示：“客户端已删除”。

8.4. 批量操作

在客户端列表中可勾选多条记录（**全选、清除全部**）；计数器显示“{count} 已选择”。对所选项可执行以下操作：

- **删除 ({count})**（POST /bulkDel）——批量删除。确认信息：“删除 {count} 个客户端？”，“每个选定的客户端将从所有已绑定的入站中移除，其流量记录将被销毁。此操作无法撤销。”。提示：“已删除客户端：{count}”，若部分失败则显示“已删除：{ok}，失败：{failed}”。

- **编辑 ({count}) / 调整 (POST /bulkAdjust)** —— 批量修改有效期和/或配额。对话框“编辑 {count} 个客户端”，提示“正值为增加，负值为减少。有效期或流量无限制的客户端在对应字段将被跳过。”。字段：**增加天数**、**增加流量 (GB)** 和 **Set flow**。逻辑：
 - **有效期**：有效期永久 (`expiryTime == 0`) 的客户端将被跳过 (“unlimited expiry”)；有截止日期的客户端，有效期向后移动指定天数；处于“首次使用后开始计时”模式 (负值有效期) 的客户端，等待时长将被调整。若减少量超过剩余时间则跳过 (“reduction exceeds remaining time/delay window”)。
 - **流量**：无限制 (`totalGB == 0`) 的客户端将被跳过 (“unlimited traffic”)；否则配额按指定量调整，不低于零。
 - **Flow**：**Set flow** 下拉列表允许同时为所有选定客户端设置或清除 XTLS flow。默认选项为 **No change** (不修改)。**Disable (clear flow)** 选项用于清除 flow，`xtls-rprx-vision` 和 `xtls-rprx-vision-udp443` 则设置对应的 vision-flow。设置 vision-flow 仅对支持 flow 的 inbound 生效；不支持的 inbound 保持不变并标记为跳过，而清除 flow 始终允许。
 - **自动启用 (自 3.4.2 起)**：仅因耗尽 (有效期到期或配额超限) 而被禁用的客户端，在续期后会自动重新启用——本地及其节点上同步——前提是调整使其重新回到限制范围内。手动禁用或仍处于耗尽状态的客户端保持禁用 (`enable` 字段现在会被显式写入，因此即使是没有 inbound 的客户端也能保持其状态)。
 - 若未指定天数、流量或 flow：“请在应用前指定天数、流量或 flow。”。提示：“已修改：{count}”/“已修改：{ok}，已跳过：{skipped}”。

示例：将所选客户端延长 30 天并增加 50 GB。 在编辑对话框中，将增加天数设为 30，增加流量 (GB) 设为 50。反之，若要减少一周并削减 10 GB 配额，则输入负值：增加天数 = -7，增加流量 (GB) = -10 (有效期或流量无限制的客户端在对应字段将被跳过)。

- **绑定 ({count}) / 解绑 ({count}) (POST /bulkAttach / bulkDetach)** —— 批量将所选客户端绑定/解绑到所选 inbound。目标仅限多用户 inbound。解绑结果：“已解绑 {detached}，已跳过 {skipped}。”自 3.5.0 起，客户端表单中的 inbound 选择器会隐藏已禁用的 inbound (客户端已绑定的除外)，“解绑”窗口中同一客户端条目重复累积的问题也已修复 (数据会在面板启动时自动修复)。
- **订阅链接 ({count})** —— 所选客户端订阅 URL 和 JSON 订阅 URL 的汇总表，带全部复制按钮。若无任何客户端有 subId：“所选客户端均无订阅 ID。”
- **加入分组和取消分组** —— 分配和取消分组标签。
- **启用 ({count}) / 禁用 ({count}) (POST /bulkEnable / bulkDisable)** —— 批量启用和禁用所选客户端。启用将激活每个所选客户端在所有已绑定 inbound 上的访问；流量配额已耗尽或已过期的客户端将被自动再次禁用。禁用立即撤销客户端的访问权限，但其记录和已积累的流量保持不变。执行前面板请求确认，操作完成后显示包含已处理客户端数量的通知，若有失败情况也会一并显示。

按状态重置流量和删除

- **重置所有客户端流量 (POST /resetAllTraffics)** —— 将面板所有客户端的 up / down 计数器清零。确认信息：“重置所有客户端的流量？”和“所有客户端的上传/下载计数器将重置为零。配额和有效期不受影响。此操作无法撤销。”。提示：“所有客户端流量已重置”。
- **删除已耗尽的客户端 (POST /delDepleted)** —— 删除所有配额已耗尽 (`total > 0 and up + down >= total`) 或已过期 (`expiry_time > 0 and expiry_time <= [?][?][?]`)

?) 且 `reset = 0` 的客户端（设有自动续期的客户端不受影响）。确认信息：“删除已耗尽的客户端？”，“所有流量配额已耗尽或已过期的客户端将被删除。此操作无法撤销。”。提示：“已删除耗尽客户端：{count}”。

导出、导入和删除未绑定客户端

当未选择任何记录时，客户端页面上的更多菜单中提供三个操作。

导出客户端（GET /clients/export）打开一个预览器，显示所有客户端的 JSON 列表，格式为 {client, inboundIds}，带有复制和下载按钮（文件名 clients-export.json）。**导入客户端**（POST /clients/import）打开一个编辑器，粘入相同格式的 JSON 后点击 **Import**：带 inboundIds 的客户端将被创建并绑定到 inbound，没有绑定的客户端将作为独立的“裸记录”恢复，而已存在的 email 永远不会被覆盖——它们会出现在跳过列表中。提示：“已导入 {count} 个客户端”，“已导入：{ok}，已跳过：{failed}”。

删除未绑定客户端（POST /clients/delOrphans）——危险操作：删除所有未绑定到任何 inbound 的客户端，连同其流量记录、IP 日志和外部链接。确认信息：“Delete clients without an inbound?”，“Removes every client that is not attached to any inbound, along with its traffic record. This cannot be undone.”。提示：“已删除 {count} 个未绑定客户端”。此操作不可撤销。

8.5. 搜索、筛选和排序

列表上方有搜索框（“搜索 email、备注、订阅 ID、UUID、密码、auth、Telegram ID...”）——可按 email、备注、subId、UUID、密码、auth 以及（自 3.5.0 起恢复支持）**Telegram ID** 进行搜索。结果计数：“显示 {shown} / {total}”。

客户端列表自动更新：面板每隔几秒抓取当前页面的最新数据，因此新连接的客户端和变化的排序顺序会自动出现，无需手动刷新（后台轮询时不显示加载指示器）。

自 3.5.0 起，表格中新增“速度”列（位于“流量”和“剩余”之间）：客户端的实时速度——蓝色标签 ↑ 100 / ↓ 100（约 5 秒滑动平均值，每 5 秒更新一次）；无流量时显示灰色短横线。在移动端，速度以单独一行显示在客户端卡片中。

客户端筛选面板支持按状态（分类）、协议、绑定 inbound、有效期范围、已用流量范围、是否有自动续期（有/无）、是否有 Telegram ID 和备注，以及分组进行筛选。在有节点的面板上，会出现节点多选框：可将列表限制为所选节点的客户端；单独的**本地面板**选项用于筛选未绑定节点的 inbound 客户端（仅在有点时显示此筛选项）。排序方式：**最早/最新、最近更新、最近在线、Email A→Z / Z→A、流量更多、剩余更多、即将到期**。

8.6. 图标和状态

状态优先级：已耗尽/已过期 → 非活跃 → 即将到期 → 活跃。

- **在线 / 离线** —— 有活跃连接（出现在当前在线列表中）且已启用的客户端。在线列表通过单独请求更新（/onlines、/onlinesByGuid）。
- **已耗尽** (depleted) —— 配额已用完（`up + down >= totalGB`）或有效期已到（`expiryTime <= 111111`）。此类客户端自动被禁用，并受删除已耗尽客户端操作影响。

- **即将到期/耗尽** (expiring) —— 已启用的客户端，距有效期到期时间低于阈值或剩余配额低于阈值（阈值在面板设置中配置）。已耗尽/已禁用的客户端不计入此状态。
 - **非活跃** (deactive) —— `enable = false` 的客户端（手动禁用或由后台任务禁用）。
 - **活跃** (active) —— 已启用、未耗尽、未过期，且距各阈值尚有余量。
-

9. 客户端分组

这是本 3X-UI 分支的一项功能。在原版 3x-ui (MHSanaei) 项目中并没有「客户端分组」这一概念——这里新增了独立的分组表、面板菜单中的**分组**页面以及相应的 API 方法。如果你将配置迁移到原版 3x-ui，分组标签将不会在任何地方被处理。

9.1. 什么是客户端分组以及为什么需要它

分组是一个具名的逻辑标签 (label)，可以贴到一个或多个客户端上。它不会创建新的连接方式，也既不是 inbound 也不是节点——它纯粹是一个组织性的标记，便于按它来筛选客户端并进行批量处理。

本分支客户端模型的核心思想是：**客户端是一个顶层实体，通过 email 标识**（clients 表中的 email 字段拥有唯一索引）。同一个客户端（拥有相同凭据的同一个 email）可以同时属于多个 inbound，甚至同时存在于多个节点上，包括使用不同协议。分组标签**每个客户端只存储一次**，因此它会自动应用到该客户端在所有 inbound 中的全部绑定上。

分组标签是用于分组的逻辑标记：

层级	存储位置	字段
客户端记录（数据库）	clients 表	group_name（默认为空字符串 ''）
分组字典（数据库）	client_groups 表	name（唯一索引，非空）
inbound 设置 (Xray)	JSON settings.clients[].group	在该客户端所属的每个 inbound 的每个客户端对象中重复存放

实际中为什么需要它：

- **一个客户端跨多个 inbound/节点。** 如果一个客户端被「售卖」为同时访问多个 inbound（例如不同协议或不同节点），分组就能把它作为一个整体来管理：重置流量、删除、重命名标签——通过一次操作就能作用于它的所有 inbound。
- **批量操作与筛选。** 在客户端页面上，列表可以按分组筛选；在分组页面上，可对分组的全部成员执行批量操作。
- **管理大量客户端。** 像 vip、trial、team-A 这样的标签有助于把成千上万的客户端归入逻辑上的「篮子」，而无需为此增设单独的 inbound。

9.2. 分组与客户端、inbound、节点和协议的关系

这是理解上最重要的一个小节，因为标签的同步并不简单。

分组描述的是客户端，而不是 inbound。 标签存在于客户端记录中（clients.group_name）。当客户端被绑定到多个 inbound 时，任何一次分组变更，面板都会遍历该客户端所属的所有 inbound，并在它们的 Xray 设置（settings.clients[]）中设置/移除 group 字段。技术上的实现方式是：通过客户端的 email 找到

它所属的所有 inbound，然后在每个这样的 inbound 的 JSON 设置中修改对应该 email 的客户端对象。因此：

- **分组与协议无关。** 同一个 email 可以在一个 inbound 中是 VLESS 客户端，在另一个 inbound 中是 Hysteria 客户端——它的分组标签仍然只有一个，并会应用到两者上（同时每个协议各自的凭据是独立的，分别保存）。
- **分组覆盖节点。** 属于节点的 inbound 与主面板的 inbound 区别在于 `nodeId` 字段（主面板 inbound 的该字段为 `null/0`）。分组标签会应用到 inbound 中的客户端对象，无论它是主面板 inbound 还是节点 inbound——只要其中存在使用该 email 的客户端即可。

分组标签对来自节点的同步以及对设置重建都具有稳定性。这一行为是特意设计的：

- 当节点发送流量快照时，它的数据**不会覆盖**面板数据库中客户端的本地 `group_name` 和 `comment`。分组和备注被视为「面板本地」字段——节点不管理它们。
- 在重建 inbound 设置时，传入数据中 `group` 的空值**不会重置**已保存的标签。分组正是通过分组页面来管理（而不是通过编辑 inbound 的 Xray 设置），因此在普通的设置重建中，「空分组」被解释为「不动它」，而不是「清空」。

实践结论：唯一会有意清空标签的操作是删除分组以及将客户端显式移出分组（见下文）。普通的 inbound 编辑或与节点的后台同步都不会丢失分组。

9.3. 分组字典与「空」分组

页面上的分组列表由两个来源合并而成：

1. **派生分组（derived）** ——客户端身上实际出现的所有非空 `group_name` 值，并附带客户端数量统计。
2. **已保存分组（stored）** ——来自 `client_groups` 表的记录。

这种合并带来一个重要效果：分组可以在**没有任何客户端的情况下**存在。这样的分组通过显式的「添加分组」按钮创建（在 `client_groups` 中写入记录），并在列表中以计数 `0` 显示。这些记录被视为**空分组**。列表始终按名称不区分大小写排序。

页面上的汇总计数：

字段 (RU)	显示内容
Всего групп	分组总数（已保存和派生的合计）。
Клиенты с группой	有非空分组标签的客户端数量。
Пустые группы	没有客户端的分组数量（计数为 <code>0</code> ）。
Клиентов в группе	某个具体分组中的客户端数量（表格列）。

9.4. 分组的字段与列

`client_groups` 表中的一条分组记录包含：

字段	类型	默认值	描述
Id	int	自增	分组记录的主键。
Name	string	–（必填）	分组名称。唯一索引，不能为空。在 UI 中为名称列。
CreatedAt	int64（毫秒）	创建时间	分组记录的创建时刻。
UpdatedAt	int64（毫秒）	修改时间	最后一次修改的时刻。

页面表格中至少显示名称和分组内客户端数两列，以及操作按钮（见下文）。

9.5. 创建分组

按钮添加分组。

步骤：

1. 点击**添加分组**。
2. 输入分组名称。
3. 确认。

后端行为（POST /panel/api/clients/groups/create，请求体 {"name": "..."}）：

- 名称会去除两侧空白。空名称会被拒绝并报错「group name is required」。
- 如果已存在同名分组——报错「group already exists」。
- 成功时在 client_groups 中创建一条记录（最初没有客户端——即空分组）。

成功消息：「分组「{name}」已创建。」

示例：通过 API 创建空分组。在客户端填充之前先准备好一套标签：

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/create' \
  -H 'Content-Type: application/json' \
  -b cookie.txt \
  -d '{"name": "vip"}'
```

成功时的响应：

```
{ "success": true, "msg": "Группа «vip» создана.", "obj": null }
```

用同一名称再次调用将返回 "success": false 和消息 group already exists。

提前创建空分组很方便，当你想先准备好一套标签，然后通过「添加客户端...」往里填充客户端时尤其如此。

9.6. 重命名分组

按钮**重命名**，对话框标题为**「重命名 {name}」**。

行为（POST /panel/api/clients/groups/rename，请求体 {"oldName": "...", "newName": "..."}）：

- 两个名称都会去除两侧空白。旧名称为空——报错「old group name is required」，新名称为空——报错「new group name is required」。
- 如果新名称与旧名称相同——不做任何操作（影响 0 个客户端）。
- 否则原子地执行重命名：
 - 重命名 client_groups 中的记录；
 - 所有 group_name = oldName 的客户端，其字段更新为 newName；
 - 在受影响客户端所属的所有 inbound（包括节点上的）中，Xray 设置里的 group 值由旧值改为新值。
- 重命名后，面板将 Xray 标记为需要重启，并发出客户端变更通知。

消息：

- 成功：「已为 {count} 个客户端重命名分组。」
- UI 中的名称冲突：「名为「{name}」的分组已存在。」

9.7. 向分组添加客户端

按钮添加客户端...，标题为**「向分组「{name}」添加客户端」**。

对话框中的逐字提示：

「选择要添加到此分组的客户端。现有的 inbound 绑定会保留；只更改分组标签。已属于此分组的客户端不会显示。」

如果没有可添加的对象，会显示**「没有其他可添加的客户端。」**

行为（POST /panel/api/clients/groups/bulkAdd，请求体 {"emails": [...], "group": "..."}）：

- 分组名称必填（否则报错「group name is required」）；空的 email 列表——操作不做任何事。
- 如果该分组在 client_groups 和派生分组中都尚不存在——它将被自动创建。
- 对所选 email 的客户端设置 group_name = group；客户端与 inbound 的绑定不变——只影响标签。然后在这些客户端的所有 inbound 中设置 group 字段。
- 返回受影响的客户端记录数量；Xray 被标记为需要重启。

成功消息：「已向 {name} 添加 {count} 个客户端。」

示例：用一个请求给多个客户端打上分组标签。客户端通过 email 指定，与 inbound 的绑定不会因此改变：

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/bulkAdd' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"emails": ["alice@example.com", "bob@example.com"], "group": "vip"}'
```

如果分组 `vip` 尚不存在，它将被自动创建。请求之后，这些客户端的记录中会被设置 `group_name = "vip"`，而它们每个 inbound 的 Xray 设置中，对应的客户端对象会获得 `"group": "vip"` 字段：

```
{ "id": "6f1b...", "email": "alice@example.com", "group": "vip", "enable": true }
```

9.8. 将客户端移出分组（不删除客户端本身）

按钮**移除客户端...**，标题为**「将客户端移出分组「{name}」」**。

逐字提示：

「选择要从此分组移除的成员。客户端本身会保留（如需彻底删除，请使用「删除分组客户端」）。」

行为（`POST /panel/api/clients/groups/bulkRemove`，请求体 `{"emails": [...]}`）：技术上这等同于以空分组执行「添加到分组」。所选客户端的 `group_name` 被清空，其 inbound 的 Xray 设置中的 `group` 字段被删除。客户端本身及其与 inbound 的绑定保持不变。

成功消息：**「已将 {count} 个客户端从 {name} 移除。」**

9.9. 重置分组流量

按钮**重置流量**。

确认对话框：

- 标题：**「重置分组 {name} 的流量？」**
- 文本：**「仅重置分组的流量计数器。各个客户端的计数器不受影响。」**

行为：仅清零分组自身显示的计数器——面板将分组当前的 `up/down` 总和记为基准点，之后在分组列表中显示 `max(0, [?] - [?])`。各个客户端的 `up/down` 计数器、配额和 `enable` 字段均不改变，无需重启 Xray。这是相对于以往版本的行为变更，此前重置会清零分组内所有客户端的流量。

请求：`POST /panel/api/clients/groups/resetTraffic`，请求体为 `{"name": "<[?]>"}`。

成功消息：**「分组 {name} 的流量已重置。」**

9.10. 删除分组与删除分组客户端

页面上有两个本质上不同的删除操作——它们很容易混淆，因此区分至关重要。

9.10.1. 删除分组（保留客户端）

按钮**「删除分组（保留客户端）」**。

对话框：

- 标题：「删除分组 {name}? 」
- 文本：「这将删除分组并清除 {count} 个客户端身上的分组标签。客户端本身不会被删除。」

行为（`POST /panel/api/clients/groups/delete`，请求体 `{"name": "..."}` ）：从 `client_groups` 中删除分组记录，清空其所有客户端的 `group_name`，并从它们的 `inbound` 中移除 `group` 字段。客户端、其连接和流量都会保留。Xray 被标记为需要重启。

成功消息：「已清除 {count} 个客户端的分组。」

9.10.2. 删除分组客户端（彻底删除）

按钮**「删除分组客户端」**。

对话框：

- 标题：「删除 {name} 中的所有客户端? 」
- 文本：「这将连同流量记录一起不可逆地删除 {count} 个客户端。分组标签也会被清除。此操作无法撤销。」

这是一项破坏性操作：它会删除客户端本身（通过按 email 批量删除，端点 `POST /panel/api/clients/bulkDel`），包括它们的流量记录，从而将它们从所有 `inbound` 中移除。

消息：

- 成功：「已删除 {count} 个客户端。」
- 部分结果：「{ok} 已删除，{failed} 已跳过」

如果分组为空，则无法对其成员执行操作——会显示**「此分组中暂无客户端。」**

9.11. 与「客户端」页面的关联

分组标签在分组页面之外也可见并被使用：

- 在客户端的精简记录中有 `group` 字段，因此客户端列表中会显示其所属分组。
- 客户端列表（`/panel/api/clients/list/paged`）接受筛选参数 `group`：可以传入一个名称，或用逗号分隔多个名称。匹配在该字段内按「或」的原则进行，不区分大小写。特殊情况：筛选分组列表中的空元素表示「无分组的客户端」（其 `group` 为空）。
- 在客户端页面的响应中会返回 `groups` 数组——现有分组名称的完整清单，以便 UI 构建筛选下拉列表。

示例：按分组筛选客户端。该请求只返回带有 `vip` 或 `trial` 标签的客户端（多个名称——用逗号分隔，「或」）：

```
GET /panel/api/clients/list/paged?group=vip,trial
```

要获取没有分组的客户端，请在列表传入一个空元素——例如筛选值 `group=`（空字符串）或 `group=vip`，（标签 `vip` 加上无分组的客户端）。

9.12. API 端点汇总

所有分组路由都挂载在 `/panel/api/clients` 下：

方法与路径	用途	请求体
GET <code>/panel/api/clients/groups</code>	列出分组及客户端计数	—
GET <code>/panel/api/clients/groups/:name/emails</code>	分组所有成员的 email（按 email 排序）	—
POST <code>/panel/api/clients/groups/create</code>	创建空分组	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/rename</code>	重命名分组	<code>{"oldName","newName"}</code>
POST <code>/panel/api/clients/groups/delete</code>	删除分组，保留客户端（清除标签）	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/bulkAdd</code>	向分组添加客户端（按 email）	<code>{"emails":[...],"group"}</code>
POST <code>/panel/api/clients/groups/bulkRemove</code>	将客户端移出分组（清除标签）	<code>{"emails":[...]}</code>
POST <code>/panel/api/clients/bulkDel</code>	彻底删除客户端（由「删除分组客户端」使用）	<code>{"emails":[...],"keepTraffic"}</code>

示例：通过 API 演示分组生命周期的典型场景。

```
# 1. 创建分组
curl -s ../panel/api/clients/groups/create -d '{"name":"trial"}'

# 2. 向分组添加客户端
curl -s ../panel/api/clients/groups/bulkAdd -d '{"emails":["u1@example.com","u2@example.com"],"group":"trial"}'

# 3. 清除分组标签
curl -s ../panel/api/clients/groups/bulkRemove -d '{"emails":["u2@example.com"]}'

# 4. 删除分组
curl -s ../panel/api/clients/groups/delete -d '{"name":"trial"}'
```

第 4 步会删除分组记录并清空其客户端的 `group_name`，但客户端本身、它们的连接和流量都会保留。要不可逆地删除客户端本身，则应改用 `bulkDel`。

会更改客户端标签的操作（`rename`、`delete`、`bulkAdd`、`bulkRemove`）会将 Xray 标记为需要重启，并发出客户端变更通知。

9.13. 按分组统计的流量

3.3.0 版本的新功能：在**分组**部分（「客户端」页面的分组管理选项卡），分组表格现在不仅显示每个分组中的客户端数量，还显示该分组的累计已用流量。该列标题为**「已用流量」**。

该列显示的内容

对于每一行分组，会显示属于该分组的所有客户端的流量总和——即所有成员的 **up + down**（发出 + 接收的流量）相加。这能快速回答「整个分组总共下载/上传了多少」的问题，而无需逐个打开客户端并手动累加。

分组表格中并排显示的还有：

列	含义
Имя	分组名称
Клиенты	该分组标记了多少客户端（此列以前叫「分组内客户端数」）
Отправлено	分组所有客户端的 up 总和（发出的流量）
Получено	分组所有客户端的 down 总和（接收的流量）
Использованный трафик	分组所有客户端的 up + down 总和

发出和接收的流量以独立的**已发送**和**已接收**两列显示，而**已用流量**列显示它们的总和。客户端数量这一列就叫**客户端**。

表格上方的汇总还会额外显示所有分组的聚合值——「**分组总数**」和「**有分组的客户端**」，并将总流量拆分为两张卡片：「**总发送 / 接收**」（带上/下箭头——分别为所有分组发出和接收的流量）和**「总流量」**（带图表图标——两者的合计）。

如何计算

计算通过对客户端表的一条 SQL 查询完成，并连接（**LEFT JOIN**）流量统计表：

- 按分组标签字段（**group_name**）对客户端进行分组并统计其数量——这就是「分组内客户端数」；
- 流量取自连接表 **client_traffics** 中 **up + down** 的总和。也就是说，对每个客户端的发出字节（**up**）和接收字节（**down**）都进行累加；
- 由于 **email** 在客户端表和流量表中都是唯一的，连接不会重复计算同一客户端的流量。

数值的特点：

- 没有流量记录的客户端**会被计入成员计数，但对总和的贡献为 0，因此刚创建的分组显示流量为 **0**。
- 空分组**（已创建但没有客户端）也会出现在列表中，计数为零、流量为零：除了从客户端标签「派生」出的分组外，结果中还会掺入显式保存的分组，之后列表会按名称不区分大小写排序。
- 没有分组标签的客户端（**group_name** 为空）不计入统计。

相关操作

从分组表格中仍然可以执行针对整个分组的操作，其中包括**「重置流量」——仅清零分组自身的计数器**（基准点），不影响各个客户端的流量（参见 [9.9](#)）。这样重置之后，该分组的「已用流量」列将显示为 **0**。

10. 订阅 (Subscription)

订阅 (subscription) 是一种机制，允许为客户端提供一个固定链接 (URL)，VPN 客户端通过该链接自动下载并定期更新完整的配置集合。无需手动向用户逐一发送每个 inbound 的链接，只需提供一个形如 `https://[?]:[?]/sub/<subId>` 的统一地址。通过该地址，面板会即时汇总该客户端绑定的所有配置，并以客户端所需的格式返回。当服务器设置发生变更 (新地址、Reality 密钥轮换、新增 inbound) 时，客户端在下次自动更新时将获取最新配置，无需用户进行任何操作。

订阅由面板内独立的 HTTP/HTTPS 服务器提供，该服务器独立于 Web 面板运行，监听独立端口。这是出于安全考虑：可以对外开放订阅端口，而无需开放面板本身的端口。

10.1. 什么是 subId 以及链接的构成方式

inbound 中的每个客户端都有一个 `subId` 字段 (界面中显示为「订阅 ID」)。该值是订阅的键：面板会在所有 inbound 中查找 `subId` 与请求匹配的客户端，并将其配置合并为一个响应。

- 如果多个客户端 (在同一个或不同的 inbound 中) 设置了相同的 `subId`，它们的配置将包含在同一个订阅中。这是通过一个链接为同一用户提供多台服务器/协议的标准方式。

示例：一个用户通过一个链接获取两台服务器的配置。 假设有两个 inbound (服务器 A 上的 VLESS 和服务 B 上的 Trojan)。若要通过一个链接向用户提供这两个配置，请为该用户的两个客户端设置相同的 `subId`：

```
Inbound 1 (VLESS): email = ivan@vpn, subId = ivan2025
Inbound 2 (Trojan): email = ivan@vpn, subId = ivan2025
```

这样，访问 `https://sub.example.com:2096/sub/ivan2025` 时，面板会同时返回两个配置。日后若添加第三个具有相同 `subId` 的 inbound，用户在下次自动更新订阅时即可获取，无需重新发送新链接。

- 如果客户端的 `subId` 字段为空，则无法共享公共访问链接。界面中会有相应提示：「该客户端没有 subId，公共访问链接不可用。」

客户端外部链接与订阅 (「Links」选项卡)

客户端表单中有一个「Links」选项卡，可以为单个客户端附加额外的配置来源，这些来源仅混入该客户端的订阅中 (支持 RAW、JSON 和 Clash 格式)：

- **Add External Link** – 第三方分享链接 (`vless://`、`trojan://`、`ss://` 等)。按原样添加到输出中，对于 JSON/Clash 还会进一步解析为配置。
- **Add External Subscription** – 外部订阅地址。面板会自动拉取该订阅 (带缓存和短超时)，并将获取到的配置合并到客户端的配置列表中。

这对于通过同一个统一链接向客户端提供除您的 inbound 之外的额外服务器非常方便。如果远程订阅的响应过大，不再静默截断：面板会返回错误并继续使用上次成功缓存的值。

- `subId` 的值不能任意设置：保存时会检查其中是否包含空格、`/`、`\` 或控制字符。相应的验证提示为：「订阅 ID 不能包含空格、`/`、`"` 或控制字符」。

最终链接的构成为 `<?>/<subPath>/<subId>`（请参阅订阅服务器设置和「反向代理 URI」字段相关章节）。如果根据 `subId` 未找到任何客户端（客户端已删除、`subId` 不存在），服务器将返回无正文的 HTTP 404。发生内部错误时返回 HTTP 500。VPN 客户端仅根据响应代码判断，因此错误正文故意留空。

inbound 链接在订阅中的排序

每个 inbound 都有一个「订阅排序」字段（`subSortIndex`）——从 1 开始的数字，用于指定该 inbound 的链接在订阅输出中的位置。数值越小排在越前面；数值相同时，按创建顺序（按 id）保持原有顺序。排序适用于所有输出格式——纯文本、订阅页面、JSON 和 Clash。该字段不影响面板中 inbound 本身的排列顺序。

该字段可在 inbound 表单中与分享地址（share address）设置一同编辑，并按常规规则同步到节点。如果至少有一个 inbound 的排序值不等于 1，Inbounds 列表中会出现紧凑的「排序」列。

10.2. 订阅服务器设置

所有订阅参数均位于面板设置的「订阅」选项卡中。以下逐一介绍每个参数；括号内标注了配置项的内部键和默认值。

该部分进一步分为以下选项卡：「面板设置」、「信息」、「配置文件」、「证书」、「Happ」和「Clash / Mihomo」。订阅标题、支持 URL、配置文件页面、公告和主题目录等字段位于「配置文件」选项卡；Happ 和 Clash/Mihomo 的路由规则位于对应的同名选项卡；订阅更新间隔位于「信息」选项卡。

基本参数

字段（界面）	键	默认值	描述
启用订阅	<code>subEnable</code>	<code>true</code> （已启用）	启动独立的订阅服务器。提示：「具有独立配置的订阅功能」。如果禁用，订阅服务器将不会启动，所有链接均不可用。
监听 IP	<code>subListen</code>	空	订阅服务器接受连接的 IP 地址。提示：「默认留空以监听所有 IP 地址」。
订阅端口	<code>subPort</code>	<code>2096</code>	订阅服务器的 TCP 端口。提示：「为订阅服务提供服务的端口号不应在服务器上被占用」——该端口必须空闲，不与面板或 Xray 冲突。
URI 路径	<code>subPath</code>	<code>/sub/</code>	提供普通订阅的路径。提示：「必须以 <code>/</code> 开头并以 <code>/</code> 结尾」。
监听域名	<code>subDomain</code>	空	允许访问订阅的域名（Host 验证）。提示：「默认留空以监听所有域名和 IP 地址」。如果设置了值，则带有其他 Host 的请求将被拒绝。

安全须知： 默认路径 `/sub/`（以及 JSON 格式的 `/json/`）广为人知，容易被猜到。面板会显示警告：「默认订阅路径 `/sub/` 广为人知——请更改它。」以及类似的 JSON 警告。建议设置自己不易猜测的路径。

TLS / 证书

字段（界面）	键	默认值	描述
订阅证书公钥文件路径	subCertFile	空	证书文件（.crt / fullchain）的完整路径。提示：「请输入以 '/' 开头的完整路径」。
订阅证书私钥文件路径	subKeyFile	空	私钥文件的完整路径。提示：「请输入以 '/' 开头的完整路径」。

如果两个路径均已设置且证书成功加载，订阅服务器将通过 **HTTPS** 运行。如果字段为空或证书无法读取，服务器将回退到 **HTTP**（错误写入日志）。有效 TLS 的存在也会影响基础 URL 的构成：当端口为 443（启用 TLS）或端口为 80（不启用 TLS）时，链接中的端口号将被省略。

更新间隔

字段（界面）	键	默认值	描述
订阅更新间隔	subUpdates	12	客户端应用程序重新获取订阅的频率（小时）。提示：「客户端应用程序中更新之间的间隔（小时）」。自 3.5.0 起，该字段接受 0-525600 并显示取值范围（此前的界面上限 168 与服务器不一致，从 2.x 升级后会阻止保存任何设置）。

该值通过 HTTP 响应头 `Profile-Update-Interval` 传递给客户端；现代客户端将其用作配置自动更新的周期。

响应格式与信息

字段（界面）	键	默认值	描述
编码	subEncrypt	true	提示：「对订阅中返回的配置进行加密」。从技术上讲，这不是加密，而是对普通订阅整个正文进行 Base64 编码 （大多数客户端所期望的格式）。禁用时，链接以纯文本形式返回，每行一个。
显示使用信息	subShowInfo	true	提示：「在配置名称后显示剩余流量和到期日期」。启用后，每个配置的名称（remark）后会附加剩余流量标记（  ）和有效期标记（例如 5D, 3H  ）；对于已过期/不可用的客户端，显示 N/A 。
在名称中包含 Email	subEmailInRemark	true	提示：「在订阅配置文件名称中包含客户端 email。」。将客户端 email 添加到配置文件的 remark 中。

备注模板（Remark Template）

订阅中每个配置的显示名称（remark）根据备注模板生成——即订阅设置「信息」选项卡中的「备注模板」字段（remarkTemplate）。之前在界面中用于单独选择 inbound/email/external proxy 各部分及分隔符的备注模型构建器已被移除；现在您可以编写任意格式的名称并在其中插入变量。默认值为 `{{INBOUND}}-`

`{{EMAIL}} | {{TRAFFIC_LEFT}} | {{DAYS_LEFT}}D`（即默认情况下配置文件名称包含客户端 email）。如果该字段留空，将使用之前的（无法通过界面配置的）备注模型。

变量按 **Client**、**Traffic** 和 **Time & status** 分组显示在字段旁边，以可点击的 `{{VAR}}` 芯片形式展现，悬停时显示提示；点击即可将令牌插入模板，并提供实时预览。每个变量在生成订阅时针对特定客户端单独替换。也支持简写形式（使用单括号，如 `{DATA_LEFT}`、`{EXPIRE_DATE}`、`{PROTOCOL}`、`{TRANSPORT}` 等）——面板会自动将其转换为内部的 `{{...}}` 格式。

可用变量：

- **客户端标识**： `{{EMAIL}}`（同义词——`{{USERNAME}}`）、`{{INBOUND}}`（inbound 本身的 remark）、`{{HOST}}`（主机 remark）、`{{ID}}`（UUID）、`{{SHORT_ID}}`（UUID 的前 8 个字符）、`{{SUB_ID}}`、`{{COMMENT}}`、`{{TELEGRAM_ID}}`、`{{PROTOCOL}}`、`{{TRANSPORT}}`。
- **流量**： `{{TRAFFIC_USED}}`、`{{TRAFFIC_LEFT}}`、`{{TRAFFIC_TOTAL}}`（以及对应的 `*_BYTES` 精确字节变体）、`{{UP}}`、`{{DOWN}}`、`{{USAGE_PERCENTAGE}}`。
- **有效期与状态**： `{{DAYS_LEFT}}`、`{{TIME_LEFT}}`、`{{EXPIRE_DATE}}`（`??-??-??` 格式）、`{{JALALI_EXPIRE_DATE}}`（波斯历日期）、`{{EXPIRE_UNIX}}`、`{{CREATED_UNIX}}`、`{{RESET_DAYS}}`、`{{STATUS}}`（active / expired / disabled / depleted）、`{{STATUS_EMOJI}}`。
- **连接 (Connection)**： `{{PROTOCOL}}` — 协议（VLESS、VMess、Trojan 等），`{{TRANSPORT}}` — 传输网络（tcp、ws、grpc 等），`{{SECURITY}}` — 传输安全性（TLS、REALITY、NONE；以大写显示）。与流量和有效期变量一样，这三个变量仅在订阅正文中生效，会自动从面板显示链接（QR 码/「信息」）和订阅信息页面的 remark 中移除。

模板可以用竖线 `|` 分割为多个片段。当某个变量在某片段中的值为「无限」（ ∞ ）时——例如没有限制的客户端的 `{{TRAFFIC_LEFT}}` 或 `{{DAYS_LEFT}}` ——该片段会自动隐藏。此外，流量和有效期信息块仅在客户端的第一个链接上显示一次，以避免在每个配置中重复出现。

示例。 模板 `{{EMAIL}} | {{TRAFFIC_LEFT}} | {{DAYS_LEFT}}D` 对于剩余 42 GB 和 7 天的客户端，将生成类似 `ivan@vpn 42.00GB 7D` 的名称，而对于无限制客户端则只显示 `ivan@vpn`（包含 ∞ 的片段被省略）。

自 3.4.2 起，`{{EMAIL}}` 变量（及同义词 `{{USERNAME}}`）仅在客户端订阅正文的**第一条**链接上输出——与流量/有效期余量块一样；同一客户端的其余链接上则省略。

在面板显示的链接（「客户端」页面上的 QR 码和「信息」窗口）以及订阅信息页面上，客户端 email 会包含在配置文件名称中：若设置了主机则显示「inbound-host-email」格式，否则显示「inbound-email」格式。流量和有效期信息（以及「连接」组的变量）不会替换到这些显示名称中——它们仅在 VPN 客户端接收到的订阅正文中生效。

如果客户端的流量统计行在删除并重新创建 inbound 后「孤立」了，`{{TRAFFIC_USED}}`（以及其他使用量指标）变量将不再显示 `0.00B`：面板会额外按客户端 email 查找统计信息，并替换正确的已用流量。`|` 备注模板 `| remarkTemplate | {{INBOUND}} | {{TRAFFIC_LEFT}} | {{DAYS_LEFT}}D` `|` 用于配置显示名称（remark）的自由模板，支持 `{{VAR}}` 变量替换。在生成订阅时针对每个客户端单独替换。之前的「备注模型」构建器（选择 inbound/email/external proxy 和分隔符）已从界面移除，仅在字段留空时作为备用方案使用。详细说明请参阅下文「备注模板 (Remark Template)」。

配置文件元数据（响应头）

这些字符串通过 HTTP 响应头传递给客户端，并在 VPN 客户端中显示为配置文件元数据。默认情况下均为空。

字段（界面）	键	响应头	描述
订阅标题	subTitle	Profile-Title (Base64 编码)	「客户端在 VPN 客户端中看到的订阅名称」。对于 Clash，也通过 Content-Disposition 用作导入配置文件的名称。
支持 URL	subSupportUrl	Support-Url	「在 VPN 客户端中显示的技术支持链接」。
配置文件 URL	subProfileUrl	Profile-Web-Page-Url	「在 VPN 客户端中显示的您的网站链接」。如果未设置，将使用实际的订阅请求 URL。
公告	subAnnounce	Announce (Base64 编码)	「在 VPN 客户端中显示的公告文本」。自 3.5.0 起，该文本（非空时）还会以信息横幅的形式显示在订阅信息页面顶部，自定义模板中可使用 announce 变量。

此外，每个响应都包含 Subscription-Userinfo 响应头，其中包含客户端的聚合流量数据：upload、download、total 和 expire（到期时间，秒）。客户端据此显示剩余流量和有效期。

路由（仅适用于 Happ 客户端）

字段（界面）	键	默认值	描述
启用路由	subEnableRouting	false	「在 VPN 客户端中启用路由的全局设置。（仅适用于 Happ）」。通过 Routing-Enable 响应头传递。
路由规则	subRoutingRules	空	「VPN 客户端的全局路由规则。（仅适用于 Happ）」。通过 Routing 响应头传递。

| 隐藏服务器设置 | subHideSettings | false | 「在订阅中隐藏服务器设置（仅适用于 Happ）」。启用后，Happ 客户端中将隐藏查看和修改服务器参数的功能。该选项仅对 Happ 客户端生效。|

Incy 路由（仅适用于 Incy 客户端）

对于 VPN 客户端 Incy，订阅设置中有一个独立的「Incy」选项卡，包含两个字段：「启用路由」开关（subIncyEnableRouting，默认关闭）和格式为 incy://routing/onadd/<base64> 的「路由规则」文本字段（subIncyRoutingRules）。当路由启用且字段已填写时，该字符串会作为单独一行追加到订阅正文（raw 格式）中——这样路由配置文件就能传递给 Incy 客户端，而不会与 Happ 客户端的 Routing 响应头冲突。这些设置仅对 Incy 客户端生效。

反向代理 URI

字段（界面）	键	默认值	描述
反向代理 URI	subURI	空	「修改订阅 URL 的基础 URI，以便在代理服务器后使用」。

如果字段为空，面板会根据订阅的域名和端口（考虑 TLS）自动构建链接的基础地址。如果订阅通过外部反向代理/CDN 在其他域名或路径上分发，则在此字段中设置最终的基础 URI，所有链接将从该 URI 构建。JSON（subJsonURI）和 Clash（subClashURI）各有对应的独立字段。

如果仅设置了通用 subURI，而 JSON 和 Clash 的独立字段留空，则这些格式在订阅页面上的链接将继承 subURI 的协议和主机（而不是订阅服务器端口和 http）——从而与反向代理地址保持一致。

示例：反向代理后的订阅。 订阅本身在 2096 端口监听，但通过 nginx/CDN 以 https://cfg.example.com/u/ 对外提供。为使响应中的链接基于外部地址而非内部的 [?]:2096 构建，请在「Reverse proxy URI」字段中设置最终的基础 URI：

```
Reverse proxy URI: https://cfg.example.com/u
```

此时最终链接将为 https://cfg.example.com/u/ivan2025。对于 JSON 和 Clash 格式，如有需要，可以同样方式分别填写 subJsonURI 和 subClashURI 字段。

10.3. 输出格式

订阅可以以三种独立格式输出，每种格式有自己的端点，可以单独启用/禁用。

输出中的服务器地址与节点

订阅链接中的服务器地址采用与面板中普通链接和 QR 码相同的链接地址策略：「listen」——可路由的绑定地址，「custom」——用户设置的自定义地址（shareAddr），「node」（默认）——节点地址。对于没有明确设置策略的 inbound，订阅输出不会改变。这使得绑定到特定公网 IP 的节点 inbound 能够向客户端提供可达地址。该策略适用于 raw、JSON 和 Clash 格式。

节点（Node）名称不会添加到订阅中配置文件的名称（remark）中：在客户端应用程序中，仅显示管理员设置的 inbound remark，不带类似 @[?][?] 的内部后缀。若要在多节点订阅中区分同名条目，请手动为其设置不同的 remark，或使用具有独立 Remark 的托管主机（Hosts）。

如果由于节点间同步问题，同一个客户端在服务 JSON inbound 中出现了两次，订阅输出会在所有三种格式中自动按 email 去重，因此输出中不会出现重复的配置文件。

托管主机（Hosts）

Hosts 部分（侧边菜单项；显示 Total/Enabled/Disabled 数量及列表的汇总页面）用于设置订阅链接中的地址覆盖。对于每个 inbound，可以添加一个或多个**主机**——这些端点将在发送给客户端的订阅链接中**替换 inbound 本身的地址、端口和 TLS 参数**。这对于通过 CDN 或中继分发流量而无需修改 inbound 本身非常方便。

自 3.5.0 版本起，主机是一个「组」：一条记录可同时覆盖多个 inbound 和多个地址。列表操作（启用/禁用/删除/排序）和 API 均以组（字符串 `groupId`）为单位；新增了批量端点 `POST /panel/api/hosts/bulk/add`（`{ "inboundIds": [...], "hosts": [...], ... }`），`list / get / update / del / setEnable` 均操作组对象。

每个主机（组）包含以下设置：

- **Remark** 和描述（Description），绑定到 **Inbounds**（带搜索的多选；至少一个），**Enable** 开关，以及节点分配（**Nodes**）。
- **Address**——自 3.5.0 起为**地址列表**（标签式输入；分隔符为逗号、分号、空格；占位符 `cdn.example.com, cdn2.example.com:443`）。每个条目可带自己的内嵌 `[::1]:443`（包括括号形式的 IPv6 `:::1:443`）；下拉提示会推荐其他主机已使用过的地址。列表为空则继承 inbound 自身的地址（在列表中显示为橙色的 **Inherits** 标签）。**Port**（`0` 则继承 inbound 端口）作为无内嵌端口条目的默认端口；**Tags**（仅在 RAW 订阅中生效）。
- **Security** 选项卡 — `same / tls / none / reality`，包含 SNI、指纹（fingerprint）、ALPN、证书绑定（pinned-cert）、`allowInsecure` 和 ECH。
- **Advanced** 选项卡 — Host 头、Path、VLESS 路由、Mux、Sockopt、Final Mask，以及从各独立订阅格式（raw / json / clash）中排除该主机。自 3.5.0 起，主机的 **Final Mask** 也会进入 **raw** 链接（`fm=`；此前仅进入 JSON/Clash）：主机的 TCP/UDP 掩码会叠加到 inbound 的掩码上，主机的 QUIC 参数仅在 inbound 没有时才使用。**Allow insecure** 开关现在对 **Hysteria/Hysteria2** 也生效（链接中的 `insecure=1`，Clash 中的 `skip-cert-verify: true`）。
- **Clash (mihomo)** 选项卡 — IP 版本、Mihomo X25519、主机混洗（Shuffle host）。

列表中的 **Endpoint** 列以芯片形式显示地址（第一个直接可见，其余收进「+N」弹出层），**Inbounds** 列则显示按协议着色的 inbound 芯片。自 3.5.0 起，主机按**全局**顺序排序（先按排序序号，再按备注），而不再局限于各自的 inbound 范围内；批量启用、禁用和删除保持可用。托管主机取代了之前的 External Proxy 数组。

VLESS 路由（VLESS route）。自 3.4.2 起，这是一个 `0-65535` 的单个数字（而非端口列表；提示——「嵌入 UUID 的单个 VLESS route 值（0-65535），例如 443；留空——不使用」，占位符 `443`）。所设的值会真正「嵌入」到每个生成的订阅 UUID 中（raw / JSON / Clash）：Xray 读取 UUID 的第 6-7 字节并在认证前将其掩码处理，因此客户端仍能匹配。值为空或不合法时不会改变 UUID。

普通链接（SUB）— Base64 / 纯文本

基本格式，端点为 `subPath`（默认 `/sub/`）。只要订阅整体处于启用状态，此格式始终开启。返回 Xray 链接列表（`vless://`、`vmess://`、`trojan://`、`ss://` 等），每行一个。启用「编码」（`subEncrypt`）选项时，整个列表以 Base64 编码；禁用时以纯文本形式返回。几乎所有客户端都支持此格式（v2rayNG、V2RayTun、Sing-box、NekoBox、Streisand、Shadowrocket、Happ 等）。

示例：禁用「编码」时的响应正文。 当 `subEncrypt = false` 时，`/sub/` 端点返回纯文本——每行一个链接：

```
vless://3c8f...@a.example.com:443?security=reality&...#srvA-ivan
trojan://p4ss@b.example.com:443?security=tls&...#srvB-ivan
```


当 `subEncrypt = true`（默认值）时，整个列表以 Base64 编码后以单行形式返回——这正是大多数客户端所期望的格式。

JSON 订阅（sing-box 及兼容客户端）

端点为 `subJsonPath`（默认 `/json/`），通过独立开关启用。

字段（界面）	键	默认值	描述
JSON 订阅	<code>subJsonEnable</code>	<code>false</code>	「独立启用/禁用 JSON 订阅端点。」。

返回完整的 JSON 配置（格式兼容 sing-box 及衍生客户端——Podkop、OpenWRT sing-box、Karing、NekoBox）。自 3.5.0 起，原生 **WireGuard inbound** 的客户端也会进入 JSON 订阅（`secretKey`、`address`、含 `publicKey` / `endpoint` / `preSharedKey` / `keepAlive` / `allowedIPs` 的 `peers[]`、`mtu`）——此前它们会被静默跳过。此格式提供额外参数（`subFormats` 选项卡）：

- **Mux**（`subJsonMux`，默认为空）— 多路复用（Mux）的 JSON 配置，将被注入 JSON 订阅中每个流的 outbound。「在单个连接中传输多个独立数据流。」。
- **Final Mask**（`subJsonFinalMask`，默认为空）— 「注入 JSON 订阅中每个流的 xray finalmask（TCP/UDP）和 QUIC 设置。需要客户端使用较新版本的 xray。」。通过子字段配置：「数据包」（`packets`）、「长度」（`length`）、「间隔」（`interval`）、「最大分片」（`maxSplit`）、「噪声」（`noises`：「类型」/ `type`、「数据包」/ `packet`、「延迟(ms)」/ `delayMs`、「应用于」/ `applyTo`，以及「+ 噪声」按钮），以及「并发」（`concurrency`）、「xudp 并发」（`xudpConcurrency`）和「xudp UDP 443」（`xudpUdp443`）。
- **路由规则**（`subJsonRules`，默认为空）— 添加到 JSON 配置中的全局规则。

Clash / Mihomo 订阅（YAML）

端点为 `subClashPath`（默认 `/clash/`），通过独立开关启用。

字段（界面）	键	默认值	描述
Clash / Mihomo 订阅	<code>subClashEnable</code>	<code>false</code>	为 Clash 和 Mihomo 客户端启用 YAML 配置生成。
启用路由	<code>subClashEnableRouting</code>	<code>false</code>	「将全局 Clash/Mihomo 路由规则添加到生成的 YAML 订阅中。」。
全局路由规则	<code>subClashRules</code>	空	「在 MATCH,PROXY 之前添加到每个 YAML 订阅开头的 Clash/Mihomo 规则。」。

响应以 `application/yaml; charset=utf-8` 类型返回。如果设置了「订阅标题」（`subTitle`），它也会通过 `Content-Disposition` 响应头（`attachment; filename*=UTF-8''<title>`）传递，以便 Clash 客户端以该名称命名导入的配置文件。

生成的链接和 YAML 格式针对现代客户端保持最新状态：Shadowsocks-2022（SS2022）不再对 `userinfo` 进行 Base64 编码；带有 http 混淆的 Shadowsocks 链接以 SIP002 格式并使用 `obfs-local` 插件输出；Clash/Mihomo 订阅实现了完整的 XHTTP 字段集。自 3.5.0 起，原生 **WireGuard inbound** 的客户端也会进入 Clash/Mihomo 订阅（字段 `private-key`、`public-key`、`pre-shared-key`、`persistent-keepalive`、`ip/`

ipv6、mtu、dns）——此前它们会被静默跳过。这些改进不需要额外设置——链接只是能被客户端更正确地识别。

注意：此版本支持三种格式——普通链接（Base64/文本）、JSON（sing-box 兼容）和 Clash/Mihomo（YAML）。订阅服务器中没有独立的 Outline 格式。

10.4. 订阅信息页面与 QR 码

如果在浏览器中打开三种订阅链接中的任意一种（raw、JSON 或 Clash；或在 URL 中显式添加参数 `?html=1` 或 `?view=html`，或发送 `Accept: text/html` 请求头），服务器将返回可视化的订阅信息页面（「订阅信息」），而不是「原始」响应。在 3.5.0 之前只有主链接 `/sub/` 如此工作，`/json/` 和 `/clash/` 在浏览器中返回的是原始 JSON/YAML。VPN 客户端仍然获得机器可读响应，因为它们不请求 HTML。

该页面（使用 Vite 构建的单页应用）显示：

- **订阅信息**（Descriptions 块）：
 - 「订阅 ID」—— `subId` 的值；
 - 「状态」——「活跃」、「非活跃」或「无限制」。如果客户端已禁用、已超出流量限制或已过期，状态显示为「非活跃」；
 - 「已下载」和「已上传」——流量用量；
 - 「总限额」——流量限制，如未限制则显示 ∞ ；
 - 「有效期」——到期日期或「永久」；
 - 剩余流量和最后在线时间。
 - 日期根据面板的「Calendar Type」设置（`datepicker`，默认 `gregorian`）以公历或波斯历显示。
- **订阅链接**：每种已启用格式各占一行，带有彩色标签（绿色 **SUB**、紫色 **JSON**、金色 **CLASH**）、复制按钮和 **QR 码** 按钮（弹出窗口，尺寸 240 px）。JSON 和 CLASH 行仅在设置中启用了相应格式时才会显示。
- **独立链接**（「复制链接」）：订阅中包含的所有独立配置的完整列表，每个配置都带有协议标签、复制按钮和 QR 码（后量子链接不生成 QR 码）。
- **「复制所有配置」按钮**（位于独立链接列表上方）：一键将所有配置链接（每行一个）复制到剪贴板，无需逐一复制；完成后显示「所有配置已复制」通知。
- **快速导入应用按钮**（按平台分类的下拉菜单）：Android 端——`v2box`、`v2rayNG`（深度链接 `v2rayng://install-config?url=...`）、`Sing-box`、`V2RayTun`、`NPV Tunnel`、`Happ`（`happ://add/...`）、`Incy`（`incy://add/...`）；iOS 端——`Shadowrocket`（通过参数 `flag=shadowrocket`）、`v2box`（`v2box://install-sub?url=...&name=...`）、`Streisand`（`streisand://import/...`）、`V2RayTun`、`NPV Tunnel`、`Happ`、`Incy`。这些按钮会打开目标应用的深度链接（已预填订阅地址），或将链接复制到剪贴板。

订阅信息页面返回时带有禁止缓存的响应头（`Cache-Control: no-cache`），以确保客户端始终看到最新的流量和有效期数据。

10.5. 自定义订阅页面模板

从 3.3.0 版本开始，可以使用自己的 HTML 模板替换标准的订阅落地页。默认情况下，访问订阅地址时返回内置页面，但如果指定了包含自定义模板的目录，面板将渲染该模板并将客户端的实时数据（流量、有效期、链接等）注入其中。

重要提示：面板**不提供**现成的模板 – 自定义主题需要自行创建。创建说明和可用变量列表见 [docs/custom-subscription-templates.md](#)。

在哪里启用

主题目录在面板设置中指定：

设置 → 订阅 → 「订阅信息」部分，字段「订阅主题目录」（subThemeDir）。

界面中的字段说明：「自定义模板（index.html/sub.html）文件夹的绝对路径，用于订阅页面（例如 /etc/3x-ui/sub_templates/my-theme/）。留空以使用默认页面。」

同一部分旁边还有相关设置，其值可在模板中使用：

「订阅主题目录」字段说明中有一个「[模板指南 ↗](#)」链接，指向创建自定义订阅页面样式模板的文档。

- 「订阅标题」（subTitle）——客户端可见的名称；
- 「支持 URL」（subSupportUrl）——技术支持链接。

配置参数

参数	默认值	用途
subThemeDir	""（空）	包含您的 HTML 模板的目录的绝对路径。留空 = 使用内置默认页面。

如何使用自定义模板

1. 在服务器上创建主题文件夹（任意位置），例如 /etc/3x-ui/sub_templates/my-theme/。
2. 在其中放置名为 index.html 或 sub.html 的 HTML 文件。

示例：主题路径。服务器上的最终文件结构和设置中的字段值：

```
/etc/3x-ui/sub_templates/my-theme/  
└─ index.html          (??sub.html - ????)  
  
?? → ?? → ??????:  
/etc/3x-ui/sub_templates/my-theme/
```

路径必须是**绝对路径**（以 / 开头）。如果文件夹中既没有 index.html 也没有 sub.html，面板将返回内置页面。3. 在面板中打开设定 → 订阅，并在「订阅主题目录」字段中填写该文件夹的绝对路径。4. 保存设置。

文件选择和渲染行为：

- 如果目录中存在 `sub.html`，则使用它；否则使用 `index.html`。即 `sub.html` 优先于 `index.html`。
- 模板由 Go 标准库 `html/template` 引擎渲染。
- 解析后的模板会被缓存，仅在文件修改时间发生变化时才从磁盘重新读取。因此，对模板的修改无需重启面板即可生效，但每次请求时不会产生读取/解析的额外开销。
- 响应在全部生成后才发送给客户端：如果模板在执行过程中失败，不完整（损坏）的页面不会发送给用户。

默认行为与回退 (fallback)

- 字段为空 → 返回内置 SPA 页面（数据注入 `window.__SUB_PAGE_DATA__`）。
- 路径不存在或不是目录 → 使用默认页面。
- 目录中既没有 `index.html` 也没有 `sub.html` → 日志中写入警告「subThemeDir set but no usable template found」，返回默认页面。
- 模板文件存在但解析失败 → 日志中写入错误「custom template parse failed」，返回默认页面。
- 模板执行错误 → 日志中写入「custom template execution failed」，返回默认页面。

也就是说，自定义模板出现任何问题都不会「破坏」订阅——面板始终会回退到内置页面。所有订阅页面（自定义页面和标准页面）均以禁止缓存的响应头（`Cache-Control: no-cache, no-store, must-revalidate`）返回，以确保客户端始终获取最新的流量和有效期数据。

可用模板变量

模板上下文中传入了订阅客户端的一组数据。通过 `{{ .? ? ? }}` 访问：

变量	类型	描述
<code>{{ .sId }}</code>	string	订阅 ID (UUID)。
<code>{{ .enabled }}</code>	bool	客户端/订阅是否已启用。
<code>{{ .download }}</code>	string	格式化的下载量（例如「2.5 GB」）。
<code>{{ .upload }}</code>	string	格式化的上传量。
<code>{{ .total }}</code>	string	格式化的总流量限制。
<code>{{ .used }}</code>	string	格式化的已用流量（下载量 + 上传量）。
<code>{{ .remained }}</code>	string	格式化的剩余流量。
<code>{{ .expire }}</code>	int64	有效期——Unix 时间戳，单位为秒（0 = 永久）。用于 JS Date 时需乘以 1000。
<code>{{ .lastOnline }}</code>	int64	最后在线时间——Unix 时间戳，单位为毫秒（0 = 从未上线）。
<code>{{ .downloadByte }}</code>	int64	下载量精确字节数。
<code>{{ .uploadByte }}</code>	int64	上传量精确字节数。
<code>{{ .totalByte }}</code>	int64	总限制精确字节数。

变量	类型	描述
<code>{{ .subUrl }}</code>	string	订阅页面 URL。
<code>{{ .subJsonUrl }}</code>	string	JSON 订阅配置 URL。
<code>{{ .subClashUrl }}</code>	string	Clash/Mihomo 配置 URL。
<code>{{ .subTitle }}</code>	string	设置中的订阅标题（可能为空）。
<code>{{ .subSupportUrl }}</code>	string	设置中的支持 URL（可能为空）。
<code>{{ .links }}</code>	[]string	配置字符串列表（VMess、VLESS 等）。遍历： <code>{{ range .links }} ... {{ end }}</code> 。
<code>{{ .emails }}</code>	[]string	订阅相关的 email 列表。
<code>{{ .datepicker }}</code>	string	面板当前日历格式：gregorian 或 jalali（来自「日历类型」设置；若为空则为 gregorian）。

使用部分变量的最简模板示例：

```
<h1>{{ .subTitle }}</h1>
<p>{{ .used }} / {{ .total }} {{ .remained }}</p>
{{ range .links }}<div>{{ . }}</div>{{ end }}
```

示例：从 `expire` 获取到期日期。`{{ .expire }}` 字段是 Unix 时间戳，单位为秒，因此在 JavaScript 中需乘以 1000；值为 `0` 表示永久有效：

```
<script>
  var exp = {{ .expire }};
  document.write(exp === 0
    ? '永久有效'
    : '到期日期: ' + new Date(exp * 1000).toLocaleDateString());
</script>
```

注意：`{{ .lastOnline }}` 已经是毫秒单位——无需再乘以 1000。

11. Xray：路由、outbounds、DNS 与扩展

「Xray 设置」部分是 Xray-core 配置模板的编辑器，面板基于该模板生成最终的 `config.json` 来启动核心。模板部分的提示：「根据模板创建 Xray 配置文件。」与 inbounds 不同（inbounds 单独存储在数据库中，在构建配置时插入模板），其余所有内容——日志、路由、outbounds、DNS、策略、统计——均在此处设置。

重要：模板的值以键 `xrayTemplateConfig` 存储在数据库中。保存时，面板会对其进行一系列自动转换（参见 11.11）。任何语法不正确的 JSON 都将被拒绝，并显示错误「`xray template config invalid`」。

菜单位置：「Outbounds」与「路由」

「Outbounds」（Outbounds）和「路由」（Routing）是侧边菜单中的独立条目（紧接「Hosts」之下、「面板设置」之上），各有独立地址：`/outbound` 和 `/routing`。直接链接到这些页面以及页面刷新均可正常工作。「Xray 配置」子菜单中则仅保留：基本、负载均衡器、DNS 和高级模板。在以下说明中，11.3 和 11.4 部分分别对应「路由」和「Outbounds」页面。

11.1. 编辑器结构：选项卡/模式

编辑器提供多种模板显示模式（按 JSON 部分过滤）：

模式	显示内容
基本	基础部分（日志、基本路由、主要设置）
高级模板	Xray 完整 JSON 模板
全部	所有部分同时显示

编辑器内的逻辑设置组：

- 主要设置（提示：「这些参数描述通用设置」）。
- 日志（参见 11.10）。
- 基本连接：封锁和直连路由。
- Inbounds（提示：「修改配置模板以连接特定客户端」）。
- Outbounds（参见 11.4）。
- 负载均衡器（参见 11.5）。
- 路由（提示：「每条规则的优先级很重要！」，参见 11.3）。
- DNS / Fake DNS（参见 11.6）。

11.2. 主要设置 (General)

Freedom Protocol Strategy

字段	标签	描述	默认值
FreedomStrategy	Freedom 协议策略设置	直连 (freedom) outbound 的网络出口策略。提示：「设置 Freedom 协议中的网络出口策略」。控制 freedom 协议 outbound 的 settings 内的 domainStrategy 字段。	在参考模板中，direct freedom-outbound 的 domainStrategy 为 AsIs （地址不在服务器侧解析，以原始形式传递）。

freedom 的 domainStrategy (Xray-core 值)：AsIs（不在服务器侧解析域名），以及 UseIP / UseIPv4 / UseIPv6 系列及其「强制」变体 ForceIP*，强制出口服务器解析域名并通过获取的 IP 进行连接。如果出口服务器没有 IPv6 或需要强制仅走 IPv4，请更改为 UseIPv4。

Freedom Happy Eyeballs (IPv4/IPv6)

字段	标签	描述
FreedomHappyEyeballs	Freedom Happy Eyeballs (IPv4/IPv6)	提示：「用于直连 (freedom) 出站的双栈拨号——在同时具有 IPv4 和 IPv6 的出口服务器上很有用。」为 freedom-outbound 启用 Happy Eyeballs 算法（同时尝试两个地址族）。

具体某个 outbound 的 **dial 设置 (sockopt) **中也有独立的 Happy Eyeballs：自 3.5.0 起它能真正启用（此前配置会被序列化为关闭状态，开关随之「弹回」）。「Try delay (ms)」的默认值现为 **250**；显式设置的 **0**（= 关闭）会被保留；此外还提供 Prioritize IPv6、interleave (1) 和 maxConcurrentTry (4)。| try delay |（提示）|「在尝试另一个地址族之前等待的毫秒数。150–250 ms 是一个不错的起点。」切换到备用地址族前的延迟。建议范围为 150–250 ms。|

Overall Routing Strategy

字段	标签	描述	默认值
RoutingStrategy	域名路由策略设置	DNS 解析的全局路由策略。提示：「设置 DNS 解析的全局路由策略」。控制 routing.domainStrategy 字段。	在参考模板中，routing.domainStrategy = AsIs 。

routing.domainStrategy 决定了如何将 IP 路由规则与域名请求进行匹配：AsIs（仅域名规则，不解析）、IPIfNonMatch（若域名未匹配规则——解析并检查 IP 规则）、IPOnDemand（遇到 IP 规则时立即解析）。要使 IP 规则（例如 geoip:*）对域名请求生效，通常需要 IPIfNonMatch。

Outbound Test URL

字段	标签	描述	默认值
outboundTestUrl	出站测试 URL	测试 outbound 连通性时使用的 URL。提示：「用于检测出站连接的 URL」。与模板分开存储，键为 xrayOutboundTestUrl。	https://www.google.com/generate_204

该值会经过清理。实际测试 outbound 时，还会额外验证其是否为公开 URL——这是 SSRF 防护：用户无法通过客户端提交任意（包括内部）URL，测试 URL 始终取自服务器设置。保存/测试时，空值会被替换为默认的 generate_204。

Default Outbound（自 3.5.0 起）

「基本路由」选项卡的第一项是 **Default Outbound** 选择器：「未匹配任何路由规则的流量将使用此 outbound（Xray 取列表中的第一个 outbound）」。列表中包含 direct、blocked 及所有现有标签；默认为 direct。选择后会将指定的 outbound 移至模板的首位；缺失的 direct/blocked 会即时创建，切换时不会被删除。

Block BitTorrent

字段	标签	描述
Torrent	封锁 BitTorrent	在 routing.rules 中添加一条规则，将 protocol: ["bittorrent"] 流量发送到 outbound blocked。该规则在参考模板中默认存在。

连接限制（Connection Limits）

提示：「0 级用户的连接级策略。留空以使用 Xray 默认值。」这些参数写入 policy.levels.0。

字段	标签	描述	默认值
connIdle	空闲超时（秒）	「在指定秒数空闲后关闭连接。减小此值可在高负载服务器上更快释放内存和文件描述符（Xray 默认值：300）。」	空 → Xray 默认 300
bufferSize	缓冲区大小（KB）	「每个连接的内部缓冲区大小（KB）。设为 0 可最小化小内存服务器的内存占用（Xray 默认值取决于平台）。」占位符：「auto」。	空 → 取决于平台；0 – 最小化

示例（policy.levels.0）。 此组中的字段写入 0 级策略。在内存较小的高负载服务器上，可如此加速资源释放：


```
"policy": {
  "levels": {
    "0": {
      "connIdle": 120,
      "bufferSize": 0
    }
  }
}
```

此处连接在空闲 120 秒后关闭（而非默认的 300 秒），`bufferSize: 0` 则最小化缓冲区内存占用。表单中留空的字段不会写入 JSON——Xray 将使用其默认值。

11.3. 路由规则（routing）

`routing.rules` 规则列表。**顺序至关重要**（「每条规则的优先级很重要！」）：规则从上到下依次评估，第一个匹配的规则生效。提示：「拖动以更改顺序」。顺序控制按钮：**第一条**、**最后一条**、**上移**、**下移**。

每条规则具有 `type: "field"`。按钮：**创建规则**、**编辑规则**。列表字段提示：「以逗号分隔的条目」。

在「路由」页面，「导入规则」和「导出规则」按钮集中在「更多」（more）下拉菜单中——与「Outbounds」页面相同。「导出规则」按钮不会直接下载文件，而是打开一个带有 JSON 预览及「复制」和「下载」按钮的模态窗口：内容可在保存前预览。「Outbounds」页面上的出站导出功能工作方式相同。

Route Tester（路由测试器）

在「路由」选项卡中有一个子选项卡 **Route Tester**——它向运行中的 Xray 查询特定连接将由哪个 outbound 处理，而不发送实际流量。指定域名或 IP、端口、网络（TCP/UDP），以及必要时的 inbound 和嗅探到的协议（`http/tls/quic/bittorrent`），然后点击 **Test Route**。结果直接来自实时路由引擎。

响应中会显示匹配到的 outbound，如果使用了负载均衡器，还会显示负载均衡器标签。若没有规则匹配，测试器会报告流量将前往默认 outbound（`outbounds` 列表中的第一个）。这对于在依赖规则顺序之前进行验证非常有用。

启用和禁用单条规则

单条路由规则可通过切换开关临时**禁用**，无需删除。规则表格中有「启用」列，带有切换开关（Switch），规则表单中也有「启用」字段——同样是切换开关。被禁用的规则不会出现在最终 Xray 配置中，但保留在模板中，随时可以重新启用。

统计服务规则（`inboundTag: ["api"] → outboundTag: "api"`）无法禁用——其切换开关被锁定，以防破坏面板的流量统计（参见 11.11）。

规则表单字段

表单字段	标签	JSON 字段	描述
来源	来源	<code>source</code>	来源 IP 地址/子网。以逗号分隔的列表。

表单字段	标签	JSON 字段	描述
来源端口	来源端口	sourcePort	来源端口。
目标	目标	domain + ip + port	目标域名、IP 和端口。域名支持前缀 domain:、full:、regexp:、keyword: 以及 geosite:*；IP 支持 geoip:* 和 CIDR。
网络	—	network	tcp、udp 或 tcp,udp。自 3.4.2 起，在配置中以小写写入（tcp / udp），而在路由表中以大写显示（TCP / UDP）；留空——任意网络。
协议	—	protocol	http、tls、bittorrent（通过嗅探确定）。
用户	用户	user	按用户 e-mail/标识符过滤。
属性 / 值	属性 / 值	attrs	用于匹配的 HTTP 请求头属性。
VLESS route	VLESS route	—	按 VLESS 的 route 字段路由。
Inbound 标签	Inbound 标签	inboundTag	规则适用的一个或多个 inbound 标签（包括内置 api 以及 DNS 设置中的 DNS 标签）。如果 inbound 设置了单独的备注，列表中显示为「tag (remark)」，否则仅显示标签；已保存的规则中仍仅存储标签。
出站标签	出站标签 / 出站连接	outboundTag	将匹配流量发送到目标。
负载均衡器标签	负载均衡器标签 / 负载均衡器	balancerTag	提示：「通过已配置的负载均衡器之一路由流量」。

outboundTag 与 balancerTag 互斥：「不能同时使用 balancerTag 和 outboundTag。同时使用时，仅 outboundTag 生效。」一条规则中只需设置出站标签或负载均衡器标签其中之一。

参考模板的内置规则

标准 config.json 的 routing 部分包含三条规则（按此顺序）：

1. inboundTag: ["api"] → outboundTag: "api" — 面板统计 gRPC API 的服务规则。
2. ip: ["geoip:private"] → outboundTag: "blocked" — 封锁私有地址段。
3. protocol: ["bittorrent"] → outboundTag: "blocked" — 封锁 BitTorrent。

api → api 规则在保存时始终自动提升至位置 0（参见 11.11），以防统计请求被上方的 catch-all 规则「吃掉」。

规则示例。 将所有发往俄罗斯网站和私有网络的流量直连（绕过代理），其余发往负载均衡器。顺序很重要：「直连」规则必须位于 catch-all 规则之上。在 routing.rules 中：

```
{
  "type": "field",
  "domain": ["geosite:category-ru", "domain:example.ru"],
  "ip": ["geoip:ru", "geoip:private"],
  "outboundTag": "direct"
}
```

要使 IP 规则（`geoip:ru`）对域名请求也能生效，通常需要在路由顶层设置 `routing.domainStrategy: "IPIfNonMatch"`（参见 11.2）。

预配置路由组（基本连接）

在「基本连接」模式下，面板可帮助从预设列表组建典型规则：

组	字段	提示
按协议/网站封锁	—	「配置此项以防止客户端访问特定协议」
按国家封锁	封锁的 IP 地址、封锁的域名	「这些设置将根据目标国家封锁流量。」
直连	直连 IP 地址、直连域名	「直连意味着特定流量不会通过其他服务器转发。」
IPv4 规则	—	「这些设置将让客户端仅通过 <i>IPv4</i> 路由到目标域名」
WARP 规则	—	「这些选项将根据特定目标通过 <i>WARP</i> 路由流量。」
NordVPN 路由	—	「这些选项将根据特定目标通过 <i>NordVPN</i> 路由流量。」

MTProto inbound：通过 Xray 路由 Telegram 流量

MTProto inbound 有一个「Route through Xray」切换开关（默认关闭）和可选的 **Outbound** 选择。启用后，面板会在 Xray 配置中添加一个带有 inbound 自身标签的回环 SOCKS 桥接，mtg 通过该桥接路由 Telegram 流量。此后，Telegram 出站流量由路由器管理：可在「路由」选项卡中通过 inbound 标签用普通规则匹配，或通过 **Outbound** 字段强制发送到指定 outbound 或负载均衡器。将 **Outbound** 留空则由路由规则决定。

11.4. Outbounds（出站连接）

`outbounds` 列表。按钮：创建出站连接、编辑出站连接。提示：「修改配置模板以定义该服务器的出站连接」。

参考模板包含两个必需的 outbound：

- `protocol: "freedom", tag: "direct"` — 直接出口到互联网（含 `domainStrategy: "AsIs"` 和 `finalRules: [{action: "allow"}]`）；
- `protocol: "blackhole", tag: "blocked"` — 丢弃被封锁流量的「黑洞」。

Outbound 表单通用字段

字段	标签	描述
标签	标签（提示：「唯一标签」）	outbound 的唯一标识符。占位符：「 <i>unique-tag</i> 」。验证：「标签为必填项」、「标签已被其他出站连接使用」。
协议	—	出站类型（见下文）。
地址 / 端口	地址 / 端口	连接目标。地址和端口均为必填项。
发送方式	发送方式	出站接口的本地 IP 地址（ <i>sendThrough</i> ）。占位符：「本地 IP」。
Target Strategy	Target Strategy	3.5.0 新增 。目标域名在连接前的解析方式。11 个值： <i>AsIs</i> （默认，不解析）、 <i>UseIP</i> 、 <i>UseIPv4</i> 、 <i>UseIPv6</i> 、 <i>UseIPv6v4</i> 、 <i>UseIPv4v6</i> （解析并带回退）、 <i>ForceIP</i> 、 <i>ForceIPv6v4</i> 、 <i>ForceIPv6</i> 、 <i>ForceIPv4v6</i> 、 <i>ForceIPv4</i> （要求解析成功）。留空 = <i>AsIs</i> （不写入配置）；对 <i>freedom</i> ，读取该值时会回退到原先的 <i>domainStrategy</i> （为兼容旧核心仍会写入该遗留键）。
Dialer proxy（链式代理）	—	提示：「通过另一个出站（按标签）连接此出站，以建立代理链。留空表示直连。」占位符：「选择用于链式连接的出站」。通过 <i>streamSettings.sockopt.dialerProxy</i> 实现。

Dialer Proxy 下拉列表不仅显示本地 outbounds，还显示订阅中的 outbound 标签——这样也可以通过订阅获取的出口建立链式连接。列表中仍排除 blackhole outbound 和当前正在编辑的 outbound。留空表示直连。

支持的 outbound 协议

表单支持的协议：

- **freedom** — 直连出口。字段：*settings.domainStrategy*、*finalRules*（见下文）、Happy Eyeballs。不支持测试（「*Outbound has no testable endpoint*」）。
- **blackhole** — 丢弃流量。字段：**响应类型**。不支持测试。
- **socks**、**http** — *settings.servers[]* 列表，含 *address / port*；字段：**授权密码**。对于 **http** 协议，在 **Username/Password** 字段下方有一个 **Headers**（请求头）编辑器——用于发送给上游 HTTP 代理的 CONNECT 请求头键值对。这些请求头在重新打开并保存 outbound 后得以保留（此前会丢失）。请注意：仅应用设置级别的请求头（*settings.headers*）；xray-core 会忽略单个服务器级别的请求头。
- **vmess** — *settings.vnext[]*（*address / port*）。
- **vless** — *settings.address / settings.port*。
- **trojan**、**shadowsocks** — *settings.servers[]*。
- **wireguard** — *settings.peers[]* 含 *endpoint*，以及密钥（参见 11.8）。
- **hysteria** — *settings.address / settings.port*（UDP 传输）。

对于 **loopback** 类型的 outbound，可使用 **Sniffing** 块，参数与 inbound 相同：启用、**destOverride**、**Metadata Only**、**Route Only** 以及**排除域名列表**。

在 **Hysteria2** 的 **UDP** 掩码（*FinalMask*）中提供了额外模式。**Salamander** 掩码有一个 **Mode** 选择器，值为 **Salamander** 和 **Gecko**：Gecko 模式为数据包添加随机填充，包含 **Min/Max** 大小字段（*packetSize*，范围

1-2048, 默认 512-1200) ——可防止基于数据包长度的指纹识别。**Realm** (UDP hole-punching) 掩码新增了可选的 **TLS Config** 块, 包含 **Server Name** (SNI)、**ALPN** (h3/h2/http/1.1)、**Fingerprint** (uTLS) 和 **Allow Insecure** 切换开关。

自 3.5.0 起新增一种 **TCP 掩码类型**——**XMC (Minecraft)** (`xmc`)：将流量伪装成 Minecraft 协议。字段：**Hostname** (可选；「握手中模拟的服务器地址」)、**Usernames** (可选列表；「向探测者展示的玩家名；默认情况下核心使用 Dream」) 以及必填的 **Password** (16 个字符, 自动生成, 带 `⌂` 按钮；「混淆密码」)。
重要：**TCP Finalmask** 与 **REALITY** 的组合在保存时会被拒绝——它会在首次连接时使 Xray-core 崩溃 (错误：「Finalmask is not supported with REALITY security...」；见 Xray-core#6453)。UDP 掩码与 REALITY 可以共存；此前保存的错误组合会在生成配置时自动「治愈」 (掩码被丢弃)。

示例：通过上游 SOCKS 建立链式代理。 `outbound upstream` 连接到外部 SOCKS5 代理, 而 `chained` 通过它 (`dialerProxy`) 发送流量, 形成链式连接。在 `outbounds` 中：

```
[
  {
    "tag": "upstream",
    "protocol": "socks",
    "settings": {
      "servers": [{ "address": "203.0.113.10", "port": 1080 }]
    }
  },
  {
    "tag": "chained",
    "protocol": "freedom",
    "streamSettings": {
      "sockopt": { "dialerProxy": "upstream" }
    }
  }
]
```

现在, 带有 `outboundTag: "chained"` 的路由规则将通过 `upstream` 将流量输出到互联网。

从分享链接导入 outbound

可从分享链接 (`vless://`、`vmess://` 等) 导入 outbound。导入时, 链接 `extra=` 块中传递的 **xmux** (XHTTP) 多路复用器设置也会保留：导入后, 其值会填入所创建 outbound 的 **XMUX** 子表单。

Mux (多路复用) 字段

最大并发数、最大连接数、最大复用次数、最大请求数、最大复用秒数、Keep Alive 周期。这些参数配置出站的 mux/XUDP 行为。

Sockopts (套接字选项)

Sockopts 组：**Keep Alive 间隔、Mark (fwmark)、接口、仅 IPv6、接受 proxy protocol、Proxy protocol、TCP 用户超时 (ms)、TCP keep-alive 空闲 (s)**。链式代理的 `dialer-proxy` 也在此设置。

Freedom finalRules（覆盖私有 IP 封锁）

对于 freedom-outbound，提供 **Final Rules** 组：

字段	标签	描述
overrideXrayPrivateIp	覆盖 Xray 默认的私有 IP 封锁	取消 Xray 内置的对私有 IP 出站的限制。
action	动作	allow（如参考模板：finalRules: [{action: "allow"}]）、redirect（Redirect）或其他。
blockDelay	封锁延迟（ms）	丢弃连接前的延迟。
redirect / fragment	Redirect / Fragment	流量重定向和分片动作。

fragment 掩码：逐段 Lengths 和 Delays

在 **fragment** 掩码（FinalMask 中的 fragment 类型，用于 TCP）中，单个 Length 和 Delay 字段被替换为 **Lengths** 和 **Delays** 列表：可为每个分片单独设置长度范围（例如 100-200）和延迟（毫秒，例如 10-20 或 0）。可以添加和删除列表行；之前保存的单个值会自动迁移为单元素数组。

其他表单字段

- **UDP over TCP** 和 **UoT 版本** – 用于类 shadowsocks 协议。
- **无 gRPC 请求头、上行 chunk 大小** – gRPC 传输参数。
- **TLS/uTLS 字段**：验证 peer 名称、Pinned SHA256、Short ID、Vision testpre，占位符「服务器名称」。

出站测试

按钮：测试、全部测试。状态：正在测试连接...、测试成功、测试失败、无法测试出站连接。结果：测试结果，延迟（毫秒）。

两种模式（提示：「TCP：仅快速拨号探测。HTTP：通过 xray 的完整请求。」）：

- **TCP**（mode=tcp）– 简单拨号到 host:port，对所有端点并行执行，超时约 5 秒。仅检查 TCP 可达性，不验证代理协议。对于 freedom/blackhole/标签 blocked 将返回「Outbound has no testable endpoint」。
- **HTTP**（mode=http 或为空）– 启动一个临时 Xray 实例，发起真实的 HTTP 请求（探测 URL = 服务器端 outboundTestUrl），测量实际延迟。权威但开销较大的模式：通过全局锁串行化（「Another outbound test is already running, please wait」）。单次尝试超时 10 秒，结果等待窗口 15 秒（已增加，以避免在慢速或隧道链路上将健康 outbounds 标记为「Failed」）。失败时，真实原因（DNS 错误、连接被拒、截止时间到、TLS 错误等）会写入面板/Xray 日志，超时提示信息指向该日志。
- **Real delay（3.5.0 新增）** – 切换器的第三种模式。使用同一个临时 Xray 实例，但报告的是**「冷」请求的完整耗时（包括隧道建立）**——这一数字接近客户端应用所显示的数值。切换器提示：「TCP：仅快速拨号探测。HTTP：通过 xray 的完整请求。Real delay：包括连接建立在内的完整耗时」。自 3.5.0 起，普通 **HTTP** 模式按「热」（keep-alive）连接测量，因此其数值更低，更接近单次请求的延迟。

UDP 协议（wireguard、hysteria）和 UDP 传输（kcp、quic、hysteria）始终以 HTTP 模式测试，即使请求的是 TCP——裸 UDP 拨号无法区分「存活」端点和「失效」端点。wireguard 在测试配置中强制设置 `noKernelTun: true`。

Egress 与 Country 列（自 3.5.0 起）

HTTP 或 Real delay 测试成功后，面板会通过同一临时路由请求 Cloudflare trace 元数据，并在 outbound 列表中显示两列：**Egress**——出口 IP（IPv4/IPv6，均藏在「眼睛」图标后；尚未测试时显示短横线并提示「执行 HTTP 测试以显示出口 IP 和国家」）；**Country**——出口国家的旗帜和名称；若出口经由 Cloudflare WARP，还会附加橙色 WARP 标记。TCP 测试不会填充这两列；在移动端，这些数据显示在 outbound 卡片中。

批量检测与阶段分解

HTTP 模式下，**测试**和**全部测试**为一批 outbound 启动一个公共临时 Xray 实例，为每个 outbound 创建一个回环 SOCKS inbound 和规则，并通过它并行发送真实 HTTP 请求；**全部测试**按批次检测 outbound。全部测试也会检测从订阅获取的 outbound（只读的「来自订阅」表格）——其行也会以测试结果高亮显示。同时，freedom（「direct」）和 dns outbound 在任何模式下均不会被测试（它们不是代理）：其测试按钮不可用，全部测试会跳过它们，服务端保护也会禁止直接调用 API 对其进行 HTTP 测试。除成功/失败外，弹出结果还显示 HTTP 响应状态码和各阶段耗时分解：**Proxy connect**（连接代理）、**TLS via outbound**（通过 outbound 的 TLS）和 **First byte**（首字节时间）——有助于定位延迟或故障发生在哪个环节。

Outbound 流量统计

面板按标签统计流量计数（up/down/total）。重置按钮可重置特定标签或所有标签（tag = "alltags-"）的计数器。账户信息和出站连接状态字段显示摘要。

11.5. 负载均衡器（Balancers）

routing.balancers 列表。按钮：创建负载均衡器、编辑负载均衡器。

「负载均衡器」选项卡有实时状态列：**Live Target** 显示当前在运行中 Xray 里负载均衡器的活跃目标，**Override** 允许手动覆盖目标选择（值 **Auto (strategy)** 恢复按策略选择）。状态通过单独按钮刷新。如果负载均衡器尚未在运行中的 Xray 中激活，面板将提示先保存更改或启动 Xray。

字段	标签	描述
标签	标签（提示：「唯一标签」）	唯一标识符。占位符：「unique-balancer-tag」。验证：「标签为必填项」、「标签已被其他负载均衡器使用」。
选择器	选择器	outbound 标签列表（按子串匹配），负载均衡器从中选择出口。至少选择一个：「请至少选择一个出站连接」。

字段	标签	描述
Fallback	Fallback	当没有选择器匹配时的备用标签。自 3.5.0 起也可以选择另一个负载均衡器（列表中此类选项带蓝色「Balancer」标记，提示「选择另一个负载均衡器作为 fallback」）。Xray 原生不支持此功能，因此面板会自动构建隐藏的 loopback outbound <code>_bl_<[?]></code> 和一条路由规则（路径：负载均衡器 → loopback → 服务器 → 目标负载均衡器 → outbound；表单中会显示关于额外「跳数」的警告）。具有循环保护（错误「无法选择——会产生循环依赖」，列表中的该选项被禁用并提示循环路径），并防止删除正在使用的负载均衡器（「无法删除——正被以下负载均衡器用作 fallback: ...」）。标签前缀 <code>_bl_</code> 已被保留（标签中禁止使用）；服务性的 <code>_bl_</code> 对象从 Outbounds/Routing 列表中隐藏，Fallback/Live Target/Override 列显示目标负载均衡器的名称而非 loopback 标签。
策略	策略	选择算法（见下文）。

策略与观察参数

策略（`strategy.type`）决定负载均衡器如何从选择器中选择 outbound。Xray-core 的值：`random`（随机）、`roundRobin`（轮询）、`leastPing`（observatory 测量的最低延迟）、`leastLoad`（最低负载）。`leastLoad` / `leastPing` 使用 `strategy.settings` 中的参数：

字段	标签	描述
<code>expected</code>	Expected	占位符：「最优节点数」——目标存活节点数。
<code>maxRtt</code>	最大 RTT	筛选候选节点时允许的最大 RTT 上限。
<code>tolerance</code>	容差	比较延迟/负载时的容差。
<code>baselines</code>	Baselines	用于节点分组的延迟阈值。
<code>costs</code>	Costs	各标签的权重系数（cost）。

策略示例。 `strategy` 块位于负载均衡器内部（JSON 中与 `tag` 和 `selector` 并列）：

```
"strategy": { "type": "random" }      // [?][?][?][?][?][?][?][?]
"strategy": { "type": "roundRobin" } // [?][?][?][?][?][?][?]
"strategy": { "type": "leastPing" }  // [?][?][?][?][?][?][?][?][?][?]
```

`leastLoad` 的参数在 `settings` 中设置：

```

"strategy": {
  "type": "leastLoad",
  "settings": {
    "expected": 2,
    "maxRTT": "1s",
    "tolerance": 0.05,
    "baselines": ["500ms", "1s", "2s"],
    "costs": [
      { "regex": false, "match": "proxy-premium", "value": 0.1 },
      { "regex": true, "match": "^proxy-cheap-.+$", "value": 5 }
    ]
  }
}

```

实际运作方式（示例）。 假设观察器测得各出口延迟：A = 250 ms、B = 280 ms、C = 700 ms、D = 1500 ms。使用上述配置，选择过程如下：

1. **maxRTT: "1s"** – 延迟超过 1 秒的出口被淘汰：D (1500 ms) 退出。剩余 A、B、C。
2. **baselines + expected** – 出口按延迟阈值分组，选取包含不少于 expected 个出口的最小阈值。阈值 500ms 已包含 A 和 B ——共 2 个 (= expected)，因此选择组 {A、B}。C (700 ms) 不参与选择，只要快速节点数量足够（它是「热备」）。
3. **tolerance: 0.05** – 在选定组内，延迟差异不超过 5% 的出口视为等价，负载均等分配。A (250) 和 B (280) 差约 12% (> 5%)，因此其他条件相同时优先选更快的 A；若差异在 5% 以内，则流量同时经由 A 和 B。
4. **costs** – 在比较之前调整各出口的「成本」：value 越小出口越优先，越大越靠后。本例中 proxy-premium 获得 0.1（变得「便宜」，更频繁被选中），而所有 proxy-cheap-*（通过正则表达式，regex: true）获得 5（变得「昂贵」，最后才使用）。这样可以在不硬性排除的情况下软性优先化出口。

结论：流量主要经由 A（延迟接近时与 B 均等分配），C 保留为备用，D 被排除直到其 RTT 降至 maxRTT 以下。

观察器：observatory 与 burstObservatory（leastPing/leastLoad 的测量数据）

leastPing 和 leastLoad 策略本身不进行测量——它们需要每个 outbound 的延迟和可用性数据。这些数据由观察器（observatory）收集：它定期「ping」每个被监控的 outbound，并记录响应时间和可用性。相同数据显示在「观察器」选项卡（状态 活跃 / 不可用、「最后活跃时间」、「最后尝试时间」）。

自 3.4.2 起，「负载均衡器」页面拆分为**「均衡器设置」（表格和添加）和「Observatory」两个选项卡，观察器改用结构化表单编辑（原先的原始 JSON 编辑器已移除）。面板会按需自动创建和删除观察器：普通 observatory ——用于 Least Ping 策略，扩展的 burstObservatory ——用于 Least Load，以及用于带指定 fallbackTag 的 Random/Round-robin**（这是 xray-core 所要求的；清空最后一个 fallbackTag 时观察器会被移除）。创建或删除观察器需要完整重启 Xray。

有两种变体：

- **observatory** – 简单版：subjectSelector + probeURL + probeInterval。
- **burstObservatory** – 高级版，通过 pingConfig 进行精细 ping 配置；适用于多个出口。

burstObservatory 块示例：

```
{
  "subjectSelector": ["WS-SE", "WS-FR", "WS-PL"],
  "pingConfig": {
    "destination": "https://www.google.com/generate_204",
    "interval": "1m",
    "connectivity": "http://connectivitycheck.platform.hicloud.com/generate_204",
    "timeout": "5s",
    "sampling": 2
  }
}
```

字段说明：

字段	作用
subjectSelector	要观察的 outbound 标签前缀列表。Xray 选取所有标签以指定字符串开头的 outbound。本例中观察 WS-SE...、WS-FR...、WS-PL... 出口。这些标签须与负载均衡器的选择器中选定的标签一致。
pingConfig.destination	通过每个 outbound 请求以测量延迟的 URL。通常使用返回 204 无正文的「轻量」页面——例如 https://www.google.com/generate_204。响应时间即为测量延迟。
pingConfig.interval	每个 outbound 的 ping 频率。时长字符串：“1m”为每分钟，也可用“30s”、“5m”等。频率越高数据越新，但后台流量越多。
pingConfig.connectivity	（可选）检查服务器基本连通性的 URL。若不可达——说明是服务器网络问题，观察器不会将 outbound 标记为不可用（防止本地故障时的误报）。通常也是返回 204 的端点。
pingConfig.timeout	单次 ping 等待响应的时长，超时则视为本次尝试失败（例如“5s”）。
pingConfig.sampling	每个 outbound 保留并平均的最近测量次数。2 表示计入最近两次 ping（平滑随机抖动）。
pingConfig.httpMethod	3.4.2 新增。探测的 HTTP 方法：HEAD（默认）或 GET。

在**「Observatory」选项卡上，这些参数通过表单（而非 JSON）设置。对于普通 observatory，包括 Watched Outbounds（subjectSelector，只读）、Probe URL（probeURL，默认 https://www.google.com/generate_204）、Probe Interval（probeInterval，1m）以及 Enable Concurrency（enableConcurrency，默认开启）。对于 burstObservatory——上述字段，外加新增的「HTTP 方法」**。如果某个均衡器同时有两种观察器，Observatory 和 Burst 表单之间可通过分段切换器切换。

如何将所有内容关联起来：

1. 在**「Observatory」**选项卡上配置观察器参数（它会根据所选策略自动创建；subjectSelector 跟随均衡器的选择器）。
2. 创建负载均衡器：策略 = leastPing，在选择器中指定相同的 outbound 标签（WS-SE、WS-FR、WS-PL）。
3. 通过路由规则将流量导向它（字段 负载均衡器标签，参见 11.3）。
4. 重启 Xray。「观察器」选项卡将显示出口状态，负载均衡器将开始选择存活节点中最快的一个。

一条规则中不能同时设置 `balancerTag` 和 `outboundTag` ——仅 `outboundTag` 生效。

删除 outbound 或负载均衡器时会发生什么（自 3.4.2 起）

删除 outbound 或负载均衡器时，面板会在同一操作中清理路由中对它的引用，并在确认对话框中显示影响预览（标题「删除时还将更新您的路由：」，随后按每条规则显示——「已删除（无剩余目标）」或「已保留（现在使用 ...）」，以及「负载均衡器 ... 已删除（无剩余目标）」）。这样可避免悬空引用

（`balancerTag` / `outboundTag` / `dialerProxy`），此前这类引用可能导致 xray-core 在下次重启时无法启动。删除负载均衡器：规则保留各自的 `outboundTag`，否则被删除。删除 outbound：其标签会从负载均衡器的选择器和 `fallbackTag` 中移除，清除其他 outbound 的 `dialerProxy`，已清空的负载均衡器连同现已失去对象的规则一并删除，观察器重新构建。

11.6. DNS

`dns` 部分。启用：启用 DNS（提示：「启用内置 DNS 服务器」）。

自 3.5.0 起，私有 IP 上的 DNS 服务器（例如同一 docker 网络中自建的 AdGuard/Pi-hole）「开箱即用」：面板会自动在拦截规则 `geoip:private` 之前插入并维护一条受管的放行规则（`direct`，且仅限该 DNS 服务器的端口），并在 `dns.servers` 变化时同步该规则。此前 Xray 自身的 DNS 查询会被该规则静默拦截，为每个新域名增加约 4 秒延迟；不再需要手动规则（手动创建的规则不受影响）。

DNS 通用参数

字段	标签	JSON	描述/提示
<code>tag</code>	DNS 标签名	<code>dns.tag</code>	「此标签将在路由规则中作为入站标签使用。」允许通过 <code>inboundTag</code> 路由 DNS 请求本身。
<code>clientIp</code>	客户端 IP	<code>dns.clientIp</code>	「用于在 DNS 请求期间通知服务器指定的 IP 位置」（EDNS Client Subnet）。
<code>strategy</code>	查询策略	<code>dns.queryStrategy</code>	「域名解析的通用策略」。值： <code>UseIP</code> 、 <code>UseIPv4</code> 、 <code>UseIPv6</code> 。
<code>disableCache</code>	禁用缓存	<code>dns.disableCache</code>	「禁用 DNS 缓存」。
<code>disableFallback</code>	禁用备用 DNS	<code>dns.disableFallback</code>	「禁用备用 DNS 查询」。
<code>disableFallbackIfMatch</code>	匹配时禁用备用 DNS	<code>dns.disableFallbackIfMatch</code>	「当 DNS 服务器的域名列表匹配时禁用备用 DNS 查询」。

字段	标签	JSON	描述/提示
enableParallelQuery	启用并行查询	–	「向多个服务器启用并行 DNS 查询以加快解析速度」。
useSystemHosts	使用系统 Hosts	dns.useSystemHosts	「使用已安装系统的 hosts 文件」。

dns 块示例。 Google 域名的请求通过 Cloudflare DoH 服务器解析，其余通过 1.1.1.1；对 Google 请求仅接受非私有 IP 的响应。在配置顶层：

```
"dns": {
  "tag": "dns-inbound",
  "queryStrategy": "UseIPv4",
  "servers": [
    {
      "address": "https://cloudflare-dns.com/dns-query",
      "domains": ["geosite:google"],
      "expectIPs": ["geoip:!private"]
    },
    "1.1.1.1"
  ]
}
```

不带字段的字符串服务器（"1.1.1.1"）是所有其他域名的默认服务器。随后可将标签 dns-inbound 用作路由规则中的 inboundTag，将 DNS 请求本身路由到所需 outbound。

过期记录缓存

字段	标签	描述
serveStale	使用过期记录	「在后台更新期间从缓存返回过期结果」。
serveExpiredTTL	过期 TTL	「缓存过期记录的有效期（秒）；0 = 永久有效」。

DNS 服务器（dns.servers 列表）

按钮：创建 DNS、编辑 DNS、全部删除（确认：「所有 DNS 服务器将从列表中删除。此操作无法撤销。」）。模板：使用模板，DNS 模板窗口，包含 家庭 预设。

点击 DNS 服务器记录（以及 Fake DNS 记录）上的 编辑 DNS 时，编辑窗口会填入已保存的服务器值，而非默认值。

DNS 服务器字段：

字段	标签	描述
address	–	DNS 地址（IP、DoH URL、localhost、fakedns 等）。
domains	域名	使用此服务器的域名列表。
expectIPs	期望 IP	仅在 IP 符合列表时接受响应。

字段	标签	描述
<code>unexpectedIPs</code>	非期望 IP	丢弃含指定 IP 的响应。
<code>skipFallback</code>	跳过 Fallback	不将此服务器用作 fallback。
<code>finalQuery</code>	最终查询	将此服务器标记为链中的最终服务器。
<code>timeoutMs</code>	超时 (ms)	服务器请求超时。

Hosts（静态记录）

Hosts 组（`dns.hosts`）。按钮 **添加 Host**；空状态 **未定义 Host**。字段：域名（占位符：「域名（如 `domain:example.com`）」）和值（占位符：「IP 或域名——输入后按 `Enter`」）。

DNS 日志

参见 [11.10](#)：日志记录部分中的 **DNS 日志** 标志（`dnsLog`）。

11.7. Fake DNS

`fakedns` 部分。按钮：**创建 Fake DNS**、**编辑 Fake DNS**。

字段	标签	描述
<code>ipPool</code>	IP 池子网	用于分配虚假 IP 的 CIDR 范围（例如 <code>198.18.0.0/15</code> ）。
<code>poolSize</code>	池大小	环形池中保留的地址数量。

Fake DNS 与 inbound 上的嗅探配合使用：核心为客户端分配虚假 IP，记录域名↔IP 映射，并在路由时还原域名。要使 Fake DNS 生效，地址为 `fakedns` 的 DNS 服务器必须添加到 DNS 服务器列表中。

示例：Fake DNS + DNS 服务器组合。 首先定义虚假地址池，然后添加 `fakedns` DNS 服务器，使域名查询从该池中获取 IP：

```
"fakedns": [
  { "ipPool": "198.18.0.0/15", "poolSize": 65535 }
],
"dns": {
  "servers": [
    { "address": "fakedns", "domains": ["geosite:geolocation-!cn"] },
    "1.1.1.1"
  ]
}
```

此外，inbound 上还需要启用包含 `destOverride: ["fakedns"]` 的嗅探，否则核心无法获取还原用的真实域名。

11.8. WireGuard / WARP / NordVPN

WireGuard 字段 (wireguard)

字段	标签	描述
secretKey	私钥	本地接口的私钥。
publicKey	公钥	peer 的公钥。
psk	预共享密钥	PreShared Key (可选)。
allowedIPs	允许的 IP 地址	路由到隧道的地址范围。
endpoint	端点	peer 的 host:port。
domainStrategy	域名策略	WireGuard outbound 的解析策略。

Cloudflare WARP (warp)

集成使用 API <https://api.cloudflareclient.com/v0a4005> (client-version a-6.30-3596)。控制器动作 (/xray/warp/:action)：config、reg、license、data、del。

步骤：

1. 创建 WARP 账户 → reg：面板生成/接受私钥 (privateKey) 和公钥 (publicKey)，在 Cloudflare 注册设备，并将 access_token、device_id、license_key、private_key (以及 client_id) 保存到 warp 设置中。
2. WARP / WARP+ 许可证密钥 → license：设置 26 字符 WARP+ 密钥 (占位符：「26 字符 WARP+ 密钥」)。出错时：「设置 WARP 许可证失败。」若尚未获取配置：「请先获取 WARP 配置。」
3. 账户信息：设备名称、设备型号、设备已启用、账户类型、角色、WARP+ data、配额、使用量。
4. 添加出站连接 – 使用获取的密钥和 Cloudflare 端点创建 WireGuard outbound。
5. 删除账户 → del：清除已保存的 WARP 数据。

NordVPN (nord / nordvpn)

集成使用 NordLynx (= WireGuard)。控制器动作 (/xray/nord/:action)：countries、servers、reg、setKey、data、del。

步骤：

1. 访问令牌 → reg：面板向 api.nordvpn.com 请求 NordLynx 凭据并提取 nordlynx_private_key。将 private_key 和 token 保存到 nord 设置中。替代方案——setKey：直接输入私钥 (不能为空)。
2. 国家 → countries 加载国家列表；城市 (或所有城市)。
3. 服务器 → servers 加载所选国家的服务器 (countryId 验证为数字——防止注入)。过滤：仅显示负载 > 7% 的服务器。若无服务器：「未找到所选国家的服务器」。若服务器无 NordLynx 公钥：「所选服务器未提供 NordLynx 公钥。」
4. 创建/更新出站连接：提示消息「NordVPN 出站连接已添加」 / 「NordVPN 出站连接已更新」。

IPv4 优先与用户空间 TUN

WARP 和 NordVPN 向导生成的 WireGuard outbounds 使用 `domainStrategy: "ForceIPv4v6"`（IPv4 优先，在纯 IPv6 主机上回退到 IPv6），而非 `ForceIP`——这解决了在 IPv6 配置不完整的主机上选择 Cloudflare 端点 AAAA 记录导致握手「卡住」的问题。此外，为其启用了用户空间 TUN（`noKernelTun: true`）而非内核 TUN：后者需要权限和 fwmark 路由，在许多 VPS 上会静默失败，而面板的内置连接检测始终通过用户空间 TUN 进行测试——现在实际流量和检测走同一路径。此变更仅对新添加或重置的 outbounds 生效；已保存的模板保留其原有设置。

11.9. Reverse 代理与 TUN

Reverse（反向代理）

Xray 配置的 `reverse` 部分。outbound 表单中有切换到 **反向代理** 类型的开关。按钮：**创建反向代理**、**编辑反向代理**。

字段	标签	描述
类型	类型	Bridge 或 Portal – Xray 反向代理的两种角色。
域名	域名	bridge↔portal 对的服务域名标签。
标签 / 连接	标签 / 连接	用于绑定 bridge 和 portal 的标签。
Reverse Tag	反向代理标签	提示：「简单 VLESS 反向代理的出站连接标签。留空则禁用。」占位符：「出站标签（空 = 禁用）」。实现简化版 VLESS 反向代理。

outbound 表单中还包含反向流字段：**反向嗅探**、**工作线程**、**保留**、**最小上传间隔（ms）**、**最大上传大小（字节）**。

TUN（tun）

字段	标签	描述	默认值
name	—	「TUN 接口名称。」	<code>xray0</code>
mtu	—	「最大传输单元。数据包的最大大小。」	1500
<code>userLevel</code>	用户级别	「通过此入站建立的所有连接将使用此用户级别。」	0

11.10. 日志与统计（Stats, metrics）

日志（log）

提示：「日志可能会降低服务器速度。请仅在需要时启用所需类型的日志！」参考模板的 `log` 部分：`access: "none"`、`error: ""`、`loglevel: "warning"`、`dnsLog: false`、`maskAddress: ""`。

字段	标签	JSON	描述	默认值
logLevel	日志级别	loglevel	「错误日志的日志级别.....」 值: debug、info、warning、error、none。	warning
accessLog	访问日志	access	「访问日志文件路径。特殊值「none」禁用访问日志。」	none
errorLog	错误日志	error	「错误日志文件路径。特殊值「none」禁用错误日志。」	"" (默认)
dnsLog	DNS 日志	dnsLog	「启用 DNS 查询日志」	false
maskAddress	地址掩码	maskAddress	「启用后, 日志中的真实 IP 地址将替换为掩码地址。」	"" (关闭)

统计 (stats / policy)

统计 组。在 policy.system 和 policy.levels 中启用计数器。参考模板中: statsInboundUplink: true、statsInboundDownlink: true、statsOutboundUplink: false、statsOutboundDownlink: false; 对于级别 0 – statsUserUplink: true、statsUserDownlink: true。

字段	标签	描述	默认值
statsInboundUplink	入站上行统计	「启用对所有入站代理出站流量的统计收集。」	true
statsInboundDownlink	入站下行统计	「启用对所有入站代理入站流量的统计收集。」	true
statsOutboundUplink	出站上行统计	「启用对所有出站代理出站流量的统计收集。」	false
statsOutboundDownlink	出站下行统计	「启用对所有出站代理入站流量的统计收集。」	false

按客户端和 inbound 统计（上行/下行）是仪表板和客户端流量显示的基础；不建议禁用。出站统计默认关闭，仅在需要按出站标签跟踪流量时才需启用。

Metrics

参考模板中包含 metrics 部分 (listen: "127.0.0.1:11111"、tag: "metrics_out") 及相应的 API metrics_out。面板使用此监听器收集指标和 observatory 快照: 它从模板解析 metrics.listen, 查询 / debug/vars 并按标签聚合延迟历史。如果更改 metrics.listen 的地址/端口, 面板将访问新地址; 删除 metrics 部分将禁用 observatory 图表收集。

HTTP 模式的 outbound 测试会启动一个**独立的临时** Xray 实例, 其 metrics 监听器使用随机端口——与主配置中的监听器不同。

11.11. 保存、重启与自动转换

按钮

按钮	动作
保存	POST /xray/update : 验证并保存模板 + <code>outboundTestUrl</code> 。
重启 Xray	使用已保存的配置重新加载服务。确认：「重启 xray?」 / 「使用已保存的配置重新加载 xray 服务。」

提示消息：成功——「Xray 重启成功」、「Xray 停止成功」；错误——「重启 Xray 时发生错误。」、「停止 Xray 时发生错误。」 **Xray 重启**输出窗口显示核心的诊断输出。

热应用更改（无需完全重启）

inbounds、outbounds 和路由规则的更改会「实时」应用：点击**保存**时，面板计算新旧配置之间的差异，并仅通过 Xray 的 gRPC API (HandlerService/RoutingService) 应用变更的部分，无需重启进程。仅当没有热重载 API 的部分发生变化时（`log`、`dns`、`policy`、`observatory` 等），才会自动执行完整重启。因此，Xray 页面上不需要单独的「重启」按钮——**保存**本身即可应用更改。必要时核心重启仍会自动执行（另见订阅更新和 WARP 轮换时的自动重载）。

恢复默认模板

端点 GET /xray/getDefaultJsonConfig 返回参考模板（内置于二进制文件的 `config.json`）。可用其将配置重置为出厂设置。

保存时的自动转换

保存 Xray 设置时，面板按以下顺序执行：

- 剥除包装** – 剥除 { "xraySetting": <?>, "inboundTags": ..., "outboundTestUrl": ... } 形式的包装（若它们意外出现在值中，否则每次保存都会叠加层数）。最多剥除 8 层。
- 配置验证** – 将 JSON 解析为 Xray 配置结构；出错则拒绝并显示「xray template config invalid」。
- 保证统计规则** – 将规则 inboundTag: ["api"] → outboundTag: "api" 强制提升到 routing.rules 的位置 0（或在缺失时添加）。这确保面板 gRPC 统计请求不会被上方的 catch-all 规则截获（否则在代理正常运行时客户端可能显示为离线且流量为零）。

由于第 3 条，请勿尝试删除或移动 api → api 规则——面板在下次保存时会将其恢复原位。这是统计基础设施的服务规则，而非用户路由。

11.12. 订阅 outbound（自动更新）

从 3.3.0 版本起，面板可直接从订阅 URL 导入 `outbound` ——格式与 VPN 提供商为客户端应用提供的格式相同。订阅在后台定期重新读取，因此服务器上的 `outbound` 集合无需手动编辑配置模板即可保持最新。自

3.5.0 起，带 query 参数（`?plugin=`、`?type=`）或结尾 `/` 的 `ss://` 链接（SIP002）能正确解析端口（此前端口会静默变为 `0`）；端口确实无效的链接会被跳过，而不会以损坏的状态导入。

该部分名为「出站订阅」，描述：「从远程订阅 URL 导入出站连接（vmess/vless/trojan/ss/...）。标签保持不变，可用于负载均衡器和路由规则。自动更新。」该部分位于 Xray 页面上 `outbound` 设置面板的上方。

工作原理

订阅与 Xray 配置模板分开存储。模板**永不被覆盖**：从订阅获取的 `outbound` 在每次生成 Xray 配置时动态添加到最终配置中。

添加订阅

「添加订阅」表单包含以下字段：

字段	键	默认值	用途
订阅 URL	<code>url</code>	–（必填）	订阅地址。占位符：「 <code>https://...</code> （base64 编码的链接列表）」。 仅接受 HTTP(S)；地址经过安全性验证。
备注	<code>remark</code>	空	任意标签（占位符「例如 HK 节点」）。
标签前缀	<code>tagPrefix</code>	<code>subN-</code>	导入的 <code>outbound</code> 标签的前缀。若留空，面板自动选取最小可用编号，格式为 <code>sub1-</code> 、 <code>sub2-</code> 等。
更新间隔	<code>updateInterval</code>	600 秒（10 分钟）	订阅重新读取的频率。UI 中以小时/分钟设置。
已启用	<code>enabled</code>	是（ <code>true</code> ）	仅启用的订阅会进入配置并自动更新。
允许私有地址	<code>allowPrivate</code>	否（ <code>false</code> ）	允许 localhost、局域网和私有 IP 的 URL。默认关闭以防 SSRF——仅对可信的本地来源启用。
置于手动出站之前	<code>prepend</code>	否（ <code>false</code> ）	启用后，此订阅的 <code>outbound</code> 将放置在模板手动 <code>outbound</code> 之前，其中一个可成为默认 <code>outbound</code> 。否则添加到之后。

「预览」按钮（`POST /outbound-subs/parse`）允许在保存前下载并解析 URL，查看将获得哪些 `outbound` 和标签；此操作不写入数据库。若 URL 未识别到任何内容，则显示「未在此 URL 找到出站连接。」

多个订阅在 `outbound` 总列表中的顺序由优先级（`priority`）决定，通过上/下箭头（`POST /outbound-subs/:id/move`）更改。

支持的订阅格式

URL 响应正文的处理方式：

- 内容首先尝试作为 **base64** 解码（标准和 URL-safe 变体，自动补全填充并去除空格/换行）。若为 base64 则解码；否则原样使用。
- 然后将正文按行拆分。每个不以 `#` 开头的非空行均作为链接解析。无法识别的行（注释、不支持的协议）静默跳过。

- 支持的链接格式：`vmess://`、`vless://`、`trojan://`、`ss://`（Shadowsocks）、`hysteria2://` / `hy2://`、`wireguard://` / `wg://`。

即支持大多数提供商使用的「base64 编码链接列表」标准订阅格式。

稳定标签

每个链接计算一个稳定的「标识」（不含片段备注的 URI 核心；vmess 为不含 `ps` 字段的内部 JSON）。

「标识→标签」的映射被保留，因此下次更新时同一服务器获得相同标签，即使备注或次要参数发生变化。这是专门为确保负载均衡器和路由规则在更新后继续正常工作而设计的：

- 负载均衡器/规则中的精确标签将继续指向同一服务器。
- 前缀/通配符选择器（例如 `hk-*`）将自动包含订阅后来返回的新服务器——这是「订阅节点池」的推荐方式。
- 若服务器从订阅中消失，其标签将从最终 `outbound` 数组中移除；若负载均衡器有 `fallbackTag`，Xray 将使用它。
- 若提供商更改了服务器的 UUID/主机/凭据，标识发生变化——这被视为带有新标签的新 `outbound`。

单次获取内部的标签通过后缀 `-N` 去重。订阅中的标签保留非 ASCII 字符（例如西里尔字母）并保持可读性：Unicode 字母和数字在 slug 中保留，标点符号替换为连字符——西里尔文名称的标签不再简化为纯数字。

自动更新工作原理

- 订阅更新的后台任务**每 5 分钟**按计划运行一次。
- 每次运行时，它遍历所有启用的订阅，仅更新那些自身间隔已到期的订阅：若订阅尚未更新过，或距上次更新已超过其 `updateInterval`，则更新该订阅。这样任务频繁检查订阅，但每个具体订阅读取频率不超过其 `updateInterval`（默认 10 分钟）。UI 中有相应提示说明这一点。
- 更新过程：URL 再次验证为公开（私有地址被封锁，除非订阅设置了 `allowPrivate`），请求通过面板代理客户端发出，携带请求头 `User-Agent: 3x-ui-outbound-sub/1.0`。重定向链限为 10 跳，每跳也检查是否为私有地址（SSRF 防护）。期望 HTTP 200；否则记录错误。
- 成功解析后，结果被保存，更新时间被记录，错误被清除。出错时，错误文本在 UI 中显示为「最后错误」，之前获取的 `outbound` 继续有效。
- 若至少有一个订阅实际更新，任务将 Xray 标记为需要重启，并发送 UI 失效通知，以便界面拉取新的 `outbound`。Xray 的实际重载发生在管理器下一个 30 秒周期。

手动更新单个订阅——按钮「**立即更新**」（`POST /outbound-subs/:id/refresh`）；它也会将 Xray 标记为需要重启。添加、修改、删除订阅同样会触发 Xray 重启标志（删除时其 `outbound` 在下次重载时从配置中移除）。UI 提示：「添加或更新后，请重启 Xray（或等待下次自动重载），以使出站连接生效。」

如何进入 Xray 配置

每次生成 Xray 配置时，活跃的订阅 `outbound` 被分为两组——`prepend`（「置于手动出站之前」标志）和其余——并与模板拼接：`[?]?prepend [?]` + `[?]?outbound` + `[?]?[?]`。各组内订阅按优先级排序。模板中的手动 `outbound` 不受影响；若模板的 `outbound` 数组因故无法解析，订阅 `outbound` 不会混入（以免丢失手动出站）。

导入的 `outbound` 还会在 `outbound` 面板中以单独的「来自出站订阅（只读）」块显示——无法在此编辑，仅可通过「出站订阅」部分管理。

11.13. WARP IP 轮换

在 3X-UI 中可以建立 WARP outbound——到 Cloudflare WARP 的 WireGuard 出站连接（Xray 配置中标签为 `warp`）。面板自动在 Cloudflare 服务器上注册设备账户，获取 WireGuard 密钥和地址，并将其填入标签为 `warp` 的 `outbound` 中。通过此 `outbound`，流量以 Cloudflare WARP 的 IP 地址出口到互联网。3.3.0 版本新增了无需手动重建 WARP 账户即可手动或按计划更换此出站 IP 的功能。

管理位于 **Xray** 部分的 WARP 卡片中（点击「创建 WARP 账户」并获取配置后；此前操作不可用——面板将提示「请先获取 WARP 配置」）。

更换 IP 时发生了什么

「更换 IP」按钮启动 IP 更换。逻辑：

1. 生成新的 WireGuard 密钥对。
2. 使用新密钥在 Cloudflare 服务器上重新注册 WARP 设备（新的 `device_id`、`access_token`、地址和 `peer` 数据）。
3. 新数据写入 Xray 配置的 WARP `outbound`：更新 `secretKey`、`address`（`v4 /32` 和 `v6 /128`）、`reserved`（来自 `client_id`），以及 `peer` 的 `publicKey` 和 `endpoint`。
4. 若之前设置了 WARP+ 许可证密钥（长度不少于 26 字符），它会自动重新应用到新账户。失败仅为日志警告——IP 更换不会取消。
5. 更换成功后，Xray 被标记为需要重启，以使新 `outbound` 生效。

成功时，界面显示「WARP IP 地址更换成功！」。

按计划自动轮换

WARP 卡片中有「自动更新 IP 地址」切换开关和「间隔（天）」字段。提示：「0 表示禁用。自动更换 IP 地址。」

参数	值
数据库中的设置	<code>warpUpdateInterval</code> （整数， ≥ 0 ）
默认值	0（禁用自动轮换）
单位	天
0	禁用自动更换
> 0	每 N 天更换一次 IP

保存间隔时记录 `warpUpdateInterval`，若值大于 0，则将「上次更新时间」重置为当前时刻——否则调度器在下一个 tick 就会立即更换 IP。

计划由每小时运行一次的后台任务执行——即面板每小时检查是否该轮换。检查算法：

- 若间隔 ≤ 0 ——不执行任何操作；
- 若「上次更新时间」为 0（例如通过直接编辑数据库设置了间隔）——这是首次运行：任务仅记录基准时间戳，不立即更换 IP；
- 若距上次更新已过去不少于 24×3600 秒——执行 IP 更换，更新时间戳，并计划重启 Xray。

重要细节：通过「更换 IP」按钮手动更换也会重置上次更新时间戳。因此，手动轮换后，自动间隔重新计时，计划中的更换不会紧随后立即触发。

「通过面板代理」

3.3.1 版本变更。 独立的「面板网络代理」（panelProxy）设置已移除。面板自身的出站流量（包括对 WARP API 的请求）现在通过所选的**面板流量 outbound**——Xray outbound 或负载均衡器——路由（参见 13 部分）。以下说明适用于 3.3.1 之前的版本。

所有对 Cloudflare WARP API 的请求（注册、获取配置、设置许可证、更换 IP）均不直接发出，而是通过超时 15 秒的面板 HTTP 客户端发出。该客户端遵守面板设置中的**「面板网络代理」**（panelProxy）设置。

根据设置说明：代理路由面板自身的出站请求（geo 数据库更新、Xray/面板版本检查、Telegram，现在还有 WARP 请求）——用于绕过服务器过滤。接受 socks5:// 或 http(s):// 格式的地址，例如 Xray 自身的本地 SOCKS inbound。若字段为空或代理设置不正确——使用直连（行为不会中断）。

对 WARP 的好处：若服务器无法直接访问 `api.cloudflareclient.com`，注册和轮换之前会失败。现在，在 panelProxy 中指定可用代理（包括自己的 Xray inbound），即可保证 WARP API 的可达性，以及手动按钮和计划轮换的正常运行。

适用场景

- 定期更换通过 WARP 走的出站 IP——降低因固定地址被封锁或追踪的风险。
- 手动「刷新」IP，当前 Cloudflare 地址被列入黑名单或速度较慢时。
- 无法直接访问 Cloudflare WARP API 的服务器：通过 panelProxy 路由请求使注册和轮换可正常工作。

12. 节点（多面板，master/slave）

节点部分将普通的 3X-UI 安装变为**中心（主）面板**，可远程监控并管理其他（子）3X-UI 面板。每个节点都是运行在独立服务器上的 3X-UI 实例；主面板通过其 HTTP API 与之通信、轮询状态，并将分配给它的 inbound 和客户端同步过去。这就是**多面板**功能：无需逐一登录每个面板，您可以在一个列表中看到所有服务器并进行集中管理。

重要原则：节点不是代理，而是完整的 3X-UI 面板。 主面板不会在节点上“安装”任何内容——它仅通过令牌连接其 API。从列表中删除节点只会停止监控；远程面板本身不受影响（提示：“这将停止对该节点的监控。远程面板本身不会受到影响”）。

12.1. 列表顶部的汇总信息

节点表格上方显示汇总计数器：

字段	描述
节点总数	列表中的节点总数。
在线	状态为 <code>online</code> 的节点数量。
离线	状态为 <code>offline</code> 的节点数量。
平均延迟	到各节点的平均延迟（ping），单位毫秒。

12.2. 添加和编辑节点

添加节点和编辑节点按钮将打开节点表单。

名称、地址、端口和 API 令牌为必填项（提示：“名称、地址、端口和 API 令牌为必填项”）。

点击“保存”时（无论是添加还是编辑），面板都会**首先验证节点的可达性**，超时时间为 6 秒。如果节点未响应，记录不会被保存并显示错误。也就是说，无法添加明显不可达的节点。

表单字段

字段	默认值	允许的值	描述
名称	—（必填）	非空字符串， 唯一	节点的内部名称。名称列具有唯一性约束——无法创建两个同名节点。占位提示： <code>de-frankfurt-1</code> 。保存时会去除首尾空格。
备注	空	任意字符串	可选的节点注释/描述。不影响功能。
协议	<code>https</code>	<code>http</code> / <code>https</code>	连接远程面板的协议。如果留空或指定无效值，规范化将设置为 <code>https</code> 。如果节点使用普通 HTTP 但协议设置为 <code>https</code> ，面板将返回友好提示：“服务器使用的是 HTTP 而非 HTTPS；请将节点协议设置为 <code>http</code> ”。
地址	—（必填）	主机名或 IP	远程面板的地址。占位符： <code>panel.example.com</code> <code>1.2.3.4</code> 。地址会被规范化；为防止 SSRF，默认禁止私有/本地地址——请参阅“允许私有地址”。

字段	默认值	允许的值	描述
端口	- (必填)	整数 1-65535	远程节点的 Web 面板端口。超出范围的值将被拒绝 ("节点端口必须在 1-65535 之间")。
基础路径	/	路径字符串	远程面板的基础路径 (web base path)，如已设置。会被规范化：保证以 / 开头和结尾 (空值 → /)。面板在查询时会在其后追加 panel/api/server/status。
API 令牌	- (必填)	远程面板的令牌	访问节点 API 的 Bearer 令牌。在 Authorization: Bearer <token> 请求头中传递。占位符："来自远程面板设置页面的令牌"。提示："远程面板在设置 → API 令牌部分显示其 API 令牌"。即令牌需在节点本身 (设置 → API 令牌) 创建，然后粘贴到此处。
启用	true	是/否	启用节点的监控和同步。已禁用的节点不会被后台任务轮询 (心跳和流量同步会跳过它们)，也不参与批量面板更新。
允许私有地址	false	是/否	解除 SSRF 保护，允许通过私有/本地地址连接节点。提示："仅为私有网络或 VPN 中的节点启用"。仅在节点确实位于私有网络或可通过 VPN 访问时才启用。

在节点端获取和重新生成令牌

令牌从远程面板的 **设置 → API 令牌** 部分获取。也可在那里重新颁发：点击 **重新生成令牌** 按钮，并附带警告："重新生成将使当前令牌失效。任何使用该令牌的中心面板将失去访问权限，直到更新令牌为止。是否继续？"。重新生成后，主面板中的旧令牌将失效——需在节点表单中更新。

出站连接 (Connection outbound)

Connection outbound (出站连接，outboundTag) 字段指定主面板到该节点 API 的请求如何离开服务器。如果选择 Xray outbound 的标签，面板对节点的请求将不是直连，而是通过指定的 outbound 路由；面板会自动在回环接口上添加桥接 inbound 到运行中的配置并实时应用，无需重启。提示："通过所选的 Xray outbound 路由此节点的面板 API 流量。回环桥接 inbound 会自动添加到运行中的配置并实时应用。留空则直连"。

选择器的工作方式类似面板的 outbound 选择：标签按 **Outbounds** (普通出站) 和 **Balancers** (负载均衡器) 分组，blackhole 出站从列表中隐藏。空值 (占位符"直连") = 直接连接到节点。

导入 inbound (选择要同步的 inbound)

节点表单中有一个 **导入 inbound** (inboundSyncMode) 设置，有两种模式：**所有 inbound** (all, 默认) 和 **选定的** (selected)。默认情况下，主面板将选择了该节点的所有 inbound 同步到节点；现有节点继续在"所有 inbound"模式下工作。

在**选定的**模式下，字段下方会出现 inbound 标签的多选框。点击**加载 inbound**——主面板将根据已输入 (尚未保存) 的连接参数向节点请求其 inbound 列表 (端点 POST /panel/api/nodes/inbounds) 并显示其标签；勾选所需的标签。面板只会将标记的标签同步并部署到节点，而直接存在于节点上的其他 inbound 将保持不变——主面板不会删除或管理它们。

示例：请求节点的 inbound 列表以进行选择性导入。请求体包含尚未保存的连接参数；响应中包含节点上可用 inbound 的标签：

```
POST /panel/api/nodes/inbounds
Content-Type: application/json

{ "name": "de-fra-1", "scheme": "https", "address": "node1.example.com",
  "port": 2053, "basePath": "/", "apiToken": "abcdef..." }
```

12.3. TLS 验证（用于 https 节点）

此字段组定义主面板如何验证节点的 HTTPS 证书。这些设置仅对 **https** 协议有效；对于 **http** 节点将被忽略。

TLS 验证——下拉列表，提示：“面板如何验证节点的 HTTPS 证书。固定或跳过——适用于自签名证书（仅 https 节点）”。

模式	值	默认	描述
验证（标准 CA）	verify	是（默认）	通过受信任的 CA 进行标准证书链验证。适用于具有公共/Let's Encrypt 证书的节点。也用于所有 http 节点。
固定证书（SHA-256）	pin	—	不验证 CA 链，但节点叶证书的 SHA-256 会与保存的指纹进行恒时比较验证。为自签名证书保留 MITM 防护。需要填写指纹字段。
跳过验证	skip	—	完全禁用证书验证。警告：“跳过验证会消除中间人攻击防护——API 令牌可能被截获。建议改为固定证书”。

在 3.4.0 中，在上述三种模式之外增加了第四种模式——**Mutual TLS（客户端证书）**（mtls），与其他模式一样，仅在 **https** 协议下可用。

模式	值	默认	描述
Mutual TLS（客户端证书）	mtls	—	除验证节点证书外，主面板还通过其自身 CA 颁发的客户端证书向节点验证自身身份。在此模式下，API 令牌对节点变为可选——节点通过证书识别主面板。选择此模式时显示提示：“此节点通过客户端证书验证面板身份。将此面板的 CA 从 Node mTLS 部分复制到节点，设置其受信任的父 CA，然后重启节点”。

要为节点启用双向 TLS：在节点端设置 **Mutual TLS** 模式，从 **Node mTLS** 部分（见下文）复制主控面板的 CA，将其作为受信任的父 CA 配置到节点上，并重启节点。

如果选择 **skip**、**pin** 或 **mtls** 之外的任何值，规范化将强制设置为 **verify**。

证书固定

选择固定证书后，将出现：

- **固定证书的 SHA-256**——输入字段。接受 **base64** 格式的指纹（Xray 的 **pinnedPeerCertSha256** 格式）或带冒号或不带冒号的 **hex** 格式（**openssl -fingerprint** 风格）。提示：“节点证书的 base64 或 hex 格式 SHA-256。点击“获取”立即从节点读取”。占位符：“base64 或 hex 格式的 SHA-256”。选择 **pin** 时，空指纹或无效指纹将在保存时触发验证错误。

示例：同一指纹的两种格式。字段接受任一变体——两者代表同一证书：

```
# base64 (Xray ❷❸pinnedPeerCertSha256 ❷❸)
607TNg3l2k0pq8R1sT2uV3wX4yZ5a6B7c8D9e0F1g2=

# ❷❸❷❸❷❸hex (openssl x509 -fingerprint -sha256 ❷❸)
E8:E2:D3:60:DE:5D:9A:4D:29:AB:CF:11:B2:7C:34:...
```

如果指纹尚不知晓，点击**获取**——主面板将通过 HTTPS 自动从节点读取并填入字段。

- **获取按钮**——在不验证证书的情况下通过 HTTPS 连接到节点，读取当前叶证书的 SHA-256（端点 `POST /certFingerprint`），并填入字段。成功后显示“已获取节点的当前证书”；失败则显示“无法获取证书”。仅适用于 https 节点。

Node mTLS（面板间的双向 TLS 认证）

节点页面有一个单独的 **Node mTLS** 部分——双向 TLS 认证设置，为“面板 → 节点”的调用在 API 令牌基础上增加第二个因素（客户端证书）。双向 TLS 是可选的；如果该部分字段为空，节点将按原有方案工作——**仅使用 API 令牌**（提示：“Mutual TLS 在节点间调用的 API 令牌基础上增加了客户端证书因素。它是可选的：留空以保持仅令牌认证”）。该部分有两项操作：

- **复制此面板的 CA**（`POST /panel/api/nodes/mtls/ca`）——将此面板的根证书（CA）复制到剪贴板。该 CA 需要传递给受管节点，使其信任面板的客户端证书；节点上随后需将 TLS 验证模式设置为 **Mutual TLS**（提示：“将此 CA 传递给此面板管理的节点，然后将其 TLS 验证设置为 Mutual TLS”）。复制后显示“CA 证书已复制到剪贴板”。
- **受信任的父 CA**（`Trusted parent CA`，`POST /panel/api/nodes/mtls/trustCA`）——当此面板本身作为上级（主控）面板的节点时使用的字段。将主控面板的 CA 粘贴到此处以要求其提供客户端证书，然后点击 **Save trust CA**。此更改需要**重启面板**（提示：“当此面板本身是节点时，将主控面板的 CA 粘贴到此处以要求其客户端证书。重启面板以应用”）。

12.4. 每个节点显示的信息

表格列和节点卡片字段（观测状态，在每次心跳轮询时填充）：

字段	描述
状态	<code>online</code> / <code>offline</code> / <code>unknown</code> ——见下文。
CPU	远程服务器的处理器负载，以百分比表示。
内存	RAM 使用率（以百分比计算，公式为 $\text{current}/\text{total} \times 100$ ）。
运行时间	服务器持续运行时间（秒）。
延迟	节点对最后一次轮询的响应时间（毫秒）。
最后 ping	最后一次成功心跳的时间（Unix 秒； <code>0</code> = “从未”；近期值显示为“刚刚”）。
Xray 版本	节点上运行的 Xray-core 版本。
面板版本	节点上的 3X-UI 版本——与最新版本比较以显示更新指示器。

字段	描述
(inbound 数)	物理部署在此节点上的 inbound 数量。
(客户端数)	节点 inbound 上的客户端数量。
(在线数)	节点上当前在线的客户端数量。
(已耗尽数)	节点上已过期或已超出流量限制的客户端数量。手动禁用的客户端不计入此计数器。
(速度)	部署在节点上的 inbound 的当前（实时）传输速度。

inbound/客户端/在线计数器通过节点的稳定 GUID（panelGuid）而非本地 id 与节点关联——这样子节点上的客户端就能归属于该子节点，而不是同步它的中间节点。

对于部署在节点上的 inbound，页面显示在线客户端、计数器和**当前传输速度**。通过稳定 GUID 的关联能正确区分具有相同 panelGuid 的“克隆”节点。

节点状态

状态	含义	何时设置
online	在线	节点对 panel/api/server/status 轮询响应 success=true。
offline	离线	节点未响应、返回 HTTP 错误、success=false 或无法识别的响应。
unknown	未知	初始值，节点尚未被轮询过。

轮询失败时，错误文本会被保存并以友好的格式显示，帮助诊断“离线”原因。

12.5. 节点操作

- **测试连接**（POST /test）——在节点表单中，使用已输入（尚未保存）的参数测试连接，超时 6 秒。结果：“连接正常（{ms} 毫秒）”或“无法连接”。便于在保存前调试地址/端口/令牌/TLS。
- **立即检查**（“立即检查”按钮，POST /probe/:id）——对已保存节点进行计划外轮询；立即更新状态和指标（CPU/内存/运行时间/延迟/版本）并记录心跳。失败时显示“检查失败”。

示例：通过主面板 API 测试和轮询节点。“测试连接”测试表单中尚未保存的参数：

```
POST /panel/api/nodes/test
Content-Type: application/json

{ "scheme": "https", "address": "de-frankfurt-1.example.com", "port": 2053,
  "basePath": "/", "apiToken": "eyJhbGciOi...", "tlsMode": "verify" }
```

对 id 为 7 的已保存节点进行计划外轮询：

```
POST /panel/api/nodes/probe/7
```

- **更新面板**（`POST /updatePanel`，请求体 `{ids:[...]}`）——在节点上启动其内置自更新器：节点下载最新的 3X-UI 版本并重启。**更新所选（{count}）按钮同时对多个选定节点执行此操作。节点旁显示指示器：有可用更新或已是最新版本**，根据节点面板版本与最新版本比较得出。

示例：通过一个请求更新多个节点。 请求体包含选定节点的 id；只有已启用且 `online` 的节点会被更新，其他节点将作为跳过返回。

```
POST /panel/api/nodes/updatePanel
Content-Type: application/json

{ "ids": [3, 7, 12] }
```

响应类似“已在 2 个节点上启动更新，1 个失败”：例如，节点 12 可能处于离线状态因而被跳过。

- 确认提示：“将 {count} 个节点更新到最新版本？每个选定节点将下载最新版本并重启。只有在线的已启用节点会被更新”。
- 只有状态为 `online` 的已启用节点会被更新。已禁用节点在结果中标记为“节点已禁用”，离线节点标记为“节点已离线”。结果：“已在 {ok} 个节点上启动更新，{failed} 个失败”。如果没有选择任何合适的节点——“请至少选择一个在线的已启用节点”。

在更新确认对话框中（无论是单节点还是批量），有一个复选框**更新到开发频道（最新提交）**。如果勾选，选定节点将安装 dev-latest 滚动版本（main 分支最新提交）而非稳定版本；未勾选时，节点按其常规频道更新。启用复选框时，下方显示警告：“开发版本跟踪 main 分支的每个提交，不是稳定版本——没有自动回滚”。dev 标志通过 `POST /panel/api/nodes/updatePanel` 传递给节点，节点随后按 dev 频道启动更新。

- **Set Cert from Panel**（辅助功能，`GET /webCert/:id`）——在节点上创建 inbound 时，允许填入节点自身 Web TLS 证书的路径（而非中心面板的证书），以确保文件确实存在于节点上。要求节点已启用且可访问。
- **删除节点**（`POST /del/:id`）——确认提示：“删除节点{name}”？这将停止对该节点的监控。远程面板本身不会受到影响。删除节点记录及其累积的流量统计；远程面板继续正常运行。**只有在节点上的所有 inbound 都已解除关联后，才能删除节点。**如果节点上仍有至少一个 inbound 通过 `node_id` 关联，面板将拒绝删除并报错，例如“cannot delete node: N inbound(s) still attached to it; detach or delete them first”——请先解除关联或删除这些 inbound，然后再删除节点。这可防止出现指向已删除节点的“孤立”inbound。

12.6. 指标历史

历史按钮/图表访问 `GET /history/:id/:metric/:bucket`。可用指标：`cpu` 和 `mem`——在每次成功的心跳时累积。聚合区间大小（`bucket`，单位秒）受白名单限制：

示例：查询历史记录。 节点 7 的 CPU 负载历史，以 60 秒区间聚合（最多返回 60 个数据点）：

```
GET /panel/api/nodes/history/7/cpu/60
```


内存和“实时”模式（2 秒）分别使用 `.../7/mem/60` 和 `.../7/cpu/2`。白名单之外的值将被拒绝（“invalid metric”/“invalid bucket”）。

Bucket（秒）	用途
2	实时模式
30	30 秒区间
60	1 分钟区间
120	2 分钟区间
180	3 分钟区间
300	5 分钟区间

最多返回 60 个数据点。无效指标或 bucket 将被拒绝（“invalid metric”/“invalid bucket”）。

12.7. inbound 和客户端如何同步

inbound 通过 `node_id` 字段“归属”于节点（在 inbound 编辑器中选择节点）：

示例：节点表单中的令牌。 令牌从子面板（设置 → API 令牌）获取并粘贴到主面板的 **API 令牌** 字段。每次轮询时主面板在请求头中发送：

```
GET https://panel.example.com:2053/panel/api/server/status
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.abc123...
```

如果子面板设置了**基础路径**（web base path），例如 `/secret/`，主面板会自动将其添加到 `panel/api/server/status` 前面 → `https://panel.example.com:2053/secret/panel/api/server/status`。

- 配置部署（reconcile）。** 当与节点关联的 inbound/客户端发生任何更改时，节点被标记为“脏”。后台任务对每个状态为 **online** 的已启用节点，在有更改的情况下将其 inbound（按 `node_id`）部署到节点，然后重置“脏”标志。已禁用、离线或“脏”的节点被视为“待处理”——部署推迟到连接恢复。
- 流量收集。** 同一任务从节点请求流量快照并合并到本地统计数据中。基于合并的流量检查限额/期限是否耗尽，并在必要时禁用客户端；节点的“已耗尽”计数器正是反映这一点。如果节点不可达，其在线客户端将被清除。

对于同时关联多个面板的客户端，主面板在同一任务中还会向节点发送该客户端**跨所有面板的总计流量**（存储在节点上的独立表中，键为主面板的 GUID；每次发送时覆盖，因此主面板端的重置也会同步）。节点上客户端的流量显示本地值和收到值中的较大者；超出总配额时，客户端在**节点本身被本地禁用**（通过与自动禁用相同的 Xray 重启机制，断开已建立的连接）。这消除了节点只看到自己那份流量、低估用量并继续为已耗尽总配额的客户端提供服务的情况。重置流量、自动续期或删除客户端时，已发送的计数器也会被清除。

首次同步部署在节点上的 inbound 时（添加新节点或重新导入 inbound），主面板用节点的真实值初始化客户端的流量计数器。以前在这种情况下，inbound 的总计数器会正确迁移，但各客户端的计数器会被清零，导致主面板低估客户端在节点接入前积累的所有历史用量。现在，如果 inbound 是在同一次同步中创建的，新的 `client_traffics` 行会继承节点上的计数器值（基线设置为该值，因此下一个增量

为零，流量不会被计算两次）。计数器预填仅适用于在同一轮创建的 inbound：在已有 inbound 下重新出现的客户端仍然从零开始（防止“幽灵”流量），而刚刚删除的客户端在重建其 inbound 时也不会“复活”。

- 心跳。独立的后台任务定期轮询所有已启用节点（有并发限制），通过 `panel/api/server/status` 更新状态/指标/版本，并在有 Web 客户端时通过 WebSocket 推送更新的节点树。

12.8. 节点链（子节点/传递节点）

拓扑可以不是扁平的：节点本身可以成为其子节点的主面板。这类下级面板在您处显示为子节点——这是从直接节点获取的只读投影。

- 提示：“只读：通过 {父节点} 可访问的下级节点。请从 {父节点} 自己的面板管理它”。即子节点无法在此处编辑、删除或更新——所有操作均从其直接父节点的面板执行。
- 子节点的身份由其 GUID 确定；因此，在线客户端和 inbound 归属于实际托管它们的物理节点，即使在 `??1 → ??2 → ??3` 的链路中也是如此（主面板通过每个直接节点向下“传递”一层）。
- 如果直接节点变得不可达，其子节点缓存会被清除，子节点从树中消失，直到连接恢复。

12.9. 3.5.0 中的修复（节点稳定性）

3.5.0 中一组节点运维者可感知的修复：

- 未做修改地保存客户端不再中断节点流量。此前任何客户端保存（哪怕「打开——保存」）都会向节点发送完整的 inbound 更新并重建 Xray 处理器——节点显示在线，但在手动重启前不再转发流量。现在无实际更改（no-op）的保存不做任何事，真正的修改则以一次轻量更新下发，不重建处理器。
- 节点的 Host 覆盖会被导入主面板：首次接受节点的 inbound 时，其 Hosts 行（SNI、指纹、ALPN 等）会被导入，订阅因此携带正确的 TLS 参数。
- 节点上的自动续期会开启新的配额窗口（「每 30 天 100 GB」之类的套餐会重新发放全额流量）。
- 在主面板删除客户端会将其从节点上完全删除（记录 + 流量），而不只是解绑；以往批量删除的残留可通过节点上的「删除未绑定客户端」操作清除。
- 节点的 inbound 在首次接受前不会被清除：保存刚添加的节点不再可能删除其现有的 inbound。
- 单个异常 inbound 不再阻断节点流量同步（同时，节点上遗留的 socks inbound 会被重命名为 mixed）。
- 端口冲突检查仅限于所在节点——修改节点 inbound 的监听地址不再因另一节点存在相同端口而被拒绝（「port ... already used by inbound ...」）。

12.10. 节点：3.3.0 中的新功能

在 3.3.0 版本中，节点部分获得了三项显著改进：多跳拓扑中流量和在线客户端的正确归属、节点间 client-IP 同步，以及当节点面板存活但其 Xray 核心崩溃时的独立状态指示器。

1. 多跳：子节点链中的正确流量归属

以前，计数器（inbound 数、在线客户端、已耗尽）在“直接”节点级别计算。如果您有 `?? → ??1 → ??2 → ??3` 这样的链路，物理位于 `??2 / ??3` 上的所有内容都会错误地归属到将其传递给主面板的 `??1`。在 3.3.0 中，归属按实际来源进行。

工作原理：

- **子节点作为独立行显示。** 每个面板发布其直接节点列表；只包含具有已知 `Guid` 的节点——需要稳定身份才能将节点归属到“上一跳”。主面板定期（从心跳任务）拉取这些列表并缓存，然后在直接节点之外添加“传递”子节点。
- **传递节点为只读。** 在 UI 中，它们标记为“**子节点**”，提示：“只读：通过 {父节点} 可访问的下级节点。请从 {父节点} 自己的面板管理它。” 此类行没有操作按钮——节点从其直接父节点的面板管理。
- **通过 GUID 建立层级。** 直接节点的 `ParentGuid` 是主面板自身的 GUID；传递节点的 `ParentGuid` 是其父节点的 GUID。这样就构建出树形结构。
- **计数器的真实来源是 inbound 上的 `origin_node_guid`。** 这是物理持有该 inbound 的节点的 `panelGuid`。它在 inbound 与节点同步时设置，**在后续跳转中保持不变**，因此深度嵌套的 inbound 归属于真实节点而非中间节点。inbound 数、在线客户端和已耗尽客户端的计数器均按此 GUID 重新计算。键的选择逻辑：

inbound 状态	归属对象
已设置 <code>origin_node_guid</code>	该 GUID（真实源节点）
为空但已设置 <code>node_id</code>	节点的合成 GUID（旧版本，尚未报告其 <code>panelGuid</code> ）
为空且 <code>node_id</code> 也为空	主面板自身的 GUID（本地 Xray 上的 inbound）

在线客户端同样按 GUID 分组，因此每个节点行只显示真正连接到它的用户。

用户看到的效果： 在扁平拓扑中（节点直接在主面板下），没有变化——按 GUID 和按 `id` 的计数器一致。但一旦出现节点链，列表中就会出现“子节点”行，每个节点的 inbound/在线/已耗尽数字现在只反映其自身负载，而不是所有经其传输的内容之和。

2. 跨节点从 `access.log` 同步 `client-IP`

IP 限制（客户端的 `limitIp`）依赖 Xray 写入其 `access.log` 的地址。以前每个节点只能看到连接到自己的请求，因此“每个客户端不超过 N 个 IP”的限制在集群中无法正常工作：客户端可以连接到不同节点从而绕过限制。在 3.3.0 中，观测到的 IP 会在整个集群中同步。

工作原理：

- 在每个节点上，后台任务解析 `access.log`，提取每行的 IP、客户端邮箱和时间戳，并存入本地表（每个邮箱一条记录，IP 存储为 JSON 数组 `{ip, timestamp}`）。本地地址 `127.0.0.1` 和 `::1` 被丢弃。
- **每 10 秒同步一次**，对每个在线的已启用节点执行双向交换：从节点拉取 IP 并合并到本地表，然后将主面板的汇总数据发送给节点。
- 合并时**不会重复计算**在多个节点上看到的同一 IP，**也不会复活过时记录**：应用与本地任务相同的过期阈值——**30 分钟**。每个 IP 保留最新的时间戳。来自其他节点的记录获得新的本地 id（节点的 id 空间是独立的）；同一邮箱的并发插入受到去重保护。
- 计算限制时，在当前本地扫描中观测到的 IP，或在同步数据库中具有非常新近时间戳（**2 分钟以内**）的 IP，被视为“活跃”。正是这一点使限制在整个集群范围内生效，即使该地址是在另一个节点上观测到的。

超出限制时，最旧的“活跃”IP 被发送到 fail2ban 日志，连接被强制断开（通过 Xray API 移除/重新添加客户端）。

用户看到的效果：IP 数量限制现在在整个集群范围内生效，而不是每个节点独立生效；面板上的客户端 IP 显示在任意节点上观测到的地址（30 分钟窗口内）。无需单独的按钮/设置——同步在后台自动进行，前提是节点已启用并可访问 access.log（IP 限制本身还需要节点上有 Fail2Ban）。

3. 独立状态指示器：节点面板在线但 Xray 已崩溃

以前节点状态基本上是“在线/离线”。如果节点面板有响应，节点就被视为在线——即使其上的 Xray 核心未运行，客户端实际上无法连接。在 3.3.0 中，面板健康状态和 Xray 核心健康状态被分开。

工作原理：

- 轮询节点时，主面板从远程 `/panel/api/server/status` 的响应中获取 `xray.state` 和 `xray.errorMsg` 字段并存储到节点中。即使面板 ping 成功但核心不健康，这些字段也会被填充——正是为了区分面板可达性和 Xray 状态。
- `xray.state` 的值：`"running"`（运行中）、`"stop"`（已停止）、`"error"`（错误）。
- 这些值转化为节点状态。在原有状态之外新增了：

状态键	显示文字	何时显示
<code>online</code>	在线	面板响应，Xray 运行中（ <code>running</code> ）
<code>offline</code>	离线	面板不可达/ping 失败
<code>unknown</code>	未知	状态尚未确定
<code>xrayError</code>	Xray 错误	面板在线，但 Xray 核心处于 <code>error</code> 状态（有 <code>errorMsg</code> ）
<code>xrayStopped</code>	已停止	面板在线，但 Xray 已停止（ <code>stop</code> ）

- 对于此类状态，UI 使用**独立的紫色指示器**（颜色不同于绿色的“在线”和红色的“离线”）。紫色直接表明：节点可以联系到，问题出在 Xray 核心本身，而不是网络或面板。

用户看到的效果：当核心崩溃时，不再显示误导性的“绿色”，节点会以**紫色**高亮显示，状态为 **Xray 错误**或**已停止**。这立即表明需要修复的是节点上的 Xray（重启核心，查看 `errorMsg`），而不是排查节点本身的可达性。同样的 `xrayState` / `xrayError` 也会传递到传递子节点（见第 1 点），因此核心的异常状态在整个链路中都可见。

13. 面板设置

「设置」区域（页面标题为 **设置**，英文 *Panel Settings*）用于管理 3X-UI Web 面板本身的行为：它监听哪个地址和端口、如何受保护、如何与 Telegram 机器人及外部服务交互、在哪个时区执行计划任务。每个参数都以「键 - 值」对的形式存储在数据库的 `settings` 表中；如果数据库中没有相应的值，则应用默认值。

重要 - 应用更改。 本页面上的任何更改都需要用 **保存** (*Save*) 按钮保存，然后重启面板才能使更改生效。原文提示：「保存更改并重启面板以使其生效。」保存时会显示通知「设置已更改」。

13.1. 保存与重启面板

元素	用途
保存 (<i>Save</i>)	将表单的所有字段写入数据库 (<code>POST /panel/setting/update</code>)。写入前，值会经过校验——不正确的地址、端口或路径将被拒绝，面板会返回错误。
重启面板 (<i>Restart Panel</i>)	重启面板的 Web 服务器 (<code>POST /panel/setting/restartPanel</code>)。重启会延迟 3 秒进行。提示：「您确定要重启面板吗？确认后，重启将在 3 秒后进行。如果面板不可用，请检查服务器日志」。成功时显示「面板已成功重启」。
恢复默认设置 (<i>Reset to Default</i>)	删除数据库中所有已保存的设置，之后面板使用默认值。此操作不会重置管理员凭据。

重启是通过向面板进程发送 `SIGHUP` 信号（或通过已注册的重启钩子）来执行的。在 Windows 上不支持通过信号自动重启。监听参数（IP、端口、路径、域名、证书、时区）的更改只有在重启面板后才会生效。

13.2. 通用设置（「面板」选项卡 / *General*）

界面语言 (*Language*)

面板 Web 界面的语言。可用语言：`en-US`（英语）、`ru-RU`（俄语）、`zh-CN`、`zh-TW`、`fa-IR`、`ar-EG`、`es-ES`、`id-ID`、`ja-JP`、`pt-BR`、`tr-TR`、`uk-UA`、`vi-VN`。这是显示设置，不影响 Xray 的工作。

日历类型 (*Calendar Type*)

- 键：`datepicker`
- 默认值：`gregorian`（公历）。
- 用途：日期选择中使用的日历类型（例如，在设置客户端有效期时）。提示：「计划任务将根据此日历执行。」备选值为波斯（贾拉里）历，这对面板的伊朗用户群体很有需求。

分页大小 (*Pagination Size*)

- 键：`pageSize`

- **默认值：** 25
- **允许值：** 从 0 到 1000 的整数。
- **用途：** 表格（连接/inbound 列表）中每页的行数。提示：「确定连接表格的页面大小。设为 0 可禁用」——当为 0 时，分页显示被禁用，所有记录以单一列表显示。
- **无需重启面板**（显示设置）。

自动停用后重启 Xray (*Restart Xray After Auto Disable*)

- **键：** `restartXrayOnClientDisable`
- **默认值：** `true`
- **用途：** 当客户端被自动停用时（因有效期到期或达到流量上限），重启 Xray，以断开该客户端已建立的连接。提示：「当客户端因有效期结束或流量上限而被自动停用时，重启 Xray。」该功能本身没有变化——只是开关位于「面板」（*General*）选项卡上，与其他通用设置并列。

备注模型与分隔符 (*Remark Model & Separation Character*)

- **键：** `remarkModel`
- **默认值：** `-ieo`
- **用途：** 设定订阅中配置名称（remark）的生成方式。该字符串由第一个字符——分隔符，以及随后的顺序字母序列组成：
 - `i` — inbound 备注 (*inbound remark*) ；
 - `e` — 客户端 email；
 - `o` — 附加标记 (*extra*) 。

默认值 `-ieo` 中分隔符为 `-`，各部分顺序为：inbound → email → extra（例如 `MyInbound-user@mail-extra`）。空的部分会被跳过。界面中的「示例备注」（*Sample Remark*）字段显示所生成名称的预览。是否将 email 纳入名称还取决于订阅设置中的「在名称中包含 Email」参数（参见订阅章节）。

示例： `remarkModel` 的值如何影响配置名称。假设 inbound 名为 `VLESS-Reality`，客户端 email 为 `alex@vpn`，附加标记为 `RU`。那么：

字段值	最终名称 (remark)
<code>-ieo</code> (默认)	<code>VLESS-Reality-alex@vpn-RU</code>
<code>_ie</code>	<code>VLESS-Reality_alex@vpn</code>
<code>-ei</code>	<code>alex@vpn-VLESS-Reality</code>
<code>o</code> (空格分隔符，仅标记)	<code>RU</code>

字符串的第一个字符始终是分隔符；其余字母决定哪些部分以何种顺序进入名称。

13.3. 面板访问：IP、端口、路径、域名、证书

这一组参数定义面板的网络入口点。此处的所有更改只有在重启面板后才会生效。

字段	键	默认值	描述
面板管理 IP 地址 (<i>Listen IP</i>)	<code>webListen</code>	"" (空)	Web 面板监听的 IP。空 = 在所有 IP 上监听。提示：「留空以允许从任意 IP 连接」。如果设置，则必须是正确的 IP 地址（否则保存会被拒绝）。
面板域名 (<i>Listen Domain</i>)	<code>webDomain</code>	"" (空)	用于按域名校验请求的面板域名。空 = 接受来自任意域名和 IP 的连接。提示：「留空以允许从任意域名和 IP 连接。」
面板端口 (<i>Listen Port</i>)	<code>webPort</code>	2053	面板运行的端口。提示：「面板运行的端口」。允许 1–65535。端口必须空闲；面板与订阅服务不能同时使用相同的 IP:?? 组合。
URI 路径 (<i>URI Path</i>)	<code>webBasePath</code>	/	面板 URL 的基础路径 (basePath)。提示：「必须以 '/' 开头并以 '/' 结尾」。保存时，如果缺少前导和结尾的 /，面板会自动添加。路径中的非法字符会被拒绝。

面板证书 (TLS / HTTPS)

字段	键	默认值	描述
面板证书公钥文件路径 (<i>Public Key Path</i>)	<code>webCertFile</code>	""	证书（链）文件的完整路径。提示：「输入以 '/' 开头的完整路径」。
面板证书私钥文件路径 (<i>Private Key Path</i>)	<code>webKeyFile</code>	""	私钥文件的完整路径。提示：「输入以 '/' 开头的完整路径」。

如果设置了证书/密钥路径中的至少一个，面板在保存时会尝试加载「证书 + 密钥」对；如果出错（文件不存在、密钥与证书不匹配），保存会被拒绝。当两个正确的路径都已设置时，面板切换到 HTTPS。两个字段都为空 = 面板按普通 HTTP 工作。

安全警告 (Security warnings)。如果面板检测到不安全的配置，会显示「您的面板可能处于暴露状态：」警告块：

- 按普通 HTTP 工作 – 「请为生产环境配置 TLS」；
- 标准端口 2053 – 「将其改为随机端口」；
- 默认基础路径 / – 「将其改为随机路径」；
- 标准订阅路径 /sub/ 和 JSON 订阅 /json/ – 「请更改它」。这些是建议，而非强制限制。

13.4. 会话、面板代理与受信任代理（「代理与服务器」选项卡 / *Proxy and Server*）

会话时长（*Session Duration*）

- 键： `sessionMaxAge`
- 默认值： 360（分钟，即 6 小时）。
- 允许值： 从 1 到 525600 分钟（1 年）。
- 用途： 管理员在无需重新登录的情况下保持登录状态的时长。单位为分钟。提示：「系统中的会话时长（单位：分钟）」。

面板流量 Outbound（*Panel Traffic Outbound*）

- 键： `panelOutbound`
- 默认值： ""（空 = 直连）。
- 用途： 设定面板通过其发送自身请求的 Xray **outbound**——版本检查和面板/Xray 下载、对 Telegram 的访问、常规的 geo 文件更新——以绕过服务器端对 GitHub/Telegram 的过滤。该字段为下拉列表：其中列出了 Xray 配置模板中的 outbound、outbound 订阅中的 outbound，以及路由负载均衡器（单独分组）。列表中排除了 blackhole 类型的 outbound——把下载路由到「黑洞」毫无意义。原文提示：「将面板自身的请求——版本检查和面板/Xray 下载、Telegram 以及常规的 geo 文件更新——通过此 Xray 出站路由，以绕过服务器端对 GitHub/Telegram 的过滤。本地桥接入站会自动添加到运行中的配置并即时生效。Xray 内置的 Geodata 自动更新不受影响；它有自己的用于下载的出站。留空以使用直连。」

工作原理。 选择 outbound 后，面板会自动向运行中的配置添加一个服务用的回环 inbound（带标签 `panel-egress` 的 SOCKS 桥接）和一条路由规则，该规则将面板自身的 HTTP 流量转发到所选 outbound。如果选择了负载均衡器，规则中会代入 `balancerTag`，面板流量将在其成员之间分配。桥接和规则会即时生效，无需完全重启面板。留空字段以使用直连。Xray 内置的 geo 数据自动更新不受此设置影响——它在 Xray 路由内部有自己的 outbound。

- 格式： `socks5://`（或 `socks5h://`）或 `http(s)://`，必要时带形如 `socks5://user:pass@host:port` 的认证。严格支持的协议为：`socks5`、`socks5h`、`http`、`https`——其他协议视为非法，此时面板会回退到直连。典型示例是 Xray 自身的本地 SOCKS 入站。
- 原文提示：「将面板自身的出站请求（geo 更新、Xray/面板版本检查、Telegram）通过此代理路由，以绕过服务器端对 GitHub/Telegram 的过滤。接受 `socks5://` 或 `http(s)://`，例如 Xray 的本地 SOCKS 入站。留空以使用直连。」
- 无效的代理不会导致保存出错——面板只是使用直连并在日志中写入警告。

字段值示例。 如果服务器上已经在端口 10808 上运行了 Xray 的本地 SOCKS 入站，可将面板的自身请求通过它转发：

```
socks5://127.0.0.1:10808
```

对于带认证的外部 HTTP 代理：


```
http://user:pass@proxy.example.com:8080
```

保存并重启后，面板将通过指定的代理拉取 geo 数据库更新、检查版本并访问 Telegram。

受信任代理 CIDR (*Trusted proxy CIDRs*)

- 键: `trustedProxyCIDRs`
- 默认值: `127.0.0.1/32,::1/128` (仅本地主机)。
- 格式: 以逗号分隔的 IP 地址或 CIDR 子网列表 (例如 `10.0.0.0/8, 192.168.1.5`)。每个元素都会作为 IP 或 CIDR 校验——不正确的值在保存时会被拒绝。
- 用途: 列出允许设置 `X-Forwarded-Host`、`X-Forwarded-Proto` 标头和真实客户端 IP 标头的来源。原文提示: 「以逗号分隔的 IP/CIDR, 允许其设置 forwarded host、proto 和 client IP 标头。」如果面板工作在反向代理 (nginx、Caddy 等) 之后, 需要配置此项, 以便正确识别客户端 IP 和协议。

示例: 面板位于反向代理之后。 如果 nginx 与面板位于同一主机并将请求代理到面板, 则仅信任本地主机 (默认值) 即可:

```
127.0.0.1/32,::1/128
```

如果代理位于内部网络 `10.0.0.0/8` 中的单独服务器上, 请添加其子网, 否则面板会忽略它传递的标头, 并看到代理的 IP 而非真实客户端的 IP:

```
127.0.0.1/32,::1/128,10.0.0.0/8
```

传递真实 IP 和协议的相应 nginx 块示例:

```
proxy_set_header X-Real-IP      $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header X-Forwarded-Host $host;
```

13.5. Telegram 机器人 (「Telegram 机器人」选项卡 / *Telegram Bot*)

启用 Telegram 机器人 (*Enable Telegram Bot*)

- 键: `tgBotEnable`
- 类型/默认值: 布尔型, `false`。
- 用途: 启用 Telegram 机器人的工作。提示: 「通过 Telegram 机器人访问面板功能」。

Telegram 令牌 (*Telegram Token*)

- 键: `tgBotToken`
- 默认值: `""`。
- 用途: 机器人令牌。提示: 「需要从 Telegram 机器人管理器 @botfather 获取令牌」。

- **安全特性：** 令牌属于机密值。在面板读取设置的响应中不会返回它（字段被清空，仅返回「已配置/未配置」标志）。如果保存时将字段留空，则之前保存的令牌将被保留（不会被覆盖）。

Telegram 机器人语言 (*Telegram Bot Language*)

- **键：** `tgLang`
- **默认值：** `en-US`。
- **用途：** 机器人消息的语言（独立于 Web 界面的语言）。可用语言列表与面板语言相同。

机器人管理员 User ID (*Admin Chat ID*)

- **键：** `tgBotChatId`
- **默认值：** `""`。
- **格式：** 一个或多个数字 Telegram User ID，以逗号分隔。
- **用途：** 通知接收者和被允许通过机器人管理面板的管理员。提示：「Telegram 机器人管理员的一个或多个 User ID。要获取 User ID，请使用 `@userinfobot` 或在机器人中使用 `/id` 命令。」

通知频率 (*Notification Time*)

- **键：** `tgRunTime`
- **默认值：** `@daily`（每日一次）。
- **格式：** **Crontab** 格式的字符串（支持标准 cron 表达式，以及 `@daily`、`@hourly`、`@every 1h` 之类的缩写）。提示：「以 Crontab 格式指定通知间隔」。控制机器人的周期性报告。

字段值示例。

值	机器人何时发送报告
<code>@daily</code>	每日午夜一次（默认）
<code>@hourly</code>	每小时
<code>@every 6h</code>	每 6 小时
<code>0 9 * * *</code>	每天 09:00
<code>30 8 * * 1</code>	每周一 08:30

时间按「时区」设置（第 13.6 节）中的时区计算。

SOCKS 代理 (*SOCKS Proxy*)

- **键：** `tgBotProxy`
- **默认值：** `""`。
- **用途：** 专门用于机器人连接 Telegram 的 SOCKS5 代理。提示：「如果连接 Telegram 需要 Socks5 代理，请按照指南配置其参数。」它专门作用于机器人的流量（不同于第 13.4 节的通用「面板网络代理」）。

Telegram API Server (*Telegram API Server*)

- **键:** `tgBotAPIServer`
- **默认值:** `""` (使用标准服务器 `api.telegram.org`)。
- **格式:** URL `http(s)://...`; 保存时会通过 URL 正确性校验——无效的地址会被拒绝。提示: 「使用的 Telegram API 服务器。留空以使用默认服务器。」用于自行部署的 Telegram Bot API server。

机器人通知 (「通知」组 / *Notifications*)

字段	键	默认值	描述
数据库备份 (<i>Database Backup</i>)	<code>tgBotBackup</code>	<code>false</code>	将数据库备份文件连同报告一起发送到 Telegram。提示: 「发送带有数据库备份文件的通知」。
登录通知 (<i>Login Notification</i>)	<code>tgBotLoginNotify</code>	<code>true</code>	在尝试登录面板时进行通知。提示: 「显示有人尝试登录您的面板时的用户名、IP 地址和时间。」
会话到期通知延迟 (<i>Expiration Date Notification</i>)	<code>expireDiff</code>	<code>0</code>	在客户端有效期到期前多少天发送通知。 <code>0</code> – 禁用。允许 ≥ 0 。提示: 「在达到阈值前接收会话有效期到期通知 (单位: 天)」。
通知的流量阈值 (<i>Traffic Cap Notification</i>)	<code>trafficDiff</code>	<code>0</code>	用于通知的剩余流量阈值。提示: 「在达到阈值前接收流量耗尽通知 (单位: GB)」。
CPU 负载阈值 (<i>CPU Load Notification</i>)	<code>tgCpu</code>	<code>80</code>	如果 CPU 负载超过阈值 (以 % 计), 通知管理员。允许 <code>0-100</code> 。提示: 「如果 CPU 负载超过此阈值, 在 Telegram 中通知管理员 (单位: %)」。

13.6. 日期与时间 (「日期与时间」选项卡 / *Date and Time*)

时区 (*Time Zone*)

- **键:** `timeLocation`
- **默认值:** `Local` (服务器的系统时区)。
- **格式:** IANA tz 数据库中的时区名称 (例如 `Europe/Moscow`、`UTC`、`Asia/Tehran`)。
- **用途:** 面板执行计划任务 (机器人报告、流量重置/检查、有效期到期) 所依据的时区。提示: 「计划任务根据此时区的时间执行」。
- **校验:** 保存时会校验时区——不存在的时区会被拒绝。如果之后数据库中出现不正确的值, 面板会在运行时回退到 `Local`, 而如果它也不可用, 则回退到 `UTC`。

13.7. 外部流量与 Xray 行为 (「外部流量」选项卡 / *External Traffic*)

字段	键	默认值	描述
外部流量通知 (<i>External Traffic Inform</i>)	<code>externalTrafficInformEnable</code>	<code>false</code>	在每次流量更新时通知外部 API。提示: 「在每次流量更新时通知外部 API。」

字段	键	默认值	描述
外部流量通知 URI (<i>External Traffic Inform URI</i>)	<code>externalTrafficInformURI</code>	<code>""</code>	面板向其发送流量更新的 URL。保存时会通过 URL 正确性校验。提示：「流量更新发送到此 URI」。
自动停用后重启 Xray (<i>Restart Xray After Auto Disable</i>)	<code>restartXrayOnClientDisable</code>	<code>true</code>	当客户端因有效期到期或超过流量上限而被自动停用时，重启 Xray。提示：「当客户端因有效期结束或流量上限而被自动停用时，重启 Xray。」该开关位于「面板」 (<i>General</i>) 选项卡 – 参见第 13.2 节；此处列出是为了完整性。

13.8. 其他：Xray 配置模板与测试 URL

Xray 配置模板 (*xrayTemplateConfig*)

- **键：** `xrayTemplateConfig`
- **默认值：** 随构建提供的内置 (embedded) JSON 模板。
- **用途：** Xray-core 配置的基础 JSON 模板，面板在其之上构建 inbound/outbound。此值在所有设置的常规输出中不会返回，且在单独的 Xray 配置页面而非面板设置字段的通用列表中编辑。默认的标准模板可通过 `GET /panel/setting/getDefaultJsonConfig` 获取。

出站测试 URL (*xrayOutboundTestUrl*)

- **键：** `xrayOutboundTestUrl`
- **默认值：** `https://www.google.com/generate_204`
- **用途：** 在检查出站 (outbound) 连接可用性时使用的 URL。设置时会作为 HTTP(S) URL 进行净化处理。

13.9. 管理员账户与 API 令牌

这些参数位于相邻的选项卡（「账户」 / *Authentication*）上，并在安全章节中详细介绍；此处为键的简要汇总。

- **更改凭据**（「当前登录名」、「当前密码」、「新登录名」、「新密码」字段）通过单独的请求 `POST /panel/setting/updateUser` 保存。需要正确的当前登录名和密码；新的登录名和密码不能为空。消息：「您已成功更改管理员凭据。」 / 「用户名或密码不正确」。
- **双因素认证 (2FA)** – 键 `twoFactorEnable`（默认 `false`）和机密 `twoFactorToken`。令牌是机密：在启用 2FA 的情况下，保存时空字段不会覆盖现有令牌。在首次启用 2FA 时，面板会使当前会话失效（提升「登录纪元」）。
- **API 令牌**由单独的端点（`/panel/setting/apiTokens...`）管理：列表、创建（`apiTokens/create`）、删除、启用/禁用。令牌本身仅在创建时显示一次，且不以可读形式存储：「立即复制此令牌。出于安全考虑，它不以可读形式存储，且不会再次显示。」

有关 2FA、密码、LDAP 同步以及订阅格式 (JSON/Clash、fragmentation、noises、mux) 的详情，已归入手册相应的单独章节。

13.10. 3.3.0 中的 API 变更（对集成很重要）

在 3.3.0 版本中，服务器 API 的路径结构发生了变化。如果您有通过 HTTP 访问面板的外部集成（脚本、机器人、中央面板、CI 任务），则需要修正它们，否则它们将停止工作。

⚠ BREAKING CHANGE: `/panel/setting/*` 和 `/panel/xray/*` 端点已迁移到 `/panel/api` 下

以前，面板设置管理和 Xray 配置分别位于路径 `/panel/setting/*` 和 `/panel/xray/*` 下。现在两组都注册在通用 API 组 `/panel/api` 内部。旧路径已完全删除——对它们的请求将返回 404。

为什么这样做：整个 `/panel/api` 组都经过统一的访问检查，也就是说这些端点现在接受与其余 API 相同的 `Authorization: Bearer <token>` 标头。API 令牌是完整的管理员访问权限，由此整个 API 面变得统一。

未发生变化的内容： Web 界面页面（SPA 路由）`/panel/settings` 和 `/panel/xray` 保持原位——只涉及服务器 API 端点。

路径对应表（旧 → 新）

以下所有路径的前缀变化——仅仅是在 `/panel/` 之后添加了 `api/`。

之前 (≤ 3.2.x)	之后 (3.3.0)	方法
<code>/panel/setting/all</code>	<code>/panel/api/setting/all</code>	POST
<code>/panel/setting/defaultSettings</code>	<code>/panel/api/setting/defaultSettings</code>	POST
<code>/panel/setting/update</code>	<code>/panel/api/setting/update</code>	POST
<code>/panel/setting/updateUser</code>	<code>/panel/api/setting/updateUser</code>	POST
<code>/panel/setting/restartPanel</code>	<code>/panel/api/setting/restartPanel</code>	POST
<code>/panel/setting/getDefaultJsonConfig</code>	<code>/panel/api/setting/getDefaultJsonConfig</code>	GET
<code>/panel/setting/apiTokens</code>	<code>/panel/api/setting/apiTokens</code>	GET
<code>/panel/setting/apiTokens/create</code>	<code>/panel/api/setting/apiTokens/create</code>	POST
<code>/panel/setting/apiTokens/delete/:id</code>	<code>/panel/api/setting/apiTokens/delete/:id</code>	POST
<code>/panel/setting/apiTokens/setEnabled/:id</code>	<code>/panel/api/setting/apiTokens/setEnabled/:id</code>	POST
<code>/panel/xray/</code>	<code>/panel/api/xray/</code>	POST
<code>/panel/xray/update</code>	<code>/panel/api/xray/update</code>	POST
<code>/panel/xray/getDefaultJsonConfig</code>	<code>/panel/api/xray/getDefaultJsonConfig</code>	GET
<code>/panel/xray/getXrayResult</code>	<code>/panel/api/xray/getXrayResult</code>	GET
<code>/panel/xray/getOutboundsTraffic</code>	<code>/panel/api/xray/getOutboundsTraffic</code>	GET
<code>/panel/xray/resetOutboundsTraffic</code>	<code>/panel/api/xray/resetOutboundsTraffic</code>	POST

之前 (≤ 3.2.x)	之后 (3.3.0)	方法
<code>/panel/xray/testOutbound</code>	<code>/panel/api/xray/testOutbound</code>	POST
<code>/panel/xray/warp/:action</code>	<code>/panel/api/xray/warp/:action</code>	POST
<code>/panel/xray/nord/:action</code>	<code>/panel/api/xray/nord/:action</code>	POST
<code>/panel/xray/outbound-subs</code> (及 <code>/outbound-subs/*</code>)	<code>/panel/api/xray/outbound-subs</code> (及 <code>/outbound-subs/*</code>)	GET/POST/ DELETE

子路径名称、请求体和响应格式本身没有改变——改变的仅是前缀。

如何修复现有集成

1. 在您的脚本/配置中找到所有出现的 `/panel/setting/` 和 `/panel/xray/`。
2. 替换前缀：在 `/panel/` 之后紧接着添加 `api/`（例如 `/panel/setting/all` → `/panel/api/setting/all`）。
3. 请求体、参数和响应格式无需修改——只更改 URL。
4. 由于设置和 Xray 配置现在位于 `/panel/api` 下，可以（并且应当）使用与 `/panel/api/inbounds/*` 及其他端点相同的 API 令牌 `Authorization: Bearer <token>` 访问它们。不要忘记对整个 `/panel/api` 组启用的 CSRF 中间件。

示例：通过 API 读取所有设置。以前 (≤ 3.2.x)：

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/setting/all" \
-H "Authorization: Bearer <token>"
```

现在 (3.3.0) ——在 `/panel/` 之后添加了 `api/`：

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/api/setting/all" \
-H "Authorization: Bearer <token>"
```

重启面板同理：`POST /panel/api/setting/restartPanel`。旧路径 `/panel/setting/restartPanel` 现在将返回 404。

类型化 API：模式与文档（Swagger / OpenAPI）

在 3.3.0 中，OpenAPI 规范变得完全类型化。以前类型化响应用空对象 `{}` 描述；现在组件和模式（`components.schemas`）直接从数据模型生成。由此：

- Swagger UI 显示真实的数据模型，而非空洞的占位符。
- 外部生成器（`openapi-generator` 等）可以根据规范构建出所需语言的现成客户端。
- 每个类型化响应都附有指向具体模型的 `$ref`，并附带了响应示例。

在哪里查看 API 文档：

- **内置 Swagger 页面。** 在面板菜单中——**「API 文档」**项（SPA 路由 `/panel/api-docs`）。这里交互式地列出了所有端点，附带描述、请求体和响应示例。

- **原始 OpenAPI 3.0 规范**在地址 `/panel/api/openapi.json` 提供。此 URL 可以直接喂给 Postman、Insomnia 或 `openapi-generator`。该规范在构建阶段嵌入到二进制文件中；当面板在非标准 `webBasePath` 下工作时，规范中的 `servers` 字段会自动重写为当前的基础路径，以便「Try it out」按钮和外部生成器命中正确的前缀。
-

14. Telegram 机器人

3X-UI 面板内置了一个 Telegram 机器人，可通过它接收服务器和客户端状态通知，并直接在即时通讯中管理客户端。该机器人采用长轮询技术（持续轮询 Telegram），因此无需外部域名或开放端口——只需服务器能访问 Telegram 服务器的出站连接即可。

机器人区分两类对话者：

- **管理员** – 其 Telegram User ID 填写在机器人设置中（字段「机器人管理员 User ID」）的用户。可访问所有功能：服务器统计、备份、客户端管理、重启 Xray。
- **客户端** – 其他任何用户，其 Telegram User ID 绑定到某个入站连接的具体客户端（客户端的 `tgId` 字段）。只能查看自己的订阅信息。

示例：将客户端绑定到 Telegram。 若要用户能查看自己的订阅统计，需将其数字 Telegram User ID 填写到客户端的 `tgId` 字段。在客户端的 JSON 配置中，如下所示：

```
{
  "email": "ivan",
  "id": "6f1e6b1a-0c3d-4f2a-9b7e-1a2b3c4d5e6f",
  "tgId": "123456789",
  "enable": true,
  "limitIp": 2,
  "totalGB": 53687091200,
  "expiryTime": 0
}
```

之后，User ID 为 123456789 的用户即可向机器人发送 `/usage ivan` 查看自己的统计数据。管理员也可通过客户端卡片上的「 设置 Telegram 用户」按钮填写该 ID，无需手动编辑 JSON。

14.1. 启用与配置机器人

所有机器人参数均在面板的 **设置** → **Telegram 机器人** 中配置。修改设置后保存即可生效——面板会立即应用，无需重启。若更改了启用标志（`tgBotEnable`）、令牌、管理员 User ID 或 API 服务器地址，面板会自动停止并以新参数重新启动机器人。此前“更改令牌后需重启面板”的规则已不再适用。

字段（界面）	配置键	默认值	说明
启用 Telegram 机器人	<code>tgBotEnable</code>	<code>false</code>	主开关。提示：「通过 Telegram 机器人访问面板功能」。关闭时，机器人不启动，通知任务也不计划。
Telegram 令牌	<code>tgBotToken</code>	（空）	机器人令牌。提示：「需从 Telegram 机器人管理器 @botfather 获取令牌」。令牌为空时机器人无法启动。
SOCKS 代理	<code>tgBotProxy</code>	（空）	连接 Telegram 的代理。提示：「如果连接 Telegram 需要 Socks5 代理，请按照指南配置其参数」。
Telegram API 服务器	<code>tgBotAPIServer</code>	（空）	备用 Telegram API 服务器。提示：「使用的 Telegram API 服务器。留空则使用默认服务器」。

字段（界面）	配置键	默认值	说明
机器人管理员 User ID	tgBotChatId	（空）	一个或多个管理员的 Telegram User ID，以逗号分隔。提示：「如需获取 User ID，请使用 @userinfobot 或在机器人中发送 /id 命令」。
机器人向管理员 发送通知的频率	tgRunTime	@daily	定期报告的 crontab 格式计划。提示：「以 Crontab 格式指定通知间隔」。
数据库备份	tgBotBackup	false	提示：「随定期报告附带数据库备份文件」。将备份附加到定期报告中。
登录通知	tgBotLoginNotify	true	提示：「当有人尝试登录您的面板时，显示用户名、IP 地址和时间」。
CPU 通知阈值	tgCpu	80	CPU 使用率阈值（百分比，校验范围 0-100）。提示：「当 CPU 负载超过此阈值时通知 Telegram 管理员（值：%）」。值为 0 时禁用 CPU 检查。
Telegram 机器人 语言	—	—	机器人生成所有消息所使用的语言。

通过 @BotFather 获取令牌

1. 在 Telegram 中打开与 @BotFather 的对话。
2. 发送命令 /newbot 并按提示操作（设置机器人名称及以 bot 结尾的唯一 username）。
3. BotFather 将返回形如 123456789:AA... 的令牌。将其复制到 Telegram 令牌 字段。

获取管理员 User ID

User ID 是账号的数字标识符（非用户名）。获取方式有两种：

- 向机器人 @userinfobot 发消息。
- 启动已配置好的机器人，发送命令 /id ——机器人会返回您的 ID。

将获得的数字填入 机器人管理员 User ID 字段。如需设置多名管理员，用逗号分隔各 ID（例如 11111111,22222222）。每个 ID 都会校验为整数；值不合法将导致机器人启动失败。

示例：「机器人管理员 User ID」字段的值。单名管理员——直接填写数字：

```
123456789
```

两名管理员用逗号分隔（空格可省略）：

```
123456789,987654321
```

每个值必须为整数。形如 @username 或 123 456（数字中间含空格）的格式不受支持——机器人将无法启动。

代理

支持 `socks5://`、`http://` 和 `https://` 协议。若代理字段留空，机器人将尝试使用面板的通用代理（若已设置且协议受支持）。协议不受支持或 URL 语法错误时，该设置将被忽略——机器人直接连接。当服务器无法直接访问 Telegram API 时，代理非常有用。

电子邮件通知 (SMTP)

除 Telegram 外，同样的事件也可通过邮件接收。该渠道在 **设置** → **Email** 的 **SMTP Settings** 选项卡中配置：

字段 (界面)	配置键	默认值	说明
Enable Email Notifications	<code>smtpEnable</code>	<code>false</code>	电子邮件通知的主开关（通过 SMTP）。
SMTP Host	<code>smtpHost</code>	(空)	SMTP 服务器主机（例如 <code>smtp.gmail.com</code> ）。
SMTP Port	<code>smtpPort</code>	<code>587</code>	SMTP 服务器端口。
SMTP Username	<code>smtpUsername</code>	(空)	SMTP 身份验证用户名，同时用作发件人地址 (From)。
SMTP Password	<code>smtpPassword</code>	(空)	SMTP 身份验证密码。以隐藏方式存储；若密码已设置，字段显示「已配置」标识，留空可保留当前密码。
Recipients	<code>smtpTo</code>	(空)	以逗号分隔的收件人列表（例如 <code>admin@example.com, ops@example.com</code> ）。
Encryption	<code>smtpEncryptionType</code>	<code>starttls</code>	连接加密类型： <code>none</code> （无加密）、 <code>starttls</code> （STARTTLS）或 <code>tls</code> （隐式 TLS）。

Send Test Email 按钮将发送一封测试邮件，并按阶段显示结果：**Connection**（连接）、**Authentication**（认证）和 **Send**（发送）。若出现问题，诊断信息会指出错误发生的阶段（例如「Authentication failed – check username and password」或「Server requires STARTTLS – change encryption type」），方便调整参数。

第二个选项卡 (**Notifications**) 用于选择哪些事件通过邮件发送——与 Telegram 使用相同的事件分组卡片（参见第 14.5 节「事件总线与通知选择」）。

Telegram API 服务器

默认情况下，机器人连接官方 Telegram API。在 **Telegram API Server** 字段中可填写自建 Bot API 服务器（`telegram-bot-api`）的地址。URL 会经过安全校验；被屏蔽或格式错误的地址将被忽略，回退使用默认服务器。

14.2. 主菜单与按钮

发送命令 `/start` 可调出菜单。按钮以 inline 键盘形式附在消息上；按钮组合取决于您是管理员还是客户端。

管理员菜单

按钮	操作
 已排序的流量使用报告	列出所有客户端，按流量排序，显示各自的消耗；无数据的「多余」email 标记为「  无结果」。
 服务器状态	服务器摘要（见第 14.5 节）。「  刷新」按钮可刷新数据。
重置所有流量	重置 所有 客户端的流量计数器。弹出确认（「您确定吗？  」），然后对每个客户端显示「  成功」或「  失败」，最后显示「  所有客户端流量重置完成」。
 数据库备份	发送数据库文件和 config.json（见第 14.6 节）。
 封禁日志	发送因超出 IP 限制而被封禁的 IP 地址日志文件。
 入站连接	所有 inbound 的摘要：备注、端口、流量、客户端数量、到期时间。
 即将到期	流量或有效期即将耗尽的 inbound 和客户端列表（见第 14.5 节）。
 命令	显示管理员命令帮助。
 在线	在线客户端数量及列表；点击 email 可打开客户端卡片。「  刷新」按钮。
 所有客户端	打开 inbound 选择，然后显示其客户端列表——用于查看/管理。
 新客户端	启动添加客户端向导（选择 inbound → 草稿 → 确认）。
订阅设置 / 独立链接 / 二维码	选择 inbound 和客户端，获取订阅链接、独立链接或二维码。

客户端菜单

客户端可使用的按钮有限：

按钮	操作
客户端统计	显示绑定到该客户端 Telegram User ID 的所有订阅数据。
 命令	显示客户端命令帮助。
订阅设置	选择自己的客户端 → 订阅链接。
独立链接	选择自己的客户端 → 独立链接。
二维码	选择自己的客户端 → 二维码。

若用户没有任何与其 Telegram User ID 绑定的客户端，机器人将回复：「 未找到您的配置！ 请让管理员在配置中使用您的 Telegram User ID。 您的 User ID：...」。请将该 ID 提供给管理员，由其填写到客户端字段中。

14.3. 机器人命令

自 3.5.0 版本起，Telegram 「/」菜单中注册了八条命令：

命令	说明（菜单中）	访问权限	功能
/start	显示主菜单	所有人	欢迎语；管理员还额外显示「  欢迎使用 管理机器人！」和主菜单。
/help	机器人帮助	所有人	显示通用欢迎语及选择菜单项的提示。
/status	检查机器人状态	所有人	回复「  机器人运行正常」。
/id	显示您的 Telegram ID	所有人	返回「  您的 User ID: ...」。便于获取自己的 User ID。
/usage	显示客户端用量：/usage email	所有人（见下文）	3.5.0 起加入菜单。
/inbound	搜索 inbound：/inbound remark (admin)	管理员	3.5.0 起加入菜单。
/restart	重启 Xray 核心 (admin)	管理员	3.5.0 起加入菜单。
/clearall	重置所有客户端流量 (admin)	管理员	3.5.0 新增： 请求确认（「取消」/「确认重置流量」按钮）并清零所有客户端的流量。

带参数命令的详细说明:

- **/usage [Email]** – 按 email 查找客户端。
 - 对**管理员**：显示完整的客户端卡片（含管理按钮）。
 - 对**客户端**：仅显示指定 email 的自有订阅（按 Telegram User ID 绑定匹配）。不带参数时，机器人提示：「 请提供要搜索的 email」。
- **/inbound [连接名称]** – 仅限管理员。按备注搜索 inbound 并显示其参数及所有客户端统计。不带参数时（或客户端使用时）回复：「 未知命令」。
- **/restart** – 仅限管理员。重启 Xray Core。可能的回复：「 Xray 核心重启成功」、「 Xray Core 未运行」（核心未运行时）、「 重启 Xray Core 时出错。<错误>」。/restart 后带任何参数均会回复未知命令提示。

在群组中， `/[?]?@botusername` 格式的命令仅在 username 与当前机器人名称匹配时才会被处理。

3.5.0 还有三处变更：**在线客服**列表中的按钮标注为 `email - inbound`（不同 inbound 中的同名客户端不再混淆）；**备份**和**封禁**日志消息以 `Hostname==...` 行开头——当一个机器人服务多个面板时很方便；按 **Telegram ID** 查找客户端卡片不再受设置 JSON 格式化方式的影响（此前来自节点或导入的「紧凑」记录无法被找到）。

管理员帮助（「命令」按钮）：

```

[?]Xray Core:/restart
[?]email [?]/usage [Email]
[?] [?]/inbound [?]
[?]Telegram User ID:/id

```

客户端帮助:

\$


14.4. 客户端管理（仅管理员）

打开客户端卡片（通过「所有客户端」、「在线」、「即将到期」或 `/usage`），管理员可查看客户端信息（email、绑定的 inbound、「启用」状态、连接状态、到期时间、流量消耗）及 inline 管理按钮：

按钮	功能
 刷新	重新加载客户端卡片。
 重置流量	清零客户端流量计数器。需确认「  确认重置流量?」。
 流量限制	设置流量限制。预设值：  无限制（0）、1/5/10/20/30/40/50/60/80/100/150/200 GB，或「  自定义」——通过内置数字键盘输入（0-9 按钮、「  」清零、「  」删除末位数字、「  确认：N」）。值以 GB 为单位。
 修改到期时间	预设选项：  无限制、「  自定义」、延长 7/10/14/20 天、延长 1/3/6/12 个月。正数延长有效期（在当前到期时间基础上累加天数，若已过期则从当前时间起算）；0 取消时间限制。
 IP 日志	显示记录的客户端 IP 地址（含时间戳，如有）。日志中提供「  刷新」和「  清除 IP」（需确认「  确认清除 IP?」）。
 IP 限制	限制同时在线的 IP 数量。选项：  无限制（0）、1-10 或「  自定义」（数字键盘）。
 设置 Telegram 用户	显示当前绑定的客户端 Telegram User ID；可清除绑定（「  删除 Telegram 用户」，需确认）。绑定新用户通过系统 Telegram 联系人选择器完成。
 启用/禁用	启用或禁用客户端。需确认「  确认启用/禁用用户?」。

所有更改配置的操作（流量/IP 限制、到期时间、绑定/解绑 Telegram 用户、启用/禁用）在必要时会标记 Xray 重启，以使更改生效。操作成功后，机器人显示「 : ...」形式的确认，并重新显示客户端卡片。

向导中输入的任何数字均限制为 < 999999。

14.5. 通知与报告

通知将发送给所有管理员（`tgBotChatId` 中的所有 User ID）。

事件总线与通知选择

通知基于统一的事件总线，支持两个投递渠道——**Telegram** 和**电子邮件（SMTP）**。每个渠道可单独选择要通知的事件类型。在 **设置** → **Telegram** 中通过 **Notifications** 选项卡配置，在 **设置** → **Email** 中通过同名选项卡配置。

事件按卡片分组；每组有一个主开关，显示已启用事件数（n/总数），部分选中时显示中间状态。可用分组：

- **Outbound** — 「Down」（`outbound.down`）和「Up」（`outbound.up`）：outbound 下线和恢复。

- **Xray Core** – 「Crash」 (`xray.crash`) : Xray 核心异常退出。
- **Nodes** – 「Down」 (`node.down`) 和 「Up」 (`node.up`) : 节点变为不可用或恢复。
- **System** – 「CPU high (%)」 (`cpu.high`) 和 「Memory high (%)」 (`memory.high`) : CPU 和内存高负载。两个事件旁均有内联阈值百分比输入框。
- **Security** – 「Login attempt」 (`login.attempt`) : 尝试登录面板。

已启用事件集分别存储: Telegram 使用 `tgEnabledEvents`, Email 使用 `smtpEnabledEvents`。两个渠道默认均启用「Login attempt」和「CPU high」(值为 `login.attempt,cpu.high`)。

面板登录通知

由 **登录通知** (`tgBotLoginNotify`, 默认启用) 复选框控制。每次尝试登录 Web 面板时, 管理员会收到消息:

- 登录成功: 「 面板登录成功。」 + 主机、用户名、IP、时间。
- 登录失败: 「 面板登录失败。」 + 主机、**原因** (例如, 输入错误的第二因素时显示「2FA 错误」)、用户名、IP、时间。

CPU 和内存负载超限

面板每分钟检查一次 CPU 和内存使用率。若 `tgCpu` > 0 且过去一分钟的平均 CPU 使用率超过阈值, 管理员将收到: 「 处理器负载为 N%, 超过阈值 M%」。内存使用率同样与 `tgMemory` 阈值 (默认 80%) 比对应「Memory high (%)」事件。

两个阈值均通过 Notifications 选项卡中 **System** 分组下「CPU high (%)」和「Memory high (%)」事件旁的内联输入框设置 (参见上文「事件总线与通知选择」)。Email 渠道使用独立的 `smtpCpu` 和 `smtpMemory` 键。阈值为 0 时, 对应检查将被禁用。

定期报告 (按计划)

按 **通知频率** (`tgRunTime`, 默认 `@daily`) 字段中的 cron 表达式计划执行。若值为空或无效, 则使用 `@daily`。报告包含:

计划构建器

机器人向管理员发送通知的**频率** 字段不是手动输入字符串, 而是通过计划构建器设置。首先在下拉列表中选择模式:

- **@every** – 按间隔重复 – 出现数字输入框和单位选择 (秒 / 分钟 / 小时); 结果组合为 `@every 6h` 形式的表达式。
- **@hourly** – 每小时、**@daily** – 每天 00:00、**@weekly** – 每周、**@monthly** – 每月 – 预设, 保存为对应宏 (`@hourly`、`@daily`、`@weekly`、`@monthly`)。
- **自定义 (crontab)** – 自定义 crontab 表达式的输入框。面板调度器启用了秒字段, 因此自定义表达式由 6 个字段组成: 秒、分、时、日、月、周 (例如, `0 30 8 * * *` – 每天 08:30:00)。切换到此模式时, 输入框会预填当前选择的 crontab 等效值, 方便在此基础上修改。

示例: 「通知频率」 (`tgRunTime`) 字段的值。支持预设缩写和完整 crontab 格式:

值	触发时间
@daily	每天午夜触发一次（默认值）
@hourly	每小时触发
@every 6h	每 6 小时触发
0 9 * * *	每天 09:00
0 9 * * 1	每周一 09:00
0 */12 * * *	每 12 小时触发（00:00 和 12:00）

crontab 字段顺序：分、时、日、月、周。

1. 一行「📄 计划报告：<计划>」及当前日期/时间。
2. 服务器状态（见下文）。
3. inbound 和客户端的「即将到期」模块。
4. 向绑定了 Telegram User ID 的客户端发送个人通知——每位非管理员客户端将收到其流量或有效期即将耗尽的订阅列表（含已禁用项）。
5. 若启用了数据库备份（tgBotBackup）——向管理员发送数据库备份。

服务器状态包含：主机名、3X-UI 和 Xray 版本、IPv4/IPv6、运行时间（天）、平均负载（Load1/2/3）、内存（当前/总量）、在线客户端数、TCP/UDP 连接计数、累计网络流量（↑/↓）及 Xray 状态。

「即将到期」显示：

- 按 inbound：已禁用数量和「即将耗尽」数量，然后列出相应 inbound（备注、端口、流量、到期时间）；
- 按客户端：同上，加上客户端卡片及 email 按钮（点击可打开客户端卡片）。

「即将耗尽」的阈值取自面板通用设置：流量余量（GB）和有效期余量（天）。当 inbound/客户端的剩余流量低于阈值或距到期时间少于阈值天数时，视为「即将耗尽」。

14.6. 备份与日志

- 数据库备份（「📄 数据库备份」按钮或定期报告中的复选框）：机器人发送备份时间、数据库文件（x-ui.db，PostgreSQL 为 x-ui.dump）及 Xray 配置文件 config.json。

机器人发送的备份文件名根据服务器地址生成：使用 webDomain 的值，若未设置则使用服务器的公网 IP。当从多个面板收集备份时，可据此判断文件来源。若无法确定地址，则使用通用名称。

- 封禁日志（「📄 封禁日志」按钮）：发送当前及上一个因超出 IP 限制而被封禁的 IP 地址日志文件。空文件不发送。

14.7. 工作特性

- 长消息会拆分为多条（阈值约 2000 字符），inline 键盘附在最后一条消息上。
- 并发性：命令和按钮点击并发处理（最多 10 个同时处理程序）。
- 发送可靠性：连接出错时，消息将以指数退避重试（1s/2s/4s，最多 3 次）。

- **缓存：**「服务器状态」数据会被缓存，以避免频繁点击「刷新」给系统带来压力。
 - **机器人重启：**保存影响机器人的设置（启用标志、令牌、管理员 User ID 或 API 服务器地址）时，面板会自动停止上一个轮询循环并以最新参数启动新循环——无需重载面板。同一时间只有一个更新接收实例运行。
-

15. 地理数据库（geoip / geosite 及自定义）

地理数据库是一些二进制 `.dat` 文件，Xray-core 用它们按国家归属（IP 段）或域名类别对流量进行路由和过滤。面板既能加载和更新标准的地理文件集合，也能加载按 URL 指定的任意用户自定义来源。所有文件都存放在 Xray 二进制文件旁边的 `bin` 目录中（默认路径为 `bin`，可通过环境变量 `XUI_BIN_FOLDER` 覆盖）。

15.1. 什么是 geoip.dat 和 geosite.dat

- **geoip.dat** – “IP 地址 → 国家/地区代码”的对应数据库。在路由规则中以 `geoip:<?>` 形式使用，例如 `geoip:ru`、`geoip:cn`，以及特殊标记 `geoip:private`（私有/本地网络）。本质上它回答的是“这个 IP 位于哪个国家”这一问题。
- **geosite.dat** – “域名 → 类别/列表”的对应数据库。以 `geosite:<?>` 形式使用，例如 `geosite:category-ads-all`（广告域名）、`geosite:google`、`geosite:ru`。本质上它是按组分类的域名列表。

这些文件用于构建诸如“所有发往俄罗斯 IP/域名的流量走直连，其余走 outbound”之类的规则。规则本身在 Xray 的路由部分定义；地理数据库只是为它们提供数据。如果没有最新的地理文件，引用 `geoip:` / `geosite:` 的规则将不会生效，或者会依据过时的列表运行。

示例：“俄罗斯域名和 IP 走直连”规则。 在路由部分中，这样的规则会把所有发往俄罗斯资源的流量导向带 `direct` 标签的 `outbound`：

```
{
  "type": "field",
  "domain": ["geosite:category-ru"],
  "ip": ["geoip:ru"],
  "outboundTag": "direct"
}
```

15.2. 标准地理文件及其更新

面板内置了一个固定的“白名单”（allowlist），包含六个标准文件，且其下载来源已硬编码。更新通过 `POST /panel/api/server/updateGeofile/:fileName` 执行（或不带文件名——用于一次性更新全部）。

示例：通过 API 更新单个文件和一次性更新全部。 仅更新 `geoip_RU.dat`：

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile/
geoip_RU.dat' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

用一个请求更新全部六个标准文件（不指定文件名）：

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

成功响应：

```
{ "success": true, "msg": "Geofile updated successfully", "obj": null }
```

文件名	来源（发布仓库）
geoip.dat	github.com/Loyalsoldier/v2ray-rules-dat (geoip.dat)
geosite.dat	github.com/Loyalsoldier/v2ray-rules-dat (geosite.dat)
geoip_IR.dat	github.com/chocolate4u/Iran-v2ray-rules (geoip.dat)
geosite_IR.dat	github.com/chocolate4u/Iran-v2ray-rules (geosite.dat)
geoip_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geoip.dat)
geosite_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geosite.dat)

更新标准文件的特点：

- **更新单个文件的按钮。** 下载前会弹出确认：“您确定要更新地理文件吗？”，并说明“这将更新文件 #filename#。”（英文 *Do you really want to update the geofile? This will update the #filename# file.*）。成功后会弹出通知“地理文件已成功更新”（英文 *Geofile updated successfully*）。
- **“Обновить все”按钮**（英文 *Update all*）会下载全部六个文件。确认提示：“这将更新所有地理文件。”（英文 *This will update all geofiles.*）。
- **条件下载。** 如果本地已存在该文件，请求中会带上 `If-Modified-Since` 头，其值为文件的修改时间。服务器返回 `304 Not Modified` 表示文件未变更——不会重复下载，只更新文件的时间戳。
- **文件名安全性。** 仅接受白名单中的名称；名称会被检查是否不含 `..`、路径分隔符 `/` 和 `\`、绝对路径，并且必须匹配模式 `^[a-zA-Z0-9._-]+\..dat$`。任何不在列表中的名称都会以错误“Invalid geofile name”被拒绝。
- **重启 Xray。** 下载地理文件后，Xray-core 会重启，以便重新读取更新后的数据库。如果重启失败，错误消息中会附加相应的说明行。

从命令行更新地理数据库（x-ui）

地理数据库也可以不通过面板更新——使用 `x-ui` 的交互式菜单（更新地理文件的菜单项），或非交互式命令 `x-ui update-all-geofiles`。对集合中的每个文件（geoip/geosite，包括 IR 和 RU 集合）都会输出单独的状态：“已更新”、“已是最新”或“下载错误”。下载失败时不会打印虚假的成功消息。只有当至少有一个文件确实被更新时，才会重启 Xray（也即断开活动连接）；如果没有任何文件发生变更（全部返回 `304 Not Modified`），面板和 Xray 都不会重启。

15.3. 通过 Xray 自动更新地理数据（Geodata Auto-Update）

按任意 URL 添加额外的 `.dat` 来源不是用面板自身的功能，而是通过 Xray-core 原生的 `geodata` 段实现。对应部分被放在 Xray 更新的模态窗口中（仪表盘 → Xray 更新，`xrayUpdates`）——即“Geodata 自动更新”选项卡（英文 *Geodata Auto-Update*）。面板在这里只是编辑 Xray 配置模板中的 `geodata` 键；文件的下载、校验和热重载由 Xray 内核本身负责。

该部分顶部显示一条提示：“Xray 会按计划下载这些文件并在不重启的情况下热重载它们。URL 必须是 HTTPS。在 Xray 能更新某个文件之前，该文件必须已存在于 bin 文件夹中。”（英文 *Xray downloads these*

files on schedule and hot-reloads them without a restart. URLs must be HTTPS. Each file must already exist in the bin folder once before Xray can update it.) 。

该部分的字段

- **计划 (cron)** (英文 *Schedule (cron)*) – 由 5 个字段组成的 cron 字符串；默认值为 `0 4 * * *` (每天 04:00)。保存时会校验该字符串恰好包含 5 个字段，否则会输出错误“Cron 必须包含 5 个字段，例如 `0 4 * * *`”。
- **通过 outbound 下载 (可选)** (英文 *Download through outbound (optional)*) – 一个下拉列表，包含可用 outbound 的标签 (外加订阅 outbound)，Xray 将通过它下载文件；协议为 `blackhole` 的 outbound 不会出现在列表中。该字段可留空——此时使用直接连接。这个选择与面板自身请求所用的 outbound (见 §11) 无关：geodata 自动更新有自己独立的下载用 outbound。
- **文件列表** – 每一行指定一对“URL + 文件名” (英文 *File name*)。URL 必须以 `https://` 开头 (否则“每个文件都需要 HTTPS URL。”)。文件名应填写简单形式，不含路径和分隔符——只能是字符 `^[A-Za-z0-9._-]+$` (否则“文件名必须是简单形式，例如 `geosite_custom.dat` (不含路径)。”)。输入 URL 时，面板会尝试根据路径的最后一段自动填入文件名。“Добавить файл”按钮 (英文 *Add file*) 会添加一行，垃圾桶按钮会删除该行。

如果列表为空，会显示提示：“尚未配置文件。在路由规则中将文件引用为 `ext:geosite_custom.dat:category`。” (英文 *No files configured. Reference files in routing rules as ext:geosite_custom.dat:category.*) 。

保存

“保存并重启 Xray”按钮 (英文 *Save & Restart Xray*) 会弹出确认“保存 geodata 设置？”，并说明“Xray 配置模板将被更新，且 Xray 将被重启。” (英文 *Save geodata settings? This updates the Xray config template and restarts Xray.*)。保存后，geodata 键会写入配置模板 (`POST /panel/api/xray/update`)，并重启 Xray (`POST /panel/api/server/restartXrayService`)。如果文件列表为空，会从模板中删除 geodata 键。

重要特点：

- **文件必须已存在于 bin 中。** Xray 只更新启动时已存在于 `bin` 文件夹中的那些 `.dat` 文件。因此，新的自定义文件要先手动放入 `bin` (或至少在那里以所需名称创建一个空的/过时的版本)，之后 Xray 才会按计划将其保持为最新状态。
- **热重载。** 在计划下载之后，Xray 会重新读取更新后的数据库，而无需完全重启进程。
- **兼容性。** 之前已下载的地理文件 (无论是标准还是自定义) 在路由规则中以 `ext:` 语法继续工作，无需更改。

如果列表为空，会显示提示：“暂无自定义 geo 来源——点击“添加”以创建” (英文 *No custom geo sources yet – click Add to create one*) 。

表格列与来源字段

字段 (UI)	JSON	默认值	描述
类型 (Type)	<code>type</code>	– (必填)	资源类型: 仅 <code>geosite</code> 或 <code>geoip</code> 。决定最终文件的名称。
别名 (Alias)	<code>alias</code>	– (必填)	来源的简短标识符。由它和类型构成文件名。
URL (URL)	<code>url</code>	– (必填)	<code>.dat</code> 文件的直接链接 (http/https)。
已启用 (Enabled)	–	–	来源在列表中是否处于活动状态的标志。
已更新 (Last updated)	<code>lastUpdatedAt</code>	<code>0</code>	上次成功更新的时间 (Unix 时间; <code>0</code> 表示尚未更新过)。
路由 (ext:...) (Routing (ext:...))	–	–	用于路由规则的现成字符串: <code>ext:<[?].dat>:tag</code> 。
操作 (Actions)	–	–	“修改”、“删除”、“立即更新”按钮。

此外, 数据库中还存储一些辅助字段: `localPath` (文件在 `bin` 目录中的实际路径)、`lastModified` (服务器返回的 `Last-Modified` 头的值, 用于条件下载)、`createdAt` 和 `updatedAt`。

文件命名

最终文件的名称由类型和别名自动生成:

- 类型 `geoip` → `geoip_<alias>.dat`;
- 类型 `geosite` → `geosite_<alias>.dat`。

例如, 类型为 `geosite`、别名为 `myads` 的来源会创建文件 `geosite_myads.dat`。

示例: 通过 API 添加来源。 将你自己的广告域名列表添加为别名为 `myads` 的 `geosite` 资源:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/customGeo/add' \
-H 'Cookie: 3x-ui=<session-cookie>' \
-H 'Content-Type: application/json' \
-d '{
  "type": "geosite",
  "alias": "myads",
  "url": "https://example.com/lists/myads.dat"
}'
```

面板会把文件下载到 `bin` 目录并命名为 `geosite_myads.dat`, 保存记录并重启 Xray。

按钮和操作

- **添加** (英文 *Add*) – 打开“添加来源”表单 (英文 *Add custom geo*)。保存按钮为“保存” (英文 *Save*)。API: `POST /add`。

- **修改**（英文 *Edit*）– “修改来源”表单（英文 *Edit custom geo*）。API: `POST /update/:id`。更改类型或别名时，旧文件会被删除，新文件会被重新下载。
- **删除**（英文 *Delete*）– 确认提示“删除此自定义 geo 来源？”（英文 *Delete this custom geo source?*）。删除数据库中的记录以及 `.dat` 文件本身。API: `POST /delete/:id`。成功时：“自定义 geo 文件「<名称>」已删除”。
- **立即更新**（英文 *Update now*）– 重新下载特定来源并更新时间戳。API: `POST /download/:id`。成功时：“地理文件「<名称>」已更新”。
- **更新全部** – 一次性更新所有自定义来源。API: `POST /update-all`。全部成功时：“所有自定义 geo 来源已更新”（英文 *All custom geo sources updated*）。如果至少有一个来源未更新成功，则该操作被视为部分失败，提示“一个或多个自定义 geo 来源更新失败”（英文 *One or more custom geo sources failed to update*），且响应中会列出成功和失败的来源。

在执行上述任一操作（添加、修改、删除、更新、在有成功项时的更新全部）之后，Xray-core 都会重启。

分步：添加来源

1. 点击“添加”。
2. 在“类型”字段中选择 `geosite` 或 `geoip`。
3. 在“别名”字段中输入标识符（仅限小写拉丁字母、数字、`-` 和 `_`；占位提示：`a-z 0-9 _ -`）。
4. 在“URL”字段中填写 `.dat` 文件的直接链接（必须以 `http://` 或 `https://` 开头）。
5. 点击“保存”。面板会立即把文件下载到 `bin` 目录，保存记录并重启 Xray。

15.4. 校验和限制

在创建和修改来源时会执行严格的检查。错误消息：

条件	消息 (RU)	消息 (EN)
类型不是 <code>geosite / geoip</code>	Тип должен быть geosite или geoip	<i>Type must be geosite or geoip</i>
别名为空	Укажите псевдоним	<i>Alias is required</i>
别名包含不允许的字符（不匹配 <code>^[a-z0-9_-]+\$</code> ）	Псевдоним содержит недопустимые символы	<i>Alias must match allowed characters</i>
别名被保留	Этот псевдоним зарезервирован	<i>This alias is reserved</i>
URL 为空	Укажите URL	<i>URL is required</i>
URL 无法解析	Некорректный URL	<i>URL is invalid</i>
协议不是 <code>http/https</code>	URL должен использовать http или https	<i>URL must use http or https</i>
主机为空/无效，或被 SSRF 防护阻止	Некорректный хост URL	<i>URL host is invalid</i>
“类型 + 别名”重复	Такой псевдоним уже используется для этого типа	<i>This alias is already used for this type</i>
来源未找到	Источник не найден	<i>Custom geo source not found</i>
下载错误	Ошибка загрузки	<i>Download failed</i>

表单中的提示（客户端校验）：“别名：仅 a-z、数字、- 和 _”（*Alias may only contain lowercase letters, digits, - and _*）以及“URL 必须以 http:// 或 https:// 开头”（*URL must start with http:// or https://*）。

额外的技术限制：

- **保留别名。** 不能使用与标准文件冲突的别名。被保留的有（比较时不区分大小写，连字符等同于下划线）：`geoip`、`geosite`、`geoip_ir`、`geosite_ir`、`geoip_ru`、`geosite_ru`。例如，`geosite-ru` 会被当作 `geosite_ru` 而被拒绝。
- **SSRF 防护。** URL 的主机会被解析为 IP，如果它指向私有/内部地址，下载会被阻止（用户会看到“Некорректный хост URL”）。这可以防止利用面板访问内部服务。
- **路径穿越防护。** 文件的最终路径必须位于 `bin` 目录内部（含符号链接解析）；尝试越出其范围会被拒绝。
- **文件最小大小。** 下载的文件只有不小于 64 字节才被视为有效；过小的文件会以下载错误被拒绝。
- **代理与条件下载。** 如果面板设置中指定了代理，下载会通过它进行；其余情况下使用具备 SSRF 安全性传输的直接连接。与标准文件一样，会应用 `If-Modified-Since / 304 Not Modified`（未变更的文件不会重复下载）。下载超时为 10 分钟，URL 可用性探测（HEAD，必要时为部分 GET）为 12 秒。

15.5. 面板启动时的自动检查

面板启动时会遍历所有自定义来源，并为每个来源检查本地文件的存在性和完整性（文件缺失、是目录或小于 64 字节）。如果文件缺失或损坏，会对来源进行探测并尝试重新下载。这保证了在重新安装或丢失 `bin` 目录后，自定义地理文件会被自动恢复。

15.6. 在路由规则中使用地理数据库

在 Xray 的路由规则中，地理数据库通过前缀在 `domain / ip` 之类的字段中使用：

- **geoip:** 用于 IP 数据库——`geoip:<??>`。示例：`geoip:ru`、`geoip:cn`、`geoip:private`。取自 `geoip.dat`（如果规则指向具体文件，则取自 `geoip_RU.dat` 等）。
- **geosite:** 用于域名数据库——`geosite:<??>`。示例：`geosite:category-ads-all`、`geosite:google`、`geosite:ru`。取自 `geosite.dat`。

示例：通过 geosite 屏蔽广告。 把所有广告域名发往“黑洞”的规则（假设有一个带 `blocked` 标签、协议为 `blackhole` 的 `outbound`）：

```
{
  "type": "field",
  "domain": ["geosite:category-ads-all"],
  "outboundTag": "blocked"
}
```

对于用户自定义文件，使用外部文件语法 `ext:`。UI 中的提示：“在路由规则中将该值用作 `ext:文件.dat:tag`（替换 tag）。”（英文 *In routing rules use the value column as `ext:file.dat:tag` (replace tag).*）。格式：

```
ext:<???.dat>:<??>
```

其中 `<???.dat>` 为 `geoip_<alias>.dat` 或 `geosite_<alias>.dat`，而 `<??>` 为文件内部的具体列表/类别。面板在“路由（ext:...）”列中提示一个现成模板，形如 `ext:geosite_myads.dat:tag`——只需把 `tag` 替换为所需的标签。这样的文件名在“Geodata 自动更新”部分（见 §15.3）的“文件名”字段中指定——例如 `geosite_custom.dat`；在规则中以 `ext:geosite_custom.dat:category` 引用它。

示例：基于自定义文件的规则。 如果添加了类型为 `geosite`、别名为 `myads` 的来源，且 `.dat` 文件内部的列表标记为 `ads` 标签，那么路由规则如下：

```
{
  "type": "field",
  "domain": ["ext:geosite_myads.dat:ads"],
  "outboundTag": "blocked"
}
```

对于 IP 来源（类型 `geoip`、别名 `mycorp`、标签 `office`），字段为 `"ip": ["ext:geoip_mycorp.dat:office"]`。

16. 日常运维：备份、日志、更新、CLI

本节涵盖面板的日常维护工作：创建与恢复数据库备份、查看面板日志和 Xray 日志、重启与停止服务、更新 Xray 及面板本身、定时任务（cron）以及卸载面板。部分操作在 Web 界面（「仪表盘」和「面板设置」页面的各标签页）中完成，部分操作则通过服务器上的 `x-ui` 控制台菜单执行。

16.1. 数据库备份与恢复

面板的所有数据（inbound、客户端、群组、节点、设置）均保存在同一个数据库中。备份管理功能位于**「仪表盘」页面的「备份」标签页，区块标题为「备份与恢复」**。

面板支持两种数据库引擎，备份行为因此有所不同：

- **SQLite**（默认）——数据保存在 `x-ui.db` 文件中。
- **PostgreSQL**——若面板配置为使用 PostgreSQL，区块中将显示如下提示：

「此面板运行于 PostgreSQL。"备份"会下载 `pg_dump` 归档文件（.dump），"恢复"则通过 `pg_restore` 将其还原。恢复也接受 SQLite 数据库（.db）或 SQLite 迁移转储，并会将其数据导入 PostgreSQL。服务器上须安装 PostgreSQL 客户端工具（`pg_dump` 和 `pg_restore`）。」

导出（创建备份）

「导出数据库」按钮（`Back Up`）会将备份文件下载到您的设备。

数据库引擎	文件名	服务器端操作
SQLite	<code>{host}_YYYY-MM-DD_HHMMSS.db</code>	先执行 WAL checkpoint 以确保文件包含最新记录，然后完整读取文件并提供下载
PostgreSQL	<code>{host}_YYYY-MM-DD_HHMMSS.dump</code>	运行 <code>pg_dump</code> ，将归档文件提供下载

自 3.4.2 起，下载的备份文件名包含您打开面板所用的地址以及日期时间：`{host}_YYYY-MM-DD_HHMMSS`，扩展名为 `.db`（SQLite）或 `.dump`（PostgreSQL）——例如 `panel.example.com_2026-06-27_000000.db`。这样文件可按服务器分组、按时间排序，同一天的多个备份也不会相互覆盖。Telegram 机器人发送的备份也使用相同的文件名（`host` 取自面板的域名/IP，备用值为 `x-ui`）。

界面提示：

- SQLite：「点击即可将包含当前数据库备份的 .db 文件下载到您的设备。」
- PostgreSQL：「点击即可将当前数据库的 PostgreSQL 转储文件（.dump）下载到您的设备。」

技术上，导出是向 `GET /panel/api/server/getDb` 发送请求。附件名称由服务器根据数据库引擎通过 `Content-Disposition` 生成。

备份文件名根据服务器地址生成，而非固定为 `x-ui.db` / `x-ui.dump`。从浏览器下载时，文件名取自地址栏中的面板地址（请求主机名）；否则取自已配置的 web 域名；若未配置域名，则取自服务器公网 IP（优先 IPv4，其次 IPv6），回退值为 `x-ui`。这样便于区分来自不同服务器的备份。扩展名保持 SQLite 用 `.db`、PostgreSQL 用 `.dump`；通过 Telegram 发送的备份同样以域名/IP 命名。

示例：通过 API 下载备份。 同样的导出也可以通过控制台请求获取——例如用于自动备份脚本。需要已授权的会话（登录 cookie）：

```
# 1) 获取 cookie
curl -s -c cookies.txt \
  -d 'username=admin&password=admin' \
  https://panel.example.com:2053/panel/login

# 2) 下载 x-ui.db 或 x-ui.dump
curl -s -b cookies.txt -OJ \
  https://panel.example.com:2053/panel/api/server/getDb
```

如果面板启用了基础路径（Web Base Path），需在 URL 中添加：`...:2053/<base_path>/panel/api/server/getDb`。

导入（恢复）

****「导入数据库」按钮（Restore）** 会打开文件选择器，并将文件上传至服务器进行恢复（POST `/panel/api/server/importDB`，表单字段 `db`）。

界面提示：

- SQLite：「点击即可从您的设备选择并上传 .db 文件或迁移转储 (.dump)，以恢复数据库。」
- PostgreSQL：「点击即可选择并上传 PostgreSQL 备份 (.dump)、SQLite 数据库 (.db) 或 SQLite 迁移转储，以恢复数据库。这将替换所有当前数据。」

自 3.5.0 起，文件类型按内容而非扩展名判断（PGDMP 归档；「SQLite format 3」文件头——.db 数据库；以 PRAGMA / BEGIN TRANSACTION 开头——文本 SQL 转储）；两种数据库引擎的选择对话框均接受 .dump, .db。上传到 PostgreSQL 面板的 SQLite 文件会以单个事务导入 PostgreSQL（见 3.14）；导入前会验证文件确为真正的面板数据库，旧 schema 会自动补齐迁移。恢复 pg_dump 归档前，面板会提前（在停止 Xray 之前）检查其可读性：若转储由更新版本的 PostgreSQL 创建，则报错并给出准确命令——例如「...run 'x-ui pgclient 17'...」。

SQLite 的导入流程（重要：该流程是原子性的，且支持回滚）：

1. 检查上传文件的格式——必须是有效的 SQLite 数据库；否则返回错误「Invalid db file format」。
2. 文件保存为临时文件 `x-ui.db.temp` 并进行完整性验证。
3. 替换数据库前**停止 Xray**。
4. 当前数据库重命名为备份文件 `x-ui.db.backup`（回滚用）。
5. 临时文件移至工作数据库位置，执行初始化和 schema 迁移，随后进行 inbound 迁移。
6. 若任何步骤失败——执行回滚：从 `x-ui.db.backup` 恢复原数据库，并在旧数据上重启 Xray。
7. 成功时删除回滚文件，**Xray 自动在恢复后的数据上重启**。

界面最终提示信息：

结果	文本
成功	「数据库导入成功」
导入错误	「导入数据库时发生错误」
文件读取错误	「读取数据库时发生错误」

恢复操作将完全替换当前数据。由于 Xray 在过程中会短暂停止，导入期间现有客户端连接将中断。

数据库引擎迁移文件 (SQLite ⇌ PostgreSQL)

除普通备份外，还有**「下载迁移文件」**功能 (Download Migration，请求 `GET /panel/api/server/getMigration`)。它生成用于切换数据库引擎的可移植文件：

自 3.5.0 起，该按钮仅在 **PostgreSQL** 面板上显示（在 SQLite 上，普通 `.db` 备份现在可以直接恢复到 PostgreSQL——无需单独导出）：

当前引擎	下载内容	文件名	用途
PostgreSQL	从 PostgreSQL 数据构建的 SQLite 数据库	<code>x-ui.db</code>	将面板迁回 SQLite

提示：「点击即可下载从 PostgreSQL 数据构建的 SQLite 数据库 (`.db`)，可直接用于在 SQLite 上运行面板。」

SQLite 的 `.db ⇌ .dump` 转换也可通过 CLI 命令 `x-ui migrateDB [file]` 执行（见第 16.7 节）。此外，自 3.5.0 起，数据库引擎之间的迁移已实现**事务化且无损**（客户端分组和全局流量计数器等也会一并迁移；导入失败不会改动目标数据库），并新增了用于更新 PostgreSQL 客户端工具的 `x-ui pgclient [版本]` 命令和 PostgreSQL 菜单中的 **10. Install/Upgrade client tools (pg_dump/pg_restore)** 菜单项（必要时会接入 PostgreSQL 官方软件源）。

通过 Telegram 机器人备份

若已配置 Telegram 机器人（见通知相关章节），它可以直接在管理员聊天中发送备份文件。通过 Telegram 发送的备份包含**两个文件**：数据库本身（`x-ui.db`，PostgreSQL 则为 `x-ui.dump`）以及 Xray 配置文件 `config.json`。消息前会有一行「📅 备份时间：...」。

获取 Telegram 备份有两种方式：

- 按需获取。** 机器人菜单中的**「📁 备份数据库」**按钮——机器人立即在当前聊天中发送文件。
- 随报告自动发送。** 机器人设置中有**「数据库备份」** (Database Backup) 开关，描述为「发送包含数据库备份文件的通知」。启用后，每次定期发送报告时，机器人会在报告之后向所有管理员发送备份文件。报告发送周期通过机器人的 cron 计划设定（见第 16.6 节）。机器人在文件之间和管理员之间会暂停一下，以免超出 Telegram 的频率限制。

通过机器人备份仅在机器人运行时有效；在 PostgreSQL 上还需要服务器上有 `pg_dump`。

16.2. 查看日志

面板有两个独立的日志查看器，均可从「仪表盘」的**「日志」**标签页打开。每个窗口均支持刷新（标题栏的「刷新」图标）以及将当前内容下载为 `x-ui.log` 文件（下载图标按钮）。

面板日志（应用程序 / syslog）

面板日志窗口（`POST /panel/api/server/logs/{count}`）。控件说明：

控件	默认值	说明
行数	20	下拉列表：20 / 50 / 100 / 500 / 1000
级别	Info	最低级别：Debug / Info / Notice / Warning / Error
SysLog（复选框）	关闭	日志来源：应用程序内部缓冲区或系统日志
自动更新（复选框）	关闭	每 5 秒重新读取日志（见下文）

行为取决于 **SysLog** 复选框：

- **关闭（默认）**：日志来自面板内部的环形缓冲区，按选定级别过滤。记录显示时附带级别（DEBUG / INFO / NOTICE / WARNING / ERROR）和来源：`X-UI`：表示面板自身消息，`XRAY`：表示转发的 Xray 消息。

不带时间戳和级别的简单通知（例如 Windows 上的系统消息「Syslog is not supported」）现在会完整原样显示。严格识别 `YYYY/MM/DD LEVEL - [?]` 格式；其余内容不加解析直接输出，因此此类行不再被截断（此前前三个词被错误解析为日期/时间/级别）。

- **开启**：面板在服务器上执行 `journalctl -u x-ui --no-pager -n <count> -p <level>`，即显示 `x-ui` 服务的系统日志。允许的行数为 1 至 10000；级别接受 syslog 值（`emerg/0`、`alert/1`、`crit/2`、`err/3`、`warning/4`、`notice/5`、`info/6`、`debug/7`）。Windows 不支持 SysLog 模式——将显示提示，需取消勾选并使用应用程序日志。若 `systemd`/服务不可用，将显示启动 `journalctl` 的错误消息。

示例：直接在服务器控制台查看同一日志。 当面板不可访问（例如无法启动）时，可直接读取系统日志——这正是面板在 SysLog 模式下执行的命令：

```
# [?] 100 [?]warning [?]
journalctl -u x-ui --no-pager -n 100 -p warning

# [?]
journalctl -u x-ui -f
```

此窗口中的级别用于过滤输出。实际写入控制台/syslog 的最低级别由面板的日志级别设置决定（环境变量，默认为 `Info`；面板写入文件时始终使用 `DEBUG` 级别）。

Xray 访问日志 (access log)

用于查看 Xray access 日志的独立窗口（`POST /panel/api/server/xraylogs/{count}`）。它解析 Xray 访问日志行，并以表格形式展示：**Date**、**From**、**To**、**Inbound**、**Outbound**、**Email**。

自 3.4.1 起，此窗口及 Xray 状态卡片上的对应按钮标注为**「访问日志」**（Access Logs）——此前均简称为「日志」。重命名是为了区分 Xray access 日志查看器和面板自身日志查看器（两者之前同名）。

控件	默认值	说明
行数	20	20 / 50 / 100 / 500 / 1000
过滤器	空	子字符串文本搜索（按 Enter 应用）
自动更新（复选框）	关闭	每 5 秒重新读取日志（见下文）
Direct（复选框）	开启	显示直连连接（通过 freedom outbound 的流量）
Blocked（复选框）	开启	显示被拦截的连接（通过 blackhole outbound 的流量）
Proxy（复选框）	开启	显示代理流量

事件类型根据日志行中的 outbound 标签自动判断：匹配 freedom 标签 → 「DIRECT」（绿色），blackhole → 「BLOCKED」（红色），其余 → 「PROXY」（蓝色）。api -> api 行和空行会被跳过。

自动更新。两个日志窗口（「日志」和「访问日志」）均有**「自动更新」**（Auto Update）复选框。启用后，日志内容每 5 秒自动重新读取一次，保留所有当前窗口设置——所选行数、级别/过滤器以及 Direct / Blocked / Proxy 复选框状态。窗口关闭或取消勾选后，轮询停止。

要让此窗口显示记录，Xray 必须启用**访问日志**并配置文件路径（不能为 none）——详见下文。若 access 日志已禁用或文件不可访问，窗口将为空（「No Record...」）。

16.3. Xray 日志级别与配置

Xray 自身的日志参数在**「Xray 配置」页面的「日志」**（Log）区块中设置，并附有警告：

「日志可能会降低服务器性能。请仅在需要时启用所需的日志类型！」

字段	翻译	默认值	说明
日志级别 (logLevel)	Log Level	warning	Xray 错误日志的详细级别。可选值：debug、info、notice、warning、error。提示：「日志级别，用于指定需要记录的错误日志信息。」
访问日志 (accessLog)	Access Log	none	访问日志文件路径。特殊值 none 禁用访问日志。提示：「访问日志文件路径。特殊值 "none" 禁用访问日志。」
错误日志 (errorLog)	Error Log	空（默认路径）	错误日志文件路径；none 禁用。提示：「错误日志文件路径。特殊值 "none" 禁用错误日志。」

字段	翻译	默认值	说明
DNS 日志 (dnsLog)	DNS Log	false (关闭)	启用 DNS 请求日志记录。提示：「启用 DNS 请求日志」。
地址掩码 (maskAddress)	Mask Address	空 (关闭)	启用后，日志中的真实 IP 地址会自动替换为掩码地址。提示：「启用后，日志中的真实 IP 地址将替换为掩码地址。」

由于默认**「访问日志」= none**，第 16.2 节中的「Xray 日志」窗口初始为空。要使其生效，请在此设置 access 日志路径并重启 Xray。

请注意：空 access 日志只影响此窗口。「仪表盘」上的在线客户端列表以及客户端表单中的 IP 数量限制**不依赖** access 日志——面板通过 Xray 核心的 online-stats API（连接统计）来判断在线客户端并统计其 IP 地址。若核心版本较旧不支持该 API，面板会自动回退到旧方式（读取 access 日志），此时仍需在此设置 access 日志路径以使 IP 限制生效。

IP 数量限制与 fail2ban。 客户端 IP 数量限制（客户端表单及批量添加中的「IP Limit」字段）仅在服务器上安装了 **fail2ban** 时才会生效——由 fail2ban 来封锁超出限制的 IP。面板会检测 fail2ban 是否存在（GET /panel/api/server/fail2banStatus）；若不存在，「IP Limit」字段将变为不可用并显示说明提示（Windows 上显示单独消息），此前设置的限制也会在此类服务器上自动清零，因为它们本就不生效。fail2ban 的封锁同时作用于 TCP 和 UDP。自 3.5.0 起，「死」连接（客户端未正常关闭 TCP 就消失）只会被记录一次，而不是在每次 10 秒扫描时重复记录：[LIMIT_IP] 行和断开周期仅在真正重新连接时才会重复，因此 fail2ban 计数器不再虚增，也无需再调高 maxretry（日志格式和 failregex 未改变）。在常规服务器上，fail2ban 现已在面板安装和更新时自动安装（见第 16.5 节）。

示例：使「Xray 日志」窗口开始显示记录的 log 配置块。 在 Xray 的 JSON 配置中，如下所示：

```
{
  "log": {
    "loglevel": "warning",
    "access": "./access.log",
    "error": "",
    "dnsLog": false,
    "maskAddress": ""
  }
}
```

关键是将 "access": "none" 改为文件路径（例如 "./access.log"）。保存后重启 Xray，「Xray 日志」窗口中的表格便会显示记录。

16.4. 管理 Xray：停止与重启

Xray 的状态通过「仪表盘」上的 Xray 卡片进行管理。当前状态显示为以下值之一：**已运行**（Running）、**已停止**（Stopped）、**未知**（Unknown）、**错误**（Error）。出错时会显示「启动 Xray 时出错」的悬停提示。

按钮	翻译	端点	操作
停止	Stop	POST /panel/api/server/stopXrayService	停止 Xray 进程。成功时显示警告通知「Xray service has been stopped」。
重启	Restart	POST /panel/api/server/restartXrayService	使用当前配置重启（或启动）Xray。成功时显示通知「Xray service has been restarted successfully」。

任意操作完成后，面板会通过 WebSocket 广播新状态，因此「仪表盘」上的状态无需刷新页面即可更新。若操作失败，Xray 状态变为「错误」，错误信息出现在通知中。

除手动重启外，面板会自动检查是否需要重启 Xray（每 30 秒一次后台任务）以及进程是否崩溃（每秒检查一次）——见第 16.6 节。

隧道健康监控（Xray 自动重启）

3.4.1 版本新增了可选的隧道健康监控功能。启用后，面板会定期检测指定 URL 的可达性，在连续多次检测失败后自动重启 Xray 核心——有助于恢复停止转发流量的隧道。默认情况下监控已禁用，且仅通过服务环境变量配置（Web 界面中没有相关设置——这是作者的设计意图）。

启用监控需设置 `XUI_TUNNEL_HEALTH_MONITOR=true`。变量 `XUI_TUNNEL_HEALTH_PROXY` 应指向本地 xray-inbound（例如 `socks5://127.0.0.1:1080`）——这样探测会经过 Xray 本身，从而真正检验隧道是否正常；不设置时仅检查主机连通性，重启也无法解决服务器网络问题。其余变量用于配置检测参数：

变量	用途	默认值
<code>XUI_TUNNEL_HEALTH_MONITOR</code>	启用监控（开/关）	<code>false</code>
<code>XUI_TUNNEL_HEALTH_PROXY</code>	探测所用代理（请指向本地 xray-inbound）	空
<code>XUI_TUNNEL_HEALTH_URL</code>	被检测的 URL	<code>https://www.cloudflare.com/cdn-cgi/trace</code>
<code>XUI_TUNNEL_HEALTH_INTERVAL</code>	检测间隔	<code>30s</code>
<code>XUI_TUNNEL_HEALTH_TIMEOUT</code>	单次检测超时	<code>10s</code>
<code>XUI_TUNNEL_HEALTH_FAILURES</code>	触发重启所需的连续失败次数	<code>3</code>
<code>XUI_TUNNEL_HEALTH_COOLDOWN</code>	两次重启之间的最小间隔	<code>5m</code>

重启 Xray 会断开所有已连接客户端的连接，因此建议将间隔和失败阈值设置得足够大，避免单次偶发探测失败触发不必要的重启。

16.5. 重启与更新面板

重启面板

****「面板设置」页面中有「重启面板」**操作（Restart Panel，POST /panel/api/setting/restartPanel）。确认后，面板将在 3 秒后重启。**

提示信息：

- 确认：「您确定要重启面板吗？确认后将在 3 秒后重启。若面板无法访问，请检查服务器日志。」
- 成功：「面板重启成功」。

在 Linux 上，重启技术上通过向面板进程发送 SIGHUP 信号（或通过注册的钩子）实现。Windows 不支持发送 SIGHUP。

面板自更新（Update Panel）

「仪表盘」上提供****「更新面板」****（Update Panel）功能——直接从 Web 界面将 3X-UI 更新至最新版本。

更新前，面板会比对版本（GET /panel/api/server/getPanelUpdateInfo），向 GitHub 请求最新的 3x-ui 版本：

字段	翻译
当前面板版本	Current panel version
最新面板版本	Latest panel version
面板已是最新 / 「已是最新」	Panel is up to date / Up to date——若无新版本则显示

启动更新——POST /panel/api/server/updatePanel。确认对话框：

- 「您确定要更新面板吗？」
- 「这将会把 3X-UI 更新至 #version# 版本并重启面板服务。」

启动后显示弹出消息「面板更新已启动」（Panel update started）；版本检查失败时显示「面板更新检查失败」（Panel update check failed）。

服务器端发生的操作：自更新仅在 Linux 上支持（其他操作系统将返回错误「panel web update is supported only on Linux installations」）。面板从 GitHub

（raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh）下载官方 update.sh 脚本并在独立进程中运行：优先通过 systemd-run 在独立单元（x-ui-web-update-<timestamp>）中运行；若无 systemd 则作为独立的分离进程运行。脚本完成后会更新组件并重启面板服务。运行需要 bash。

若更新过程中脚本生成了新的随机 Web Base Path，x-ui 服务会自动重启，以使新路径立即生效。（若不重启，服务器将继续使用旧路径，而界面显示新路径，导致新地址在手动重启前无法访问。）

Dev 渠道（滚动），自 3.4.2 起。「仪表盘」上的面板版本按钮现在始终打开更新窗口（此前在最新稳定版构建上会跳转到 GitHub 发布页），而****「Dev 渠道」****开关（devChannelEnable）始终显示——即使在稳定版构建上也是如此。启用后，下次更新时即可切换到「滚动」dev 渠道（按 main 分支每次提交构建）。启

用时会显示警告：「Dev 构建跟踪 main 的每次提交，并非稳定版——无自动回滚」。在用户未明确操作前，不会更新任何内容。

开发频道更新（滚动构建）

除更新至稳定版本外，还有可选的**「开发频道」（Dev）。该开关仅在 dev 构建**（按单独提交构建的 CI 版本）上的更新窗口中显示；稳定版本中不可见。启用后，面板将更新至 dev-latest 滚动构建，该构建跟踪 main 分支的每个提交，并非稳定版——会显示警告，提示 dev 构建不稳定且无法自动回滚。在 dev 模式下，窗口显示「当前提交」/「最新提交」，而非版本号。此功能仅在带有 systemd 的 Linux 上可用。

在 dev 构建中，面板版本显示为 dev+<[?][?][?][?]>，而非误导性的稳定版本号——出现在侧边栏徽章、「仪表盘」卡片、更新窗口、Telegram 机器人状态报告以及 x-ui -v 命令输出中。稳定版本的版本显示方式不变。

节点（nodes）上运行的同一 3x-ui 面板可通过 POST /panel/api/nodes/updatePanel 集中更新——见节点相关章节。

自动安装 fail2ban

为使客户端 IP 数量限制（第 16.3 节）开箱即用，面板在常规服务器上安装和更新时现已自动安装并配置 fail2ban（此前仅在 Docker 镜像中自动安装）。行为由环境变量 XUI_ENABLE_FAIL2BAN 控制：若变量未设置或值为 true 则执行配置。也可通过命令 x-ui setup-fail2ban 手动运行。fail2ban 配置失败不会中断面板的安装或更新。

在仅 IPv6 主机上安装与更新

install.sh 和 update.sh 脚本现已在纯 IPv6 服务器上正常工作：下载版本文件、x-ui.sh 脚本和服务文件时不再强制使用 IPv4（curl -4），而是使用可用的协议。因此，面板可以在没有 IPv4 地址的主机上安装和更新。

3.5.0 中的脚本修复

- **RHEL 系**（Rocky/Alma/RHEL/Oracle）：从菜单本地安装 PostgreSQL 现在可以正常工作（改用密码认证而非 ident），用于 IP 限制的 fail2ban 从 **EPEL** 安装。
- **Arch/Manjaro**：安装和更新不再执行完整的系统升级 pacman -Syu ——只更新软件包数据库。
- **32 位 ARM**：下载的 Xray 二进制文件命名为 xray-linux-arm32 ——与面板启动它所用的名称一致（此前更新会以其他名称保存文件，面板继续运行旧核心）。
- **IP 证书**：签发前会显示自动检测到的公网 IPv4 供确认（按 Enter 接受）；拒绝时改为经过校验的手动输入。
- **选择 ACME 端口**：按 Enter（接受默认端口 80）不再误报「Your input is invalid」。

通过 XUI_PORT 变量覆盖面板端口

Web 面板的监听端口可通过环境变量 `XUI_PORT` 覆盖——该变量仅在当前进程运行期间生效，**不修改**数据库中保存的 `webPort` 值。允许的值为 1 至 65535；空值、无效值或超出范围的值将被忽略（使用 `webPort`）并在日志中输出警告。这在部署时（尤其是 Docker 中）很有用：使用 bridge 网络时，容器发布端口必须与 `XUI_PORT` 一致——例如 `XUI_PORT=8080` 对应 `ports: "8080:8080"`。

更新与切换 Xray-core 版本

同样在「仪表盘」上，可以独立于面板管理 Xray-core 的版本。

- **Xray 更新**（Xray Updates）/选择版本（Version）——可用版本的下拉列表。提示：「选择所需版本」以及警告「重要：旧版本可能不支持当前配置」。
- 安装/切换版本——`POST /panel/api/server/installXray/{version}`。对话框：「切换 Xray 版本」/「您确定要切换 Xray 版本吗？」。成功时显示「Xray 更新成功」。

示例：通过 API 切换 Xray-core 版本。 版本以 XTLS/Xray-core 的版本标签指定（带 `v` 前缀）。例如，切换至 `v1.8.24`：

```
curl -s -b cookies.txt -X POST \
  https://panel.example.com:2053/panel/api/server/installXray/v1.8.24
```

（`cookies.txt` 为第 16.1 节示例中的 cookie 文件。）安装后，Xray 将自动以所选版本重启。

服务器端切换版本时，先停止 Xray，从 GitHub（XTLS/Xray-core）下载所需版本的归档，解压并替换二进制文件，然后在验证归档/二进制文件校验和后重启 Xray。

16.6. 定时任务（cron）

面板在启动时注册了若干后台任务。其计划是固定的（UI 中不可配置，Telegram 报告计划和 LDAP 同步除外）。以下为与运维相关的任务。

任务	计划	用途
检查 Xray 是否运行	每 1 秒	监控 Xray 进程是否在运行
检查是否需要重启 Xray	每 30 秒	若配置被标记为已更改则重启
收集 Xray 流量	每 5 秒（启动后 5 秒开始）	统计 inbound/客户端流量
检查客户端 IP	每 10 秒	通过日志控制 IP 限制
节点心跳与流量同步	每 5 秒	与节点（nodes）通信
清理日志	每天（@daily）	清理 IP 限制日志和持久访问日志，将当前日志轮转为 <code>*.prev.log</code> 。自 3.5.0 起，每日清理也涉及 Xray 的 error 日志（此前仅 access 日志）

任务	计划	用途
限制 Xray 日志增长	每 10 分钟 (@every 10m)	3.5.0 新增 。当 Xray 的 access 或 error 日志任一超过 64 MiB 时将其截断；已禁用的日志 (none /为空) 不受影响
按周期重置流量	@hourly 、 @daily 、 @weekly 、 @monthly	重置已设置相应自动重置周期的 inbound (及其客户端) 的流量计数器
Telegram 报告	在机器人设置中配置 (默认 @daily)	向管理员发送报告；若启用了相应选项，则附带数据库备份文件 (第 16.1 节)
重置 Telegram 哈希存储	每 2 分钟	仅在机器人启用时
监控 CPU 负载 (Telegram)	每 10 秒	仅在设置了 CPU 阈值 > 0 时

补充说明：

- **定期流量重置**仅对设置了相应自动重置模式 (每小时/每天/每周/每月) 的 inbound 生效。任务会重置 inbound 本身及其所有客户端的流量。
- **到期与耗尽检查**。客户端到期和流量耗尽后的禁用操作在流量统计过程中执行： expiry_time 已过期或流量已耗尽的客户端会被标记并禁用，必要时计算下一个到期时间 (针对循环限制和「首次使用起计」模式)。「仪表盘」和列表中以「已到期」/「已耗尽」/「即将到期」状态反映。
- **Telegram 自动备份**是报告任务的副作用，没有单独仅用于备份的 cron 计划。因此，自动备份的频率与机器人报告的频率相同。

16.7. 控制台菜单与 CLI (x-ui)

在服务器上，面板通过 x-ui 命令管理。不带参数时打开「3X-UI Panel Management Script」交互式菜单；带参数时执行指定子命令。与运维相关的菜单项：

菜单编号	菜单项	操作
1	Install	安装面板 (下载并运行 install.sh)
2	Update	将所有 x-ui 组件更新至最新版本，不丢失数据；完成后自动重启
3	Update to Dev Channel (latest commit)	更新至 dev-latest 滚动构建 (main 分支最新提交)，需确认 (见第 16.5 节)
4	Update Menu	仅更新 x-ui 菜单脚本本身
5	Legacy Version	按输入的版本号安装指定的旧版面板 (例如 2.4.0)
6	Uninstall	完全卸载面板和 Xray (见第 16.8 节)
7	Reset Username & Password	重置管理员用户名/密码
8	Reset Web Base Path	重置 Web Base Path
9	Reset Settings	将设置重置为默认值
10	Change Port	修改面板端口

菜单编号	菜单项	操作
11	View Current Settings	查看当前设置
12-14	Start / Stop / Restart	启动、停止、重启面板服务
15	Restart Xray	仅重启 Xray
16	Check Status	当前服务状态
17	Logs Management	查看与清理日志（见下文）
18-19	Enable / Disable Autostart	启用/禁用操作系统启动时自动运行服务
27	Update Geo Files	更新地理文件（GeoIP/GeoSite）
25	PostgreSQL Management	管理 PostgreSQL

菜单项编号在 3.4.1 版本中有所变化：由于新增了第 3 项「Update to Dev Channel」，其后的所有项目编号均后移了一位。菜单共 28 项，选择范围为 [0-28]。

在 CLI 中管理日志（第 16 项）

「Logs Management」子菜单现在通过第 17 项打开（此前为第 16 项）：

- **Debug Log**——以流式方式查看服务日志：`journalctl -u x-ui -e --no-pager -f -p debug`（在 Alpine 上使用 `grep` 读取 `/var/log/messages`）。
- **Clear All logs**——清理系统日志：`journalctl --rotate + journalctl --vacuum-time=1s`，之后重启服务。（Alpine 上不可用。）

x-ui 直接子命令

所有可用子命令：

命令	说明
<code>x-ui</code>	打开管理菜单
<code>x-ui start</code>	启动面板
<code>x-ui stop</code>	停止面板
<code>x-ui restart</code>	重启面板
<code>x-ui restart-xray</code>	重启 Xray
<code>x-ui status</code>	当前状态
<code>x-ui settings</code>	显示当前设置
<code>x-ui enable</code>	启用开机自启
<code>x-ui disable</code>	禁用开机自启

命令	说明
<code>x-ui log</code>	查看日志
<code>x-ui banlog</code>	查看 Fail2ban 封禁日志
<code>x-ui setup-fail2ban</code>	安装并配置 fail2ban 以实现 IP 限制（见第 16.5 节）
<code>x-ui update</code>	更新面板

| `x-ui update-dev` | 将面板更新至开发频道（`dev-latest` 滚动构建） || `x-ui update-all-geofiles` | 更新所有地理文件（并随后重启） || `x-ui migrateDB [file]` | 转换数据库 `.db` 至 `.dump`（SQLite） || `x-ui legacy` | 安装旧版本 || `x-ui install` | 安装面板 || `x-ui uninstall` | 卸载面板 |

命令 `x-ui update` 会下载并运行官方 `update.sh`（与第 16.5 节中 Web 更新使用的脚本相同），并请求确认：「This function will update all x-ui components to the latest version, and the data will not be lost.」完成后面板自动重启。

setting 子命令中的 `-webCert` / `-webCertKey` 标志。Web 面板的证书和私钥路径可直接通过子命令 `x-ui setting -webCert <[?]> -webCertKey <[?]>` 设置——指定其中任意一个标志即可保存相应路径（与单独的 `cert` 子命令效果相同），面板将立即切换至 HTTPS。

通过 CLI 获取 API 令牌

通过 CLI（菜单项/`x-ui` 命令）获取 API 令牌时，不会显示之前已签发的令牌。API 令牌仅以哈希形式存储，因此无法以明文形式获取现有令牌。若已配置令牌，命令会告知令牌数量，建议在面板中管理令牌（**Settings** → **API Tokens**，见 API 令牌相关章节），并立即生成一个名称格式为 `cli-fallback-<timestamp>` 的新备用令牌并输出，以便在不登录界面的情况下仍能通过 CLI 使用。

16.8. 卸载面板

卸载通过 CLI 执行——菜单第 **5 项（Uninstall）** 或命令 `x-ui uninstall`。卸载前会请求确认（默认为「否」）：「Are you sure you want to uninstall the panel? xray will also uninstalled!」。

确认后，脚本将：

1. 停止服务并禁用自启（`systemctl stop/disable x-ui`，Alpine 上使用 `rc-service / rc-update`），删除服务 `unit` 文件并重新加载 `systemd` 配置。
2. 删除数据目录和应用程序目录（`/etc/x-ui/`、安装目录）以及服务环境文件（`/etc/default/x-ui`、`/etc/conf.d/x-ui` 或 `/etc/sysconfig/x-ui`——取决于发行版）。
3. 删除 `x-ui` 脚本本身，并输出「Uninstalled Successfully.」以及重新安装的命令。

若面板使用了 PostgreSQL（环境文件中 `XUI_DB_TYPE=postgres`），删除面板文件后，脚本会额外询问是否同时删除 PostgreSQL 服务器及其所有数据库：「Also purge PostgreSQL and delete all of its data?」。该请求需要明确确认（默认拒绝），并附有警告：删除将影响该机器上所有 PostgreSQL 数据库，包括属于其他应用程序的数据库，且操作不可逆。若拒绝，PostgreSQL 及其数据保持不变。

卸载操作不可逆：面板、Xray 以及所有数据（包括数据库）将一并删除。若数据可能日后用到，请提前导出数据库（第 16.1 节）。

16.9. x-ui migrateDB 命令

自 3.3.0 版本起，管理脚本 `x-ui.sh` 新增了 `migrateDB` 子命令——这是对内置二进制文件 `x-ui`（`x-ui migrate-db`）的封装，用于在两种格式之间转换面板的 SQLite 数据库：二进制 `.db` 文件和可移植文本转储 `.dump`（普通 SQL 文本）。

命令功能

该命令支持两个方向，且方向根据输入文件自动判断：

方向	名称	操作
<code>.db → .dump</code>	dump（导出）	将二进制 SQLite 数据库导出为 SQL 文本文件
<code>.dump → .db</code>	restore（恢复）	从 SQL 文本文件重新构建二进制 SQLite 数据库

底层脚本调用面板二进制文件：

- 导出：`x-ui migrate-db --src <[?]?> --dump <[?]?>`
- 恢复：`x-ui migrate-db --restore <[?]?> --out <[?]?>`

调用语法

```
x-ui migrateDB [file.db|file.dump] [output]
```

- **[file.db|file.dump]** ——输入文件（第一个参数）。若未指定，则使用面板的默认安装数据库：`/etc/x-ui/x-ui.db`。
- **[output]** ——输出文件路径（第二个参数）。可选：若未指定，则在输入文件旁自动选择名称（见下文）。

示例：

```
x-ui migrateDB                               # [?]? /etc/x-ui/x-ui.db -> /etc/x-ui/x-
ui.dump
x-ui migrateDB /etc/x-ui/x-ui.db backup.dump
x-ui migrateDB backup.dump restored.db      # [?]?[?]?[?]?[?]? .db
```

如何判断方向

脚本根据输入文件的扩展名判断：

- `*.db`、`*.sqlite`、`*.sqlite3` → **dump** 模式（导出为文本）；
- `*.dump`、`*.sql` → **restore** 模式（构建数据库）。

若扩展名无法识别，脚本读取文件前 16 字节：签名 `SQLite format 3` 表示二进制数据库（dump 模式），否则将文件视为转储文件（restore 模式）。

若未指定第二个参数，输出文件名如下：

- 导出时——与输入文件同名，扩展名改为 `.dump`；
- 恢复时——与输入文件同名，扩展名改为 `.db`。

保护性检查与行为

- **二进制文件是否存在。** 若找不到 `x-ui` 二进制文件或其不可执行，则输出错误「`x-ui binary not found ... Is the panel installed?`」。
- **构建是否支持该功能。** 脚本通过 `x-ui migrate-db -h` 检查二进制文件是否支持 `migrate-db --dump/--restore`。若不支持，建议先通过 `x-ui update` 更新面板。
- **输入文件是否存在。** 若输入文件不存在，输出错误信息和调用语法说明。
- **覆盖输出文件。** 若输出文件已存在，将请求确认（默认为「否」）；若不确认，操作取消。恢复时会预先删除旧的输出文件。
- **保护「活动」数据库。** 恢复到默认数据库 `/etc/x-ui/x-ui.db` 时，若面板正在运行，操作将被拒绝，并要求先停止面板（`x-ui stop`）或选择其他输出路径。这可防止在服务运行期间覆盖工作数据库。
- 若数据库构建失败，不完整的输出文件将被删除。

使用场景

- **备份。** 文本 `.dump` 文件具有可读性，便于存储在版本控制系统中，并可对比数据库内容的差异。
- **迁移。** 转储文件可在不同机器间传输，且对 SQLite 文件格式版本差异具有鲁棒性——在新服务器上可从中构建可用的 `.db`。
- **诊断。** 通过 `.dump` 文件可直接查看面板的结构和数据，无需 SQLite 工具。

交互式模式

除直接调用外，转换功能也可从交互式菜单使用。在 PostgreSQL 子菜单（`x-ui` → PostgreSQL 管理部分）中，有第 9 项「**Convert SQLite `.db` <=> `.dump`**」：该项会询问输入文件路径（默认 `/etc/x-ui/x-ui.db`）和输出文件路径（可留空以自动命名），方向同 CLI 模式一样自动判断。

本文档根据 3X-UI 源代码整理。若您版本中的某个界面项与此处有出入，以面板本身的行为和 UI 提示为准。